



Guidewire InsuranceSuite™ for Guidewire Cloud

Gosu Rules Guide

Release: Latest



© 2026 Guidewire Software, Inc.

For information about Guidewire trademarks, visit <https://www.guidewire.com/legal-notices>.

Guidewire Proprietary & Confidential — DO NOT DISTRIBUTE

Product Name: Guidewire InsuranceSuite

Product Release: Latest

Document Name: Gosu Rules Guide

Document Revision: 01-April-2026

Contents

Support.....	11
1 Gosu rules: A background.....	13
Guidewire rule types.....	14
Gosu rule hierarchy.....	15
Gosu rule execution.....	15
Gosu rule management.....	17
2 About Gosu rules.....	19
Rule structure.....	19
Gosu rule syntax.....	20
Rule members.....	20
Understanding Gosu rule conditions.....	21
Understanding Gosu rule actions.....	22
Exiting a Gosu rule.....	22
How to use exitToNextRoot in ClaimCenter rules.....	23
How to use exitToNextRoot in PolicyCenter rules.....	23
Gosu annotations and Gosu rules.....	24
The effect of annotations on Gosu rules.....	24
Determine the annotations on elements of Gosu code.....	24
Invoking a rule from Gosu code.....	24
3 Naming Gosu rules.....	25
Creating unique rule names.....	25
ClaimCenter rule naming example.....	26
Parent and child rules in ClaimCenter.....	26
Operating system length restrictions.....	27
4 Using the rules editor.....	29
View or edit a rule.....	29
Changing rule order.....	29
Create a new rule set category.....	30
Create a new rule set.....	31
Create a new rule.....	31
Accessing a rule set from Gosu code.....	32
Renaming or deleting a rule.....	32
About renaming a rule.....	32
Making a rule active or inactive.....	32
5 Advanced rule writing.....	33
Accessing fields on subtypes.....	33
Preventing repeated actions in ClaimCenter rules.....	34
Taking action on more than one subitem.....	34
Checking entity permissions.....	35
Determining the original attributes of a changed entity.....	35
About changing the root entity of a rule.....	36
Change the root entity of a rule.....	37

6	Rule testing and debugging.....	39
	Generating logging information.....	39
	Returning rule names in messages.....	41
7	BillingCenter rule set categories.....	43
	Archive.....	43
	Assignment.....	43
	Event message.....	44
	About messages and messaging events.....	45
	Generating messages.....	46
	Event fired rule example – constructing a message.....	46
	Event fired rule example – asynchronous document storage failure.....	47
	Exception.....	47
	Activity escalation rules.....	47
	Preupdate.....	48
	How BillingCenter triggers preupdate rules.....	48
	Triggering Preupdate rules on custom entities.....	49
	Transaction Preupdate rule example – creating a history event.....	49
	Validation.....	50
8	ClaimCenter rule set categories.....	51
	Approval routing.....	52
	Approval rules and Transaction approval rules.....	52
	BulkInvoice Approval Assignment Rules.....	54
	Archive.....	54
	Assignment.....	55
	Default Group Activity Assignment rules.....	55
	Default Group Claim Assignment rules.....	56
	Default Group Exposure Assignment rules.....	56
	Default Group ServiceRequest Assignment rules.....	57
	Global Activity Assignment rules.....	57
	Global Claim Assignment rules.....	59
	Global Exposure Assignment rules.....	60
	Global Matter Assignment rules.....	61
	Global ServiceRequest Assignment rules.....	61
	BulkInvoice.....	61
	Bulk Invoice Approval rules.....	61
	Closed.....	62
	Activity Closed rules.....	62
	Claim Closed rules.....	62
	Exposure Closed rules.....	63
	Matter Closed rules.....	64
	Event message.....	64
	About messages and messaging events.....	64
	About modifying data in Event Fired rules and messaging plugins.....	66
	Event Fired rule example.....	66
	ISO Event message rules.....	67
	MetroReport Event message rules.....	68
	System console logger Event message rule.....	68
	Contact Auto Synchronization Failure Event message rule.....	68
	Policy System Notification Event message rules.....	69
	Asynchronous Document Storage Event message rule.....	69

Contact Event message rules.....	70
Instant check message rules.....	71
Payment and recovery on account message rules.....	72
Solr Destination Event message rules.....	72
Exception.....	74
Activity Escalation rules.....	74
Claim Exception rules.....	75
Group Exception rules.....	77
User Exception rules.....	78
Initial Reserve.....	78
Initial Reserve rules for auto vehicle damage.....	78
Initial Reserve rules for travel losses.....	79
Loaded.....	80
Claim Loaded rules.....	80
Exposure Loaded rules.....	81
Postsetup.....	81
Activity Postsetup rules.....	81
Claim Postsetup rules.....	81
Exposure Postsetup rules.....	82
Matter Postsetup rules.....	82
Transaction Postsetup rules.....	83
Presetup.....	83
About claim and exposure setup.....	83
Creating Transaction Set Presetup rules.....	85
Preupdate.....	87
How ClaimCenter triggers Preupdate rules.....	88
How ClaimCenter triggers Preupdate rules on custom entities.....	88
Activity Preupdate rules.....	89
Claim Preupdate rules.....	89
Contact Preupdate rules.....	94
Exposure Preupdate rules.....	94
Group Preupdate rules.....	95
Matter Preupdate rules.....	96
Policy Preupdate rules.....	96
RIAgreementPreupdate rules.....	96
RITransactionSetPreupdate rules.....	97
TransactionSet Preupdate rules.....	97
User Preupdate rules.....	99
Purge.....	99
Reopened.....	99
Claim Reopened rules.....	99
Exposure Reopened rules.....	100
Matter Reopened rules.....	100
Segmentation.....	100
Claim Segmentation rules.....	101
Exposure Segmentation rules.....	102
Handling a strategy separately from a segment.....	105
Transaction Approval.....	106
Transaction Approval rules.....	106
Validation.....	107
About ClaimCenter validation levels.....	108

About validation warnings and errors.....	109
Adding new validation levels.....	109
How ClaimCenter triggers validation rules.....	109
How ClaimCenter triggers Validation rules on custom entities.....	110
ClaimCenter Validation and Preupdate rules.....	110
Determining the validity of an object.....	110
Validation error reject methods.....	111
Activity Validation rules.....	113
Claim Closed Validation rules.....	113
Claim Reopened Validation rules.....	114
Claim Validation rules.....	114
Contact Validation rules.....	118
Exposure Closed Validation rules.....	118
Exposure Reopened Validation rules.....	119
Exposure Validation rules.....	120
Group Validation rules.....	123
Matter Closed Validation rules.....	123
Matter Reopened Validation rules.....	123
Matter Validation rules.....	123
Person Validation rules.....	124
Policy Validation rules.....	124
RIAgreement Validation rules.....	125
RITransactionSet Validation rules.....	126
ServiceRequest Validation rules.....	127
TransactionSet Validation rules.....	127
User Validation rules.....	130
Workplan.....	130
Gosu methods to use with Workplan rules.....	130
Claim Workplan rules.....	130
Exposure Workplan rules.....	132
Matter Workplan rules.....	133
9 PolicyCenter rule set Categories.....	135
Assignment.....	135
Audit.....	136
Reporting Trend Analysis rule set.....	136
Event Message rules.....	136
Messaging events.....	137
Message destinations and message events.....	137
Message destinations and message plugins.....	137
Generating messages.....	138
Listening for object changes that trigger events.....	138
Example Event Fired rule.....	138
Example asynchronous document storage failed Event Fired rule.....	139
Exception.....	139
Activity Escalation rules.....	139
Group Exception rules.....	140
Policy Exception rules.....	140
Premium Report Escalation rules.....	142
User Exception rules.....	143
Renewal.....	143
Renewal AutoUpdate.....	143

Validation.....	143
Validation in PolicyCenter.....	144
PolicyCenter validatable entities.....	145
How an entity triggers Validation rules.....	146
Triggering Validation rules on custom entities.....	146
The validate method.....	146
Account Validation rule example.....	147
10 Rule reports	149
About the Rule Repository report.....	149
Generate a Rule Repository Report.....	149
About the Profiler Rule Execution report.....	150
Generate a Profiler Rule Execution Report.....	150
Understanding the Profiler Rule Execution Report.....	150
View rule information in the Profiler Chrono report.....	151
11 Assignment in InsuranceSuite applications.....	153
Primary and secondary assignment.....	153
Primary (user-based) assignment entities in ClaimCenter and PolicyCenter.....	154
Secondary (role-based) assignment entities in ClaimCenter and PolicyCenter.....	155
Assignment within the Assignment rules.....	156
Assignment outside the Assignment rules.....	157
Global and default Assignment rules in BillingCenter and ClaimCenter.....	157
Global Assignment rules.....	158
Default Group Assignment rules.....	159
Assignment execution session in BillingCenter and ClaimCenter.....	159
Role assignment in PolicyCenter.....	160
Role assignment and Submission Jobs.....	160
Role assignment and non-Submission jobs.....	161
Gosu support for assignment entities in PolicyCenter.....	161
Assignment success or failure in BillingCenter and ClaimCenter.....	162
Assignment success or failure within the context of rules.....	162
Assignment success or failure outside the context of rules.....	162
Logging assignment activity.....	163
Assignment success or failure in PolicyCenter.....	163
Logging assignment activity.....	164
Assignment cascading in BillingCenter and ClaimCenter.....	164
Assignment events.....	164
Assignment method reference.....	165
Assignment by assignment engine in BillingCenter and ClaimCenter.....	165
Claim-based assignment in ClaimCenter.....	166
Condition-based assignment in ClaimCenter and PolicyCenter.....	167
Dynamic assignment.....	170
Group assignment (within the Assignment rules) in BillingCenter and ClaimCenter.....	172
Immediate assignment.....	176
Manual assignment in ClaimCenter.....	177
Proximity-based assignment in ClaimCenter.....	178
Queue assignment in ClaimCenter and PolicyCenter.....	182
Round-robin assignment.....	183
Round-robin assignment of similar exposures in ClaimCenter.....	183
Workload Count in ClaimCenter.....	186

12	ClaimCenter rules-based validation.....	187
	Overview of rules-based validation.....	187
	Entity validation order.....	188
	About the validation graph.....	188
	How ClaimCenter traverses the validation graph.....	188
	How ClaimCenter works with owned arrays.....	189
	Top-level entities that trigger validation.....	190
	Overriding validation triggers.....	191
	Validation performance issues.....	192
	View the validation graph in the server log.....	193
13	Detecting claim fraud in ClaimCenter.....	195
	The Special Investigation Details screen.....	195
	Special Investigation trigger rules.....	196
	SI rule evaluation sequence.....	196
	Using the Special Investigation rules.....	196
	Set the initial life cycle stage.....	197
	Advance the life cycle stage.....	197
	Evaluating SI triggers.....	198
	Escalating a claim to a supervisor.....	199
	Assigning an activity to a SIU group.....	200
	Adding an assigned user in a SIU role.....	200
14	Transaction Set validation in ClaimCenter.....	201
	Transaction Set validation rules.....	201
	TXV01000 – Total payments not greater than exposure limit.....	202
	TXV02000 – Check exposure limit when increasing reserves.....	203
	TXV03000 – Total payments not greater than incident limit.....	204
	TXV04000 – Check incident limit when increasing reserves.....	204
	TXV05000 – PIP.....	204
	TXV06000 – Reserve threshold.....	205
	TXV08000 – Check aggregate limits.....	206
	TXV13000 – Warning if adverse party pays non-subrogation recovery.....	206
	TXV17000 – Reserve lines compatible with service requests.....	206
	TXV18000 – Currencies compatible with service requests.....	207
	TXV24000 - Disallow When Policy Agreement is Inconsistent.....	207
	sumForPIPAgg enhancement methods.....	207
15	Working with queues in ClaimCenter.....	209
	Example: Using a queue to handle FNOL claims.....	209
	Assign a claim to an intake group.....	210
	Assign a claim to a group supervisor.....	210
	Create a FNOL review activity.....	211
	Assign a FNOL review activity to an intake group.....	211
	Assign a FNOL review activity to a queue.....	212
	Example: Using a queue to void/stop checks.....	212
	Create a stop/void queue.....	213
	Assign an activity to the appropriate group.....	213
	Add an activity to a queue.....	214
16	PolicyCenter Rule-based validation.....	215
	About the validation graph.....	215
	How PolicyCenter traverses the validation graph.....	216

How PolicyCenter works with owned arrays.....	217
Top-level entities that trigger full validation.....	217
Overriding validation triggers.....	218
Validation performance issues.....	218
View a text version of the validation graph.....	219

Support

For assistance, visit the Guidewire Community.

Guidewire customers

<https://community.guidewire.com>

Guidewire partners

<https://partner.guidewire.com>

Gosu rules: A background

Designing Gosu rules

In general, Guidewire strongly recommends that you develop and document the functional logic of rules before attempting to turn that logic into rules within an application. In a large implementation, there can be a large number of rules, so it is extremely beneficial to organize the basic structure of the rules in advance. Use this guide to understand how rules work. It can also help you make decisions about changing your rules as your use of the InsuranceSuite application evolves over time.

Note: In the following topics, the *application* refers to an InsuranceSuite application such as BillingCenter, ClaimCenter, PolicyCenter, or ContactManager when the information may apply to more than one InsuranceSuite application.

Note: The following topics include information about all Gosu rules up to and including the latest release of InsuranceSuite. To view reports on any new or changed Gosu rules in ClaimCenter or ContactManager, refer to the *InsuranceSuite Release Notes* for those applications.

Gosu sample rules

Guidewire provides a set of sample rules as examples and for use in testing. These are sample rules only, and Guidewire provides these rules merely as a starting point for designing your own rules and rule sets. You access sample rules (and other Studio resources) through the Studio interface.

Gosu rule terminology

Guidewire uses the following terminology in discussing Gosu rules.

assignment engine (BillingCenter and ClaimCenter only)

In BillingCenter, the assignment engine handles assignment of trouble tickets and activities to users by repeatedly asking BillingCenter to evaluate assignment rules for the item. Guidewire structures the assignment rule sets so that the assignment engine first asks to which group the assignment goes. It then asks to which person within the group (with each of these decisions governed by a different rule set).

In ClaimCenter, the assignment engine handles assignment of claims, exposures, and activities to users by repeatedly asking ClaimCenter to evaluate assignment rules for the item. Guidewire structures the assignment rule sets so that the assignment engine first asks to which claim handling office the assignment goes. It then asks to which group, and finally to which person (with each of these decisions governed by a different rule set).

entity

An entity is a type of business data configured in the data model configuration files. For example, a Policy in PolicyCenter, a Claim in ClaimCenter, an ABCContact in ContactManager, and an Account in BillingCenter. You

can use the *Data Dictionary* to review the entity types and their properties. For an entity type, this documentation refers to an instance as an *entity instance* or, for brevity, *object*.

Guidewire Studio

Guidewire Studio is the Guidewire administration tool for managing the application resources, such as PCF pages, Gosu rules, and Gosu classes.

job (PolicyCenter only)

A job encapsulates a specific set of tasks that PolicyCenter can perform on a `PolicyPeriod` object.

library

A library is a collection of functions (methods) that you can call from within your Gosu programs. Guidewire provides a number of standard library functions (in the `gw.api.*` packages, for example).

object

An object refers to any of the following:

- An instance of an *entity type*, such as `Activity`. For an entity type, Guidewire also calls an object an *entity instance*.
- An instance of an Gosu class
- An instance of a Java class, such as `java.util.ArrayList`.

policy (PolicyCenter only)

A Policy encapsulates all the information about a risk underwritten by an insurer. You can also think of a policy as a container of logical policy periods (with a specific range of effective time) for a policy.

policy period (PolicyCenter only)

A logical period of time for which a policy is in effect. For example, a typical year-long personal auto policy is one logical policy period. If you modify that policy several times, PolicyCenter keeps each version, but represents all of them with the same *logical policy period*.

PolicyCenter represents each logical policy period by a single identifier called a fixed ID. PolicyCenter represents each *version of the logical period* by a `PolicyPeriod` entity instance, which stores its period ID in a field called `PeriodID`.

Although similar, a policy period as a concept and the `PolicyPeriod` entity type are not exactly the same.

rule

A rule is a single decision in the following form:

```
If {some conditions}
Then {take some action}
```

The following examples illustrate the logic of a validation rule:

In `ClaimCenter`, if the claim has no loss date or the claim has a loss date in the future, mark the loss date field as invalid. In `PolicyCenter`, if the auto policy does not list a Vehicle Identification Number (VIN) for all covered automobiles, then mark the policy as invalid. In `BillingCenter`, if the contact information does not include a valid country code for the listed home phone number, then raise an error.

rule set

A rule set combines many individual rules into a useful set to consider as a group.

workflow

A workflow is a Guidewire mechanism to run custom business processes asynchronously, optionally with multiple states that transition over time. You create and develop workflow within Guidewire Studio.

Guidewire rule types

Guidewire provides the following types of rules in the base configuration of each InsuranceSuite application.

Gosu rules

You create and manage Gosu rules entirely in Guidewire Studio. Gosu rules require in-depth domain knowledge and technical expertise to create. After you make changes to Gosu rules, you typically need to restart the application server.

ClaimCenter business rules

You use business rules to create, edit, and manage activities. You work with business rules in the **Business Rules** screens, in ClaimCenter by navigating to **Administration > Business Settings**.

The target users of business rules are ClaimCenter administrators and business analysts. It is possible to create and deploy ClaimCenter business rules without starting the application server.

ClaimCenter provides the following types of business rules:

- Activity rules
- Exposure rules
- Reserve rules

These rules perform different functions. For example, you use activity business rules for tasks such as the following:

- Adding an activity to the workplan
- Creating activities more selectively than through Gosu rules
- Changing an activity assignment
- Adding a new activity pattern to the workplan
- Disabling an activity

PolicyCenter business rules

You use business rules to create, edit, and manage underwriting issues. You work with business rules in the **Business Rules** screens, in PolicyCenter by navigating to **Administration > Business Settings**.

The target users of business rules are PolicyCenter administrators and business analysts. It is possible to create and deploy PolicyCenter business rules without restarting the application server.

You use business rules for defining underwriting issue types. In the underwriting rules you can:

- Test for the existence of an underwriting issue on a policy
- Define the course of action to take for an underwriting issue
- Define whether to block the progress of a policy transaction until the underwriting issue receives approval.

Gosu rule hierarchy

A rule set can be thought of as a logical grouping of rules that are specific to a business function within the application. You typically organize these rules sets into a hierarchy that fits your business model. Guidewire strongly recommends that you implement a rule-naming scheme as you create rules and organize these rules into a hierarchy.

Prior to implementing rules, it is important to first understand the rule hierarchy that groups the rules. The rule hierarchy is the context in which the application groups all rules. You can implement a rule hierarchy in several formats, depending on the needs of your organization. However, it is important to outline this hierarchy up-front before creating the individualized rules to reduce potential duplicates or unnecessary rules. You can create multiple hierarchies within the application. However, make each hierarchy specific to the rule set to which it belongs.

Gosu rule execution

The hierarchy of rules in the application mirrors a decision tree that you might diagram on paper. The application considers the rules in a very specific order, starting with the first direct child of the root. (The first direct child is the first rule immediately following the line of the rule set.) The application moves through the hierarchy according to the

following algorithm. Recursively navigating the rule tree, it processes the parent and then its children, before continuing to the next peer of the parent.

- Start with the first rule
- Evaluate the rule's conditions. If true...
 - Perform the rule's actions
 - Start evaluating the rule's children, starting with the first one in the list.

You are done with the rule if a) its conditions are false, or b) its conditions are true and you processed its actions and all its child rules.

- Move to the next peer rule in the list. If there are no more peers, then the rules at the current level in the tree are complete. After running all the rules at the top level, rule execution is complete for that rule set.

In ClaimCenter, to illustrate how to use a rule hierarchy, take segmentation logic as an example. You would probably expect to describe a series of questions to figure out in which segment to put a claim. For convenience, you would want to describe these questions hierarchically. For example, if you answer question #1 as No, then skip directly to question #20 because questions #2-19 only pertain if you answered #1 as Yes.

You might go through logic similar to the following:

- **If this is an auto claim**, then go through a list of more detailed segmentation rules for auto claims:
 - If this has glass damage only, then segment as "Auto Glass" [done]
 - Otherwise, if this was a collision, then consider collision segmentation rules
 - If there are injuries, then segment as "injury accident" [done]
 - Else if there are more than two cars, segment as "multi-car accident" [done]
 - Else if there are two cars, segment as "2 car accident" [done]
 - Else segment as "single car accident" [done]
 - Otherwise, consider non-collision segmentation rules
 - If there are third-party exposures, segment as "non-accident liability" [done]
 - Else, if the cause of loss is theft, then segment as "theft" [done]
 - Else, segment as "non-accident vehicle damage" [done]
- **If this is a homeowner's claim**, then go through a list of more detailed segmentation rules for homeowner's claims...
- **If all else fails [Default]**, segment this as "unknown" [done]

You can see from this example that your decision tree follows a hierarchy that maps to the way ClaimCenter keeps rules in the hierarchy. If the first rule checks whether the claim is an auto claim, then ClaimCenter does not need to check any of the follow-up (child) rules. (That is, unless the claim is actually an auto claim.)

In PolicyCenter, to illustrate how to use a rule hierarchy, take the business logic used to determine the type and number of forms that a certain policy requires as an example. You would probably expect to describe a series of questions to figure out in which forms are mandatory and required by each policy. For convenience, you would want to describe these questions hierarchically. For example, if you answer question #1 as No, then skip directly to question #20 because questions #2-19 only pertain if you answered #1 as Yes.

You might go through logic similar to the following:

- **If this is a Workers' Compensation policy**, go through a list of more detailed rules for this policy line ...
 - If this is the state of Washington, [done]
 - Otherwise... If this is the state of California and business is not construction-related ...
 - If payroll less than \$100,000[done]
 - Else, if payroll less than \$1,000,000 [done]

- Else, if payroll more than or equal to \$1,000,000 [done]
- Otherwise, consider construction-related (high hazard) rules ...
 - If the number of employees less than 50 [done]
 - Else, if the number of employees less than 150 [done]
 - Else, if the number of employees greater than or equal to 150 [done]
- **If this is a BOP policy**, then go through a list of more detailed rules for this policy line ...
- **If all else fails [Default]**, use the default rule action [done]

You can see from this example that your decision tree follows a hierarchy that maps to the way PolicyCenter keeps rules in the hierarchy. If the first rule checks whether the policy line is Workers' Compensation, then PolicyCenter does not need to check any of the follow-up (child) rules. (That is, unless the policy is actually a Workers' Compensation policy.)

Gosu rule management

Guidewire strongly recommends that you tightly control access to changing rules within your organization. Editing rules is a complicated process. Because the application uses rules to make automated decisions regarding many important business objects, you need to be careful to verify rule changes before moving them into production use.

Typically, the following kinds of people manage Gosu rules.

Business analysts

One or more business analysts who own decision-making for making the necessary rules. Business analysts must understand the normal business process flow and must understand the needs of your business organization and how to support these needs through rules in the application.

Technical rule writers

One or more rule writers who are generally more technical than business analysts. Possibly, this can be someone on the business side with a good technical aptitude. Or possibly, this can be someone within IT with a good understanding of the business entities that are important to your business. Rule writers are responsible for encoding rules and editing the existing set of rules to implement the logic described by business analysts. The rule writers work with the business analysts to create feasible Gosu rules. These are rules that you can actually implement with the information available to the application.

About Gosu rules

In the base configuration, Guidewire provides a number of sample rules as examples of how to use rules to perform business logic. Guidewire divides the sample rule sets into categories, or large groupings of rules that center around a certain business process, such as assignment or validation. In the rules hierarchy rule set categories consist of rule sets, which, in turn, further subdivide into individual rules. Rule sets are logical groupings of rules specific to a business function with each Guidewire InsuranceSuite application. Rules contain the Gosu code (condition and action) that perform the actual business logic.

Rule structure

Every Gosu rule has the following basic syntax:

```
IF {some condition}
THEN {do something}
```

Each Gosu rule divides the rule functionality into different blocks of code, with a specific label. The following list summarizes each of these blocks, in the order in which a block occurs in a rule.

USES

A list of packages or classes that this rule uses, for example:

```
uses java.util.HashSet
uses gw.lang.reflect.IType
```

CONDITON

A Boolean expression (that is, a series of questions connected by AND and OR logic that evaluates to TRUE or FALSE). For example:

```
(This is an auto claim) AND (the policy includes collision coverage)
```

It is also possible to insert a statement list, instead of a simple expression, for example:

```
var o = new HashSet<IType>() {A, B, C, ...}
return o.contains(typeof(...))
```

The CONDITION block must contain a return statement that returns a Boolean value.

ACTION

A list of actions to take. For example:

- Create a history event in BillingCenter
- Add a follow-up activity in BillingCenter
- Mark the claim as flagged in ClaimCenter
- Add an activity for the adjuster to follow up in ClaimCenter
- Mark the renewal as flagged in PolicyCenter
- Add an activity for the underwriter to follow up in PolicyCenter

The best way to add rules to the rule set structure is to right-click in the Studio rule pane. After you do this, Studio opens a window containing a tree of options from which to select. As you use the right-click menu, the application gives you access to conditions and actions that are appropriate for the type of your current rule.

Gosu rule syntax

Guidewire Gosu rules use a programming language called Gosu, which is a high-level language tailored for expressing Gosu rule logic. The syntax supports rule creation with business terms, while also using common programming methods for ease of use. Gosu syntax can be thought of in terms of both statements and expressions. Before you begin to write rules, Guidewire strongly recommends that you make yourself completely acquainted with the Gosu programming language.

Statements are merely phrases that perform tasks within Studio. Examples of statements include the following:

- Assignment statements
- If statements
- Iteration statements

All expressions use a dot notation to reference fields, subobjects, and methods.

For example, in ClaimCenter, to reference the license plate data from a vehicle associated with a claim, the field path is:

```
claim.Vehicles[0].LicensePlate
```

A statement can consist of one or many expressions.

For example, in PolicyCenter, to retrieve the latest PolicyPeriod (of possibly several branches), do the following:

```
var period = Job.LatestPeriod
```

An expression can consist of one or many statements.

For example, in BillingCenter, to reference the group assigned to a trouble ticket, do the following:

```
var thisGroup = troubleTicket.CurrentAssignment.AssignedGroup
```

IMPORTANT: Use only Gosu expressions and statements to create Gosu rules in the application. For example, do not attempt to define a Gosu function in a Gosu rule. Instead, define a Gosu function in a Gosu class or enhancement, then call that function from a Gosu rule. Guidewire expressly does not support the definition of functions—and especially nested functions—in Gosu rules.

Rule members

As described previously, a rule consists of a set of conditions and actions to perform if all the conditions evaluate to true. It typically references the important business entities and objects in the application such as claims in ClaimCenter.

Rule conditions

Rule conditions are a collection of expressions that provide true/false analysis of an entity. If the condition evaluates to true, then Studio runs the activities in the ACTION block. For example, you can use the following condition to test whether the contact information includes a home phone number:

```
contact.HomePhone == null
```

Rule actions

Rule actions are a collection of action expressions that perform business activities such as making an assignment or marking a field as invalid. These actions occur only if the corresponding condition in the CONDITION block evaluates to true. For example, if a contact information is missing a home phone number, you can use the following code to raise an error in the application:

```
contact.rejectField("HomePhoneCountry",  
    ValidationLevel.TC_LOADSAVE, displaykey.Validator.Phone.Home.CountryCode.Null, null, null)
```

Rule APIs

Rule APIs are a collection of Gosu methods accessed through the `gw.api.*` package. They include many standard mathematic, date, string, and financial methods. For example, you can use the following code to determine if an activity (act) is more than 15 days old and thus needs additional handling:

```
gw.api.util.DateUtil.differenceInDays(act.CreateTime, gw.api.util.DateUtil.currentDate()) > 15
```

Rule entities

Rule entities (objects) supported by Guidewire Studio all use the familiar “dot” notation to reference fields and objects.

In ClaimCenter, you can use the following object conditions to verify if the loss is an Auto claim involving a vehicle collision without the weather field entered:

```
Claim.LossType == "auto" and Claim.LossCause == "vehcollision" and Claim.Weather == null
```

In PolicyCenter, you can use the following object condition to automatically raise evaluation issues for the specified location:

```
PolicyPeriod.autoRaiseLocationEvalIssues(location)
```

In BillingCenter, you can use the following code to assign a trouble ticket to a member of the assigned group:

```
troubleTicket.CurrentAssignment.assignUserByRoundRobin(false, troubleTicket.AssignedGroup)
```

Understanding Gosu rule conditions

The simplest kind of condition looks at a single field on the object or business entity, for example:

```
activity.ActivityPattern.Code == "AuthorityLimitActivity"
```

This example demonstrates some important basics:

1. To reference an object (for example, an Activity object, or, in this case, an ActivityPattern object) and its attributes, you begin the reference with a *root object*. While running rules on an activity, you reference the activity in question as Activity. Other root objects, depending on your Guidewire application, are Account, PolicyPeriod, Producer, Invoice, TroubleTicket, and Charge.
2. The application uses dot notation to access fields (for example, Activity.ActivityPattern) or objects (Activity.ActivityPattern.Category, for example), starting from the root object.

Combining rule conditions

For more complicated conditions, you can combine simple conditions using standard Boolean logic operators (and, or, not), for example:

```
assignedGroup != null && activity.assignUserByRoundRobin(true, assignedGroup)
```

IMPORTANT: The rule condition statement must evaluate to either Boolean true or false. If you create a condition statement that evaluates to null, the application interprets this as false. This can happen inadvertently, especially if you create a condition statement with multiple conditions to evaluate. If your condition evaluates to null (false), the application never executes the associated rule actions.

Defining a statement list as rule conditions

It is also possible to use a statement list, instead of a simple expression, in the **CONDITION** block. Every **CONDITION** block must contain a return statement that evaluates to a Boolean value of true or false, for example:

```
var o = new HashSet<IType>() {A, B, C, ...}
return o.contains(typeof(...))
```

Understanding Gosu rule actions

Within the rule **ACTION** block, you create the outcome for the criteria identified in the rule **CONDITION** block. Actions can be single statements or they can be strung together to fulfill multiple criteria. You can add any number of actions to a rule.

In **ClaimCenter**, suppose that you want an exposure to fail validation if a certain coverage exists but the claim does not contain that coverage. The following syntax creates this action:

```
for( Exposure in claim.Exposures ) {
    if( Exposure.OtherCoverage==true and (Exposure.OtherCoverageInfo==null or Exposure.OtherCoverageInfo=="") ) {
        // Mark each exposure in which this occurs as invalid
        Exposure.rejectSubField( Exposure, "OtherCoverageInfo", "loadsave", "You need to provide information
            about claimant's other coverage.", null, null )
    }
}
```

In **PolicyCenter**, suppose that you want to verify that the country field exists on a contact's home phone number and is valid. The following syntax creates this action:

```
if (contact.HomePhoneCountry == null) {
    contact.rejectField("HomePhoneCountry", ValidationLevel.TC_LOADSAVE, displaykey.Validator.Phone.Home
        .CountryCode.Null, null, null)
}

if (contact.HomePhoneCountry == PhoneCountryCode.TC_UNPARSEABLE) {
    contact.rejectField("HomePhoneCountry", ValidationLevel.TC_LOADSAVE, displaykey.Validator.Phone.Home
        .CountryCode.Unparseable, null, null)
}
```

In **BillingCenter**, suppose that you want to assign a trouble ticket using round-robin assignment. The following syntax creates this action:

```
if (Activity.CurrentAssignment.assignUserByRoundRobin(...)) {
    actions.exit()
}
```

Exiting a Gosu rule

At some point in the rule decision tree, the application makes the decision that you want. At this point, it is important that the application not continue to execute the remaining rules in the rule set. Indeed, if rule checking did continue, and if the application processed the rule set default rule, it might overwrite the decision that came earlier. Therefore, you need to be able to instruct the application at what point to stop considering any further rules.

Guidewire Studio provides several options for this flow control, with the simplest version simply meaning:

Exit – Stop everything! I am done with this rule set.

The following list describe the methods that you can add as an action for a rule to tell the application what to do next.

Flow control action	Description
<code>actions.exit</code>	This is the simplest version of an exit action. The application stops processing the rule set as soon as it encounters this action.
<code>actions.exitAfter</code>	This exit action causes the application to stop processing the rule set after processing any child rules.
<code>actions.exitToNext</code>	This exit action causes the application to stop processing the current rule and immediately go to the next peer rule. The next peer rule is one that is at the same level in the hierarchy. You use this exit action only rarely, within very complicated action sections.
<code>actions.exitToNextParent</code>	This exit action causes the application to stop processing the current rule and immediately go to the next rule at the level of the parent rule. It thus skips any child rules of the current rule.
<code>actions.exitToNextRoot</code>	This exit action causes the application to stop processing the current rule and immediately go to the next rule at the top level. It thus skips any other rules in the entire top-level branch containing this rule.

Usually, a rule lists an exit action as the last action in the rule so that it occurs only after the rule executes all other defined actions.

How to use `exitToNextRoot` in ClaimCenter rules

It is useful to employ the `exitToNextRoot` method if you set up your rule set to make two separate decisions. For example, suppose that you set up the segmentation rules to do the following sequence of actions:

1. First, determine the severity rating of an injury.
2. Then, decide on the segmentation using the results of this determination.

To do this, you can structure the rule set as follows:

- Always process my child rules to evaluate severity...
 - If (conditions) Then (Set severity) [Done, but go to next top-level rule]
 - Otherwise...
- Always process my child rules to evaluate categorization...
 - If (conditions) Then (Set segment) [Done, no need to go further]
 - Otherwise...

After ClaimCenter makes the first decision (setting the severity), the decision logic is complete. However, ClaimCenter needs to move on to the next major block of decision-making, which is deciding on the categorization. After ClaimCenter sets the segment, the rule set is finally complete, so the rule can just use the simple exit method.

How to use `exitToNextRoot` in PolicyCenter rules

It is useful to employ the `exitToNextRoot` method if you set up your rule set to make two separate decisions.

For example, suppose that you set up an Add Forms rule to determine the issuing state for a policy. Then, you decide on the forms to attach using the results of this determination. You can structure the rule set as follows:

- Always process my child rules to evaluate the policy line...
 - If (conditions) Then (Add forms) [Done, but go to next top-level rule]
 - Otherwise...
- Always process my child rules to evaluate the policy state...
 - If (conditions) Then (Add forms) [Done, no need to go further]
 - Otherwise...

After PolicyCenter makes the first decision (setting forms for that policy line), the decision logic is complete. However, PolicyCenter needs to move on to the next major block of decision-making, which is deciding if the giving state requires further forms. After PolicyCenter sets the segment, the rule set is finally complete, so the rule can just use the simple exit method.

Gosu annotations and Gosu rules

Guidewire uses annotation syntax to add metadata to Gosu classes, constructors, methods, and properties. For example, you can add an annotation to a method to indicate its return type or what exceptions it might throw.

The variation that annotations cause in the visibility of classes, constructors, methods, and properties can make it seem that your Gosu code is incorrect. The location of your Gosu code might be the issue, not the code itself.

The effect of annotations on Gosu rules

Guidewire marks certain Gosu code in the base application with the `@scriptable-ui` annotation. That annotation restricts the usage, or *visibility*, of the code to non-rules Gosu code. The converse is the `@scriptable-all` annotation, which makes a class, constructor, method, or property visible in Gosu code everywhere.

Within the Studio Rules editor, the Gosu compiler ignores a class, a property, or a method marked as `@scriptable-ui`. For example, suppose that you attempt to access a property in Studio that has a `@scriptable-ui` annotation. The Rules editor does not recognize the property descriptor for that property. However, the Gosu compiler does recognize the property in other editors in Studio, such as the Gosu editor for classes and enhancements.

Determine the annotations on elements of Gosu code

About this task

To determine the annotations on an element of Gosu code, such as a class, constructor, method, or property, do the following:

Procedure

1. Place your cursor at the beginning of the line directly above the affected code.
2. Type an `@` sign.
Studio displays a list of valid annotations.

Invoking a rule from Gosu code

It is possible to invoke Gosu rules in a rule set from Gosu code. To do so, use the following syntax:

```
rules.[rule set category].[rule set name].invoke([root entity])
```

It is important to understand that the use of the `invoke` method on a rule set triggers all of the rules in that rule set. You cannot invoke individual rules within a rule set using the `invoke` method. The application executes all of the rules in the invoked rule set in sequential order. If a given rule's `CONDITION` block expression evaluates to true, then the application executes the Gosu code in the `ACTION` block for that rule.

Naming Gosu rules

It is important to appoint someone in your organization to develop and enforce simple and consistent naming conventions for rule categories, rule sets, and rule names. This helps to ensure that naming standards are followed to make sure that rule identifiers readily identify specific rules within the total catalog of rules in your Guidewire instance. A good naming convention can help you quickly identify each rule within the complex hierarchy of rules in your Guidewire instance during testing and in production.

General Principles

Guidewire recommends that you implement the following principles in constructing rule names.

Required

Make each rule name unique within a rule set.

Suggested

Use sequential numbering in rule names to mimic the order of rules in the fully expanded set.

Example

The following example in ClaimCenter demonstrates these principles.

```
Claim Validation Rules
CV001000 - Future loss date
CV002000 - Policy expiration date after effective date
CV002500 - Not Set: Coverage in question
CV003000 - Injury
  CVI03100 - Workers Compensation
    CVIW3110 - Claimant exists
    CVIW3120 - Not Set: Injury description
  CV103900 - Default
CV004000 - Expected recovery exceeds 100
```

Creating unique rule names

Each rule name within a rule set must be unique. One way to achieve unique rule names is to adhere to the following basic format in naming rules.

Identifier - Description

Follow these conventions for *Identifier* and *Description*:

- Separate *Identifier* from *Description* with a space, followed by hyphen, followed by a space.
- Limit *Identifier* to eight alphanumeric characters.

IMPORTANT: The application truncates Identifier values that exceed eight characters if you include the actions.ShortRuleName property in rule actions to display rule names in messages that you log or display. The application also truncates Identifier values that exceed eight characters in automatic log messages if you enable the RuleExecution logging category and set the server run mode to Debug.

- Begin *Identifier* with up to four capital letters to identify the rule set or parent rule of which the rule is a member.
- End *Identifier* with at least four numerals to identify the ordinal position of the rule within the hierarchy of rules in the set.
- For *Description* values, keep them simple, short, and consistent in their conventions.
- Limit the total length of rule name to 60 characters.

For example:

```
ARR0100 - Assign Round Robin (BillingCenter)
CV000100 - Future loss date (ClaimCenter)
RAU0100 - Business Auto (PolicyCenter)
```

ClaimCenter rule naming example

Consider the following example rule set, *Claim Validation Rules*. The identifiers of rules in this set all begin with CV, a code to identify “Claim Validation” rules.

```
Claim Validation Rules
CV001000 - Future loss date
CV002000 - Policy expiration date after effective date
CV003000 - Injury
  CVI03100 - Workers' Compensation
  CVI03900 - Default
CV004000 - Expected recovery exceeds 100
```

The rule set contains four root rules, with identifiers CV001000, CV002000, CV003000, and CV004000. The numbers at the end of the identifiers, 1000, 2000, 3000 and 4000, are units of one thousand. This spread of numbers lets you add new root rules between existing ones without renumbering. You want identifier numbers for rules in a set to remain in sequential order to mimic the order of rules within the fully expanded set.

For example, you want to add a rule between CV002000 and CV003000. Assign the new rule the identifier CV002500.

```
Claim Validation Rules
CV001000 - Future loss date
CV002000 - Policy expiration date after effective date
CV002500 - Not Set: Coverage in question
CV003000 - Injury diagnosis validity dates
  CVI03100 - Workers' Compensation
  CVI03900 - Default
CV004000 - Expected recovery exceeds 100
```

Parent and child rules in ClaimCenter

Many rule sets achieve their business function with simple rules in the root of the set. In a previous example, rules CV001000, CV002000, CV002500, and CV004000 were simple root rules. Frequently however, rule sets achieve their business function *only* with a hierarchy of parent and child rules. In the example, rule CV003000 is a parent rule with two child rules.

When you add child rules to a parent, follow these conventions:

- Expand the beginning code for the child rules with an additional letter to identify their parent.
- Assign each child rule an ending number that falls between the number of the parent and the sibling rule that follows the parent.
- Assign the children a spread of numbers so you can add more children later without renumbering.

In the preceding example, the identifiers for the child rules of CV003000 all begin with CVI, a code to identify “Claim Validation Injury” rules.

```
Claim Validation Rules
...
CV003000 - Injury
  CVI03100 - Workers' Compensation
  CVI03900 - Default
CV004000 - Expected recovery exceeds 100
```

The spread of numbers for child rules of a root parent rule generally are units of one hundred. This spread of numbers lets you add new child rules between existing ones without renumbering. Most importantly, the numbers of child rules must fall between the numbers of their parent rule and the sibling rule that follows their parent. In this example, the numbers for child rules satisfy both conventions.

The parent and child naming convention applies to another, third level of children. For example, you want to add two new child rules to the rule CVI03100 - Workers' Compensation. Begin the child identifiers with CVIW, a code to identify “Claim Validation Injury Workers' Compensation” rules. At the third level of a rule set hierarchy, the spread of numbers for the child rules generally are units of ten.

```
Claim Validation Rules
...
CV003000 - Injury
  CVI03100 - Workers' Compensation
    CVIW3110 - Claimant exists
    CVIW3120 - Not Set: Injury description
  CVI03900 - Default
CV004000 - Expected recovery exceeds 100
```

Operating system length restrictions

Guidewire stores rules in files within a directory structure that mimics the rule set category structure in Studio. The fully qualified path for a rule file can be quite long, depending on:

- The length of the path to your Guidewire installation directory
- The length of the path from your installation directory to the root of the rules directory, which is 34 characters:
modules/configuration/config/rules
- The depth of the hierarchy of rule categories, rule sets, rules, and parent/child rules
- The length of individual rule names

In addition, you must add four characters to the path for the name of each rule category, rule set, and parent rule.

You must understand the file system implications of rule names. Windows file systems have a file path limit of 255 characters. To avoid exceeding the limit:

- Avoid a long path to your Guidewire installation directory.
- Avoid deep hierarchies of rule set categories.
- Avoid hierarchies of parent and child rules within a rule set deeper than three levels.
- Avoid long names for rule set categories, rule sets, and rules.

Using the rules editor

The application organizes and displays rules as a hierarchy in the center pane of Guidewire Studio, with the rule set appearing at the root, the top level, of the hierarchy tree. Studio displays the **Rule** menu only if you first select a rule set category in the **Project** window. Otherwise, it is unavailable.

There are a number of operations involving rules that you can perform in the Studio Rules (Gosu) editor.

View or edit a rule

Procedure

1. Navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets

Alternatively, you can use the 'breadcrumb' functionality (which bypasses the Studio **Project** window) located directly under the screen toolbar to navigate to a rule, for example:

configuration > config > rules > ...

2. Expand the **Rule Sets** folder, and then expand the rule set category.
3. Select the rule set you want to view or edit.

All editing and saving in the Rule Editor occurs at the level of a rule set.

Changing rule order

If you want to change the order of your Gosu rules, you can drag and drop rules within the rule hierarchy in Guidewire Studio. If you move rules using drag and drop, Guidewire Studio moves the chosen rule and all its children as a group. This behavior makes it easy to reposition entire branches of the hierarchy.

Guidewire Studio also supports standard cut, copy, and paste commands, which you can use to move rules within the hierarchy. If you paste a cut or copied rule, Studio inserts the rule as if you added a new rule. It becomes the last child of the currently selected rule.

IMPORTANT: Exercise caution when changing rule order as the application will process rules according to their execution sequence (top to bottom) in the rule hierarchy. Ensure any 'default' rules are always last in the rule order. See “Gosu rule execution” on page 15 for information on this sequence.

Additionally, if rules are moved, you will need to rename them accordingly to adhere to “Creating unique rule names” on page 25 guidelines.

The following list describes the actions you can take with rules.

Move a rule

Click the rule that you want to move, and then hold down the mouse button and move the pointer to the new location for the rule. Studio then displays a line at the insertion point of the rule. Release the mouse button to paste the rule at that location.

Make a rule a child of another rule

Select the rule you want to be the child, and then choose **Edit > Cut**. Click on the rule that you want to be the parent, and then choose **Edit > Paste**.

Move a rule to a different level

Select the rule to move and drag the rule next to another rule at the desired level in the hierarchy (the reference rule). Notice how far left the insertion line extends:

- If the line ends before the beginning of the reference rule's name, Studio inserts the rule as a child of the reference rule.
- If the line extends all the way to the left, Studio inserts the rule as a peer of the reference rule.

By moving the cursor slightly up or down, you can indicate whether you want to insert the rule as a child or a peer.

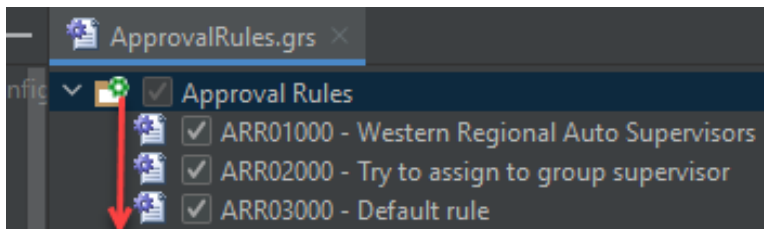
order.txt

Any changes to rule ordering, rule addition, or rule removal in Guidewire Studio is recorded in a file (`order.txt`) in each rule set directory. This file is a simple text file that lists each rule in a rule set in their execution sequence, where the first rule executed is at the start of the file, and the last rule is at the end.

For example, consider the TransactionSet Approval Assignment rule set in ClaimCenter. In the base configuration, `modules/configuration/config/rules/ApprovalRouting/ApprovalRules_dir/order.txt` is as follows:

```
ARR01000WesternRegionalAutoSupervisors
ARR02000Trytoassigntogroupsupervisor
ARR03000Defaultrule
```

This also appears in Guidewire Studio, where these rules are shown with the same top to bottom sequence:



If you moved the rule `ARR01000...` in Guidewire Studio and placed it after the rule `ARR02000...`, `order.txt` is modified as follows:

```
ARR02000Trytoassigntogroupsupervisor
ARR01000WesternRegionalAutoSupervisors
ARR03000Defaultrule
```

In this example, in Studio, you must also rename `ARR02000 - Try to assign to group supervisor` to `ARR01000 - Try to assign to group supervisor` as it is now executed first, and rename `ARR01000 - Western Regional Auto Supervisors` to `ARR02000 - Western Regional Auto Supervisors` as it is now executed second. You do not need to edit these names directly in `order.txt` as any renaming in Studio is also recorded in this file.

Create a new rule set category

Procedure

1. Navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets

2. Right-click **Rule Sets**, and then click **New > Rule Set Category**.
3. Enter a name for the rule set category.

Results

Studio inserts the rule set category in the category list in alphabetic order.

Create a new rule set

Procedure

1. Navigate to the following location in the Studio **Project** window:.

configuration > config > Rule Sets

Although the label is **Rule Sets**, the primary children of this folder are actually rule set categories.

2. Do one of the following:
 - If an appropriate rule set category for your rule set does not exist, first perform the steps listed in “Create a new rule set category” on page 30. After creating a rule set category, move to the next step.
 - If an appropriate rule set category for your rule set does exist, move to the next step.
3. Select a rule set category, then select **New > Rule Set** from the context menu.
4. Enter the following information:

Field	Description
Name	Studio displays the rule name in the middle pane. In general, if you create a rule set for a custom entity named <code>Entity_Ext</code>, you must name your rule set <code>Entity_Ext<RuleSet></code>. Thus, if you want the custom entity to invoke the <code>Preupdate</code> rules, then name your rule set <code>Entity_ExtPreupdate</code>. There are some variations in how to name a rule set. See the existing rule sets in that category to determine the exact string to append and follow that same pattern with new rule sets in that category.
Description	Studio displays the description in a tab at the right of the Studio if you select the rule set name in the middle pane. Guidewire recommends that you make the description meaningful, especially if you have multiple people working on rule development. In any case, a meaningful rule description is particularly useful as time passes and memories fade.
Entity Type	The application uses the entity type as the basis on which to trigger the rules in this rule set. For example, suppose that you select a rule set, then a rule within the set. Right-click and select Complete Code from the menu. Studio displays the entity type around which you base the rule actions and conditions.

Create a new rule

Procedure

1. Select the rule set to contain the new rule in the Studio **Resources** pane.
2. Do one of the following:
 - If the new rule is to be a top-level rule, select the rule set name in the middle pane.
 - If the new rule is to be a child rule, expand the rule set hierarchy in the middle pane and select the parent rule.
3. Select **New Rule** from the **Rule** menu, or right-click and select **New Rule**.
4. Enter a name for the new rule in the **New Rule** dialog box.

Results

Studio creates the new rule as the last child rule of the currently selected rule (or rule set).

Accessing a rule set from Gosu code

You can access a rule set within a rule set category (and thus, all the rules within the rule set) by using the following Gosu invoke method. You can use this method to invoke a rule set in any place that you use Gosu code.

```
rules.RuleSetCategory.RuleSet.invoke(entity)
```

You can only invoke a rule set through the Gosu invoke method, not individual rules. Invoking the rule set triggers evaluation of every rule in that rule set, in sequential order. If the conditions for a rule evaluate to true, then the application executes the actions for that rule.

Renaming or deleting a rule

Use the following commands to rename a rule or to delete it entirely from the rule set. You access these commands by right-clicking a rule and selecting the command from the drop-down list.

Command	Description	Actions to take
Rule > Rename Rule	Renames the currently selected rule	- Select the rule to rename in the center pane of Studio. - Select Rename Rule from the Rule menu, or, right-click and select Rename Rule - Enter the new name for the rule in the Input dialog box. You must save the rule for the change to become permanent.
Edit > Delete	Deletes the currently selected rule	- Select the rule to delete in the center pane of Studio. - Select Delete from the Edit menu, or right-click and select Delete. The application does not require you to confirm your decision before deleting the rule. You can use the Delete command to delete a rule only.

About renaming a rule

At a structural level, the application stores each rule as a separate Gosu class, with a `.gr` extension. The name of the Gosu class corresponds to the name of the rule that you see in the Studio **Project** window. The application stores the rule definition classes in the following location in the installation directory:

```
modules/configuration/config/rules/
```

If you rename a rule set, the application renames the class definition file in the directory structure and any internal class names. It also renames the directory name if the rule has children. Thus, the application ensures that the rule class names and the file names are always in synchronization. Always use Guidewire Studio to rename a rule. Never rename a file using your local file system.

Making a rule active or inactive

The application skips any inactive rule, acting as if its conditions are false. (This causes it to skip the child rules of an inactive rule, also.) You can use this mechanism to temporarily disable a rule that is causing incorrect behavior (or that is no longer needed) without deleting it. Sometimes, it is helpful to keep the unused rule around in case you need that rule or something similar to it in the future.

To make a rule active, set the check box next to it. To make a rule inactive, clear the check box next to it.

Advanced rule writing

The following examples describe ways to perform more complex or sophisticated actions in rules, for example:

- Accessing a field that exists on a subtype of an entity, but, not on the entity itself.
- In ClaimCenter, using a custom event to prevent repeating the same action more than once in a rule
- Using rule logic to iterate over the elements of a set of items
- Using permissions to restrict user actions
- In ClaimCenter, determining the original values of a changed entity

Accessing fields on subtypes

Various entities have subtypes, and a subtype may have fields that apply only to it, and not to other subtypes. For example, a Contact object has a Person subtype, and that subtype contains a DateOfBirth field. However, DateOfBirth does not exist on a Company subtype. Similarly, only the Company subtype has the Name (company name) field.

Because these fields apply to particular subtypes only, you cannot reference them in rules by using the primary root object.

For example, in BillingCenter and PolicyCenter, the following illustrates an invalid way to reference the primary contact for an account:

```
Company.PrimaryContact.LastName.compareTo("Smith") //Invalid
```

For example, in ClaimCenter, the following illustrates an invalid way to reference the lienholder of a vehicle:

```
Exposure.lienholder[0].FirstName == "Joe" // invalid
```

To access a field that belongs to a subtype, you must “cast” (or convert) the primary object to the subtype by using the `as` operator.

For example, in BillingCenter and PolicyCenter, you would cast a contact to the Person subtype using the following syntax:

```
(Company.PrimaryContact as Person).LastName.compareTo( "Smith" )
```

For example, in ClaimCenter, you would cast a contact to the Person subtype using the following syntax:

```
(Exposure.lienholder[0] as Person).FirstName == "Joe" // valid
```

In ClaimCenter, as another example, consider transaction validation rules. A transaction set has several subtypes, such as a ReserveSet, CheckSet, and others. To validate that primary checks being sent by mail have a mailing address that is not null, you can use the following in the rule condition:

```
TransactionSet.Subtype == "checkset" &&
    (transactionSet as CheckSet).PrimaryCheck.DeliveryMethod == "send" &&
    (transactionSet as CheckSet).PrimaryCheck.MailToAddress == null
```

It is important to cast the type of an object correctly, otherwise, a Runtime Error can occur.

Preventing repeated actions in ClaimCenter rules

Many times, during exception rule execution, you want to take action on an exception the first time you discover the exception only. For example, suppose that you want to remind the adjuster to set the fault rating on a claim that is open for more than 30 days. If this action has not been completed within 30 days, then you might want to add an activity on day 31 to remind the adjuster. On day 32, if the adjuster still has not set the rating, you do not want to add another activity, even though ClaimCenter finds the same exception again.

One way to handle this is to use ClaimCenter methods for noting a custom event in the Claim History. The first time ClaimCenter finds an exception, you need to instruct ClaimCenter to note it in the claim's history. Then, in the rule, include a check to verify if this event appeared before in the rule's exceptions. If the event already exists, then do not execute the actions again.

The syntax for this is similar to the following:

```
CONDITION (claim : entity.Claim):
return (gw.api.util.DateUtil.daysSince( claim.ReportedDate ) > 30)
    and (!exists (hist in claim.History where hist.CustomType == "a_n_f_r"))

ACTION (claim : entity.Claim, actions : gw.rules.Action) :
claim.createCustomHistoryEvent( "a_n_f_r", "description" )
claim.createActivity( parameterList)
```

In this example, ClaimCenter determines if the event occurred previously. You can also determine if the event occurred within a certain time period. In this way, you can repeat the rule's actions if enough time has passed since ClaimCenter first noted the exception.

Taking action on more than one subitem

ClaimCenter

A common situation in ClaimCenter to handle is raising the priority on every activity on a claim that is overdue. Gosu provides a `for(... in ...)` syntax and an `if(...) { then do something }` syntax that you can use to construct fairly complicated actions:

```
for (act in claim.Activities) {
    if ( act.Status == "open" and act.TargetDate < gw.api.util.DateUtil.currentDate() ) {
        act.Priority = "urgent"
    }
}
```

Notice that the curly braces (`{}`) mark the beginning and end of a block of Gosu statements:

- The outer pair of braces contain statements to perform “for” each activity on the claim.
- The inner pair of braces contain statements to perform “if” the activity is overdue.

PolicyCenter

Gosu provides a `for(... in ...)` syntax and an `if(...) { then do something }` syntax that you can use in PolicyCenter to construct fairly complicated actions. In the following example, the rule iterates through the various lines on a policy to determine if a line contains a certain Coverage Group.

```
for (line in policyPeriod.Lines) {
    for (cov in line.AllCoverages) {
```

```
    if (cov.CoverageCategory == "BAPHiredGrp") { }
  }
}
```

Notice that the curly braces ({}) mark the beginning and end of a block of Gosu statements:

- The outer pair of braces contain statements to perform “for” each line on the policy.
- The outer pair of braces contain statements to perform “if” the specified coverage group is found.

You can use the Length method on a subobject to determine how many subobjects exist. For example, if there are five lines on this PolicyPeriod, then the following expression returns the value 5:

```
PolicyPeriod.Lines.Length
```

BillingCenter

BillingCenter provides a for(... in ...) syntax and an if(...) { *then do something* } syntax that you can use to construct fairly complicated actions. In the following example, the rule iterates through the various invoices on an account to determine if the invoice due date is earlier than the current date. If it is, the rule sets the invoice due date to the current date.

```
var account1 : Account //Some account

for (var inv in account1.Invoices) {
  if (inv.PaymentDueDate < gw.api.util.DateUtil.currentDate()) {
    inv.PaymentDueDate = gw.api.util.DateUtil.currentDate()
  }
}
```

Notice that the curly braces ({}) mark the beginning and end of a block of Gosu statements:

- The outer pair of braces contain statements to perform “for” each line on the policy.
- The outer pair of braces contain statements to perform “if” the specified coverage group is found.

Checking entity permissions

The application provides a Gosu mechanism for checking user permission on an object by accessing properties and methods off the object in the perm namespace.

- The application exposes static permissions that are non-object-based (like the permission to create a user) as Boolean properties.
- The application exposes permissions that take an object (like the permission to edit a claim) as methods that take an entity as their single parameter.
- The application exposes application interface permissions as typecodes on the perm.System object.

All the properties and methods return Boolean values indicating whether or not the user has permission to perform the task. The application always evaluates permissions relative to the current user unless specifically instructed to do otherwise. You can use permissions anywhere that you can use Gosu (in PCF files, rules, and classes) and there is a current user.

You can check that any given user has a specific permission using the following Gosu code:

```
var u : User = User( "SomeUser" /* Valid user name*/ )
var hasPermission = u.Roles.hasMatch(\role -> role.Role.Privileges.hasMatch(\perm -> perm.Permission == p))
```

If using this code in a development environment, you must connect Studio to a running development application server before Studio recognizes users and permissions.

Determining the original attributes of a changed entity

The following ClaimCenter example illustrates how to determine if the original field values on an entity have changed, and, for the changed values, what are the original values. For example, suppose that the Loss Location on the claim

changes for some reason. You might want to retrieve the original values to compare with the current values to determine how the change in the Loss Location state.

It is not sufficient to determine the original values simply by knowing that there was a change to the Loss Location. The following expression does not provide the correct value, as it does not point back to the original Loss Location.

```
Claim.LossLocation.getOriginalValue("State")
```

Instead, to determine the original attributes of a changed object, you must first re-instantiate the original object.

In this example, the rule executes if the `isFieldChanged("LossLocation")` becomes `true`. Then, if the new Loss Location state is different from the original state, the rule creates a custom history event identifying the change.

```
CONDITION (claim : entity.Claim):
return claim.isFieldChanged("LossLocation")

ACTION (claim : entity.Claim, actions : gw.rules.Action):
var add = Address( claim.getOriginalValue("LossLocation") )

if (add.State != Claim.LossLocation.State) {
    claim.createCustomHistoryEvent("DataChange",
        displaykey.Rules.Log.DataChange(add.State,claim.LossLocation.State ) )
}
```

Additional changed entity methods

Guidewire provides a number of methods that you can use to determine if an entity changes. (For information on each, place your cursor on the method signature and press **Ctrl+Q**.)

```
Entity.getOriginalValue(fieldname : String)
Entity.getAddedArrayElements(arrayFieldName : String)
Entity.getChangedArrayElements( arrayFieldName : String)
Entity.getRemovedArrayElements( arrayFieldName : String)
Entity.isArrayElementChanged(arrayFieldName : String)
Entity.isArrayElementAddedOrRemoved(arrayFieldName : String)
Entity.isFieldChanged(fieldname : String)
```

You can also use the read-only `Changed` property on the entity to determine if the entity contains changed fields. The `Changed` property becomes `true` only after a modification to the entity.

```
Entity.Changed
```

These methods are available in all Gosu code, not just Gosu rules.

See also

- For information on how to determine if an entity changed, see the *Gosu Reference Guide*.

About changing the root entity of a rule

The application bases each Gosu rule on a specific business entity. In general, the rule set name reflects this entity. For example, in the Validation rule set category, you have Contact Validation rules and Person Validation rules. These rule set names indicate that the root entity for each rule set is—respectively—the Contact object and the Person object.

The application provides the ability to change the root entity of a rule through the use of the right-click **Change Root Entity** command on a rule set. The intent of this command is to enable you to edit a rule that you otherwise cannot open in Studio because the declarations failed to parse. Do not use this command in any other circumstances.

For example, suppose that you have the following sequence of events:

1. You create a new entity in the application, for example, `TestEntity`. Studio creates a `TestEntity.eti` file and places it in the following location:
modules/configuration/config/extensions
2. You create a new rule set category called **TestEntityRuleSetCategory** in **Rule Sets**, setting `TestEntity` as the root entity. Studio creates a new folder named `TestEntityRuleSetCategory` and places it in the following location:
modules/configuration/config/rules/rules

3. You create a new rule set under **TestEntityRuleSetCategory** named **TestEntityRuleSet**. Folder **TestEntityRuleSetCategory** now contains the rule set definition file named **TestEntityRuleSet.grs**. This file contains the following (simplified) Gosu code:

```
@gw.rules.RuleName("TestEntityRuleSet")
class TestEntityRuleSet extends gw.rules.RuleSetBase {
    static function invoke(bean : entity.TestEntity) : gw.rules.ExecutionSession {
        return invoke( new gw.rules.ExecutionSession(), bean )
    }
    ...
}
```

Notice that the rule set definition explicitly invokes the root entity object: **TestEntity**.

4. You create one or more rules in this rule set that use **TestEntity** object, **TestEntityRule**, for example. Studio creates a **TestEntityRule.gr** file that contains the following (simplified) Gosu code:

```
internal class TestEntityRule {
    static function doCondition(testEntity : entity.TestEntity) : boolean {
        return /*start00rule*/true/*end00rule*/
    }
    ...
}
```

Notice that this definition file also references the **TestEntity** object.

5. Because of upgrade or other reasons, you rename your **TestEntity** object to **TestEntityNew** by changing the file name to **TestEntityNew.eti** and updating the entity name in the XML entity definition:

```
<?xml version="1.0"?>
<entity xmlns="http://guidewire.com/datamodel"
    entity="TestEntityNew" ... >
</entity>
```

This action effectively removes the **TestEntity** object from the data model. However, this action does not remove references to the entity that currently exist in the rules files.

6. You update the database by stopping and restarting the application server.
7. You stop and restart Studio.

As Studio reopens, it presents you with an error message dialog. The message states that Studio cannot parse the listed rule set files. It is at this point that you can use the **Change Root Entity** command to shift the root entity in the rule files to the new root entity. After you do so, Studio recognizes the new root entity for these rule files.

Change the root entity of a rule

About this task

The intent of the Rules editor **Change Root Entity** menu command is to enable you to edit a rule that you otherwise cannot open in Studio. For example, this can happen if the application is unable to parse the rule declarations. Do not use this command in any other circumstances.

Procedure

1. Select a rule within a rule set.
2. Right-click the rule name and select **Change Root Entity** from the drop-down menu. Studio prompts you for an entity name.
3. Enter the name of the new root entity.

Results

After you complete this command:

- Studio performs a check to verify that the provided name is a valid entity name.
- Studio replaces occurrences of the old entity in the function declarations of all the files in the rule set with the new entity. This replacement only works, however, if the old root type is an entity.

- Studio changes the name of the entity instance passed into the condition and action of each rule.
- Studio does not propagate the root entity change to the body of any affected rule. You must correct any code that references the old entity manually.

Rule testing and debugging

It is a very useful practice to add logging statements to your rules to identify the currently executing rule, and to provide information useful for debugging purposes. Guidewire recommends that you use the following means of providing information extensively.

Print error and log messages

BillingCenter: Use class `gw.api.system.BCLoggerCategory` to print out helpful error messages to the application log files.

ClaimCenter: Use class `gw.api.system.CCLoggerCategory` to print out helpful error messages to the application log files.

PolicyCenter: Use class `gw.api.system.PCLoggerCategory` to print out helpful error messages to the application log files.

Add comments to rules

Embed comments within rules to provide useful information while troubleshooting rules. Print these comments to the screen and/or the application log.

Generating logging information

To trigger the flow of information from a Gosu rule to a log file, you need to configure the following:

- The appender definition (`<RollingFile>`) for the rule log file in `log4j2-json.xml`.
- The logger definition (`<Logger>`) associated with the rule file in `log4j2-json.xml`.
- The activation trigger for the logging category in a Gosu rule using `gw.api.system.BCLoggingCategory` (BillingCenter), `gw.api.system.CCLoggingCategory` (ClaimCenter), or `gw.api.system.PCLoggingCategory` (PolicyCenter).

Logging category definitions

A logging category defines how the Apache log4j logging utility handles different types of logging requests. In the base configuration, Guidewire provides a number of readily usable logging categories.

To define how the application interacts with the ASSIGNMENT category, add something similar to the following code in file `log4j2-json.xml`.

```
<!-- Rolling file definition... -->
<RollingFile name= "RuleEngineLog" fileName="${guidewire.logDirectory}/ruleengine.log"
    filePattern="${guidewire.logDirectory}/ruleengine.log%d{.yyyy-MM-dd}">
  <PatternLayout pattern="${file.defaultPattern}" charset="UTF-8"/>
  <TimeBasedTriggeringPolicy/>
```

```
</RollingFile>

<!-- Logger definition... -->
<Logger name="Assignment" additivity="false" level="info">
  <AppenderRef ref="RuleEngineLog">
</Logger>
```

Notice that this code does the following:

- It defines a rolling log file named `ruleengine.log` in the `<RollingFile>` element.
- It then links the `ruleengine.log` file to the `Assignment` logger category in the `<Logger>` element.
- It sets a default logging level of `INFO`.

Base configuration logging categories

There are several ways to view the base configuration logging categories:

- From the **Set Log Level Server Tools** screen.
- By running the `system_tools` command from a command prompt and adding the `-loggercats` option.

Triggering a logging category from a Gosu rule

For example, in `ClaimCenter`, to use this API in Gosu code to perform assignment logging, do something similar to the following:

```
uses gw.api.system.CCLoggerCategory
...
var logger = CCLoggerCategory.ASSIGNMENT
...
logger.info("Print out this message.")
```

This code snippet is identical in `BillingCenter` or `PolicyCenter` except for the use of `gw.api.system.BCLoggingCategory` in `BillingCenter` and `gw.api.system.PCLoggingCategory` in `PolicyCenter`.

Logging example

The following code examples show the use of a logging category in a Gosu rule in each application. This code assumes that the necessary appender and logger definitions exist in file `log4j2-json.xml`.

BillingCenter:

```
var logger=gw.api.system.BCLoggerCategory.ASSIGNMENT

//Attempt to assign the group for a trouble ticket
if (troubleTicket.CurrentAssignment.assignGroupByRoundRobin(GroupType.TC_GENERAL, false,
troubleTicket.AssignedGroup)) {
  actions.exit();
} else {
  logger.info( "Global Activity Assignment rules could not assign Trouble Ticket");
}
```

ClaimCenter:

```
var logger=gw.api.system.CCLoggerCategory.ASSIGNMENT

// Pass the user search criteria to the assignUserByProximityWithSearchCriteria function
if (activity.assignUserByProximityWithSearchCriteria(usc, -1, true,
activity.AssignedGroup)) {
  logger.debug( DisplayKey.get("Rules.Assignment.DefaultGroup.Activity", actions.ShortRuleName) )
  logger.debug( DisplayKey.get("Rules.Assignment.AssignedUserIs", activity.AssignedUser) )
  actions.exit()
}
```

PolicyCenter:

```
var logger=gw.api.system.PCLoggerCategory.ContextualLogger

// If there is an address, assign by location using that address
if (addr != null) {

  if ( policy.CurrentRoleAssignment.assignGroupByLocation("branch", addr, false,
assignment.CurrentAssignment.AssignedGroup) ) {
```



```
// Then attempt to assign to the proper group within this branch
if ( policy.CurrentRoleAssignment.assignGroupByRoundRobin("branchuw", true,
    assignment.CurrentAssignment.AssignedGroup) ) {
    logger.debug("##### This is the Global Pre-renewal Assignment rule " + actions.getRule().DisplayName)
    logger.debug("Assigned Group is " + policy.CurrentRoleAssignment.AssignedGroup.DisplayName)
    actions.exit()
}
}
}
```

Returning rule names in messages

In working with application messages, Guidewire recommends that you obtain and show the name of any rule called during the course of creating that message. Following a consistent scheme for rule names helps you to identify specific rules in messages to users, in print statements for testing, and in log messages. Using an alphabetic beginning of a rule identifier helps you find the rule set or parent rule that contains the rule. Adding a numeric ending helps you determine the order of a rule in a rule set or parent rule.

Note: In actual practice, make all String values in the following example into display keys.

Example

BillingCenter Example

In BillingCenter, suppose that you want to test a regional validation rule, RGV01000 - No zones in lookup. The first portion of the rule name begins with RGV. That identifies the rule as a member of the RegionValidationRules rule set. The rule name ends with a numeric identifier, 01000, which indicates that rule is near the beginning of the rule set. The rule has the following definition.

```
RGV01000 - No zones in lookup
Rule Conditions:
Region.getOrganization() = null

Rule Actions:
producerCode.reject(null, null, "loadsave",
    "There are zones that do not appear in the zone lookup dataset."
)
```

As written, the rejection message that the rule action displays makes it difficult to determine exactly which rule caused an update to fail. To help identify the specific rule in the rejection message, use the `actions.getRule().DisplayName` property to include the identifier portion of the rule name in the message.

```
Rule Actions:
Region.reject( null, null, "loadsave"
    "There are zones that do not appear in the zone lookup dataset. Rule:" +
    actions.getRule().DisplayName.substring(8)
)
```

By including the rule display name in the rule action, users see the following statement in the **Validation** window when the rule action executes.

```
There are zones that do not appear in the zone lookup dataset. Rule: RGV01000
```

Example

ClaimCenter Example

In ClaimCenter, suppose that you want to test an exposure validation rule for vehicular exposures, EXV01000 - Incident not null. The first portion of the rule name begins with EXV. That identifies the rule as a member of the Exposure Validation rule set. The rule name ends with 01000, which indicates that rule is near the beginning of the rule set. The rule has the following definition.

```
EXV01000 - Incident not null
Rule Conditions:
exposure.Vehicle.Vin == null or exposure.Vehicle.Vin == ""
```

```

Rule Actions:
exposure.rejectField ("Vehicle.Vin", null, null, "newloss",
    "The vehicle VIN must have a value."
)

```

As written, the rejection message that the rule action displays makes it difficult to determine exactly which rule caused an update to fail. To help identify the specific rule in the rejection message, use the `actions.getRule().DisplayName` property to include the identifier portion of the rule name in the message.

```

Rule Actions:
exposure.rejectField ("Vehicle.Vin", null, null, "newloss",
    "The vehicle VIN must have a value. Rule: " + actions.getRule().DisplayName.substring(8)
)

```

By including the rule display name in the rule action, users see the following statement in the **Validation** window when the rule action executes.

```
VIN: The Vehicle VIN should have a value. Rule: EXV01000
```

Example

PolicyCenter Example

In PolicyCenter, suppose that you want to test a producer code validation rule, `PCV01000` - No producer code roles. The first portion of the rule name begins with `PCV`. That identifies the rule as a member of the `ProduceCodeValidationRules` rule set. The rule name ends with `01000`, which indicates that rule is near the beginning of the rule set. The rule has the following definition.

```

PCV01000 - No producer code roles
Rule Conditions:
producerCode.ProducerCodeRoles.Count == 0

Rule Actions:
producerCode.rejectField("ProducerCodeRoles", null, null, "loadsave",
    "A role is required for producer code " + producerCode.Code + "."
)

```

As written, the rejection message that the rule action displays makes it difficult to determine exactly which rule caused an update to fail. To help identify the specific rule in the rejection message, use the `actions.getRule().DisplayName` property to include the identifier portion of the rule name in the message.

```

Rule Actions:
producerCode.rejectField("ProducerCodeRoles", null, null, "loadsave",
    "A role is required for producer code " + producerCode.Code +
    ". Rule: " + actions.getRule().DisplayName.substring(8)
)

```

By including the rule display name in the rule action, users see the following statement in the **Validation** window when the rule action executes.

```
A role is required for producer code ACME235. Rule: PCV01000
```

BillingCenter rule set categories

As part of the BillingCenter base configuration, Guidewire provides sample rules in the following rule set categories.

Archive

Archive a policy period and its related data.

Assignment

Determine the responsible party for an activity or trouble ticket.

Event Message

Handle communication with integrated external systems.

Exception

Specifies an action to take if an Activity is overdue and is in an escalated state.

Preupdate

Trigger automatic actions anytime that an account or other high-level entity changes.

Validation

Validate a small subset of the BillingCenter entities.

Archive

Archiving is the process of removing the unneeded data of a closed policy period from the BillingCenter database and saving it to an external storage device.

It is possible to define custom eligibility checks in the form of Gosu rules to use in determining the eligibility of a policy period for archiving. Guidewire provides the archiving Saved Distribution sample rule in the base configuration to illustrate how to implement a custom eligibility rule:

Assignment

In the BillingCenter base configuration, Guidewire provides a number of sample rule sets that provide business logic for assignable entities. Guidewire refers to certain business entities as assignable, meaning that it is possible to determine the party responsible for that entity.

At the minimum, an entity must implement the `Assignable` delegate to be assignable. It is not possible to make an entity that is not assignable in the base configuration to be assignable. However, you can extend an entity and make the new entity assignable.

In the BillingCenter base configuration, Guidewire defines the following entities as assignable:

- Activity
- AgencyBillWorkflow
- BCWorkflow
- DelinquencyProcess
- DelProcessWorkflow
- Hold
- TroubleTicket
- UserRoleAssignment

You use Gosu assignment methods to set an assigned group and user for an assignable entity. These assignment methods run either in the context of the Assignment engine (within the Assignment rule set) or simply within Gosu code (within a class or enhancement, for example).

Event message

IMPORTANT: BillingCenter runs the Event Message rules as part of the database bundle commit process. Only use these rules to create integration messages.

In the base configuration, Guidewire provides a single rule set—Event Fired—in the Event Message rule category. The rules in this rule set:

- Perform event processing
- Generate messages about events that have occurred

BillingCenter calls the Event Fired rules if an entity involved in a bundle commit triggers an event of interest to a message destination. A message destination registers an event to indicate its interest in that event.

As part of the event processing:

- BillingCenter runs the rules in the Event Fired rule set a single time for every event of interest to a message destination.
- BillingCenter runs the Event Fired rule set a single time for each destination that is listening for that particular event. Thus, it is possible for BillingCenter to run the Event Fired rules sets multiple times for each event, one time for each destination interested in that event.

In a normal bundle commit any new or updated entity can generate an event. The event messaging rules can detect and use the event to create a new message for communicating with an external system. BillingCenter uses bulk updates to maintain many denormalized values. If the only change to an entity is a bulk update change then that entity does not appear as a modified entity in the bundle. Therefore the entity change does not fire an event.

Typically, the update of a denormalized field is not important to business logic as BillingCenter uses denormalization to cache information that it already stores elsewhere. Write rules to detect changes to the original entity rather than side effects of those changes. For example, look for TransactionAdded rather than looking for changes in the AmountDenorm of a TAccount.

A Word of Warning

Be extremely careful about modifying entity data in Event Fired rules and messaging plugin implementations. Use these rule to perform only the minimal data changes necessary for integration code. Entity changes in these code locations do not cause the application to run or re-run validation or preupdate rules. Therefore, do not change fields that might require those rules to re-run. Only change fields that are not modifiable from the user interface. For example, set custom data model extension flags only used by messaging code.

Guidewire does not support the following:

- Guidewire does not support (and, it is dangerous) to add or delete business entities from Event Fired rules or messaging plugins (even indirectly through other APIs).
- Guidewire does not support—even within the Event Message rule set—calling any message acknowledgment or skipping methods such as `reportAck`, `reportError`, or `skip`. Use those methods only within messaging plugins.
- Guidewire does not support creating messages outside of the Event Message rule set.

About messages and messaging events

BillingCenter automatically generates certain events for most top-level objects. Guidewire calls these events standard events. In general, a standard event occurs for any addition, modification, or removal (or retirement) of a top-level entity. BillingCenter automatically generates the following events on the `Activity` object:

- `ActivityAdded`
- `ActivityChanged`
- `ActivityRemoved`

It is also possible to create a custom event on an entity by using the `addEvent` method. This method takes a single parameter, `eventName`, which is a `String` value that sets the name of the event.

```
entity.addEvent(eventName)
```

IMPORTANT: Do not create custom code that generates update events for any entity type that can possibly be part of an archive bundle.

Message destinations and message events

You use the Studio **Messaging** editor to define message destinations and message events.

- A *message destination* is an external system to which it is possible to send messages.
- A *message event* is an abstract notification of a change in BillingCenter that is of possible interest to an external system. For example, this can be adding, changing, or removing a Guidewire entity.

Using the **Messaging** editor, it is possible to associate (register) one or more events with a particular message definition. For example, in the base configuration, BillingCenter associates the following events with the PAS (Policy Administration System) (such as PolicyCenter) message destination (ID=100):

```
PAS_(\w)*
```

If you modify, add, or remove a `Contact` object (among others), BillingCenter generates the relevant event. Then, during a bundle commit of the `Contact` object, BillingCenter notifies any Event Message rule interested in the event of the occurrence of the event. You can use this information to generate a message to send to an external system or to the system console for logging purposes, for example.

Message destinations and message plugins

Each message destination encapsulates all the necessary behavior for an external system, but uses three different plugin interfaces to implement the destination. Each plugin handles different parts of what a destination does. Thus:

- The message request plugin handles message pre-processing.
- The message transport plugin handles message transport.
- The message reply plugin handles message replies.

You must first register a new messaging plugin in the Studio **Plugins** editor. During this process, Studio prompts you for a plugin interface name, and, in some cases, a plugin name. Use that plugin name in the **Messaging** editor in Studio to register each destination.

Note: You must register your plugin in two different editors in Studio, first in the **Plugin** registry, and then in the **Messaging** editor.

IMPORTANT: After the BillingCenter application server starts, BillingCenter initializes all message destinations. BillingCenter saves a list of events for which each destination requested notifications. As this happens at system start-up, you must restart BillingCenter if you change the list of events or destinations.

Generating messages

Use method `createMessage` on Java class `MessageContext` to create the text of a message, which can be either a simple text message or a more involved constructed message. The following code is an example of a simple text message that uses the Gosu in-line dynamic template functionality to construct the message. Gosu in-line dynamic templates combine static text with values from variables or other calculations that Gosu evaluates at run time. For example, the following `createMessage` method call creates a message that lists the event name and the entity that triggered this rule set.

```
messageContext.createMessage("${messageContext.EventName} - ${messageContext.Root}")
```

The following is an example of a constructed message that provides more detailed information if BillingCenter adds an account successfully.

```
if (messageContext.EventName == "AccountAdded") {
    var account = messageContext.Root as Account
    messageContext.createMessage("Account [" + account.AccountNumber + "] added")
}
```

Listening for object changes that trigger events

It is possible that you want to listen for a change in an object that does not automatically trigger an event if you update it. For example, suppose that you want to listen for a change to the `User` object (an update of the address, perhaps). However, in this case, the `User` object itself does not contain an address. Instead, BillingCenter stores addresses as an array on `UserContact`, which is an extension of `Person`, which is a subtype of `Contact`, which points to `User`. Therefore, updating an address does not directly in itself touch the `User` object.

However, in an Event Message rule, you can listen for the `ContactChanged` event that BillingCenter generates every time that the address changes. The following example illustrates this concept.

```
USES:
uses gw.api.util.ArrayUtil

CONDITION (messageContext: entity.MessageContext):
//DestID 68 is the Console Message Logger
return messageContext.DestID == 68 and messageContext.EventName == "ContactChanged"

ACTION (messageContext: entity.MessageContext, actions : gw.rules.Action):
var message = messageContext.createMessage( "Event: " + messageContext.EventName )
var contact = messageContext.Root as Contact
var fields = contact.PrimaryAddress.ChangedFields
```

Event fired rule example – constructing a message

In the following example, an Event Fired rule constructs a message containing the account number any time that the application adds an account.

```
CONDITION (messageContext : MessageContext):
// Only fire if adding an account
return messageContext.EventName == "AccountAdded"

ACTION (messageContext : MessageContext, actions : gw.rules.Actions):
// Create handle to root object.
var account = messageContext.Root as Account

// Create the message
messageContext.createMessage("Account [" + account.AccountNumber + "] added")
```

Event fired rule example – asynchronous document storage failure

Guidewire recommends that, in working with asynchronous document storage, that you create an Event Fired rule to handle the failure of the database to store a document properly. If a document fails to store properly, BillingCenter creates a `FailedDocumentStore` event on `Document`.

To be useful, you need to add the `FailedDocumentStore` event to a messaging destination in `messaging-config.xml`. For example, you can add this event to message destination 324, which already tracks the `DocumentStore` event.

You then need to create an event rule to process the event as you see fit. You can, for example:

- Email a notification message to the creator of the `Document` entity.
- Create an activity on the primary or root object.
- Attempt to correct the document issues and create a new `DocumentStore` message.

The following example rule illustrates a possible Event Fired rule to handle the failure of the database to store a document.

```
CONDITION (messageContext : entity.MessageContext):
return messageContext.DestID == 324 and messageContext.EventName == "FailedDocumentStore"

ACTION (messageContext : entity.MessageContext, actions : gw.Rules.Action):
var document = messageContext.Root as Document
var fixedIssue = false
var sendEmail = false

if (fixedIssue) {
    messageContext.createMessage( "DocumentStore" )
}
else if (document.CreateUser.Contact.EmailAddress1 != null && sendEmail) {
    gw.api.email.EmailUtil.sendEmailWithBody(document,
    document.CreateUser.Contact.EmailAddress1,
    document.CreateUser.Contact.getDisplayName(),
    null, null, "Failed Document Store",
    //CREATE MESSAGE BODY HERE )
}
```

Exception

Use the Exception rule sets to specify an action to take if an Activity is overdue and is in an escalated state. You can use this rule set to send email, create an activity, or, to set a flag, among other actions.

BillingCenter runs the Exception rule sets at regularly scheduled intervals. Batch process Activity Escalation marks activities for escalation.

Activity escalation rules

An activity has two dates associated with it:

Due date	The target date to complete this activity. If the activity is still open after this date, it becomes overdue.
Escalation date	The date at which an open and overdue activity becomes escalated and needs urgent attention.

BillingCenter runs scheduled process `activityesc` every 30 minutes (by default). This batch process runs against all open activities in BillingCenter to find activities to escalate, using the following criteria:

- The activity has an escalation date that is non-null.
- The escalation date is in the past.
- BillingCenter has not yet escalated the activity.

BillingCenter marks each activity that meets the criteria as escalated. After the batch process runs, BillingCenter runs the escalation rules for all the activities that have hit their escalation date.

Use this rule set to specify what actions to take anytime that the activity enters the escalated condition. For example, you can use this rule set to reassign escalated activities. The simplest approach to escalated activities is to add an

activity for the appropriate supervisor to intervene and check on why the work is not yet complete. You can also decide that there is no follow-up action needed for certain kinds of activities.

In the following example, if an activity associated with a trouble ticket becomes escalated, the rule sets the activity priority to high.

```
CONDITION (activity : entity.Activity):  
return activity.TroubleTicket.Escalated  
  
ACTION (activity : entity.Activity, actions: gw.rules.Activity):  
activity.TroubleTicket.Priority ="high"
```

Preupdate

Use the Preupdate rule sets to perform domain logic or validation that you want to commit to the database before BillingCenter commits the entity in question to the database. For example, use these rule sets to handle changes to the delinquency process that occur as a result of other entity changes. Only a change to an entity marked as validatable in its metadata definition can trigger a preupdate rule.

It is important to understand that the Preupdate rules are non-recursive. The Preupdate rules do not run a second time on objects modified during their initial execution.

Guidewire provides a number of Preupdate rule categories in the base configuration. However, most of these categories are empty by default. Guidewire expects you to add Preupdate rules as necessary to meet your business logic.

Prioritized Preupdate handler

The `gw.plugin.preupdate.prioritized` package contains a set of classes and interfaces that simplify working with the preupdate handler plugin. Instead of directly working in Gosu rules, you can instead use Gosu code that calls the `IPreupdateHandler` plugin.

See the *BillingCenter Configuration* documentation and *Plugins, Prebuilt Integrations, and SOAP APIs* for more.

How BillingCenter triggers preupdate rules

The following BillingCenter base configuration root entities all trigger preupdate rules:

- Account
- Activity
- BaseMoneyReceived
- BillingInstruction
- Charge
- Contact
- DelinquencyProcess
- Disbursement
- History
- Hold
- Invoice
- OutgoingPayment
- PaymentRequest
- Plan
- Policy
- PolicyPeriod
- Producer
- ProducerPayment
- ProducerStatement
- SuspensePayment

Transaction
 TroubleTicket
 User
 Writeoff

For an entity to trigger a preupdate rule, that entity must implement the `Validatable` delegate. This is true for an entity in the base BillingCenter configuration or for a custom entity that you create.

BillingCenter runs the preupdate rules:

- After the creation or modification of an instance of a validatable entity.
- After the creation or modification of a subentity of a validatable entity and a `triggersValidation="true"` link connects the subentity to the validatable entity.

IMPORTANT: In running the Preupdate rule set, BillingCenter first computes the set of objects on which to run the preupdate rules. BillingCenter then runs the rules on this set of objects. If a preupdate rule then modifies an entity, the preupdate rules for that entity do not fire—unless the schedule for preupdate rules already includes this entity. In other words, changing an object in a preupdate rule does not then cause the preupdate rules to run on that object as well.

This is also the case for entities newly created within a preupdate rule. For example, if you create a new activity within a preupdate rule, BillingCenter does not then run the Activity preupdate rules on that activity.

Triggering Preupdate rules on custom entities

It is possible to create Preupdate rules for custom entities, meaning those entities that you create yourself and which are not part of the base Guidewire BillingCenter configuration.

For an entity to trigger a preupdate rule:

1. The entity must implement the `Validatable` delegate. In short, any extension entity that you create that you want to trigger a preupdate rule must implement the following code:

```
implementsEntity name="Validatable"
```

Note: The `Validatable` entity must be a root entity, not a subtype of an entity

2. A preupdate rule must exist in the Preupdate rule set category that conforms to the following naming convention:
`<custom_entity_name>Preupdate`

Naming a custom Preupdate rule set

A preupdate rule must exist in a preupdate rule set that reflects the name of the custom entity with the rule name ending with `Preupdate`. In other words, you must create a rule set with the same name as the extension entity and add the word `Preupdate` to the rule name as a suffix. You then place your rules in this rule set.

For example, if you create an extension entity named `NewEntityExt`, then you need to create a rule set to hold your preupdate rules and name the rule set `NewEntityExtPreupdate`.

Transaction Preupdate rule example – creating a history event

IMPORTANT: Do not add large numbers of rules to the Transaction Preupdate rule set. This rule set can seriously affect performance as, typically, there are many transactions active in the system.

In the following example, the rule first determines if this transaction is of type `AccountAdjustment`. If this is true, it creates a history event object (`history`) to track the change.

```
CONDITION (transaction : entity.Transaction):
return transaction typeis AccountAdjustment

ACTION (transaction : entity.Transaction, actions : gw.rules.Action):
```

```
var adjustment = transaction as AccountAdjustment
var history = new AccountHistory(adjustment)

history.Account = adjustment.Account
history.EventDate = adjustment.TransactionDate
history.EventType = HistoryEventType.TC_ACCOUNTADJUSTED
history.Transaction = adjustment
history.Description = adjustment.LongDisplayName

adjustment.Account.addToHistory(history)
```

Validation

In the base configuration, BillingCenter provides the ability to write validation rules against the following entities:

- Activity
- Contact
- Group
- Region
- User

Configuration parameter `EnableWorkQueueValidation` in file `config.xml` determines whether BillingCenter performs validation on the BillingCenter-specific work queues.

In the base configuration, Guidewire provides Validation rule set categories for the following entities:

- Contact
- Person
- Region

For the most part, Guidewire disables any sample Validation rules that exist in the base configuration. Guidewire expects you to add Validation rules as necessary to meet your business logic.

ClaimCenter rule set categories

Note: It is important to understand that you edit Gosu rules in Guidewire Studio. Certain Gosu rules relate to the business rules that you create and manage in the ClaimCenter Business Rules editor. For business rules that are in the same category as Gosu rules, such as Preupdate rules, ClaimCenter runs the Gosu rules first.

As part of the ClaimCenter base configuration, Guidewire provides Gosu sample rules in the following rule set categories.

Approval Routing

Route approval activities for a transaction that requires approval to the appropriate user.

Archive

Manage the claim archive process.

Assignment

Determine the responsible party for an activity, claim, exposure, matter, or service request.

BulkInvoice Approval

Define the approval requirements for Bulk Invoices.

Closed

Perform an action whenever you close an activity, claim, exposure, or matter—for example, sending a notification.

Event Message

Handle communication with integrated external applications.

Exception

Specify an action to take under certain circumstances. For example, if no one touches a claim for a certain number of days, then create a review activity.

Initial Reserve

Determine the initial financial reserves that to create for a new exposure.

Loaded

Trigger automatic actions whenever you load a new claim using the FNOL import interface.

Postsetup

Trigger automatic actions whenever you add a new claim, activity, exposure, matter, or transaction within ClaimCenter. These rules run after the Assignment and Workplan rules complete their execution.

Presetup

Trigger automatic actions whenever you add a new claim, exposure, matter, or transaction within ClaimCenter. These rules run prior to the execution of rules in the Segmentation, Assignment, and Workplan rule set categories.

Preupdate

Trigger automatic actions whenever a claim or other high-level entity changes.

Purge

Specify additional conditions or actions to take in purging archived entities.

Reopened

Trigger automated actions on reopening a claim or exposure.

Segmentation

Categorize a new claim or an exposure based on complexity, severity of damage, or other attributes.

Transaction Approval

Verify that a user has the appropriate authorization to submit a financial transaction.

Validation

Check for missing information or invalid data on validatable entities, Activity, Claim, Contact, Exposure, objects, for example.

Workplan

Add initial activities to a claim, exposure, or matter as a checklist of work that various people need to do on the claim.

Approval routing

Approval routing rules route an approval activity for a transaction that requires approval. ClaimCenter uses approval routing rules to determine which person to assign the approval activity to. Typically, a supervisor is the approving authority. The transaction set and its transactions remain in a *pending approval* state until someone approves the activity.

Approval rules and Transaction approval rules

The Approval Rules rule set and the rule sets in the Transaction Approval rule set category form a pair. ClaimCenter calls these rule sets sequentially.

- ClaimCenter calls rule sets in the Transaction Approval rule set category whenever you submit any kind of financial transaction.
- ClaimCenter calls the Approval Rules rule set if the outcome of transaction approval is Yes or if the transaction exceeds the transaction authority limit, causing the transaction to need approval.

Infinite-loop approval rules

Do not create approval rules that generate an infinite loop. For example, suppose that a rule first assigns the transaction to a supervisor for approval, who subsequently approves it. Then, the rule assigns the transaction approval to someone who does not have the authority to approve the transaction. ClaimCenter runs the rule set again, and again reassigns the approval to the supervisor, generating a never-ending loop.

Approval rules for TransactionSet

In the base configuration, Guidewire provides the following sample TransactionSet Approval Gosu rules. To work with these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > ApprovalRouting > ApprovalRules

In the base configuration, this rule set contains the following rules.

ARR01000 - Western Regional Auto Supervisors

This rule sets the approving user to the supervisor of the Western Auto Group if the last approval was by a supervisor of one of the subordinate teams. The rule then exits the rule set. If all the conditions are false, the next rule executes.

This rule is an example of how to encapsulate complex logic in reusable, external class methods instead of implementing all logic inside the rule.

ARR02000 - Try to assign to group supervisor

This rule calls the `transactionSet.approveByGroupSupervisor` method. This method first verifies that `ClaimCenter` has not yet assigned the associated claim or exposure. It also verifies that the current user making the request is not the group supervisor. If so, the method then assigns approval to the group supervisor and the rule exits the rule set. If not, the method returns `false` and the next rule executes.

ARR03000 - Default Rule

This rule runs if neither of the previous rules succeeded in making an assignment to a group supervisor. The rule assigns approval to a higher level supervisor.

Methods to set the approver of a transaction

The following list describes the main methods used to set the person who can approve the transaction. These methods are available on the `TransactionSet` object.

`approveByGroupSupervisor`

Use to assign an approval activity to a group supervisor based on the number of exposures associated with the transaction:

- If the transaction set has transactions associated with only one exposure, the method assigns the approval activity to the supervisor of the group assigned to the exposure.
- If the transaction set has transactions associated with multiple exposures, the method assigns the approval activity to the supervisor of the group assigned to the associated claim.

In either case, if the supervisor of the group is the same as the user requesting approval, then the activity remains unassigned.

Note: If you use this method, you must explicitly handle this last case.

`approveByUserSupervisor`

Use to assign an approval activity to a group supervisor based on the user who submitted the transaction:

- If the submitting user is in the owning group and is not the supervisor, then approval goes to the group supervisor.
- If the submitting user is the group supervisor, then `ClaimCenter` escalates the approval to the group supervisor of the next group up the group hierarchy.
- If the submitting user is a group supervisor someplace higher in the hierarchy, then the approval escalates to the supervisor of the next group up the hierarchy.
- If the submitting user is in *one and only one* group, then the approval goes to the supervisor of that group. Otherwise, approval goes to the supervisor of the owning group.

Finally, if `ClaimCenter` worked its way through the previous conditions to the supervisor of the root group without making an assignment, then `ClaimCenter` assigns the approval to user *defaultowner*.

`setApprovingUser`

Use to assign an approval activity to a specified user. You must provide a user and group as parameters. The method checks whether the user is retired or inactive (`user.credential.Active != true`) and returns `false` if either of these conditions exist. It also returns `false` if the user or group is missing. It returns `true` otherwise.

IMPORTANT: Call this method only from within the Approval Routing rule set.

Note: The `BulkInvoice` object has the same set of methods available to set the person who can approve the bulk invoice.

Example TransactionSet Approval rule

In the following example Transaction Set Approval rule, the rule assigns the approval to the supervisor, unless the supervisor is the one who entered the transaction.

```

USES:
uses gw.api.system.CCLoggerCategory

CONDITION (transactionSet : entity.TransactionSet):
return true

ACTION (transactionSet : entity.TransactionSet, actions : gw.rules.Action):
var exp = transactionSet.AllTransactions*.Exposure
// Verify that the supervisor has not already approved this TransactionSet
if (not exists( act in transactionSet.Claim.Activities where (act.TransactionSet == transactionSet
    and (act.ActivityPattern.Code == "approve_reserve_change"
    or act.ActivityPattern.Code == "approve_payment" )
    and act.Approved == true and act.AssignedUser == exp[0].AssignedGroup.Supervisor ))) {
    if ( transactionSet.approveByGroupSupervisor() ) {
        CCLoggerCategory.ASSIGNMENT.debug(" Assign to supervisor")
        CCLoggerCategory.ASSIGNMENT.debug(
            transactionSet.AllTransactions*.Exposure[0].AssignedGroup.Supervisor.Contact.DisplayName )
        actions.exit()
    }
}

```

Note: In actual practice, Guidewire recommends that you make String values into display keys or constant variables, as appropriate.

BulkInvoice Approval Assignment Rules

The BulkInvoice Approval Assignment Rules and the BulkInvoice Approval Rules rule sets form a pair. These rules work together similarly to the rule sets in the Approval Routing rule set category and the Transaction Approval rule set category. ClaimCenter calls these rule sets sequentially.

ClaimCenter runs the BulkInvoice Approval Assignment rule set for each bulk invoice to determine to whom to assign it for approval. See “Bulk Invoice Approval rules” on page 61.

To work with these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > ApprovalRouting > BulkInvoiceApprovalAssignmentRules

In the base configuration, this rule set contains the following rules. You can customize these rules to meet your business needs.

ARB01000 - Western Regional Auto Supervisors

This rule sets the approving user to the supervisor of the Western Auto Group if the last approval was by a supervisor of one of the subordinate teams. The rule then exits the rule set. If all the conditions are false, the next rule executes.

This rule is an example of how to encapsulate complex logic in reusable, external class methods instead of implementing all logic inside the rule.

ARB02000 - Try to assign to group supervisor

This rule calls `bulkInvoice.approveByGroupSupervisor`. This method first verifies ClaimCenter has not yet assigned the associated claim or exposure. It then verifies that the current user making the request is not the group supervisor. If so, the method then assigns approval to the group supervisor and the rule exits the rule set. If not, the method returns false and the next rule executes.

ARB03000 - Default Rule

This rule runs if neither of the previous rules succeeded in making an assignment to a group supervisor. The rule assigns approval to a higher level supervisor.

Archive

IMPORTANT: Guidewire must enable claim archiving for you to be able to use this feature.

Archiving is the process of moving a closed claim and associated data from the active ClaimCenter database to a document storage area. Archived claims occupy less space in the active database. You can still search for and retrieve archived claims.

For information on archive-related rules, see the *Data Archiving* guide.

Assignment

Certain business entities are *assignable*. For these entities, it is possible to determine—either through the use of assignment rules or through the use of Gosu assignment methods—the party responsible for that entity. In the base configuration, ClaimCenter defines the following entities as assignable:

- Activity
- Claim
- Exposure
- Matter
- ServiceRequest
- UserRoleAssignment

Note: UserRoleAssignment does not have a rule set defined for it in the base configuration. It is in this list because it is assignable.

If an entity is not assignable in the base configuration, you can extend it and make the new entity assignable. To make an entity assignable, it must implement either the Assignable or the CCAssignable delegate class.

You use Gosu assignment methods to set an assigned group and user for an assignable entity. These assignment methods run either in the context of the Assignment rule set or simply as Gosu code, such as in a class or enhancement.

In general, ClaimCenter uses the Global assignment rules to determine the group to which to assign the entity. It then runs the Default Group assignment rules to assign the entity to a user in the chosen group. ClaimCenter runs the assignment rules until it either completes the assignment or runs out of rules to run.

To work with the Assignment rule sets, navigate in the Studio **Project** window:

configuration > config > Rule Sets > Assignment

Default Group Activity Assignment rules

The Gosu Default Group Activity Assignment rules determine how ClaimCenter assigns activities within a group. To work with these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Assignment > DefaultGroupActivityAssignmentRules

In the base configuration, this rule set contains the following rules.

DGA01000 - SI Assign claim review activity to supervisor

If the activity is for a special investigation review (SI_review), the rule assigns the review activity to the current group's supervisor. If the assignment is successful, the rule exits the rule set.

DGA02000 - SI Assign Escalation Activity to named SIU user

If the activity is for a special investigation escalation (SIU_escalation), the rule assigns the review activity to a user who has the SIU Investigator role. If the assignment is successful, the rule exits the rule set.

DGA03000 - SI Default SI Escalation activity routing

If the activity is for a special investigation escalation (SIU_escalation), assignment to a user was not successful in the previous rule. This rule assigns the activity to a user in the current group by round-robin. If the assignment is successful, the rule exits the rule set.

DGA04000 - Assign reinsurance review activity to reins user

If the activity is for a reinsurance review or synchronization, and if the claim specifies a reinsurance manager, attempt to assign it to the reinsurance manager. If there is no reinsurance manager on the claim, assign the activity by round-robin to the group assigned to the claim associated with the activity. If the assignment is successful, the rule exits the rule set.

DGA05000 - Assign to Subro group

If the activity is to determine the opportunity for a recovery (subro_check), the rule assigns the activity to the user who is the subrogation owner. If the assignment is successful, the rule exits the rule set.

DGA06000 - Test Assignment by Proximity

The rule first determines if it can use the geocoding feature locally and if there is a primary address for the claimant. If so, the rule searches for a nearby user that it can assign the activity to. In the base configuration, the search radius is five miles and the default user name is "b". If the assignment is successful, the rule exits the rule set.

DGA07000 - Assign activity to named user

This rule determines if the activity for assignment has a user role associated with it and if there is already a user with that role on the claim. If so, the rule assigns the activity to a user with the proper role and then exits the rule set.

DGA08000 - Default - if users in group

This rule determines if there is at least one user in the group assigned to the activity. If so, the rule assigns the activity by round-robin and then exits the rule set.

DGA09000 - Default

If there is no user assigned to the activity, this rule assigns it to the group's supervisor for manual assignment to a user.

DGA10000 - Assignment by Proximity and Attribute Example

Guidewire disables this rule in the base configuration by default. It is a sample rule that shows how to use a custom attribute and proximity search to find a user and assign an activity to that user. The example also uses logger categories to log the results.

Default Group Claim Assignment rules

The Gosu Default Group Claim Assignment rules determine how ClaimCenter assigns claims within a group.

To work with these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Assignment > DefaultGroupClaimAssignmentRules

In the base configuration, this rule set contains the following rules.

DGC00500 - Balanced workload within group

If enabled, the rule first determines if weighted workload assignment is also enabled. If so, it looks for users in the current group that are members and who are available for work, those who are active and not on vacation. If the rule finds a user, it assigns the claim by weighted workload. Whether assignment is successful or not, the rule exits the rule set.

DGC01000 - Default - if users in group

If there are users in the group assigned to the activity, assign the claim by round-robin. If the assignment is successful, the rule exits the rule set.

DGC02000 - Default - if no users in group

If the claim is still not assigned to a user, this rule assigns it to the group's supervisor for manual assignment to a user.

Default Group Exposure Assignment rules

The Gosu Default Group Exposure Assignment rules determine how ClaimCenter assigns exposures within a group. These rules apply only if the claim owner does not assign the exposure.

To work with these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Assignment > DefaultGroupExposureAssignmentRules

In the base configuration, this rule set contains the following rules.

DGE00500 - Balance workload within group

If enabled, the rule first determines if weighted workload assignment and exposure-level weighted workload assignment is enabled. If so, it looks for users in the current group that are members and who are available for work, those who are active and not on vacation. If the rule finds a user, it assigns the exposure by weighted workload. Whether assignment is successful or not, the rule exits the rule set.

DGE01000 - Group exposures with same adjuster

This rule determines if anyone in the assigned group has already been assigned an exposure on this claim. If so, the rule assigns that user to the exposure, too. If the assignment is successful, the rule exits the rule set.

DGE02000 - Default - users in team

If there is no assigned user for the exposure, this rule determines if the assigned group for the exposure contains any users. If so, the rule assigns the exposure by round-robin.

DGE03000 - Default - no users in team

If there is still no assigned user for the exposure, this rule assigns the exposure to the group's supervisor for manual assignment to a user.

Default Group ServiceRequest Assignment rules

The Gosu Default Group Service Request Assignment rules determine how ClaimCenter assigns service requests to users within a group.

To work with these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Assignment > DefaultGroupServiceRequestAssignmentRules

In the base configuration, this rule set contains the following rule.

DGS01000 - Default

This rule assigns the service request to the claim's assigned user and to the default group for that user. The rule exits regardless of whether the assignment is successful.

Global Activity Assignment rules

The Gosu Global Activity Assignment rules determine how ClaimCenter first assigns activities to a group. After these rules run, the Default Group Activity Assignment rules run to assign the activity to a user in the group.

To work with these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Assignment > GlobalActivityAssignmentRules

In the base configuration, this rule set contains the following rules.

GAA01000 - SI - Assign claim review to claim owner's group

If the activity is for a special investigation review (SI_review), the rule assigns the review activity to the claim's assigned group. If the assignment is successful, the rule exits the rule set.

GAA02000 - SI - Assign SIU escalation activity to SIU group

If the activity is for a special investigation escalation (SIU_escalation), in the base configuration, the rule assigns the review activity to an SIU group:

1. The group of the SIU Investigator on the claim, if there is such a user.
2. The Western SIU group if the first assignment was not possible.

If the assignment is successful, the rule exits the rule set.

GAA03000 - Subro - Assign to Subro group

If the activity is to determine the opportunity for a recovery (subro_check), in the base configuration, the rule assigns the review activity to a subrogation group:

1. The group of the Subrogation Owner on the claim, if there is such a user.
2. The HQ Subro Unit group if the first assignment was not possible.

If the assignment is successful, the rule exits the rule set.

GAA04000 - Assign reinsurance review to Reinsurance group

If the activity is for a reinsurance review or synchronization, attempt to assign it to the reinsurance manager and group. If a reinsurance manager does not exist on the claim, attempt to assign the activity by round-robin to a user in the Reinsurance group. If the assignment is successful, the rule exits the rule set.

GAA05000 - Homeowners PolicyType

If the policy for the claim that is related to the activity is a Homeowners policy, execute the following child rules.

GAA05100 - Damaged items list

If the activity pattern is for damaged items, attempt to assign it to the owner of the claim or exposure, whichever is relevant for the activity. If the assignment is successful, the rule exits the rule set.

GAA05200 - Contact insured about living expenses

If the activity pattern is to contact the insured about living expenses, attempt to assign it to the owner of the claim or exposure, whichever is relevant for the activity. If the assignment is successful, the rule exits the rule set.

GAA05300 - Get property inspected

If the activity pattern is to inspect the damage property, attempt to assign it to the owner of the claim or exposure, whichever is relevant for the activity. If the assignment is successful, the rule exits the rule set.

GAA05400 - Get claimant medical reports

If the activity pattern is to get medical report from the claimant, attempt to assign it to the owner of the claim or exposure, whichever is relevant for the activity. If the assignment is successful, the rule exits the rule set.

GAA05500 - Verify coverage

If the activity pattern is to verify the coverage, attempt to assign it to the owner of the claim or exposure, whichever is relevant for the activity. If the assignment is successful, the rule exits the rule set.

GAA06000 - Activity Exceptions

This rule serves as a folder to group child rules that process assignment of some special activities.

GAA06100 - Legal

If the activity pattern is to initiate a legal review, attempt to assign the activity in the following order:

1. Attempt to assign the activity to an assigned claim user role matching that of the activity, and to the group associated with that user. If successful, exit the rule set.
2. If the previous assignment was not successful, attempt to assign the activity to a group in a location near the insured party's primary address. If successful, exit the rule set.
3. If the previous assignment was not successful, attempt to assign the group by round-robin, starting with the group type associated with the activity pattern and including subgroups if necessary. If successful, exit the rule set.

GAA06200 - Independent appraisal

If the activity pattern is to initiate an independent appraisal, attempt to assign the activity in the following order:

1. Attempt to assign the activity to an assigned claim user role matching that of the activity, and to the group associated with that user. If successful, exit the rule set.
2. If the previous assignment was not successful, attempt to assign the activity to a group in a location near the loss location. If successful, exit the rule set.

3. If the previous assignment was not successful, attempt to assign the activity to a group in a location near the primary address of the claimant on the exposure. If successful, exit the rule set.
4. If the previous assignment was not successful, attempt to assign the activity to a group in a location near the insured party's primary address. If successful, exit the rule set.

GAA06300 - Nurse Visit

If the activity pattern is to schedule a visit from a nurse, attempt to assign the activity in the following order:

1. Attempt to assign the activity to an assigned claim user role matching that of the activity, and to the group associated with that user. If successful, exit the rule set.
2. If the previous assignment was not successful, attempt to assign the activity to a group in a location near the insured party's primary address. If successful, exit the rule set.

GAA06400 - Property inspection

If the activity pattern is to schedule a property inspection, attempt to assign the activity in the following order:

1. Attempt to assign the activity to an assigned claim user role matching that of the activity, and to the group associated with that user. If successful, exit the rule set.
2. If the previous assignment was not successful, attempt to assign the activity to a group in a location near the insured party's primary address. If successful, exit the rule set.

GAA06500 - Police

If the activity pattern is to obtain a police report, attempt to assign the activity in the following order:

1. Attempt to assign the activity to an assigned claim user role matching that of the activity, and to the group associated with that user. If successful, exit the rule set.
2. If the previous assignment was not successful, attempt to assign the activity to a group in a location near the loss location. If successful, exit the rule set.
3. If the previous assignment was not successful, attempt to assign the activity to a group in a location near the primary address of the claimant on the exposure. If successful, exit the rule set.
4. If the previous assignment was not successful, attempt to assign the activity to a group in a location near the insured party's primary address. If successful, exit the rule set.

GAA06600 - Assignment Review

If the activity pattern is to review the assignment of the activity, if there is an assigned group for the claim, assign the activity to the group's supervisor.

GAA07000 - Default

If the activity is still unassigned to a group, this rule assigns it the owner of the claim or exposure, as appropriate.

Global Claim Assignment rules

The Gosu Global Claim Assignment rules determine how ClaimCenter first assigns claims to a group.

To work with these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Assignment > GlobalClaimAssignmentRules

In the base configuration, this rule set contains the following rules.

GCA00010 - Claim

This rule serves as a folder to group its child rules.

GCAA00010 - Auto

If the claim loss type is Auto, this rule uses the loss type and claim segment to get the top two matching group type choices for assigning this claim. The rule then tries the following assignment options, in order, and exits the rule set if one of them succeeds:

1. Primary group type, near the claim's loss location
2. Secondary group type, near the claim's loss location
3. Primary group type, near the insured party's primary address
4. Secondary group type, near the insured party's primary address

GCAP00010 - Property

If the claim loss type is property (PR), this rule performs the same steps to assign the claim as the previous rule.

GCAL00010 - Liability

If the claim loss type is liability (GL), this rule performs the same steps to assign the claim as the previous rule.

GCAWC0010 - WC

If the claim loss type is Workers' Compensation (WC), this rule performs the same steps to assign the claim as the previous rule.

The rule does further processing if the attempts to assign the claim all fail. For example, there is neither a primary address for the insured party nor a loss location. The rule then tries to assign the claim to the first group to which the claim creator belongs.

GCATR0040 - Travel

If the claim loss type is travel (TRAV), this rule performs the same steps to assign the claim as the previous rule.

For this rule, the first four assignment attempts often fail. For example, the Travel group's location rarely matches the loss location, and it often does not match the insured party's primary address. As with the previous rule, the rule then tries to assign the claim to the first group to which the claim creator belongs.

Global Exposure Assignment rules

The Gosu Global Exposure Assignment rules determine how ClaimCenter first assigns exposures to a group.

To work with these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Assignment > GlobalExposureAssignmentRules

In the base configuration, this rule set contains the following rules.

GEA010000 - Auto Claims

This rule determines if the exposure has a loss type of Auto and that there is an exposure on the claim that has either of the following properties:

- The loss party is the insured party on the claim.
- The exposure type is not bodily injury damage.

If the exposure matches these criteria, it is processed by the following child rule. Otherwise, if the claim has only third-party injury exposures, this claim was already assigned to a liability adjuster, and all exposures go to that adjuster.

GEA01100 - 3rd-party injury exposures

This rule determines if the exposure has both the following properties:

- The loss party is third party
- The exposure type is bodily injury damage

If so, the rule uses the claim's loss type and the exposure's segment to get the top two matching group type choices for assigning this claim. The rule then tries the following assignment options, in order, and exits the rule set if one of them succeeds:

1. Primary group type, near the insured party's primary address
2. Secondary group type, near the insured party's primary address

GEA02000 - Non-auto claims

This rule determines if the exposure has a loss type other than Auto. If so, the child rule executes.

GEO02100 - Default

This rule assigns the exposure to the assigned owner and group for the claim. It then exists the rule set.

GEO03000 - Default

If the exposure is still not assigned, this rule assigns the exposure to the assigned owner and group for the claim. It then exits the rule set.

Global Matter Assignment rules

The Gosu Global Matter Assignment rules determine how ClaimCenter first assigns matters to a group.

To work with these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Assignment > GlobalMatterAssignmentRules

In the base configuration, this rule set contains the following rule.

GMA01000 - Default

This rule assigns the matter to the assigned owner and group for the claim. It then exits the rule set.

Global ServiceRequest Assignment rules

The Gosu Global Service Request Assignment rules determine how ClaimCenter first assigns service requests to a group.

To work with these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Assignment > GlobalServiceRequestAssignmentRules

In the base configuration, this rule set contains the following rule.

GSA01000 - Default

This rule assigns the service request to the assigned owner and group for the claim. It then exits the rule set.

BulkInvoice

In ClaimCenter, the approval process for a bulk invoice is much the same as the approval process for transaction sets. However, there is one critical difference: There are no authority limits for bulk invoices, although authority limits are verified on the individual checks in the bulk invoice. The Bulk Invoice Approval rules alone control whether a bulk invoice itself requires approval.

The BulkInvoice Approval Rules and the BulkInvoice Approval Assignment Rules rule sets form a pair. These rules work together similarly to the rule sets in the Approval Routing and Transaction Approval rule set categories. ClaimCenter calls these rule sets sequentially.

Bulk Invoice Approval rules

ClaimCenter runs the BulkInvoice Approval rules for each bulk invoice to determine whether the bulk invoice requires approval.

In the base configuration, ClaimCenter provides a sample approval rule for bulk invoices that always requires approval. To work with these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > BulkInvoiceApproval > BulkInvoiceApprovalRules

ClaimCenter provides a single rule in this rule set in the base configuration, BIA010000 - Always Require Approval. Guidewire disables this rule in the base configuration. This rule contains code showing how to require approval each time a bulk invoice is submitted.

Writing an approval rule for a bulk invoice is similar to writing an approval rule for a transaction set. However, keep the following important differences between the two types of rules in mind in working with these rule types.

Approval rules

For a typical `TransactionSet` approval rule, the rule condition requires verification of the subtype of the transaction set, such as `TransactionSet.subtype == "checkset"`. This verification is required because `ClaimCenter` passes all created transaction sets, regardless of subtype, to the same approval rules. In contrast, the bulk invoice approval rules have no need for subtype verification in their rule conditions. The only implicit entity that `ClaimCenter` ever passes to the bulk invoice approval rules is a bulk invoice.

Bulk invoice rules

Bulk invoice approval rules have an implicit reference to the bulk invoice submitted for approval. You can access this reference in a rule condition or action block through the `BulkInvoice` entity. This behavior is different from the rules in the Transaction Approval rule set category, which have an implicit reference to the transaction set being approved.

Automatic approval of bulk invoices

If none of the approval rules determine that the bulk invoice requires approval, `ClaimCenter` automatically and immediately approves the bulk invoice. `ClaimCenter` then does the following:

- Marks the bulk invoice as `Approved` and sets the approval date to the current day.
- Transitions the bulk invoice status to `Pending` upon bulk invoice item validation.
- Transitions all the bulk invoice items that have a status of `Draft` to `Approved` status.

Closed

The `Closed` rule sets fire whenever you close an activity, claim, exposure, or matter. They fire immediately on closing one of these entities, not before. You can use these rules to send a notification or to generate additional activities.

To work with this rule set, navigate in the Studio **Project** window to **configuration > config > Rule Sets > Closed**.

The following examples, which work in conjunction with one another, show some aspects of how to use these rule sets:

- “Example – closing a claim from exposure closed rule” on page 63
- “Example – closing claim activities while closing a claim” on page 63

Activity Closed rules

`ClaimCenter` executes the Activity Closed rules immediately after an activity closes. Use these rules to create follow-up actions and activities.

To work with these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Closed > ActivityClosed

In the base configuration, this rule set contains the following sample rule.

CAC01000 - Salvage

The rule first verifies that an activity to salvage a vehicle is complete. If so, the rule sets the vehicle recovery date to the current date.

Claim Closed rules

As the name suggests, the Claim Closed rules execute immediately after a claim closes. Use these rules to create follow-up actions and activities.

To work with these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Closed > ClaimClosed

In the base configuration, this rule set contains the following rules.

CCL01000 - Notify external systems

This rule notifies external systems that the claim is closed.

CCL02000 - Suspend AutoSync for Related Contacts

Any Contact on a closed claim must have its AutoSync status set to Suspended to prevent attempts by ContactManager and ClaimCenter to update the closed claim. This action helps facilitate the synchronization of contact information between ClaimCenter and ContactManager.

CCL04000 - Set archive eligibility date

Sets `claim.DateEligibleForArchive` to a date created by adding the value of configuration parameter `DaysClosedBeforeArchive` to the current system date.

This rule works in conjunction with the claim reopened rule CRO04000 - Clear archive eligibility date.

IMPORTANT: Guidewire must enable claim archiving for you to be able to use this feature.

CCL05000 - Set archived entity purge date

Guidewire disables this rule in the base configuration. Sets `claim.ClaimInfo.ArchivedEntityPurgeDate` to the current date plus seven years. The purge date is configurable.

Example – closing claim activities while closing a claim

The following example Closed rule, which you would add to the Claim Closed rule set, runs whenever ClaimCenter closes a claim. The rule works in conjunction with the “Example – Closing a Claim from Exposure Closed Rule” example, which closes a claim whenever the last exposure closes. This example rule closes all open activities associated with the claim that are not available to closed claims.

```
CONDITION (claim : entity.Claim):
return true

ACTION (claim : entity.Claim, actions : gw.rules.Action):
for( act in claim.Activities) {
  if (act.Status == ActivityStatus.TC_OPEN and act.ActivityPattern != null
    and act.ActivityPattern.ClosedClaimAvlble == false) {
    act.complete()
    if (act.Description == null) { act.Description = "Closed by ClaimCenter." }
    else { act.Description = act.Description + "Closed by ClaimCenter." }
  }
}
```

Note: In actual practice, Guidewire recommends that you make String values into display keys or constant variables, as appropriate.

Exposure Closed rules

The Exposure Closed rules execute immediately after the rules in the Exposure Closed Validation Rules rule set run. Use these rules to take automated actions on the closure of the exposure.

To work with this rule set, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Closed > ExposureClosed

In the base configuration, this rule set contains the following rule.

CEX01000 - Notify external systems

Notifies external systems that the exposure is closed.

Example – closing a claim from exposure closed rule

The following example Closed rule, which you would add to the Exposure Closed rule set, runs whenever ClaimCenter closes an exposure. This rule closes the claim file from the Exposure Closed Rules rule set if all other exposures on the claim are closed.

```
CONDITION (exposure : entity.Exposure):
return !exists (exp in exposure.Claim.Exposures where exp.State != TC_CLOSED)
```



```
ACTION (exposure : entity.Exposure, actions : gw.rules.Action):
exposure.Claim.close( "completed", "Claim number " + exposure.Claim.ClaimNumber
+ " closed by ClaimCenter because last open exposure was closed." )
```

Note: In actual practice, Guidewire recommends that you make String values into display keys or constant variables, as appropriate.

Matter Closed rules

A matter is the set of data organized around a single lawsuit or potential lawsuit. This data includes information on the attorneys involved, the trial details, and the lawsuit details. These rules execute immediately after a matter closes. Use these rules to take automated actions on the closure of a matter.

To work with this rule set, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Closed > MatterClosed

In the base configuration, this rule set contains the following rule.

CMC01000 - Set Litigation Status

This rule first ensures that all matters on the associated claim have a status of Closed. If so, this rule sets the litigation status on the associated claim to "complete".

Event message

IMPORTANT: ClaimCenter runs the Event Message rules as part of the database bundle commit process. Use these rules only to create integration messages.

In the base configuration, there is a single rule set, Event Fired, in the Event Message rule category. To work with these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > EventMessage > EventFired

The rules in this rule set:

- Perform event processing.
- Generate messages about events that have occurred.

ClaimCenter calls the Event Fired rules if an entity involved in a bundle commit triggers an event of interest to a message destination. A message destination registers an event to indicate its interest in that event.

As part of the event processing:

- ClaimCenter runs the rules in the Event Fired rule set a single time for every event of interest to a message destination.
- ClaimCenter runs the Event Fired rule set a single time for each destination that is listening for that particular event. Thus, it is possible for ClaimCenter to run the Event Fired rules sets multiple times for each event, one time for each destination interested in that event.

About messages and messaging events

ClaimCenter automatically generates standard events for most top-level objects. In general, event generation occurs for any addition, modification, removal, or retirement of a top-level entity. For example, ClaimCenter automatically generates the following events on the Activity object:

- ActivityAdded
- ActivityChanged
- ActivityRemoved

It is also possible to create a custom event on an entity by using the `addEvent` method. This method takes a single parameter, `eventName`, which is a `String` value that sets the name of the event.

```
entity.addEvent(eventName)
```

IMPORTANT: Do not create custom code that generates update events for any entity type that can possibly be part of an archive bundle.

Message destinations and message events

You use the Studio Messaging editor to link one or more events to a message destination:

- A *message destination* is an external system to which it is possible to send messages.
- A *message event* is an abstract notification of a change in ClaimCenter that is of possible interest to an external system. For example, this change can be adding, changing, or removing a Guidewire entity.

Using the Studio Messaging editor, it is possible to associate (register) one or more events with a particular message destination. For example, in the base configuration, ClaimCenter associates the following events with the `metro` message destination (DestID 67):

- `MetroReportAdded`
- `MetroReportChanged`

If you modify or add a `MetroReport` object, ClaimCenter generates the relevant event. Then, during a bundle commit of the `MetroReport` object, ClaimCenter notifies any Event Message rule interested in the event of the occurrence of the event. You can use this information to generate a message to send to an external system or to the system console for logging purposes, for example.

Message destinations and message plugins

Each message destination, which encapsulates all the necessary behavior for delivering a notification to an external system, uses three different plugin interfaces to implement the destination. Each plugin handles different parts of what a destination does, as follows:

- The message request plugin handles message pre-processing.
- The message transport plugin handles message transport.
- The message reply plugin handles message replies.

You register new messaging plugin implementation classes in Studio first in the Plugins Registry editor. Whenever you first create a new plugin registry, Studio prompts you for a name for the registry and a plugin interface name. Use that plugin registry name in the Messaging editor in Studio to register each destination.

Note: You must register your plugin in two different editors in Studio, first in the Plugin Registry, and then in the Messaging editor.

IMPORTANT: After the ClaimCenter application server starts, ClaimCenter initializes all message destinations. ClaimCenter saves a list of events for which each destination requested notifications. Because this process happens at system start-up, you must restart the ClaimCenter application if you change the list of events or destinations.

Generating messages

Use the `messageContext.createMessage` method to create the message text, which can be either a simple text message or a more involved constructed message. The following code is an example of a simple text message that uses the Gosu in-line dynamic template functionality to construct the message. Gosu in-line dynamic templates combine static text with values from variables or other calculations that Gosu evaluates at run time. For example, the following `createMessage` method call creates a message that lists the event name and the entity that triggered this rule set.

```
messageContext.createMessage("${messageContext.EventName} - ${messageContext.Root}")
```

The following is an example of a constructed message that provides more detailed information if a claim changes.

```
var session = messageContext.SessionMarker
var claim = messageContext.Root as Claim

// Create a message for this claim.
var msg = messageContext.createMessage("Message for ClaimChanged")
msg.putEntityByName( "assigneduser", claim.AssignedUser )

// Add assigned user to session map, so it's available in the session.
session.addToTempMap( "assigneduser", claim.AssignedUser )

// Create a message for each exposure.
for (exposure in claim.Exposures) {
    msg.messageRoot = exposure
    var user : User = session.getFromTempMap( "assigneduser" ) as User
    msg.putEntityByName( "assigneduser", user )
    msg.putEntityByName( "claim", claim )
}
```

Detecting object changes in an Event Fired rule

You might want to listen for a change to an object that does not automatically trigger an event if you update it. For example, suppose that you want to listen for a change to the User object, such as an update of the address. The User object itself does not contain an address. Instead, ClaimCenter stores addresses as an array on UserContact, which is an extension of Person, which is a subtype of Contact, which points to User. Therefore, updating an address does not directly touch the User object.

However, in an Event Fired rule, you can listen for the ContactChanged event that ClaimCenter generates every time that the address changes. The following example illustrates this concept.

```
USES:
uses gw.api.util.ArrayUtil

CONDITION (messageContext : entity.MessageContext):
//DestID 68 is the Console Message Logger
return messageContext.DestID == 68 and messageContext.EventName == "ContactChanged"

ACTION (messageContext : entity.MessageContext, actions : gw.Rules.Action):
var message = messageContext.createMessage( "Event: " + messageContext.EventName )
var contact = messageContext.Root as Contact
var fields = contact.PrimaryAddress.ChangedFields
```

About modifying data in Event Fired rules and messaging plugins

Be very careful about modifying entity data in Event Fired rules and messaging plugin implementations. Use these rules to perform only the minimal data changes necessary for integration code. Entity changes in these code locations do not cause the application to run or re-run validation or preupdate rules. Therefore, do not change fields that might require those rules to run. Instead, change only the fields that are not modifiable from the user interface. For example, set custom data model extension flags used only by messaging code.

ClaimCenter does not support the following:

- Adding or deleting business entities from Event Fired rules or messaging plugins, even indirectly through other APIs.
- Calling, even in the Event Message rule set, any message acknowledgment methods or any skipping methods, such as reportAck, reportError, or skip. Use those methods only in messaging plugins.
- Creating messages outside the Event Message rule set.

Event Fired rule example

In the following example, the rule creates a CashReceiptDocument whenever a user creates a new Recovery transaction in ClaimCenter.

```
USES:
uses java.util.HashMap

CONDITION (messageContext : entity.MessageContext):
```

```

return messageContext.Root typeis Recovery and (messageContext.Root).Status == "submitting"

ACTION (messageContext : entity.MessageContext, actions : gw.Rules.Action):
//This code uses custom fields that must be added to the Recovery and Document entities
var rec = messageContext.Root as Recovery
var values = new HashMap()
var template : gw.plugin.document.IDocumentTemplateDescriptor

values["InsuredName"] = rec.Claim.Insured
values["ReceivedFrom"] = rec.Payer
values["DateReceived"] = rec.CreateTime
values["AmtReceived"] = rec.Amount
values["PmntNo"] = null
values["CheckDate"] = rec.TransactionDate
values["CompletedBy"] = rec.UpdateUser
values["ClaimNo"] = rec.Claim.ClaimNumber
values["PolicyNo"] = rec.Claim.Policy.PolicyNumber
values["CostCat"] = rec.CostCategory
values["RecoveryCat"] = rec.RecoveryCategory
values["ReceiptNo"] = rec.CashReceiptNumberExt //Custom field

var doc = new Document(rec.Claim)
doc.MimeType = "application/pdf"
doc.Claim = rec.Claim
doc.Name = "CashReceivedTicket"
doc.Exposure = rec.Exposure
doc.TypeExt = "Recovery" //Custom field
doc.SubTypeExt = "Other" //Custom field
doc.PrivilegedExt = "No" //Custom field
doc.ProcessMethodExt = "Sent_to_File" //Custom field
doc.Claimant = rec.Claim.claimant
doc.Status = "approved"
doc.DocUID = "ID-" + java.util.Calendar.getInstance().getTimeInMillis()

gw.document.DocumentProduction.createDocumentSynchronously(template, values, doc)

```

ISO Event message rules

ClaimCenter uses event rules to determine if changes to an object require that information be sent to ISO.

To work with these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > EventMessage > EventFired > EFR01000 - ISO

In the base configuration, ClaimCenter provides the following event message rules for ISO integration.

EFR01000 - ISO

This rule determines if the message context destination ID is 66. If so, the destination is ISO and the child rules can run.

EFR01100 - Exposure

Determines if the object that fired the message event is Exposure. If so, the child rule can run.

EFR01110 - Exposure Changed

If the message event indicates that the exposure is new, changed, or has been marked valid, call a method that sends a message to ISO. For details, see the method `checkForExposureChanges` in the file `ISO.gs`.

EFR01200 - Claim

Determines if the object that fired the message event is Claim. If so, the child rule can run.

EFR01110 - Claim Changed

If the message event indicates that the claim is changed or has been marked valid, call a method that sends a message to ISO. For details, see the method `checkForClaimChanges` in the file `ISO.gs`.

EFR01300 - Claim Contact Role

Determines if the object that fired the message event is ClaimContactRole. If so, the child rule can run.

EFR01310 - Claim Contact Role Changed

If the message event indicates that the claim contact role is new, changed, or removed, call a method that sends a message to ISO. For details, see the method `checkForClaimChanges` in the file `ISO.gs`.

EFR01400 - Policy

Determines if the object that fired the message event is Policy. If so, the child rule can run.

EFR01410 - Policy Changed

If the message event indicates that the policy has changed, call a method that sends a message to ISO. For details, see the method `checkForClaimChanges` in the file `ISO.gs`.

MetroReport Event message rules

ClaimCenter uses event rules to check the status of a metropolitan police report request.

- If the status is `InsufficientData`, ClaimCenter does not send the report request to the Metropolitan Reporting Bureau. Instead, it creates an activity as described in “Claim Preupdate rule for metropolitan police report requests” on page 89.
- If the status is `Sending Order`, ClaimCenter fires an event and sends a message to the Metropolitan Reporting Bureau requesting a report.

To work with these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > EventMessage > EventFired > EFR02000 - Metro

In the base configuration, ClaimCenter provides the following event message rules for metropolitan police report integration.

EFR02000 - Metro

Basically a folder that contains the Metro Report rules. Performs a check to verify the Metropolitan Reporting Bureau destination (`DestID 67`) before proceeding.

EFR02100 - Metro Report

Filter for the following two child messages. Checks for changes to the status of the `MetroReport`.

EFR02110 - Send Order Message

ClaimCenter sends out the initial report request to Metropolitan Reporting Bureau with all the claim information and the type of the report it requests.

EFR02120 - Send Inquiry Message

ClaimCenter sends an inquiry on a scheduled basis to Metropolitan Reporting Bureau for each outstanding report requested. The interval for sending the inquiry is set to hourly in the base configuration.

System console logger Event message rule

ClaimCenter uses the event rules to determine whether to send a message to the system console message logger.

To work with this rule, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > EventMessage > EventFired > EFR04000 - Console

In the base configuration, ClaimCenter provides the following event message rule for the system console message logger.

EFR04000 - Console

This rule determines if the message context destination ID is 68, indicating that the system console message logger is the destination. If so, the rule creates a message that says that this event, `messageContext.EventName`, was fired.

Contact Auto Synchronization Failure Event message rule

ClaimCenter uses an event rule to send a message indicating that ClaimCenter failed to synchronize a contact with the contact management system.

To work with this rule, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > EventMessage > EventFired > EFR05000 - Contact Auto Sync Failure

In the base configuration, ClaimCenter provides the following event message rule for a contact auto sync failure.

EFR05000 - Contact Auto Sync Failure

This rule determines if the message context destination ID is 80, the Contact Auto Sync Failure destination. If so, the rule creates a message indicating the AddressBookUID of the contact that failed to synchronize. The logic to detect events is defined in the messaging destination, `messaging-config.xml`. The event this destination listens for is `ContactAutoSyncFailed`.

Policy System Notification Event message rules

ClaimCenter uses an event rule to send a message indicating that ClaimCenter must synchronize with the policy system. In the base configuration, Guidewire disables these rules.

To work with this rule, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > EventMessage > EventFired > EFR06000 - Policy System Notification

In the base configuration, ClaimCenter provides the following event message rules for a sending notifications to the policy system.

EFR06000 - Policy System Notification

This rule and its child rules are disabled in the base configuration. If enabled, this rule determines if the message context destination ID is 69, indicating that ClaimCenter needs to send a message to the policy system. If so, the child rules can execute.

EFR06100 - Claim

This rule and its child rules are disabled in the base configuration. If enabled, this rule determines if the object that fired the message is a `Claim`. If so, the child rules can run.

EFR06110 - Claim Resync

Guidewire disables this rule in the base configuration. If enabled and the message event name is `ClaimResync`, this rule sends a message that resynchronizes the claim with the policy.

EFR06120 - Notification

Guidewire disables this rule in the base configuration. If enabled, this rule determines if there is a policy system notification event registered. If so, the rule retrieves the message event and creates a new message that uses the message context. In the base configuration, there is one such event defined, an event for Claim Exceeds Large Loss.

Asynchronous Document Storage Event message rule

ClaimCenter uses an event rule to send a message indicating that ClaimCenter must store a document asynchronously, at some point in the future.

To work with this rule, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > EventMessage > EventFired > EFR07000 - Async Document Storage

In the base configuration, ClaimCenter provides the following event message rule for storing a document asynchronously.

EFR07000 - Async Document Storage

This rule determines if the message context destination ID is 324, a document storage message, and that the event involves storing a document (`DocumentStore`). If so, the rule sends a message that identifies the document that ClaimCenter is to process asynchronously.

Example asynchronous document storage failed Event Fired rule

Guidewire recommends that, in working with asynchronous document storage, that you create an Event Fired rule to handle the failure of the database to store a document properly. If a document fails to store properly, ClaimCenter creates a `FailedDocumentStore` event on `Document`.

To be useful, you need to add the `FailedDocumentStore` event to a messaging destination in `messaging-config.xml`. For example, you can add this event to message destination 324, which already tracks the `DocumentStore` event.

You then need to create an event rule to process the event as you see fit. You can, for example:

- Email a notification message to the creator of the Document entity.
- Create an activity on the primary or root object.
- Attempt to correct the document issues and create a new DocumentStore message.

The following example rule illustrates a possible Event Fired rule to handle the failure of the database to store a document.

```
CONDITION (messageContext : entity.MessageContext):
return messageContext.DestID == 324 and messageContext.EventName == "FailedDocumentStore"

ACTION (messageContext : entity.MessageContext, actions : gw.Rules.Action):
var document = messageContext.Root as Document
var fixedIssue = false
var sendEmail = false

if (fixedIssue) {
    messageContext.createMessage( "DocumentStore" )
}
else if (document.CreateUser.Contact.EmailAddress1 != null && sendEmail) {
    gw.api.email.EmailUtil.sendEmailWithBody(document,
    document.CreateUser.Contact.EmailAddress1,
    document.CreateUser.Contact.getDisplayName(),
    null, null, "Failed Document Store",
    //CREATE MESSAGE BODY HERE )
}
```

Contact Event message rules

ClaimCenter uses an event rule to send a message indicating that ClaimCenter must synchronize with the contact management system. In the base configuration, Guidewire disables these rules.

To work with this rule, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > EventMessage > EventFired > EFR08000 - Contact

In the base configuration, ClaimCenter provides the following event message rules for handling contact messages with the contact management system.

EFR08000 - Contact

This rule determines if the message context destination ID is 99, indicating that ClaimCenter needs to send a message to the contact management system. If so, the child rules can execute. These messages are used if ClaimCenter is integrated with ContactManager.

EFR08100 - Linked

This rule determines if a contact fired the message, and, if so, if all these additional conditions are true:

- The contact is stored in the contact management system—has an AddressBookUID.
- Automatic synchronizing of contacts is enabled.
- The contact is not an internal user, represented by the UserContact entity.
- The claim is not in draft save mode, indicated by the state of the New Claim Wizard.

If so, the child rules can execute.

EFR08110 - Update or add

Guidewire disables this rule in the base configuration. It is deprecated because it is for use only with ContactManager 7.0, and that version of ContactManager cannot be integrated with ClaimCenter 8.0.

EFR08115 - Tags changed

Guidewire disables this rule in the base configuration. It is deprecated because it is for use only with ContactManager 7.0, and that version of ContactManager cannot be integrated with ClaimCenter 8.0.

EFR08120 - UpdateInAB

This rule determines if ClaimCenter has updated a contact synchronously in ContactManager. If so, the rule sends a message that causes ClaimCenter to update all copies of the contact.

Whenever ClaimCenter updates a contact in ContactManager, ClaimCenter does not get a notification back from ContactManager that the contact was updated. This behavior prevents infinite looping of update messages between the applications.

However, ClaimCenter can have multiple local instances of a given linked contact. Whenever ClaimCenter updates one contact instance synchronously in ContactManager, ClaimCenter triggers the `ContactUpdatedInAB` event for that contact update. That event enables ClaimCenter to create an `AutoSyncWorkItem` to update all the other local instances of the contact.

This rule is not used in the base configuration because contact updates now go through an asynchronous API. However, the rule remains enabled in ClaimCenter to support any custom code that might continue to use synchronous update.

EFR08160 - Update

This rule determines if the contact has been updated or changed, if ContactManager is integrated, and if the contact has changes that need to be synchronized with ContactManager. If so, and the change did not come from ContactManager, the rule sends the changes to ContactManager.

EFR08200 - Not Linked

This rule determines if a contact fired the message, and, if so, if all these additional conditions are true:

- The contact is not stored in the contact management system—does not have an `AddressBookUID`—and therefore is not linked.
- The contact is not an internal user, represented by the `UserContact` entity.
- The claim is not in draft save mode, indicated by the state of the New Claim Wizard.

If so, the child rule can execute.

EFR08210 - Update Or Add

This rule determines if the contact has been updated or changed and if ContactManager is integrated. Because the contact is not linked, there is no need to determine if automatic contact synchronization is on.

If these conditions are true and the change did not come from ContactManager, the rule sends the changes to ContactManager.

EFR08300 - ClaimContact

This rule determines if a `ClaimContact` object fired the message, and, if so, if the event indicates that the claim contact was removed from a claim. If so, the child rule can execute.

EFR08310 - Removed

This rule first determines the following:

- That the claim contact is stored in the contact management system—has an `AddressBookUID`.
- That automatic synchronization is enabled for this contact.
- That the contact is not an internal user, represented by the `UserContact` entity.

If these conditions are true, and the change did not come from ContactManager, and the contact is on only one claim, the rule sends the change to ContactManager.

Instant check message rules

The ClaimCenter instant disbursements feature integrates with a payment gateway. This integration sends checks from ClaimCenter to a payment gateway, where they are processed as payouts. The `Check` entity in ClaimCenter is used to represent the payout when a check has a payment method of *instant*. ClaimCenter uses event rules to send a message indicating that ClaimCenter is requesting an instant payout from the payment gateway. The process of moving an instant check from `Requesting` to `Requested` is handled by the Instant Check Transport message queue.

To work with these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > EventMessage > EventFired > EFR09000 - Instant Check Transport

In the base configuration, ClaimCenter provides the following event message rules for the instant payment gateway integration:

EFR09000 - Instant Check Transport

This rule determines if the instant payment integration feature is enabled and the message context destination ID is 71. If so, the destination is the Instant Check Transport message queue and the child rule can run.

EFR09100 - Check Added Or Changed

Child rule that determines if the object that fired the message event is a Check being added or updated. If so, the `shouldMakeOutboundPayment()` method in the `InstantCheckUtil.gs` class is called to determine if a Message entity must be created and the rule checks that the Status field on Check has changed. In the `InstantCheckTransportPlugin.gs` (Destination ID: 71) plugin class, the Message entity is processed, and an API call to the payment gateway provider initiates the payout.

Payment and recovery on account message rules

The ClaimCenter payment on account and recovery on account features integrate with BillingCenter, a billing management system.

This integration sends checks and recoveries from ClaimCenter to BillingCenter, where they are processed as billing instructions.

To work with these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > EventMessage > EventFired > BillingSystem

In the base configuration, ClaimCenter provides the following event message rules for the payment on account and recovery on account:

BillingSystem

This rule determines if the message context destination ID is 63. If so, the destination is the Billing Message Transport message queue and the child rules can run.

ClaimPayment

Child rule that uses methods in a Gosu utility class (`BillingMessageUtil`) to determine if the object that fired the message event is a Check being added or updated. In addition, the utility class checks if the payment on account feature is enabled in `config.properties` or Guidewire Settings Vault. If so, a Message entity is created and processed, and check data is sent to BillingCenter to be processed as a billing instruction.

ClaimRecovery

Child rule that uses methods in a Gosu utility class (`BillingMessageUtil`) to determine if the object that fired the message event is a Recovery being added or updated (and not an offset recovery). In addition, the utility class checks if the payment on account feature is enabled in `config.properties` or Guidewire Settings Vault. If so, a Message entity is created and processed, and recovery data is sent to BillingCenter to be processed as a billing instruction.

Solr Destination Event message rules

ClaimCenter uses Event Fired rules to update the Solr index that supports free text search for contacts on claims. These rules run whenever a change to contacts can be caused by the following events:

- A claim is added, changed, removed, or purged.
- A claim contact is added to, changed in, or removed from a claim.
- A claim role is added to, changed in, or removed from a claim.
- Any claim information is removed or purged.

To work with these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > EventMessage > EventFired > SolrDestFilter

In the base configuration, ClaimCenter provides the following event message rules for communicating contact changes to the Solr free text search system.

SolrDestFilter

This rule determines if both the following Solr message requirements are true:

- The message context destination ID is the same as the one being used by the Solr system.
- Free text search is enabled.

If both conditions are true, the child rules can execute.

Claim

This rule determines if a `Claim` object fired the message. If so, its child rules can run.

ClaimAdded

This rule determines if the event that fired the message was that a claim was added. If so, the rule sends a message to Solr to handle the claim added event.

ClaimChanged

This rule determines if the event that fired the message was that a claim was changed. If so, the rule sends a message to Solr to handle the claim changed event.

ClaimRemoved

This rule determines if the event that fired the message was that a claim was removed. If so, the rule sends a message to Solr to handle the claim removed event.

ClaimPurged

This rule determines if the event that fired the message was that a claim was purged. If so, the rule sends a message to Solr to handle the claim purged event.

ClaimContact

This rule determines if a `ClaimContact` object fired the message. If so, its child rules can run.

ClaimContactAdded

This rule determines if the event that fired the message was that a claim contact was added. If so, the rule sends a message to Solr to handle the claim contact added event.

ClaimContactContactChanged

This rule determines if the event that fired the message was that a claim contact was changed. If so, the rule sends a message to Solr to handle the claim contact changed event.

ClaimContactRemoved

This rule determines if the event that fired the message was that a claim contact was removed. If so, the rule sends a message to Solr to handle the claim contact removed event.

ClaimContactRole

This rule determines if a `ClaimContactRole` object fired the message. If so, its child rules can run.

ClaimContactRoleAdded

This rule determines if the event that fired the message was that a claim contact role was added. If so, the rule sends a message to Solr to handle the claim contact added event.

ClaimContactRoleChanged

This rule determines if the event that fired the message was that a claim contact role was changed. If so, the rule sends a message to Solr to handle the claim contact role changed event.

ClaimContactRoleRemoved

This rule determines if the event that fired the message was that a claim contact role was removed. If so, the rule sends a message to Solr to handle the claim contact role removed event.

ClaimInfo

This rule determines if a `ClaimInfo` object fired the message. If so, its child rules can run.

ClaimInfoChanged

This rule determines if the event that fired the message was that claim information was changed. If so, the rule sends a message to Solr to handle the claim information changed event.

The `ClaimInfo` entity changes only if `ClaimCenter` archives the claim or restores the claim from archive. In either case, it is important to update the Solr search index to either remove or restore the claim.

ClaimInfoPurged

This rule determines if the event that fired the message was that claim information was purged. If so, the rule sends a message to Solr to handle the claim information purged event.

Exception

Use the Exception rule sets to specify an action to take if an activity or claim is overdue and is in the escalation state. Additionally, there are rule sets to handle user exceptions and group exceptions. Neither of these rule sets contain sample rules in the base configuration. You can use Gosu rule sets in the Exception category to perform action such as sending an email, creating an activity, or setting a flag.

`ClaimCenter` runs the exception rule sets at regularly scheduled intervals.

Activity Escalation rules

`ClaimCenter` runs scheduled process `activityesc` every 30 minutes (by default). This batch process runs against all open activities in `ClaimCenter` to find activities to escalate, using the following criteria:

- The activity has an escalation date that is non-null.
- The escalation date is in the past.
- `ClaimCenter` has not yet escalated the activity.

`ClaimCenter` marks each activity that meets the criteria as escalated.

After the batch process runs, `ClaimCenter` runs the escalation rules for all the activities that have hit their escalation date. Use this rule set to specify what actions to take if the activity becomes escalated. For example, you can use this rule set to reassign escalated activities.

An activity has two dates associated with it.

Due date

The target date to complete this activity. If the activity is still open after this date, it becomes overdue.

Escalation date

The date at which an open and overdue activity becomes escalated and needs urgent attention.

Note: `ClaimCenter` does not run Activity Escalation rules on activities or users who are external to `ClaimCenter`. For example, `ClaimCenter` ignores any activity assigned to a third-party claims adjuster who is external to `ClaimCenter` and flagged as such in `ClaimCenter`.

The simplest approach to escalated activities is to add an activity on the claim for the appropriate supervisor to determine why the work is not yet complete. You can mark the claim as a flagged claim as well. You can also decide that there is no follow-up action needed for certain kinds of activities.

To work with the Activity Escalation rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Exception > ActivityEscalationRules

In the base configuration, this rule set contains the following rules.

AER01000 - High priority activities

If the activity has a high priority, set a flag on the claim for an overdue high priority activity and indicate the subject of this activity.

AER02000 - Urgent priority activities

If the activity has an urgent priority, set a flag on the claim for an overdue urgent priority activity and indicate the subject of this activity.

Claim Exception rules

ClaimCenter monitors claims for unusual conditions or conditions of potential concern to make it possible to alert adjusters or supervisors. This process has multiple parts:

- Determining the list of claims to evaluate
- Running claim exception rules for each claim on the list to find exceptions and take appropriate actions

For ClaimCenter to evaluate a claim, the claim must meet one of the following criteria.

Significant change

The claim must have changed in some significant way since the last time ClaimCenter ran the exception checking process. Significant changes include editing the claim basics, adding or editing an exposure, and adding or changing a financial transaction. For example, ClaimCenter does not consider adding notes or documents, a significant change because the exception rules do not typically evaluate the content of a new note or document.

Idle for a length of time

The claim is idle, aging without any significant change, for a certain length of time. Several configuration parameters, defined in file `config.xml`, impact how ClaimCenter determines a claim is idle:

- Configuration parameter `IdleClaimThresholdDays` defines the number of days after which ClaimCenter identifies a non-modified claim as idle. In the base configuration, this parameter is set to seven days.
- Configuration parameter `IdleClosedClaimThresholdDays` defines the number of days after which ClaimCenter identifies a closed claim as idle. In the base configuration, this parameter is set to seven days.

To work with these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Exception > ClaimExceptionRules

In the base configuration, this rule set contains the following rules.

CER01000 - Setting SIU Life Cycle State

Advances the claim life cycle stage based on the number of days that have passed since claim inception.

CER02000 - At least one activity for claim owner

This Gosu rule is disabled in the base configuration because it is handled in the Business Rules editor.

CER03000 - At least one activity for exposure owner

This Gosu rule is disabled in the base configuration because it is handled in the Business Rules editor.

CER04000 - Recalculate Claim Metrics

This rule verifies that all claim metrics, exposure metrics, and claim indicators are created on every claim. Any missing metrics or indicators are added to the claim.

Guidewire disables this rule in the base configuration. Enable this rule only if a new metric or indicator that has been added to the system is to be added to all claims.

CER05000 - BizRules

This rule executes the business rules that are set to trigger on a claim exception. This rule runs after the completion of the other Gosu Claim Exception rules.

Checking for recently modified claims

Guidewire defines *recently modified* as any claim for which the update time is newer than the last time the exception rules ran on the claim. The check for recently modified claims examines the following:

- The update time on the claim
- The update time on any exposure on the claim
- The update time on any transaction on the claim
- The update time on any claim contact or claim contact role on the claim

- The update time on any matter on the claim

About exception batch processing

The following list describes the batch processes that work with changed and idle claims.

Batch process	Description	Default time
claimexception	The Claim Exception batch process runs the claim exception rules on claims that have been modified significantly since the last time that ClaimCenter ran this process. The claims processed are: • Claims that have been updated since the last time the batch process processed it • Claims with any of the following updates: ◦ exposure ◦ transaction ◦ claim contact ◦ claim contact role ◦ matter • Claims that have never been processed by the batch process For all these cases, the Claim Exception batch process creates a <code>ClaimException</code> row in the database for the claim if none currently exists. If the row exists, the batch process updates the <code>ExCheckTime</code> field of the <code>ClaimException</code> to the current time.	Every day at 2:00 a.m.
idleclaimexception	The Idle Claim Exception batch process runs the claim exception rules on claims that are idle. A claim is idle if the exception rules have not been run on it in the number of days specified by the <code>IdleClaimThresholdDays</code> configuration parameter. In the base configuration, the value of this configuration parameter is 7 days. More specifically, the Idle Claim Exception batch process evaluates all claims that have a foreign key to an existing <code>ClaimException</code> row in the database. If the <code>ExCheckTime</code> field of the <code>ClaimException</code> row is older than Today minus <code>IdleClaimThresholdDays</code>, the batch process runs the claim exception rules on that claim.	Every Sunday at 12:00 noon.

To run separate schedules for changed claims and idle aging claims, set configuration parameter `SeparateIdleClaimExceptionMonitor` to true in file `config.xml`.

Configuration parameters that affect changed and idle claims

The following list describes the configuration parameters that specifically affect changed and idle claims.

Configuration parameter	Description	Default value
<code>IdleClaimThresholdDays</code>	Number of days without significant change after which ClaimCenter considers a claim idle.	7
<code>IdleClosedClaimThresholdDays</code>	Number of days without significant change after which ClaimCenter considers a closed claim idle.	7
<code>SeparateIdleClaimExceptionMonitor</code>	If true, ClaimCenter runs separately scheduled processes for determining changed claims (<code>claimexception</code>) and idle aging claims (<code>idleclaimexception</code>). If false, ClaimCenter runs the <code>idleclaimexception</code> process also as it runs the <code>claimexception</code> process.	true

Claim exceptions and Gosu rules

There are many kinds of exceptions for which you can write rules:

- Work that has not been completed within a certain number of days, but for which there is no activity-based escalation

For example, you can look for auto claims that are more than 30 days since being reported but still do not have a fault rating set.

- Unusual reserve changes or values

For example, you can look for claims with three or more reserve increases.

- Items of interest

You can decide to flag claims that have important new information. For example, if a claim notes the existence of a plaintiff attorney, you can flag the claim for potential litigation automatically.

Flagging a claim as an exception

IMPORTANT: Avoid writing exception rules that cause ClaimCenter to report the same exceptions repeatedly.

One action to take in regard to escalated claims is to flag a claim as an exception. The following list describes Claim fields and one method that you can use with flagged claims. ClaimCenter displays the three fields in the **New Loss** screen for the claim.

Property or method	Description
Flagged	Sets the flagged status of a claim to a typecode defined in the FlaggedType typelist: • isflagged • neverflagged • wasflagged Used to determine a status of a claim.
FlaggedDate	Indicates the date on which a flag was set.
FlaggedReason	A text field that provides a reason the flag was set. Helps the adjuster understand the nature of the exception noticed.
clearFlag	A method that toggles the Flagged field and clears the FlaggedReason field. Use this method after the claim is no longer considered to have any exceptions.

You can use the `setFlag` method in a rule to set the value of the flagged reason. For example:

```
claim.setFlag( "Claim reported within 30 days of opening policy." + " Claim referred to SIU for investigation." )
```

In actual practice, Guidewire recommends that you make String values into display keys or constant variables, as appropriate.

Group Exception rules

ClaimCenter runs the Group Exception rules on a scheduled basis to look for certain conditions on groups that might require further attention. You can use these rules to define the follow-up actions for each exception found. This rule set is empty in the base configuration.

To work with this rule set, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Exception > GroupExceptionRules

In the base configuration, Guidewire schedules the Group Exception work queue to run the group exception rules on all groups in ClaimCenter every day at 4:00 a.m. However, Guidewire disables the Group Exception work queue by

default. You must enable the work queue by setting the number of workers for this work queue to a number greater than zero in file `work-queue.xml`.

User Exception rules

ClaimCenter runs the User Exception rules on a scheduled basis to look for certain conditions on users that might require further attention. You can use these rules to define the follow-up actions for each exception found. This rule set is empty in the base configuration.

To work with this rule set, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Exception > UserExceptionRules

In the base configuration, Guidewire schedules the User Exception work queue to run the user exception rule sets on all users in ClaimCenter every day at 3:00 a.m. However, Guidewire disables the User Exception work queue by default. You must enable the work queue by setting the number of workers for this work queue to a number greater than zero in file `work-queue.xml`.

Initial Reserve

ClaimCenter uses Initial Reserve rules to create initial financial reserves on new exposures. The Initial Reserve rule category, which executes immediately after the Postsetup rules, automates creation of reserves for new exposures.

Note: If your enterprise does not automate creation of reserves, you probably do not need to use this rule category.

ClaimCenter uses these rules to determine the initial financial reserves to create for a new exposure. Each segment type has specific reserve requirements. For example, the reserve amount you establish for a moderate vehicle damage exposure is likely to be less than the reserve you create for a serious bodily injury exposure.

The rules use the `createInitialReserve` method to automate the creation of the reserve.

Note: Reserves created by using these rules are not submitted for approval. Authority limit validation is skipped, and they go straight into **Submitting** status.

ClaimCenter does not associate a `TransactionSet` with any reserves added through these rules until it processes the new reserves. The most common approach for creating initial reserves is to write a rule or rule set for each exposure segment type.

In the base configuration, ClaimCenter provides Initial Reserve rules for Auto Vehicle Damage, Travel Baggage Delay or Loss, Trip Cancellation, and Travel Medical Payments. To work with these Gosu rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > InitialReserve

Initial Reserve rules for auto vehicle damage

The Initial Reserve (Auto) rules create an initial reserve for an Auto loss type. To work with these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > InitialReserve > InitialReserve > Auto

In the base configuration, this rule set contains the following rules.

IRR01000 - Auto

This rule determines if the loss type is Auto. If so, the child rules execute.

IRR01100 - Vehicle damage

This rule determines if the exposure is Vehicle Damage. If so, the child rules execute.

IRR01110 - Minor

This rule determines if the exposure segment is Auto Low, if the claim is not open, and if there is not exactly one exposure and one transaction. If so, and if there are no financial holds, the rule creates an initial reserve for an

exposure. The call to `createInitialReserve` specifies a cost type of `claim_cost`, a cost category of `body`, and the value of the script parameter `InitialReserve_AutoMinorVehicleDamageBody`.

IRR01120 - Medium

This rule determines if the exposure segment is Auto Medium, if the claim is not open, and if there is not exactly one exposure and one transaction. If so, and if there are no financial holds, the rule creates an initial reserve for an exposure. The call to `createInitialReserve` specifies a cost type of `Claim Cost`, a cost category of `Body`, and the value of the script parameter `InitialReserve_AutoMediumVehicleDamageBody`.

Additionally, regardless of the status of financial holds, the rule creates an initial reserve with a cost type of `Adjusting and Other Expenses` to cover inspection of the auto damage. This initial reserve value is specified in the script parameter `InitialReserve_AutoMediumVehicleDamageInspection`.

IRR01130 - High complexity

This rule determines if the exposure segment is Auto High, if the claim is not open, and if there is not exactly one exposure and one transaction. If so, if and there are no financial holds, the rule creates an initial reserve for an exposure. The call to `createInitialReserve` specifies a cost type of `Claim Cost`, a cost category of `Body`, and the value of the script parameter `InitialReserve_AutoMajorVehicleDamageBody`.

Additionally, regardless of the status of financial holds, the rule creates an initial reserve with a cost type of `Adjusting and Other Expenses` to cover inspection of the auto damage. This initial reserve value is specified in the script parameter `InitialReserve_AutoMediumVehicleDamageInspection`.

Initial Reserve rules for travel losses

The Initial Reserve (Travel) rules create an initial reserve for a Travel loss type. To work with these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > InitialReserve > InitialReserve > Travel

In the base configuration, this rule set contains the following rules.

IRR02000 - Travel

This rule determines if the loss type is Travel. If so, the child rules execute.

IRR02100 - Baggage

This rule determines if the exposure type is Baggage. If so, the child rules execute.

IRR02110 - Baggage Delay

This rule determines if the incident for this Baggage exposure is Delay Only. If so, and, if there are no financial holds:

- If baggage is delayed for more than 4 hours and less than a day, the rule:
 - Sets the initial reserve to the value of the `InitialReserve_TravelBaggageLoss` script parameter if this value is lower than the exposure limit.
 - Otherwise, sets the initial reserve to the exposure limit.
- If baggage is delayed for more than a day, the rule multiplies the value of the `InitialReserve_TravelBaggageLoss` script parameter by the number of days lost.
 - If this product is lower than the exposure limit. the rule sets the initial reserve to this product.
 - Otherwise, the rules set the initial reserve to the exposure limit.

Note: The rule hardcodes the values of 4 and 24 for hours delayed.

IRR02120 - Baggage Loss

This rule determines if the incident for this Baggage exposure is not Delay Only, which means that the baggage was lost. If so, and if there are no financial holds:

- If there is an exposure limit, the rule creates an initial reserve for that amount.

- If there is no exposure limit, create an initial reserve for the approved content total if the value is greater than zero.

IRR02200 - Trip Cancellation

This rule determines if the exposure type is Trip Cancellation or Delay. If so, and if there are no financial holds:

- The rule sets the initial reserve to the total financial impact if that value is available and is lower than the exposure limit.
- Otherwise, the rule sets the initial reserve to the exposure limit.

IRR02300 - Medical payments

This rule determines if the exposure type for this Travel loss type is Medical Payments. If so, and if there are no financial holds, the rule sets the initial reserve to the exposure limit.

Loaded

Rules in the Loaded rule set category execute only in concert with the FNOL (First Notice of Loss) import interface and relate only to imported claims. To add rules to this category, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Loaded

Note: Place rules that apply to claims that ClaimCenter creates, or, to claims that must execute before Segmentation in the Presetup rule set category.

Claim Loaded rules

Claim Loaded rules execute whenever an FNOL import loads a new claim. ClaimCenter executes Loaded rules after it finishes importing, or *loading*, the claim from an external FNOL source. Some uses of this rule category are:

- To make any changes necessary to convert initial claim data from an external format into a suitable structure for ClaimCenter, such as:
 - Sanity checks
 - Data aggregations and transformations
 - Data and naming reconciliations
- To record the import of claims
- To assign review of the new FNOL to a user prior to running the full automated setup process

A typical application of the Loaded rules is to ensure that ClaimCenter makes a record of claims that it imports from an external FNOL application. Making a record typically consists of logging the transaction. The following rule illustrates this application.

```
USES:
uses gw.api.system.CCLoggerCategory

CONDITION (claim : entity.Claim):
return Claim.State != null and Claim.State != TC_DRAFT

ACTION (claim : entity.Claim, actions : gw.rules.Action):
CCLoggerCategory.IMPORT.debug("In Claim No. : "+ claim.ClaimNumber + " - In " + actions.getRuleSet()
+ " - In Rule " + actions.getRule() )
actions.exit()
```

Note: In actual practice, Guidewire recommends that you make String values into display keys or constant variables, as appropriate.

Exposure Loaded rules

ClaimCenter does not execute the Exposure Loaded rules. Guidewire intends these rule to trigger after an FNOL import into ClaimCenter for the Claim entity only. If you add rules to this rule set, you must trigger the execution of the rules manually.

In the following example, the rule generates a logging message if the import loads an exposure linked to an existing claim. As with any Exposure Loaded rules, you must manually trigger this rule.

```
USES:
uses gw.api.system.CCLoggerCategory

CONDITION (exposure : entity.Exposure):
return (exposure.State == null or exposure.State == TC_DRAFT) and exposure.Claim.State == TC_OPEN

ACTION (exposure : entity.Exposure, actions : gw.rules.Action);
CCLoggerCategory.IMPORT.debug( " " )
CCLoggerCategory.IMPORT.debug( "Loaded Exposure Loaded default rules " + actions.getRule() )
```

Note: In actual practice, Guidewire recommends that you make String values into display keys or constant variables, as appropriate.

Postsetup

Postsetup rules run automatically whenever ClaimCenter adds a new claim, exposure, activity, matter, or transaction. ClaimCenter runs these rules after the Assignment and Workplan rules complete their execution. The Postsetup rules run immediately before the rule sets in the Initial Reserve rule set category and the Preupdate rule set category.

Postsetup rules can perform a wide variety of actions, depending on your business requirements. Postsetup rules often depend on the results of the rules executed during setup or other tasks. You can design these rules to trigger based on specific conditions that the setup rules create.

Note: ClaimCenter runs the Postsetup rules as part of the “save and setup” operation on claims and exposures.

Activity Postsetup rules

The Activity Postsetup rules fire just prior to completing the setup process and saving any changes to activities. They run after the assignment rules have run. To work with these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Postsetup > ActivityPostsetup

In the base configuration, this rule set contains the following sample rules.

APS01000 - Add user in SIU Role to claim if necessary

If the current activity is an SIU escalation, this rule determines whether the claim already has an SIU investigator assigned. If there is no assigned SIU investigator, and the owner of the activity is an SIU investigator, the rule assigns the owner of the activity to the claim.

APS02000 - Subro - Add user to Role on claim if necessary

This rule applies if the current activity is a subrogation check and the claim does not already have a subrogation owner. If so, and there is an assigned user for this activity, assign that user to the claim as the subrogation owner.

APS03000 - Reins - Add user to Role on claim

This rule applies if the current activity is a claim reinsurance review and there is an assigned user for this activity. In that case, assign that user to the claim as the reinsurance manager.

Claim Postsetup rules

The Claim Postsetup rules fire just prior to completing the setup process and saving any changes to claims. To work with these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Postsetup > ClaimPostsetup

In the base configuration, this rule set contains the following sample rule.

CPS01000 - Add Auto-body shop

If the current claim is an Auto claim, attempt to find and add an auto body shop as a vendor service to the claim. Guidewire disables this rule in the base configuration.

Example Claim Postsetup rule

In the following example, the Claim Postsetup rule creates a new activity for FNOL (first notice of loss).

```
CONDITION (claim : entity.Claim):
return claim.isValid(TC_NEWLOSS, true)

ACTION (claim : entity.Claim, actions : gw.rules.Actions):
var strClaimNumber = claim.DuplicateClaimNumbers

if ( strClaimNumber != null ) {
    /* Create new claim activity if there are possibly duplicate claims listed. */
    var description = "Check that claim is not a duplicate of " + strClaimNumber
    claim.createActivity( null, ActivityPattern.finder.getActivityPatternByCode( "fnol" ),
        "Check that duplicate claim does not exist", description, null, null, null, null )
} else {
    /* Create new claim activity */
    claim.createActivityFromPattern( null, ActivityPattern.finder.getActivityPatternByCode( "fnol" ) )
}
```

Note: In actual practice, Guidewire recommends that you make String values into display keys or constant variables, as appropriate.

Exposure Postsetup rules

The Exposure Postsetup rules fire just prior to completing the setup process and saving any changes to exposures, after assignment and workplan rules have run. Guidewire does not provide any sample rules in the base ClaimCenter configuration.

To add rules to this rule set, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Postsetup > ExposurePostsetup

Matter Postsetup rules

The Matter Postsetup rules fire just prior to completing the setup process and saving any changes to matters, after assignment and workplan rules have run. To work in this rule set, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Postsetup > MatterPostsetup

In the base configuration, this rule set contains the following sample rule.

MPS01000 - Set Litigation Status

If a matter is created, set the litigation status of the claim to Litigated.

Example Matter Postsetup rule

In the following example, the Matter Postsetup rule sets up an activity for the claim owner if the matter has a bodily injury exposure in its array of exposures. The activity is to initiate a legal review of a bodily injury claim 10 days before a scheduled mediation meeting with the insured.

```
CONDITION (matter : entity.Matter):
return true

ACTION (matter : entity.Matter, actions : gw.rules.Actions):
var exp = matter.Exposures*.Exposure.firstWhere( \ elt -> elt.ExposureType ==
    ExposureType.TC_BODILYINJURYDAMAGE )

if (exp !=null) {
    var act = matter.Claim.newActivity( ActivityPattern.finder.getActivityPatternByCode( "legal_review" ), exp)
    act.TargetDate = gw.api.util.DateUtil.addDays( act.Matter.MediationDate, -10 )
}
```

```
act.assignToClaimOwner()  
}
```

Transaction Postsetup rules

The Transaction Postsetup rules fire under the following conditions:

- Immediately after the Exposure Closed rule set, after closing an exposure that has a transaction associated with it.
- Immediately after the Transaction Approval Rules rule set except if you are in the process of creating a reserve.

These rules also fire under the following conditions:

- ClaimCenter approves a TransactionSet
- ClaimCenter voids, stops, or escalates a check
- ClaimCenter records a payment
- ClaimCenter voids a recovery
- ClaimCenter closes an exposure

To work on this rule set, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Postsetup > TransactionPostsetup

In the base configuration, this rule set contains the following rules.

TPS01000 - Check Aggregate Limits

If the transaction set is within 20 percent of its limit, check to see if there is already an activity assigned to check for approaching aggregate limits. If not, create a general reminder activity to check on the transaction within 5 days.

TPS02000 - Check Aggregate Deductible

If the aggregate deductibles are within 20 percent of the limit, check to see if there is already an activity assigned to check for approaching aggregate deductible limits. If not, create a general reminder activity to check on the deductibles within 5 days.

Presetup

ClaimCenter runs the Presetup rules automatically whenever ClaimCenter adds a new activity, claim, exposure, matter, or transaction. Presetup rules do not apply to modifications made to existing claims. You use Presetup rules to perform actions on new claims before claim or exposure setup. *Setup* typically encompasses running rules in the Segmentation, Assignment, and Workplan rule set categories. If you need to modify a new claim before beginning setup, the Presetup rule category provides a place for the rules that perform the actions.

To work on Presetup rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Presetup

In the base configuration, ClaimCenter provides a single Presetup rule:

CPR01000 - WCDefaultExposures

About claim and exposure setup

Guidewire uses the term *setup* as a generic term for invoking multiple rules (Presetup, Segmentation, Assignment, Workplan, and Postsetup, for example). The term *setup* also applies to initialization work that ClaimCenter performs whenever it first creates a Claim or Exposure as an open item. ClaimCenter typically performs this kind of setup under the following circumstances:

- At the end of the **New Claim** wizard, ClaimCenter sets up the new claim and all its exposures.
- After creating a new exposure through ClaimCenter, ClaimCenter sets up the new exposure after you click **Update**.

- Whenever you add a new claim through the addFNOL SOAP call, ClaimCenter performs a setup before it commits data to the database.

About adding a new claim using ClaimAPI

ClaimCenter performs the following setup tasks whenever you add a new claim through the ClaimAPI web service method addFNOL:

1. It adds the FNOL claim.
2. It invokes the Loaded rule set category rules on the FNOL claim. See “Loaded” on page 80.
3. It creates an import history event.
4. It performs the sequence of actions classified as *save-and-setup*, setting the claim status to open at the end of the process.
5. It commits the claim. This commit includes all exposures, activities, and other linked objects created during the save-and-setup process.
6. At commit time, it runs the rules in the Preupdate, Validation, and Event Message rule set categories, validating the claim at the loadsave level.

About save and setup processes for an FNOL claim

Guidewire ClaimCenter performs the following steps for a save-and-setup operation on an FNOL claim:

1. It creates a new claim number (if one does not exist).
2. It creates a claim snapshot.
3. It runs the rules in the Presetup rule set category on the claim and each exposure.
4. It runs the rules in the Segmentation rule set category on each exposure, then the claim.
5. It runs the rules in the Assignment rule set category on the claim, then each exposure.
6. It runs the rules in the Workplan rule set category on the claim, then each exposure. This process generates the workplan.
7. It runs the rules in the Postsetup rule set category on the claim and each exposure.
8. It updates history timestamps so that client-added history events can be correctly stamped.
9. It sets the status of the claim and each exposure to open.
10. It ensures that each exposure has a valid claim order field set.
11. It sets initial reserves.

At the end of the setup process for an FNOL claim, ClaimCenter returns the public ID of the newly created claim.

About save and setup processes for exposures

ClaimCenter performs the following save-and-setup pre-processing operations on an exposure after the Exposure.saveAndSetup method creates it:

1. It sets the claim order on the exposure.
2. It runs rules in the Presetup rule set category on the exposure.
3. It runs the segmentation engine on the exposure.
4. It runs the strategy engine on the exposure.
5. It runs the assignment engine on the exposure.
6. It runs the workplan engine on the exposure.
7. It runs rules in the Postsetup rule set category on the exposure.

8. Sets the exposure's status to open.
9. It creates initial reserves.
10. It commits the exposures bundle, which contains the exposure, activities, and other objects creating during the call.

Creating Transaction Set Presetup rules

The Transaction Set Presetup rules execute prior to transaction setup. *Setup* for transactions encompasses submitting a `TransactionSet` for approval and updating the T-accounts. In the base configuration, ClaimCenter does not provide any rules for this rule set.

The Transaction Set Presetup rule set is useful primarily for a limited purpose, making the same modifications to the following transactions:

- Transactions created in the ClaimCenter user interface
- Transactions created in rules or outside rules in Gosu

Whenever you use the rules for this purpose, you can modify transaction values that affect financial calculations. These values include the transaction Amount, the Eroding state, the Exposure, CostType, or CostCategory, or the ScheduledSendDate on a payment's check.

If you create or modify transactions as described in the following cases, you do not need to use Presetup rules on them:

- If you need to modify transactions created only in the ClaimCenter user interface, rather than using rules, you can modify them in the `beforeCommit` attribute of the PCF page. Use the financial APIs for transactions in your Gosu code.
- If you need to modify transactions created only in rules or Gosu, use the financial APIs to create the transactions correctly in the first place instead of modifying them later.

About financial APIs for transactions

In the base configuration, ClaimCenter contains a set of classes and methods that provide extensive control over payments and transaction line items as you set up checks. If used correctly, these APIs also update the T-accounts associated with the payments and transactions.

Guidewire recommends that you use the methods on the following class if you want to create, modify, or remove transactions programmatically:

```
gw.api.financials.CheckCreator
```

See the *ClaimCenter Configuration* documentation for information on this class and its methods.

IMPORTANT: Guidewire does not provide a mechanism for editing previously saved checks or recoveries exclusively with rules or Gosu. The editing of a check must be initiated in the user interface.

About ClaimCenter execution of Transaction Set Presetup rules

ClaimCenter executes the Transaction Set Presetup rule set under the following circumstances:

- If you click the **Finish** at the end of the **Check** wizard
- If you click **Save** or **Update** on the **New Reserves**, **New Recovery Reserves**, or **New Recoveries** screen
- If you create a claim in the **New Claim** wizard in **Auto - First and Final** mode, which creates a check

ClaimCenter also runs the Transaction Set Presetup rules:

- During `CheckCreator.prepareForCommit`.
- During `BulkInvoiceItem` processing of its placeholder check.

ClaimCenter does not execute these Presetup rules if you use the following methods on Claim, Exposure, or ReserveLine to create a transaction:

- `createRecovery`
- `setAvailableReserves`
- `setOpenRecoveryReserves`

Using the Transaction Set Presetup rules

You use the rules in the Transaction Set Presetup rule set to modify transactions and checks before they go through the setup process. Through these rules, you can:

- Add to or remove a payment from a check.
- Add to or remove a line item from a transaction.
- Modify the amount of a line item.
- Add or remove a Recovery, a Reserve, or RecoveryReserve from a TransactionSet of the appropriate subtype.

Note: Reserves and recovery reserves only have one line item.

The Transaction Set Presetup rules provide support for modifying the amounts of single and recurring checks from rules just before the checks are set up. They also provide support for adding new payment line items through the user interface just before setting up the transaction. For example, after a payment is submitted, but before it is set up and then saved, you might want to add a new payment to a check. You might want to add the payment both from the user interface and programmatically to the check while it is in Awaiting Submission status. You might want to add the payment, for example:

- To adjust a future payment
- To add a line item for a certain type of tax computed in rules

Performing your business logic in the Transaction Set Presetup rule set does cause ClaimCenter to update the T-accounts properly.

Guidewire provides the following methods that you can use in this rule set:

- `Transaction.addNewLineItem(Amount, Comments, LineCategory)`
- `RecoveryReserveSet.newRecoveryReserve(Exposure, CostType, CostCategory)`
- `RecoverySet.newRecovery()`
- `ReserveSet.newReserve(Exposure, CostType, CostCategory)`
- `Check.addNewPayment(Exposure, CostType, CostCategory)`

If you want to change a line item, use the standard `getLineItems` method to retrieve the line item. Then, in this rule set, call any of the standard setters on the line item—for example, `lineitem.setAmount`—so that ClaimCenter correctly computes the T-accounts.

IMPORTANT: If you use financial API methods in a Presetup rule, do not call `prepareForCommit` in the rule as ClaimCenter prepares the transactions for commit as part of the setup process. In addition, do not call the constructors of financial transaction entities, such as `new Payment` or `new RecoveryReserve`, in Presetup rules. Instead, use the financial APIs to create these entities outside rules.

About modifying the check scheduled send date

It is possible to modify the check property for the schedule send date, `ScheduledSendDate`, in the Transaction Presetup rules or from Gosu called from the application user interface (PCF) code. The date determines the place to include the check amount. For example, the date might be used:

- In Future Payments (tomorrow or later)

- In Awaiting Submission (today), which is included in Total Payments and similar financial calculations.

If you update the date inappropriately, ClaimCenter throws an exception with the following message:

```
Check contains a payment whose LifeCycleState is inconsistent with the Check's Scheduled Send Date.
```

Workers' Compensation Claim Presetup example

One common application of Presetup rules is to create exposures for a new Workers' Compensation claim. Workers' Compensation claims typically have only three possible exposure assignments:

- An employee is injured on the job and loses some work time. The employee is entitled to reimbursement for both medical costs and lost wages, and the employer's workers' compensation insurance covers the loss.

This type of claim has two exposures, one for Medical Details and another for Time Loss.

- An employee is injured on the job, but misses no work time. The employee is entitled to reimbursement only for medical costs, and the employer's workers' compensation insurance covers the loss.

This type of claim has only a single Medical Details exposure.

- An employee is injured and the employer's insurance does not cover the loss. The claimant is therefore obliged to seek compensation in court.

This type of claim has only a single Employer Liability exposure.

In the base configuration, Guidewire provides a Claim Presetup rule to create the standard workers' compensation exposures that you see in setting up a Workers' Compensation claim in ClaimCenter. To view this rule, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Presetup > ClaimPresetup > CPR01000 - WCDefaultExposures

This rule contains the following code.

```
CONDITION (claim : entity.Claim):
return claim.LossType == TC_WC

ACTION (claim : entity.Claim, actions : gw.rules.Action):
claim.createRelevantWorkCompExposures()
```

The condition determines that the rule applies to any Workers' Compensation claim. The action calls the method `createRelevantWorkCompExposures`, defined in an enhancement to the claim entity in `FNOL.gsx`. The method contains the instructions for evaluating and assigning exposures for workers' compensation claims. To see this method, navigate in the Studio **Project** window to **configuration > gsrc > libraries > FNOL**.

Preupdate

Use the Preupdate rule sets to perform domain logic or validation that must be performed before the entity in question is committed. Only a change to an entity marked as `Validatable` in its definition file can trigger a preupdate rule.

Typically, the Preupdate rules execute before the rules in the Validation rule set category.

The Preupdate rules can perform a wide variety of actions. One application of the preupdate rules is to create an activity based on a condition. For example, if a vehicle covered in a claim has been declared a total loss, a rule can create an activity to inspect the car for its salvage potential.

To access this rule set category, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Preupdate

Prioritized Preupdate handler

The `gw.plugin.preupdate.prioritized` package contains a set of classes and interfaces that simplify working with the preupdate handler plugin. Instead of directly working in Gosu rules, you can instead use Gosu code that calls the `IPreupdateHandler` plugin.

See the ClaimCenter *ClaimCenter Configuration* documentation and *Plugins, Prebuilt Integrations, and SOAP APIs* for more.

How ClaimCenter triggers Preupdate rules

IMPORTANT: In running the Preupdate rule set, ClaimCenter first computes the set of objects on which to run the preupdate rules. It then runs the rules on this set of objects. If a preupdate rule then modifies an entity, the preupdate rules for that entity do not fire unless the schedule for preupdate rules already includes this entity. In other words, changing an object in a preupdate rule does not then cause the preupdate rules to run on that object as well.

The following ClaimCenter root entities can all trigger preupdate rules in the ClaimCenter base configuration:

Activity
Claim
Contact
Exposure
Group
Matter
Policy
RIAgreement
RITransactionSet
Subrogation
TransactionSet
User

Note: Not all these entities have Preupdate rules in the base configuration.

For an entity to trigger a preupdate rule, that entity must implement the `Validatable` delegate. This requirement applies both to entities in the base ClaimCenter configuration and to custom entities that you create.

Entities that implement the `Validatable` delegate trigger Validation rules as well.

ClaimCenter runs the preupdate and validation rules in response to the following events:

- An instance of a validatable entity is created or modified.
- For an entity connected by an array or foreign key to a validatable entity:
 - The validatable entity's array key or foreign key is defined with `triggersValidation="true"`.
 - The entity to which the array or foreign key connects is created or modified.

The same behavior also applies to entities newly created in a preupdate rule. For example, if you create a new activity in a preupdate rule, ClaimCenter does not then run the Activity preupdate rules on that activity.

How ClaimCenter triggers Preupdate rules on custom entities

It is possible to create preupdate rules for custom entities, which are entities that you create yourself and are not part of the base Guidewire ClaimCenter configuration.

For an extension entity to trigger a preupdate rule, all of the following must be true:

1. The entity must be a root entity, not a subtype of an entity.
2. The entity must implement the `Validatable` delegate.

For any extension entity that you create, if you want it to trigger a preupdate rule, the entity must implement the following code:

```
<implementsEntity name="Validatable"/>
```


3. A rule set must exist in the Preupdate rule set category that conforms to the following naming convention:
`<custom_entity_name>Preupdate`

For example, if you create an extension entity named `NewEntityExt`, create a rule set in the Preupdate rule set category to hold your preupdate rules and name it `NewEntityExtPreupdate`

Activity Preupdate rules

Use the Activity Preupdate rules to modify activities and related entities. In the base configuration, whenever you reassign a claim, ClaimCenter reassigns activities that are associated with the claim and belong to the previous owner to the new claim owner. You can write Activity Preupdate rules that provide different behavior.

In the base configuration, there are no rules in this rule set. To access the rule set, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Preupdate > ActivityPreupdate

Claim Preupdate rules

Use the Claim Preupdate rules to modify claims and related entities. To work with these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Preupdate > ClaimPreupdate

Claim Preupdate rules for document creation

There are several Claim Preupdate rules that create documents whenever the claim description field changes. Guidewire does not enable these rules in the base configuration.

CPU01000 - Create Document Synchronously

Sample rule that shows how to create and store a document synchronously. Guidewire disables this rule in the base configuration.

CPU02000 - Create Document Asynchronously

Sample rule that shows how to create and store a document asynchronously. Guidewire disables this rule in the base configuration.

Claim Preupdate rules for Special Investigation Unit (SIU)

The Claim Preupdate SIU rules and the Claim Exception SIU rules work together to assist in determining claim fraud. These rules identify certain characteristics of a claim that increase the suspicion that fraud has occurred and assign points to each of these characteristics.

Claim Preupdate rule for metropolitan police report requests

IMPORTANT: ClaimCenter integrates with Metropolitan Reporting Bureau in the base configuration. However, you must have an account with Metropolitan Reporting Bureau for report requests to work.

The Metropolitan Reporting Bureau at www.metroreporting.com is a nationwide service in the United States for obtaining accident and incident reports. The Claim Preupdate rule set has one rule that works with this service, CPU08000 - Metro Report Request. This rule starts an internal process to determine the type of each report requested by a claim, one at a time. The rule verifies that the claim is in draft state and then calls `libraries.Metro.validateAndStartReportFlows(claim)`. This method verifies that the claim contains the data required to request each of the requested reports.

- If there are any missing fields, ClaimCenter creates an activity called Metropolitan Report Request Failed and assigns it to the user that created the MetroReport request. The **Description** text area of the activity contains information on the type of report that was requested. It also contains any data that must be supplied to successfully make the request. For each MetroReport request for which the required data is not defined on the claim, ClaimCenter marks the status as `insufficientData`.

- If the required data is specified and the claim update completes successfully, ClaimCenter changes the MetroReport status to validated. After the report status becomes valid, ClaimCenter changes the Metropolitan report status to Sending Order.

Claim Preupdate rules for catastrophes

In the base configuration, Guidewire provides the following Claim Preupdate rules related to catastrophe categorization in the Claim Preupdate rule set.

CPU09000 - Related to Catastrophe

This Gosu rule is disabled in the base configuration because it is handled in the Business Rules editor.

CPU13000 - Catastrophe History

Determines if there is a change to the claim **Catastrophe** field. If there is, the rule updates History and marks any open *Review for Catastrophe* activity complete.

CPU19000 - Geocode Catastrophe Claims

Determines if the claim **Catastrophe** field is set (not null) and if the claim's loss location has never been geocoded. If so, the rule schedules the claim's loss location for geocoding. This rule takes effect only if geocoding is enabled.

ClaimCenter uses these rules in conjunction with the catastrophe profile that an administrative user creates through the **Administration > Catastrophes** screen.

Claim Preupdate rules for changed primary diagnosis code

In the base configuration, Guidewire provides the following Claim Preupdate rules to record if a primary diagnosis for an injury incident has changed.

CPU10000 - Primary Diagnosis Code Changed

This rule screens for injury incidents that have injury diagnoses, for the following rules.

CPU10100 - WC

If the loss type is Workers' Comp, this rule determines:

- If the diagnosis change was to set the diagnosis to primary for the first time. In this case, the rule saves the primary diagnosis ICD code only to the history.
- If an existing primary diagnosis has changed. In this case, the rule saves both the original code and the new code to history.

CPU10200 - Default

This rule has the same logic as the WC rule, except that it applies to all other injury incidents if the WC rule does not apply.

Claim Preupdate rules for subrogation

In the base configuration, Guidewire provides the following Claim Preupdate rules that apply to subrogation.

CPU10800 - Create Subro Summary

This rule is conditional on a claim subrogation that both exists, as determined by the method call `claim.ActivateSubroModule`, and has an empty subrogation summary. If there is such a subrogation on the claim, this rule creates a summary for it.

The method `ActivateSubroModule` is a Claim enhancement defined in `GWSubroNonFinancialClaimEnhancement.gsx` that applies various tests to see if a subrogation exists and returns a Boolean value based on these tests.

CPU12000 - Subro Status - Close

This rule is conditional on a change of the subrogation status to `closed`. If it has been, this rule sets the close date on the summary to today's date and creates a history event for the closure.

CPU14000 - Subro Adverse Party role

This rule is conditional on a claim subrogation that has a summary. If so, it iterates over the subrogation adverse parties. If a party has a changed `AdverseParty` field, the rule iterates over the `claim.Contact` objects to find the one with the same contact as that of the adverse party. If a claim contact is found that does not have the role "adverseparty", the rule adds this role as a new `ClaimContactRole` to this claim contact.

Subrogation rules and scenarios

You can find the financial rules governing subrogation in Guidewire Studio. In the Studio **Project** window, navigate to **configuration > config > Rule Sets > Preupdate > ClaimPreupdate**.

The following descriptions are scenarios for subrogation rules you could add to your application.

Flag a claim as a possible subrogation opportunity

- **Condition** – The claim's subrogation status has not been set and the loss cause is a rear-end collision.
- **Action** – Set the claim's subrogation status to **Review** and the reason to **Rear-end collision**.

You can add more conditions that flag a claim for possible subrogation, or you can add an action to create an activity to review the claim.

Create an activity to send the first subrogation letter

- **Condition** – The **Strategy** is **Pursue**, and this activity does not exist.
- **Action** – Create an activity that creates and sends the first dunning letter with a particular template.

Create an activity to send the second subrogation letter

This rule can be part of the **Activity Closed** rule set. An activity to send a third demand letter would be similar.

- **Condition** – The previous activity is complete and a certain time has passed.
- **Action** – Create an activity that completes and sends the second dunning letter by using its particular template.

Claim Preupdate rule for auto first and final

In the base configuration, Guidewire provides a single Claim Preupdate rule that applies to an auto first and final claim. This kind of claim is a simple auto claim, such as a cracked windshield or glass damage, that requires only a first and final payment.

CPU15000 - AutoFirstAndFinal

This rule is conditional on a claim of type `FirstAndFinal` that is open, has a loss type of `Auto`, and has one exposure and one payment. If all of these conditions are met, the rule auto-completes all non-approval activities and then sets the `FaultRating` to `nofault` (only if the `FaultRating` is null).

Claim Preupdate rules for travel

In the base configuration, Guidewire provides the following Claim Preupdate rules that apply to Travel claims with at least one incident.

CPU16000 - Travel

This rule filters for claims with a loss type of `TRAV` and passes them to its child rules for processing.

CPU16100 - Trip Incident

This rule filters for travel related claims that have at least one trip incident and passes them to the next rule for processing. A *trip incident* concerns cancellation or delay of the trip. For example, if a trip is canceled, it might be necessary to reimburse transportation and accommodation costs that were prepaid.

CPU16110 - Trip Segment

This rule looks for trip segments in the claim's trip incidents and calls the method `onPreUpdate` for each segment found, passing it a value indicating transportation as the expense. The method, declared in

`GWTripExpenseDelegateEnhancement.gsx`, evaluates the `Assessment` field of the claim's trip expenses. Based on the value it finds in this field, the method can create history events and activities for further action.

CPU16120 - Trip Accommodation

This rule looks for trip accommodations in the claim's trip incidents and calls the method `onPreUpdate` for each accommodation found, passing it a value indicating accommodation as the expense. See the previous rule for a description of what `onPreUpdate` does.

Claim Preupdate rules for reinsurance notifications

In the base configuration, Guidewire provides the following Claim Preupdate rules that apply to reinsurance notifications.

CPU17000 - RI Notifications

This rule serves as a folder grouping two reinsurance notification rules.

CPU17100 - Exceeds Notification Threshold

This rule calls the method `setReinsuranceNotificationIfGrossTotalIncurredOverThreshold`, a Claim enhancement defined in `GWClaimReinsuranceNotificationEnhancement.gsx`. The method determines if the notification threshold has been exceeded for each reinsurance agreement. If so, the method can create an activity to review the claim for reinsurance, as appropriate.

CPU172000 - Set Reinsurance Fields from User

This rule can add notes to the claim based on user changes:

- A change in `ReinsuranceReportable` status that is not accompanied by a change in `ReinsuranceFlaggedStatus`
- A change in the `Text` field that occurs with either setting (flagging) or clearing (unflagging) the `ReinsuranceFlaggedStatus` field.

Claim Preupdate rule for claim health metric updates

In the base configuration, Guidewire provides a single Claim Preupdate rule that applies to claim health updates.

CPU18000 - Update Claim Health

This rule calls `claim.scheduleHealthUpdate` to update the claim health metrics.

Claim Preupdate rules for coverage in question

In the base configuration, Guidewire provides the following Claim Preupdate rules that apply to coverage being in question.

CPU20000 - Coverage in question

This rule filters for claims that have coverage in question or claims for which the coverage in question has changed and passes them to its child rules.

CPU20100 - Set Coverage in Question

This rule applies tests to determine if it will set `claim.CoverageInQuestion` to `true`. All these tests must be true for the rule to apply:

- The original Boolean value of `CoverageInQuestion` was not `true`.
- There is no history entry indicating that coverage is in question.
- A call to `claim.isCoverageInQuestion` returns `true`.

CPU 20200 - Activities

This rule applies tests to determine if it will create an activity related to coverage being in question. The rule applies if `claim.CoverageInQuestion` is `true` and there is no activity already for checking coverage in question or for checking on a recovery opportunity:

- If no cost payments have been made, the rule creates an activity for checking coverage in question.
- If cost payments have been made, the rule creates an activity for checking into a recovery opportunity.

CPU 20300 - Create history

This rule makes a history entry if there has been a change to the `claim.CoverageInQuestion` field:

- If `claim.isCoverageInQuestion` returns true, the new history entry says that coverage is in question for the claim.
- If coverage is not in question and the previous value of `claim.CoverageInQuestion` was true, the new history entry says that the user turned off coverage in question.

Claim Preupdate rule to unverify policy if loss date changes

In the base configuration, Guidewire provides a single Claim Preupdate rule that removes policy verification if the claim's loss date changes.

CPU 21000 - Unverify Policy if Loss Date Changed

This rule changes the policy from verified to unverified if the loss date, `claim.LossDate`, changes.

Claim Preupdate rules to balance workload assignment

In the base configuration, Guidewire provides the following Claim Preupdate rules that balance workload assignments.

CPU30000 - Workload Assignment Balancing

This rule tests to see if the weighted workload assignment feature is enabled in configuration. If `gw.api.system.CCConfigParameters.WeightedAssignmentEnabled.Value` is true, its child rules run.

CPU30100 - Claim Closed

If the claim's state has changed to closed, this rule makes an appropriate log entry. The rule then updates the current user's workload to account for the closed claim.

CPU30200 - Claim Reassignment

If the claim has been reassigned to the current user, this rule makes an appropriate log entry. The rule then updates the current user's workload to account for the reassignment.

CPU30300 - Claim Reopened

If the claim has been reopened after previously being closed, this rule makes an appropriate log entry. The rule then updates the current user's workload to account for the reopened claim.

CPU30400 - New Claim

If the claim is new or has just been assigned for the first time, this rule makes an appropriate log entry. The rule then updates the current user's workload to account for the new claim.

CPU30500 - Claim Workload Affected

This rule applies if the workload has changed but the assigned user has not. In particular:

- The claim is open.
- A field on the claim that affects workload has changed.
- The assigned user has not changed.
- The claim's state is not affected.

If all these conditions are true, the rule makes a log entry indicating that the workload has changed for the user for this claim. The rule then updates the current user's workload to account for the change.

Claim Preupdate rule to set workers' comp class code

In the base configuration, Guidewire provides the following Claim Preupdate rules that set the Workers' Compensation class code.

CPU31000 - Workers Comp

This rule filters for a claim loss type of workers' compensation. In the base configuration, it has one enabled child rule.

CPU31100 - Class Code Selection

This Gosu rule is disabled in the base configuration because it is handled in the Business Rules editor.

CPU31200 - Compensability Decision

This rule adds a history entry whenever there is a change in Workers' Compensation compensability for the claim. It also updates the claim's DateCompDcsnMade field to the current date.

Claim Preupdate rule to execute business rules

In the base configuration, Guidewire provides the following Claim Preupdate rule that triggers execution of business rules that trigger on an update to a claim.

CPU40000 - BizRules

This rule executes the business rules that are set to trigger on update of a claim. Guidewire recommends that you always run this rule after the completion of the other Gosu Claim Preupdate rules.

Contact Preupdate rules

Use the Contact Preupdate rules to ensure correct modification of contacts that are added to or already parties to claim. To work with these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Preupdate > ContactPreupdate

In the base configuration, this rule set contains the following rules.

COP01000 - Update Check Address

This rule applies whenever ClaimCenter receives a change of address from the contact management system for a contact that is in sync with that system. This rule ensures that all future dated checks with this contact as a payee have the new contact address, and that the old address is removed from the check.

COP02500 - Set all Vendors to auto sync

All contacts added to the claim are set to automatically synchronize with the contact management system.

COP03000 - Add default tags

This rule ensures that the required, minimum contact tags are applied to each contact added to a claim. In the base configuration, every claim contact of any type must have the Claim Party tag, and every vendor contact must have the Vendor tag.

You can change the minimal tags applied to contacts by configuring the class CCContactMinimalTagsImpl.

Exposure Preupdate rules

Use the Exposure Preupdate rules to modify exposures and related entities. If a rule encounters an exception, the bounding database transaction is rolled back, which prevents the update of the exposure.

To work with these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Preupdate > ExposurePreupdate

In the base configuration, this rule set contains the following rules.

EPU01000 - Assign default coverage

This rule ensures that coverage, if not already set, is set properly for three workers' compensation coverage subtypes:

- WCWorkersCompWAGES
- WCEmpLiabCov

- WCWorkersCompMED

EPU02000 - Salvage

This Gosu rule is disabled in the base configuration because it is handled in the Business Rules editor.

EPU03000 - SI Triggers

This rule is the parent for two Exposure Preupdate SIU rules, which are disabled in the base configuration.

EPU04000 - Update Deductible on Updated Exposure Coverage

This rule checks for a deductible on coverage. If there is one, then the rule creates a deductible tracking object on the claim.

EPU05000 - Update Deductible on Updated Coverage Deductible

This rule calls methods that detect if a deductible has changed and ensures that the deductible is updated if there is a need to do so.

EPU10000 - Workload Assignment Balancing

This rule tests to see if the weighted workload assignment feature is enabled in configuration. If `gw.api.system.CCConfigParameters.WeightedAssignmentEnabled.Value` is true, the following rules run.

EPU10100 - Exposure Closed

If the exposure's state has changed to closed, this rule makes an appropriate log entry. The rule then updates the current user's workload to account for the closed exposure.

EPU10200 - Exposure Reassignment

If the exposure has been reassigned to the current user, this rule makes an appropriate log entry. The rule then updates the current user's workload to account for the reassignment.

EPU10300 - Exposure Reopened

If the exposure has been reopened after previously being closed, this rule makes an appropriate log entry. The rule then updates the current user's workload to account for the reopened exposure.

EPU10400 - New Exposure

If the exposure is new or has just been assigned for the first time, this rule makes an appropriate log entry. The rule then updates the current user's workload to account for the new exposure.

EPU10500 - Exposure Workload Affected

This rule applies if the workload has changed but the assigned user has not. In particular:

- The exposure is open.
- A field on the exposure that affects workload has changed.
- The assigned user has not changed.
- The exposure's state is not affected.

If all these conditions are true, the rule makes a log entry indicating that the workload has changed for the user for this exposure. The rule also:

- Updates the current user's workload to account for the change.
- Creates a new `SITrigger` if none exists.
- Updates the claim's `SIScore` by the number of points specified in the rule.

EPU20000 - BizRules

This rule executes the business rules that are set to trigger on update of an exposure. The rule runs after the completion of the other Gosu Exposure Preupdate rules.

Group Preupdate rules

Use Group Preupdate rules to modify groups and related entities. In the base configuration, there are no rules in this rule set. To work with these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Preupdate > GroupPreupdate

ClaimCenter runs the Group Preupdate rules upon a save of an instance of a Group entity. Any exceptions found cause the bounding database transaction to roll back, thus effectively vetoing the update.

After running these rules, ClaimCenter runs the Group Validation rules.

Matter Preupdate rules

Use the Matter Preupdate rules to modify matters and related entities. This rule set is empty in the base configuration. To work with these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Preupdate > MatterPreupdate

In the following example, the rule creates a new activity to review the claim file 30 days before the date set to litigate the claim in court.

```
CONDITION (matter : entity.Matter):
where (Activity.ActivityPattern != ActivityPattern.finder.getActivityPatternByCode( "trial_review" ) ) )

ACTION (matter : entity.Matter, actions : gw.rules.Action):
var act = matter.Claim.newActivity( ActivityPattern.finder.getActivityPatternByCode( "trial_review" ), null )
act.TargetDate = gw.api.util.DateUtil.addDays( act.Matter.TrialDate, -30 )
act.assignToClaimOwner()
```

Policy Preupdate rules

Use the Policy Preupdate rules to modify policies and related entities. To access these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Preupdate > PolicyPreupdate

In the base configuration, this rule set has one rule that is disabled by default, PPU01000 - Assign Default Coverage. The rule is disabled because its function, linking workers' compensation exposures to policy coverages, is now handled in the **New Claim** wizard. See the comments for this rule for more information.

In the following example, the rule standardizes the policy number by using standard Gosu library functions. The nested functions trim any spaces in the number and turn all alpha characters to lower case.

```
CONDITION (policy : entity.Policy):
return true

ACTION (policy : entity.Policy, actions : gw.rules.Action):
policy.PolicyNumber =
    gw.api.util.StringUtil.toLowerCase( gw.api.util.StringUtil.trim( policy.PolicyNumber ) )
```

RIAgreementPreupdate rules

ClaimCenter runs the RIAgreement Preupdate rule set whenever a user adds or edits a reinsurance agreement. Use this rule set to modify RIAgreement entities and related entities. Exceptions cause the bounding database transaction to roll back, effectively vetoing the update. This rule set is empty by default in the base configuration.

To access these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Preupdate > RIAgreementPreupdate

Modifying an RIAgreement object

Modifying financial values on an RIAgreement object that is used by the IReinsuranceTransactionPlugin to set up RITransaction objects does not affect the calculations in the current commit. The reason is that the IReinsuranceTransactionPlugin has already run and created the RITransaction objects. Instead, modify RIAgreement objects between the invocation of the IReinsurancePlugin and the invocation of the IReinsuranceTransactionPlugin. IReinsurancePlugin retrieves the agreements from an external system. IReinsuranceTransactionPlugin creates RITransaction objects based on the regular financials values and the RIAgreement objects.

RITransactionSetPreupdate rules

ClaimCenter runs the RITransactionSet Preupdate rule set whenever a user adds or edits a reinsurance transaction. This rule set is empty by default in the base configuration.

To access these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Preupdate > RITransactionSetPreupdate

Do not modify the amount of an RITransaction in this rule set. Exceptions cause the bounding database transaction to roll back, effectively vetoing the update. You can use rules in this rule set to update extension fields. However, there are limitations on using this rule set.

Changes to RITransaction entities

Do not use the RITransactionSet Preupdate rules to update the following fields on an RITransaction entity:

- ClaimAmount
- ReportingAmount
- ReserveLine
- RIAgreement

Changes made in this rule set to RITransaction entities do not affect automatic reinsurance financials calculations for RITransaction entities as the calculations are updated whenever the RITransaction object is created. You can modify the IReinsuranceTransactionPlugin implementation to set the ClaimAmount, ReportingAmount, ReserveLine, and RIAgreement fields correctly whenever an RITransaction object is created.

TransactionSet Preupdate rules

ClaimCenter runs the Transaction Set preupdate rules whenever a user adds or edits a transaction. To access these rules, navigate in the Studio **Project** window:

configuration > config > Rule Sets > Preupdate > TransactionSetPreupdate

In working with the Transaction Set preupdate rules, keep the following in mind:

- Do not add a large number of rules to this rule set. This rule set can seriously affect performance because there are typically many transactions active in the system.
- Do not modify fields that affect financial calculations, such as the transaction Amount, the Eroding state, the Exposure, CostType, or CostCategory, or the ScheduledSendDate on a payment's check. You can modify other fields that do not affect financial calculations, including extension fields
- Do not create new transactions in preupdate rules. Preupdate rules do not run on transactions created through preupdate rules. Instead, create the new transactions in a class that implements the IPreUpdateHandler plugin interface, which runs immediately before preupdate rules. In the base implementation, ClaimCenter provides a class that you can use for this purpose, CCPreupdateHandlerImpl, which is registered in the plugin registry IPreupdateHandler.gwp.

In the base configuration, this rule set contains the following rules.

TPU01000 - Create Activity After Check Denial

This Gosu rule is disabled in the base configuration because it is handled in the Business Rules editor.

TPU02000 - Create Activity After Recovery Denial

This Gosu rule is disabled in the base configuration because it is handled in the Business Rules editor.

TPU03000 - Add Budget Lines to Matter

This rule runs whenever you write a check to make a payment on a matter. It checks to see if you have set the **Budget Lines** screen to display for each matter. If so, and the line item category for the check matches a line category used by this screen, the rule adds a budget line item record for the matter.

More specifically, the rule runs on new CheckSet transactions if the script parameter `UtilizeMatterBudget` has been set to true. The rule looks for transactions that include a payment with a cost category of "legal" and have an associated Matter. For each of these transactions, if there is no budget line record for the line item category, the rule adds one.

TPU04000 - Reinsurance

Guidewire disables this rule in the base configuration. It is designed to find transaction sets that have not exceeded the claim's reinsurance reporting threshold so the two reinsurance rules that follow can process these transaction sets.

TPU04100 - Large Loss Identification

Guidewire disables this rule in the base configuration. It calls `transactionSet.Claim.setReinsuranceIfTotalIncurredOverThreshold`, which is defined in `GWClaimReinsuranceEnhancement`.

TPU04200 - Continuous WC Injury Payments Exceeds 12 Months

Guidewire disables this rule in the base configuration. It runs if the loss type is workers' comp and the transaction set is a CheckSet. The rule calls `transactionSet.Claim.setReinsuranceIfContinuousWCInjuryPaymentsExceed12Months`, which is defined in `GWClaimReinsuranceEnhancement`.

TPU05000 - Large Loss Notification

Guidewire disables this rule in the base configuration. It determines if the claim's large loss notification status is either null or set to "None" and the claim exceeds the large loss threshold. If so, the rule calls `transactionSet.Claim.addLargeLossEvent` to notify the policy system. The method is defined in `GWClaimReinsuranceEnhancement`.

TPU06000 - Unlink Deductible After Check Denial

This rule runs on checks that have been denied. For each check in the check set, if the status changed to "Denied", the rule calls `check.unlinkDeductibles`. This method, defined in `GWCheckDeductibleEnhancement`, finds all the deductible line items for payments on the check and unlinks them from their deductibles.

TPU07000 - Update Claim Health

This rule calls `claim.scheduleHealthUpdate` to update the claim health metrics, which might have changed as a result of running the previous rules in this rule set.

TPU08000 - Clean Up Payment Method Related Check Fields

This rule is useful for cleaning up data if the payment method changes. It nulls out unwanted fields if the user changes the payment method between check and EFT (Electronic Check Transfer).

This rule calls the method `gw.entity.GWCheckEnhancement.removeUnusedPaymentMethodRelatedFields` on the Check object. In the base configuration, this method nulls out fields related to a specific payment method.

For payment by Check, the method nulls the following fields:

- `BankAccountNumber`
- `BankName`
- `BankAccountType`
- `BankRoutingNumber`

For payment by EFT, the method nulls the following fields:

- `MailingAddress`
- `MailTo`
- `DeliveryMethod`

TPU04000 - RI Notification Threshold Exceeded

This rule calls `transactionSet.Claim.setReinsuranceIfTotalIncurredOverThreshold`, which is defined in `GWClaimReinsuranceNotificationEnhancement`. The method determines if the notification threshold has been

exceeded for each reinsurance agreement. If so, the method can create an activity to review the claim for reinsurance, as appropriate.

TPU05000 - BizRules

This rule executes the business rules that are set to trigger on update of a transaction set. The rule runs after the completion of the other Gosu Transaction Set Preupdate rules.

TPU06000 - Escalate Instant Payment Check Status

This rule escalates an instant check in `Awaiting` submission status to `Requesting` if the check's scheduled send date has been reached and the instant disbursements (instant payment integration) feature is enabled in `config.xml`. An instant check is a check with a payment method of `Instant` and represents a payment processed by an external payment gateway to an individual.

User Preupdate rules

User Preupdate rules modify `User` and related entities prior to user validation. ClaimCenter runs the User Preupdate rules on a save of the `User` entity. Any exceptions found in a rule in this rule set cause the bounding database transaction to roll back, thus effectively vetoing the update.

To access these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Preupdate > UserPreupdate

In the base configuration, this rule set contains a single rule only.

UPU00005 - User Workload Resync On Status Change

The rule first verifies that weighted workload is enabled and that the user's vacation status or active status has changed. It then logs the changes that were made and resynchronizes the workload for the group to which the user belongs. See the rule description in Guidewire Studio for more information.

ClaimCenter runs the User Validation rule set after this rule set.

Purge

IMPORTANT: Guidewire must enable claim archiving for you to be able to use this feature.

Use the Purge rules to specify additional conditions or actions to take in purging archived entities. For information on archive-related rules, see the *Data Archiving* guide.

Reopened

The Reopened rule sets provide a means of taking automatic action whenever reopening one of the following business objects:

- Claim
- Exposure
- Matter

Claim Reopened rules

The Claim Reopened rule set executes after a claim has been reopened, after Claim Reopened Validation rules run but before preupdate and validation rules run. These rules take automated actions on reopening a claim.

To access these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Reopened > ClaimReopened

In the base configuration, this rule set contains the following rules.

CRO01000 - Notify external systems

If a claim is reopened, this rule adds a Claim Reopened event to notify external systems.

CRO02000 - Allow AutoSync for Related Contacts

Helps facilitate the synchronization of contact information between ClaimCenter and ContactManager.

CRO04000 - Clear archive eligibility date

If claim archiving is enabled, this rule removes the archive eligibility date on the reopened claim.

IMPORTANT: Guidewire must enable claim archiving for you to be able to use this feature.

Exposure Reopened rules

The Exposure Reopened rule set executes after an exposure has been reopened, after Exposure Reopened Validation rules run, but before preupdate and validation rules run. These rules take automated actions on reopening an exposure.

To access these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Reopened > ExposureReopened

In the base configuration, this rule set contains the following rule.

ERO01000 - Notify external systems

If an exposure is reopened, this rule adds an Exposure Reopened event to notify external systems.

Example Exposure Reopened rule

In the following example, the rule uses a script parameter to set the reserve amount for the logged-in user if a Workers' Compensation exposure is reopened. It also uses try... catch... exception handling to deal with potential exceptions.

```
CONDITION (exposure : entity.Exposure):
return exposure.ExposureType == TC_WCINJURYDAMAGE

ACTION (exposure : entity.Exposure, actions : gw.rules.Action):
try {
    exposure.setAvailableReserves(TC_CLAIMCOST, TC_MEDICAL, ScriptParameters.InitialReserve_ReopenClaim,
        User.util.CurrentUser)
} catch (e) {
    CCLoggerCategory.Rules.error("Caught exception: " + e)
}
```

Matter Reopened rules

The Matter Reopened rule set executes after a matter has been reopened, after Matter Reopened Validation rules run, but before preupdate and validation rules run. These rules take automated actions on reopening the matter.

To access these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Reopened > MatterReopened

In the base configuration, this rule set contains the following rule.

MRO01000 - Set Litigation Status

If a matter is reopened, this rule sets `matter.Claim.LitigationStatus = "litigated"`.

Segmentation

The purpose of the segmentation rules is to set the Segment property on new claims and exposures based on the complexity, severity of damage, and other attributes. Optionally, you can also use these rules to set the Strategy property. Segment values are defined in the **ClaimSegment** typelist.

ClaimCenter runs segmentation rules just prior to running assignment rules if automated assignment is selected for a new claim or exposure. Typically, values set for the segmentation for a new claim or exposure are used in determining the assignment of the claim or exposure.

To arrive at a decision on the segment of an exposure, ClaimCenter examines the following:

- The fields on the exposure, such as, for an injury, severity, body part injured, nature of injury, first as opposed to third-party claimant
- Possibly, the fields on the claim, such as the cause of loss and the time between the loss date and the time that the claim was reported

It is easier to make decisions about the segmentation of the claim as a whole after each exposure has been categorized. For example, an auto claim could be categorized as complex if there are any third-party injury exposures. For this reason, the default ClaimCenter workflow first segments each exposure and then segments the claim.

Claim Segmentation rules

You can use the Claim Segmentation rules to categorize each claim as a whole based on complexity, severity of damage, and other attributes. The results set the Segment field of a Claim.

To access these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Segmentation > ClaimSegmentationRules

In the base configuration, this rule set contains the following rules.

CSG01000 - Auto

Filters for auto claims, which have a loss type of "AUTO", and passes these claims to its child rules, the next two rules. If the loss type is not "AUTO", the rule CSG02000 - Property, executes.

CSG01100 - Auto

Finds an exposure with the most important auto segmentation. If not found, the code repeats for the rest of the segmentations in reverse order of importance.

For the first exposure found by these searches, the rule sets the claim segment to that of the found exposure and then exits the claim segmentation rules.

If no exposure is found, this rule drops into the default rule.

CSG01200 - Auto default

This rule sets the claim segment to the lowest auto segment value, "auto_low", and then exits the claim segmentation rules.

CSG02000 - Property

Filters for property claims, which have a loss type of "PR", and passes these claims to its child rules, the next two rules. If the loss type is not "PR", the rule CSG03000 - WC, executes.

CSG02100 - Property

Declares an array of property segment values to be used to find an exposure with the most important segmentation. This array is used to order the segments from lowest to highest. If you add new segment types, you must update the array declared in this rule.

Finds the exposure with the highest segment value in the array and assigns the claim segment to that exposure segment value.

If no exposure is found, this rule drops into the default rule.

CSG02200 - Property default

This rule sets the claim segment to the lowest property segment value, "prop_low", and then exits the claim segmentation rules.

CSG03000 - WC

Filters for workers' comp claims, which have a loss type of "WC", and passes these claims to its child rules, the next two rules. If the loss type is not "WC", the rule CSG04000 - GL, executes.

CSG03100 - WC

Finds an exposure with the most important workers' comp segmentation. If not found, the code repeats for the rest of the segmentations in reverse order of importance.

For the first exposure found by these searches, the rule sets the claim segment to that of the found exposure and then exits the claim segmentation rules.

If no exposure is found, this rule drops into the default rule.

CSG03200 - WC default

This rule sets the claim segment to the lowest workers' comp segment value, "wc_med_only", and then exits the claim segmentation rules.

CSG04000 - GL

Filters for liability claims, which have a loss type of "GL", and passes these claims to its child rules, the next two rules. If the loss type is not "GL", the next rule, CSG05000 - Travel, executes.

CSG04100 - GL

Finds an exposure with the most important liability segmentation. If not found, the code repeats for the rest of the segmentations in reverse order of importance.

For the first exposure found by these searches, the rule sets the claim segment to that of the found exposure and then exits the claim segmentation rules.

If no exposure is found, this rule drops into the default rule.

CSG04200 - GL

This rule sets the claim segment to the lowest liability segment value, "liab_low", and then exits the claim segmentation rules.

CSG05000 - Travel

Filters for travel claims, which have a loss type of "TRAV", and passes these claims to its child rules, the next two rules. If the loss type is not "TRAV", the rule set exits the claim segmentation rules.

CSG05100 - Travel

Finds an exposure with the most important travel segmentation. If not found, the code repeats for the rest of the segmentations in reverse order of importance.

For the first exposure found by these searches, the rule sets the claim segment to that of the found exposure and then exits the claim segmentation rules.

If no exposure is found, this rule drops into the default rule.

CSG05200 - Travel default

This rule sets the claim segment to the lowest travel segment value, "travel_low", and then exits the claim segmentation rules.

Exposure Segmentation rules

You can use the Exposure Segmentation rules to categorize each exposure based on complexity, severity of damage, and other attributes. The results set the Segment field of an Exposure. These rules run before the Claim Segmentation rules so that claim segmentation rules can use the results of exposure segmentation.

To access these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Segmentation > ExposureSegmentationRules

In the base configuration, this rule set contains the following rules.

ESG01000 - Auto

Filters for a claim associated with this exposure that has a loss type of "AUTO", and passes this claim to its child rules, the next four rules. If the loss type is not "AUTO", the rule ESG02000 - Property, executes.

ESG01100 - Auto low

Tests for the following conditions:

- Exposure type is "vehicledamage", "BodilyInjuryDamage", "PIPDamages", "PropertyDamage", or "TowOnly".
- Severity of the incident is "minor" or "moderate-auto".

If the tests evaluate to true, the rule sets `exposure.Segment = "auto_low"` and exits the rule set.

If none of these conditions are true, the next rule executes.

ESG01200 - Auto medium

Tests for the following conditions:

- Exposure type is personal injury damages or loss party is third party.

If the tests evaluate to true, the rule sets `exposure.Segment = "auto_mid"` and exits the rule set.

If none of these conditions are true, the next rule executes.

ESG01300 - Auto high

Tests to see if any one of the following is true:

- Severity of the incident is a severe injury, a major injury, or a fatality.
- The vehicle was a total loss.

If any of the tests evaluate to true, the rule sets `exposure.Segment = "auto_high"` and exits the rule set.

If none of these conditions are true, the default rule executes.

ESG01400 - Auto default

If the exposure segment has not been set yet, this rule sets it to "auto_mid" and exits the rule set.

ESG02000 - Property

Filters for property claims, which have a loss type of "PR", and passes these claims to its child rules, the next four rules. If the loss type is not "PR", the rule ESG03000 - WC, executes.

ESG02100 - Property Low

Tests to see if both of the following conditions are true:

- Exposure type is either "PersonalPropertyDamage" or "PropertyDamage".
- Severity of the incident is "minor".

If the tests evaluate to true, the rule sets `exposure.Segment = "prop_low"` and exits the rule set.

If none of these conditions are true, the next rule executes.

ESG02200 - Property Medium

Tests to see if any one of the following is true:

- Severity of the incident is moderate property,
- Exposure type is loss of use damage.
- The loss party was a third party.

If any of the tests evaluate to true, the rule sets `exposure.Segment = "prop_mid"` and exits the rule set.

If none of these conditions is true, the next rule executes.

ESG02300 - Property High

Tests to see if both of the following conditions are true:

- Severity of the incident is either "major-prop" or "fatal".
- The loss party is the insured party.

If both tests evaluate to true, the rule sets `exposure.Segment = "prop_high"` and exits the rule set.

If none of these conditions is true, the default rule executes.

ESG02400 - Property default

If the exposure segment has not been set yet, this rule sets it to "prop_mid" and exits the rule set.

ESG03000 - WC

Filters for workers' comp claims, which have a loss type of "WC", and passes these claims to its child rules, the next three rules. If the loss type is not "WC", the rule ESG04000 - GL, executes.

ESG03100 - Medical

If the exposure type is workers' comp injury damage, this rule initially assigns the segment "wc_complex_med_only" and exits the rule set. If not, the next rule executes.

This rule also evaluates the severity of the injury incident on the claim and may segment the exposure as "wc_med_only" if any of the following applies:

- There were no fatalities and medical attention was not required.
- Medical attention was required but the severity was minor.
- The severity is 'became medical only', or is null, and the cause of injury was 'cut, puncture, scrape, injured by' (cut typecode in the LossCause typelist).

[See also](#)

- *Application Guide*

ESG03200 - Lost Wages

If the exposure type is lost wages, this rule assigns the segment "wc_lost_time". If not, the default rule executes.

ESG03300 - Default

Sets the segment to "wc_liability" and then exits the rule set.

ESG04000 - GL

Filters for liability claims, which have a loss type of "GL", and passes these claims to its child rules, the next four rules. If the loss type is not "GL", the rule ESG05000 - Travel, executes.

ESG04100 - GL Low

Tests to see if any of the following is true:

- Exposure type is general damage or loss of use damage.
- Severity of the incident is minor.

If any of the tests evaluate to true, the rule sets exposure.Segment = "liab_low" and exits the rule set. If none of these conditions are true, the next rule executes.

ESG04200 - GL Medium

Tests to see if any of the following is true:

- Exposure loss party is a third party.
- Severity of the incident is "moderate-gen".

If any of the tests evaluate to true, the rule sets exposure.Segment = "liab_mid" and exits the rule set. If none of these conditions are true, the next rule executes.

ESG04300 - GL High

If the exposure severity is "severe-gen", this rule assigns the segment "liab_high" and exits the rule set. If not, the default rule executes.

ESG04400 - GL Default

Sets the segment to "liab_mid" and exits the rule set.

ESG05000 - Travel

Filters for travel claims, which have a loss type of "TRAV", and passes these claims to its child rules, the next four rules. If the loss type is not "TRAV", the rule set exits the Claim Segmentation rules.

ESG05100 - Travel Low

Tests to see if any of the following is true:

- Exposure type is baggage or trip cancellation or delay.
- Severity of the incident is minor.

If any of the tests evaluate to `true`, the rule sets `exposure.Segment = "travel_low"` and exits the rule set.

If none of these conditions are `true`, the next rule executes.

ESG05200 - Travel Medium

Tests to see if any of the following is true:

- Exposure type is "MedPay" or "PersonalPropertyDamage".
- Severity of the incident is "moderate-gen".

If any of the tests evaluate to `true`, the rule sets `exposure.Segment = "travel_mid"` and exits the rule set.

If none of these conditions are `true`, the next rule executes.

ESG05300 - Travel High

Tests to see if any of the following is true:

- Exposure type is "VehicleDamage", "PropertyDamage", or "PIPDamage".
- Severity of the incident is "severe-gen" or "major-gen"

If any of the tests evaluate to `true`, the rule sets `exposure.Segment = "travel_high"` and exits the rule set.

If none of these conditions are `true`, the default rule executes.

ESG05400 - Travel Default

Sets the segment to `"travel_low"` and exits the rule set.

Handling a strategy separately from a segment

There are a number of reasons for defining the handling strategy for a claim or exposure separately from the segment:

- There can be many different categories of claims for which it is interesting to report statistics separately. However, there is usually a much smaller number of strategies (also known as handling approaches), such as fast track as opposed to investigate. The segment can capture the more detailed understanding of categorization and govern how ClaimCenter assigns the work, while the strategy can provide an assessment of how much effort is put into adjusting the claim.
- You can experiment with different handling strategies for the same segment of claim. For example, you decide to employ a new approach for a certain segment of claims in one office. You want to be able to analyze the results later to determine if average outcomes were better following the new strategy.

Setting the strategy property

Note: The strategy property can be set independently of any claim or exposure segment. In this example, the segment is used to set the strategy, which is a common approach, but, there are other ways that a strategy can be set.

You can set the strategy value after setting the segment by using the following property:

```
claim.Strategy = strategy_value
```

The `ClaimStrategy` typelist defines the valid strategy values. In the simplest form, the segmentation rules simply use the claim's segment and map a segment to a strategy. For example, the following rule could run after the segmentation is set for an Auto claim.

Note: For this rule to run after the two existing auto claim segmentation rules, you would have to change the two rules to not exit if each rule set completes.

The following code sample illustrates a claim segmentation rule.

```
CONDITION (claim : entity.Claim):  
return true
```

```

ACTION (claim : entity.Claim, actions : gw.rules.Action):
if (claim.Segment == TC_AUTO_HIGH or claim.Segment == TC_AUTO_MID) {
    claim.Strategy = TC_AUTO_NORMAL
}
else {
    claim.Strategy = TC_AUTO_FAST
}
actions.exit()

```

Transaction Approval

Transaction approval rules ensure that a user has authorization to submit certain financial transactions. A transaction set contains one or more transactions that are submitted as a group for approval. If a user attempts to save a transaction set, ClaimCenter can ensure that the transaction be marked as requiring approval.

ClaimCenter calls this rule set to handle transaction approvals of all kinds, not just those that involve authority limits, such as reserves or payments. Guidewire recommends that you create rules in this rule set that are used only to determine whether a transaction requires approval or not.

The Approval Routing and Transaction Approval rule set categories form a pair. ClaimCenter calls these rule sets sequentially.

- ClaimCenter calls rules in the Transaction Approval rule set category whenever you submit any kind of financial transaction.
- ClaimCenter calls rules in the Approval Routing rule set category if the transaction approval rules mark the transaction as requiring approval. It also calls this rule set if the transaction authority limit fails, thus requiring that the transaction needs approval. See “Approval routing” on page 52.

You must ensure that if you approve the activity, you approve the transaction also. If a transaction requires approval, ClaimCenter uses the Approval Routing rules to determine who must give the approval.

In working with the Approval Routing rules:

- Use the `TransactionSet.requireApproval` method to require approval.
- Use the `TransactionSet.getRequestingUser` method to determine the person on whose behalf ClaimCenter runs the rules. For example, suppose that Andy Applegate initiates the reserve, and ClaimCenter routes the transaction to Sue Smith for approval. After Sue approves the transaction, they become the `requestingUser` for the next person in the approval chain.

IMPORTANT: Often, the Transaction Approval rules iterate through all the transactions on the `TransactionSet` entity. Do not use these rules to set a claim to a new value in the course of working with a transaction.

Transaction Approval rules

The Transaction Approval rules determine the approval requirements for financial transactions. ClaimCenter runs these rules for each transaction to determine if the transaction requires approval. In a typical `TransactionSet` approval rule, the rule condition verifies the subtype of the transaction set, such as `TransactionSet.subtype == "checkset"`. This subtype verification is required because all created transaction sets, regardless of subtype, are passed to the same approval rules.

To access these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > TransactionApproval > TransactionApprovalRules

In the base configuration, this rule set contains the following rules.

TAP01000 - PIP

This rule determines if approval is needed for a transaction with an exposure that has reached its limit for incident, exposure, or several measures of personal injury coverage (PIP).

The rule first verifies that the transaction set subtype is `checkset` and that there is at least one transaction in the transaction set with a cost type of `claimcost`.

The rule then tests each transaction in the transaction set to see if the related exposure's authority limit has been reached for any of the following:

- Incident limit
- Exposure limit
- PIP non-medical aggregate limit
- PIP replacement services aggregate limit
- PIP per-person aggregate limit
- PIP claim aggregate limit

If one of these limits has been reached and the supervisor has not already approved payment for the transaction, an activity is created for that purpose.

TAP02000 - Sample - Manual Auth Limit Checking

This sample rule is disabled by default in the base configuration.

If enabled, the rule checks authority limits on `ReserveSet` and `CheckSet` objects, excluding bulk invoice checks. It provides an alternative to automatic authority limit checking, which does check bulk invoice checks.

Note: This rule is appropriate only if the system configuration parameter `CheckAuthorityLimits` is `false`.

TAP03000 - Sample - Custom Calculation

This sample rule is disabled by default in the base configuration.

The rule uses a customized total incurred value. For more information on Total Incurred Value, see the class `util.financials.CustomCalcs`.

TAP04000 - Claim and Exposures Should Be At Ability to Pay

This rule detects claims and exposures associated with the transaction set that are not at a specified validation level. The transaction set must be a check set. If the associated claim or exposure is not at Ability to Pay, the check set is sent for approval.

TAP05000 - Optionally Check for Duplicate Checks

This rule can look for duplicate checks that might exist if a check is created programmatically, through the web service `ClaimFinancialsAPI.creatCheckSet`.

Note: If a check is created in the ClaimCenter user interface, the check for duplicates has already happened and has been handled there. In this case, `CheckSetRequiresApprovalForDuplicateChecks` is set to `false`.

The rule first determines if the transaction set is a check set. If it is and `CheckSetRequiresApprovalForDuplicateChecks` set to `true`, the rule looks for duplicate checks in the transaction set. If the rule finds a duplicate check, it sends the check set for approval.

Validation

ClaimCenter uses validation levels to ensure that certain conditions are met before it is possible to move to a critical stage in the processing workflow of claim objects. For example, these validation levels apply as you create a new claim or make a payment. You can view these stages as maturity levels—as a claim gains data, it achieves new levels of maturity. A ClaimCenter object can always achieve a new and higher level of maturity, but it can never go backwards to a lower validation level.

A validation rule sets the maturity level to which the rule applies. ClaimCenter then enforces rule errors only if set at or below the level the object has achieved.

To access these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Validation

Notes:

- ClaimCenter runs the validation rules at many different points in the ClaimCenter workflow. ClaimCenter also runs these rules whenever it commits a claim or other validatable business entity to the database.
- ClaimCenter does not run validation rules on claims that are in the in-process Draft state in the New Claim wizard.

About ClaimCenter validation levels

In the base configuration, ClaimCenter has five levels of validation. Some of these validation levels are internal to ClaimCenter, and you cannot change them. Others are editable, and you can access them through Guidewire Studio by using the `ValidationLevel` typelist.

Validation Priority Levels

ClaimCenter uses the `priority` attribute of the typekeys to order the validation levels. The lower the priority number, the higher the validation level. Thus, in the base configuration, `newloss` with a priority of 300 is more restrictive than `loadsave` with a priority of 400. The following list describes the base configuration validation levels, listed in increasing order of maturity.

Level	Name	Priority
loadsave	Load and save	400
newloss	New loss completion	300
ISO	Valid for ISO	250
external	Send to external system	200
payment	Ability to pay	100

System Validation Levels

In the base configuration, ClaimCenter defines three validation levels that you cannot delete, although you can override their properties. ClaimCenter requires that these validation levels exist for internal code use. In the `ValidationLevel` Typelist editor, you see the following three validation levels in gray text, indicating that they are not editable.

Level	Priority	Description
loadsave	400	The loosest possible validation level against which ClaimCenter validates claims. All data must pass loadsave to be saved to the database.
newloss	300	The lowest level at which a claim can exit the New Claim wizard.
payment	100	The lowest level at which it is possible to pay a claim or an exposure. For example, in the base configuration, ClaimCenter does not permit you to enter the New Check wizard if the claim is not in Ability to Pay.

Non-system Validation Levels

ClaimCenter provides other validation levels in the base configuration, listed in the table that follows. Because Guidewire defines these levels in the `validationLevel1.ttx` typelist, it is possible to modify, remove, or even add to them by using the Studio Typelist editor.

Level	Priority	Description
iso	250	The level at which the claim passes ISO validation.
external	200	The level at which a claim can be submitted to an external system. Validation occurs whenever an external submission or decline is triggered by a workflow button.

Note: It is possible to add additional validation levels.

About validation warnings and errors

It is possible for ClaimCenter to generate validation warnings that provide information only. To generate this type of warning, the issue must not affect the claim or exposure, but it can affect other business practices. For example, if an automobile claim is missing weather information, a warning can remind the user to enter the information. The following code sample, added to the Claim Validation Rules rule set, illustrates this concept.

```
CONDITION (claim : entity.Claim):
return claim.LossType==TC_AUTO and claim.Weather==null

ACTION (claim : entity.Claim, actions : gw.rules.Action):
claim.rejectField("Weather", null, null, TC_NEWLOSS,
    "Remember to enter weather information for an auto claim!")
```

It is also possible for ClaimCenter to generate validation errors that prevent the claim or exposure from being saved in ClaimCenter. For example, it is not possible for a user to save a claim in which the loss date is in the future. The following code sample is from the Claim Validation Rule CLV03000 - Future loss date.

```
CONDITION (claim : entity.Claim):
return Claim.LossDate > gw.api.util.DateUtil.currentDate()

ACTION (claim : entity.Claim, actions : gw.rules.Action):
Claim.rejectField("LossDate", TC_NEWLOSS, displaykey.Rules.Validation.Claim.ProvideLossDateNotInTheFuture,
    null, null)
```

In this example, ClaimCenter marks the `Claim.LossDate` field as invalid and prompts the user to fix it.

Adding new validation levels

It is possible to add additional validation levels to meet your business needs.

System Validation Levels

You can override the name and the priority of the three system validations levels `loadsave`, `newloss`, and `payment`. For example, you can change the name of the `payment` validation level from *Ability to pay* to *Ability to pay expenses*. You can also modify the priority so this level is easier to reach. However, you cannot delete the three system validation levels.

Non-system Validation Levels

You can modify or delete any non-system validation level.

Custom Validation Levels

You can add new validation levels. If you do so, then you must provide a linkage to the actions you would like to control through the new validation level and to the validation level itself. You do this by calling the following method on a `Claim` or `Exposure` object or any other validatable object:

```
isValid(validationLevel, includeExposures)
```

For example:

```
Claim.isValid("newLoss", false)
```

The `isValid` method determines if the claim or exposure is valid for the action you are attempting to perform. Calling this method is required for checking actions triggered in `preupdate` rules or from within ClaimCenter. It is important to understand that ClaimCenter runs the Validation rules after the `preupdate` rules, so the `Claim.ValidationLevel` property reflects the previous validation level. However, this is not true for rules in the Event Message rule set category, which ClaimCenter runs after validation.

How ClaimCenter triggers validation rules

For an entity to trigger the validation rules, it must implement the `Validatable` delegate. This is true for an entity in the base ClaimCenter configuration or for a custom entity that you create.

ClaimCenter runs the validation (and preupdate) rules:

- Whenever an instance of a validatable entity is created or modified.
- Whenever a subentity of a validatable entity is created or modified and the validatable entity is connected to the subentity through a `triggersValidation="true"` link.

How ClaimCenter triggers Validation rules on custom entities

It is possible to create validation rules for custom entities, entities that you create yourself and that are not part of the base Guidewire ClaimCenter configuration.

For an extension entity to trigger a validation rule, all of the following must be true:

1. The entity must be a root entity, not a subtype of an entity.
2. The entity must implement the `Validatable` delegate.

For any extension entity that you create, if you want it to trigger a validation rule, the extension entity must implement the following code:

```
<implementsEntity name="Validatable"/>
```

3. A rule set must exist in the Validation rule category that conforms to the following naming convention:

```
custom_entity_nameValidationRules
```

For example, if you create an extension entity named `NewEntityExt`, create a rule set in the Validation rule category to hold your validation rules and name it `NewEntityExtValidationRules`.

ClaimCenter Validation and Preupdate rules

With the exception of in-process claims in the New Claim wizard, ClaimCenter runs preupdate and validation rules every time that it commits data to the database. Guidewire calls the process of committing data to the database a *database bundle commit*.

Validation rules execute after ClaimCenter runs all preupdate rules and internal preupdate callbacks. Execution of the validation rules is followed by event fired rules before ClaimCenter writes data to the database.

It is possible for the preupdate rules to modify objects during execution, requiring changes to the database. However, ClaimCenter does not permit you to modify objects during validation rule execution that require changes to the database. To allow this would make it impossible to ever completely validate data—data validation would be an ever-shifting target.

New Claim wizard

Claims being processed in the New Claim wizard are saved in Draft state until the user completes the wizard. Whenever ClaimCenter saves the claim in Draft state, it runs only the preupdate rules. Whenever the user finishes the wizard, ClaimCenter runs both the preupdate and validation rules on the claim as described in this topic. In fact, at the end of the New Claim wizard, it explicitly validates the claim and any existing exposures. This results in claim validation rules always running twice at the end of the New Claim wizard. It may also result in exposure validation rules running once or twice, if exposures were added or modified before the wizard is complete.

Determining the validity of an object

ClaimCenter provides a `validate` method that you can call on a number of different business objects to determine if that object is valid. For example, the `validate` method on `Claim` executes the Claim Validation Rules on the supplied claim. The method then returns a `ValidationResult` object containing any errors. If the validation was successful, the method returns an empty validation object. Also, if `validateExposures` is true, then the method validates the claim and all the exposures on the claim. Otherwise, the method validates the claim only.

```
claim.validate(validateExposures)
```

This method works on a single object at a time and returns a validation object that you must then handle. For this reason, Guidewire does not recommend that you use this method for bulk operations. For example, if you want to

process a large number of converted claims by escalating them to whatever level they can achieve, use the `ClaimAPI` method, `startClaimValidation`, as follows:

```
ClaimAPI.startClaimValidation(loadCommandPublicID)
```

The `startClaimValidation` method does not perform any validation. Instead, it launches a batch process that creates a separate work item to validate each claim. The batch process runs unattended and does not return a result. The `loadCommandPublicID` parameter is a `String` value that identifies the conversion batch process that imported the claims. The `loadCommandPublicID` value is available through the `TableImportResult` object returned from a table import operation. For example:

```
TableImportAPI.integrityCheckStagingTableContentsAndLoadSourceTables()
```

To determine if a claim and its exposures are valid, call the following method on a `Claim` or `Exposure` object:

```
claim.isValid(validationLevel, includeExposures)
exposure.isValid(validationLevel)
```

Validation error reject methods

IMPORTANT: Although it is possible to access business entities other than the declared type within a rule, call the `reject` method only on the declared entity in the rule set. Studio does not detect violations of this guideline. If you do call `reject` on an entity that is not the declared type, `ClaimCenter` can lose this reject call. This loss can result from `ClaimCenter` traversing the set of validatable objects that resets the previous result for the entity.

You can use a form of the `reject` method to prevent continued processing of the object and to inform the user of the problem and how to correct it. The `reject` methods are available on all validatable entities. In the base configuration, these entities are:

- Activity
- Claim
- Contact
- Exposure
- Matter
- Person
- Policy
- RIAgreement
- RITransactionSet
- ServiceRequest
- TransactionSet
- User

The following table describes the various forms of the `reject` method.

Method form	Description
<code>reject</code>	Indicates a problem and provides an error message, but does not point the user to a specific field.
<code>rejectField</code>	Indicates a problem with a particular field or field path, provides an error message, and directs the user to the correct field to fix.
<code>rejectSubField</code>	Similar to <code>rejectField</code> , but also indicates that the problem is with a field on a subobject.

The method signature can take one of the following forms:

```
reject( errorLevel, strErrorReason, warningLevel, strWarningReason )
rejectField( strRelativeFieldPath, errorLevel, strErrorReason, warningLevel, strWarningReason )
rejectSubField( relatedBean, strRelativeFieldPath, errorLevel, strErrorReason, warningLevel,
               strWarningReason )
```

IMPORTANT: Guidewire ClaimCenter currently supports the listed reject methods only, even though additional methods can show in Studio. Specifically, ClaimCenter does not support reject methods that take a `flowtStepId` argument. ClaimCenter ignores this argument if you attempt to set it.

The following table lists the reject method parameters and describes their use. Identified issues or problems are either errors (blocking) or warnings (non-blocking). Note that you can indicate failure as both an error and a warning simultaneously. However, if the failure is both an error and a warning, use different error and warning levels for the failure.

Method parameter	Description
<code>errorLevel</code>	Corresponding level to be set as a result of the validation error.
<code>relatedBean</code>	Object related to the root entity that is causing the error.
<code>strErrorReason</code>	Message to show to the user indicating the reason for the error.
<code>strRelativeFieldPath</code>	Relative path from the root entity to the field that failed validation. Using a relative path (as opposed to a full path) sets a link to the problem field from within the message shown to the user. It also identifies the invalid field on-screen.
<code>strWarningReason</code>	Message to show to the user indicating the reason for the warning.
<code>warningLevel</code>	Corresponding level effected by the validation warning.

Validation Rules and Entity Types

You define a validation rule in a rule set that is declared for a specific entity type. For example, the rule set would be for `Claim`, `User`, or one of the other business entities. A rule declares its primary object in the rule `CONDITION` statement.

About error text strings

Guidewire recommends that you use one of the following ways to insert an error text string into a reject:

- Method `getMessage`
- Display keys

Method `getMessage`

The `getMessage` method returns an instance of `UserMessage`, which represents all the information needed to render a user message. Guidewire recommends that you also call the `localize` method to handle any localization issues. The following `ClaimContact` Role Configuration example for exposure validation checks roles at the exposure level and generates an error message for each invalid role.

```
/* If anything relevant has changed, check for invalid role
   configurations at the exposure level and report each individually
   on the Contacts array if they exist
*/
var b = \ bean : KeyableBean -> typeof bean == entity.ClaimContact ||
      typeof bean == entity.ClaimContactRole
if (!exposure.isValid(TC_NEWLOSS))
    || exposure.Bundle.InsertedBeans.toList().hasMatch(b)
    || exposure.Bundle.UpdatedBeans.toList().hasMatch(b)
    || exposure.Bundle.RemovedBeans.toList().hasMatch(b)) {
    var failures = exposure.validateRoles()
    for (failure in failures) {
        exposure.reject(TC_NEWLOSS, failure.getMessage().localize(), null, null)
    }
}
```


Display Keys

You can also use display keys to return a value as the error text string. A display key provides globalization support. For example, in the `display.properties` file for US English, the following display key is defined as User has duplicate roles:

```
displaykey.Java.Admin.User.DuplicateRoleError
```

You can also use display keys that require a parameter or parameters. In the following example, the `display.properties` file defines the following display key with placeholder `{0}`:

```
Java.UserDetail.Delete.IsSupervisorError = Cannot delete user because they are supervisor  
of the following groups\: {0}
```

Suppose then, that you use the following display key code for a rule validating Groups. In this case, ClaimCenter has already retrieved the value of `GroupName`.

```
displaykey.Java.UserDetail.Delete.IsSupervisorError( GroupName )
```

This display key returns the following from the `display.properties` file:

```
Cannot delete user because they are supervisor of the following groups: AutoClaim1
```

Activity Validation rules

Use the Gosu Activity Validation rules to provide validation for Activity objects. To access these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Validation > ActivityValidationRules

In the base configuration, this rule set contains a single rule.

ACV01000 - WC Compensability

This rule tests for an activity of type Claim Acceptance with a Status of Complete and an associated workers' comp claim with a Compensable status of either null or Pending.

For any matching activity, the rule rejects the activity, but does not close it, and displays the following message:

```
Prior to completing the activity, {0}, please change the Compensability status to either  
Accepted or Denied
```

In the base configuration, ClaimCenter defines this message in file `display.properties` as the following display key:

```
displaykey.Rules.Validation.Activity.Compensable.Status(activity.Subject)
```

Claim Closed Validation rules

The Claim Closed Validation rules execute whenever ClaimCenter closes a claim. These rules execute:

- Before ClaimCenter commits the data to the database.
- Before ClaimCenter executes the Claim Closed rule set.
- Before ClaimCenter executes the Claim Preupdate rule set.
- Before ClaimCenter executes the Claim Validation rule set.

You can use these rules to display validation errors or warnings if ClaimCenter must not close a claim for some reason. To access these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Validation > ClaimClosedValidationRules

In the base configuration, this rule set contains the following rules.

CCV01000 - Open exposures

If the claim being closed has any open exposures, this rule prevents the claim from being closed. Open exposures must be closed before closing the claim.

CCV02000 - Open activities

If the claim being closed has open activities that are not allowed on a closed claim, this rule prevents the claim from being closed. All the open activities must first be completed or skipped before the claim can be closed.

The rule does not prevent the claim from being closed for activities that are allowed on closed claim.

CCV03000 - Property checks

This rule checks if the claim being closed has a loss type of PR, but the rule has no actions. Its purpose is to filter for property claims in case you want to add child rules that perform claim closed validation on property claims.

CCV04000 - At Fault must be determined for non-WC

In the base configuration, the claims that can require a determination of which party is at fault are claims with a loss type of AUTO, PR, and GL. For example, these claims can be on policies for personal auto, commercial auto, general liability, other liability, businessowners, personal umbrella, commercial property, Homeowners, and inland marine.

This rule first checks if the claim being closed has one of these three loss types—AUTO, PR, or GL—and `FaultRating` is not set but payments have been made. If the claim matches these criteria, this rule prevents the claim from being closed. The fault rating must be determined before closing the claim.

CCV05000 - Subrogation Status must not be Review

This rule first checks if the claim being closed is being reviewed for subrogation—`claim.SubrogationStatus == "review"`. If so, the rule prevents the claim from being closed. The subrogation must be set to Open or Closed before the claim can be closed.

CCV06000 - if Third party Liability indicated

This rule first checks if the claim being closed has third party liability and has not been reviewed for subrogation. If so, the rule prevents the claim from being closed. The claim's third party liability requires that subrogation be closed before the claim can be closed.

CCV07000 - Open matters

If the claim being closed has any open matters, this rule prevents the claim from being closed. All open matters must first be closed before the claim can be closed.

CCV08000 - Open Service Requests

If the claim being closed has any active service requests that have not been completed, this rule prevents the claim from being closed. All open service requests must be closed before the claim can be closed.

Claim Reopened Validation rules

The Claim Reopened Validation rules execute whenever ClaimCenter reopens a claim. In particular, they execute:

- After the Claim Validation rules execute.
- Before ClaimCenter commits the data to the database.
- Before ClaimCenter inserts a new entity or updated entity into the database.
- Before ClaimCenter executes the Claim Reopened rules.

The Claim Reopened Validation rules perform their actions only if the entity is valid.

You can use these rules to raise validation errors or warnings if ClaimCenter cannot reopen a claim for some reason. To access these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Validation > ClaimReopenedValidationRules

This rule set is empty in the base ClaimCenter configuration.

Claim Validation rules

Use the Claim Validation rules to ensure that claim data stored in ClaimCenter is sufficient and valid for downstream processing. To access these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Validation > ClaimValidationRules

In the base configuration, this rule set contains the following rules.

CLV01000 - Not set - Coverage in Question

Guidewire disables this rule in the base configuration. If the claim has a null `CoverageInQuestion` property, the rule prevents the claim from being used until the property is set.

CLV02000 - Coverage indicated but no details provided

This rule is disabled in the base configuration because exposures are validated in a dedicated exposure rule set. The rule serves only as an example.

The rule tests each exposure of the claim to determine if the `OtherCoverage` property is set but the detail field for other coverage is empty. For every exposure that matches these criteria, the rule calls `claim.rejectSubField`. This method call sets the claim to New Loss and displays a message saying to indicate the details of the other coverage.

CLV03000 - Future loss date

If the claim has a loss date that is later than today's date and time, the rule calls `claim.rejectSubField`. This method call sets the claim to New Loss and displays a message saying to change the date to one that is not in the future.

CLV04000 - ClaimContact Role Configuration

This rule determines if anything that is relevant to contact roles has changed for the claim. If so, the rule checks for invalid role configurations for the claim and reports each one it finds individually on the `Contacts` array.

CLV05000 - Policy expiration date after effective date

If the policy for the claim expired before the policy was in effect, this rule prevents the claim from being saved. The policy must be in effect before it expires for a claim to be filed on it.

CLV06000 - Policy original effective date

If the policy for the claim has an original effective date that is later than the current effective date, `claim.Policy.EffectiveDate`, this rule prevents the claim from being saved. The original effective date of the policy must not be later than the policy effective date.

CLV07000 - Liability can not exceed 100

This rule first determines if the claim has gone to subrogation and that there are adverse parties involved. It prevents the claim from proceeding if either of the following is the case:

- Total liability for adverse parties exceeds 100%.
- Total liability for the insured and the adverse parties together exceeds 100%.

CLV08000 - Expected Recovery exceeds 100

This rule first determines if the claim has gone to subrogation and that there are adverse parties involved. It prevents the claim from proceeding if both the following are the case:

- There was a change to the expected recovery amount.
- The expected recovery from the adverse parties exceeds 100%.

CLV09000 - ISO Validation

This rule tests if the current claim has ISO enabled. If so, it passes the claim to its child rules.

CLV09100 - Not draft

This rule checks the claim to see if it is still in Draft state. If so, the rule prevents the claim from going to ISO. The state must be at least ISO for ISO to be enabled.

CLV09200 - Policy

If there is no policy related to the claim, this rule prevents the claim from going to ISO. There must be an associated policy for ISO to be enabled.

CLV09300 - LossLocation

If there is no loss location for the claim, this rule prevents the claim from going to ISO. There must be a loss location for ISO to be enabled.

CLV09400 - PolicyNumber

If there is no policy number for the policy that is related to the claim, this rule prevents the claim from going to ISO. There must be a policy number for the associated policy for ISO to be enabled.

CLV09500 - PolicyType

If there is no policy type for the policy that is related to the claim, this rule prevents the claim from going to ISO. There must be a policy type for the associated policy for ISO to be enabled.

CLV09600 - ClaimNumber

If there is no claim number for the claim, this rule prevents the claim from going to ISO. There must be a claim number on the claim for ISO to be enabled.

CLV09700 - LossDate

If there is no loss date for the claim, this rule prevents the claim from going to ISO. There must be a loss date for the claim for ISO to be enabled.

CLV09800 - State

This rule tests if the current claim is missing a State entry for the loss location. If the state is missing and the country US or CA (Canada) or there is no country, the rule prevents the claim from going to ISO.

CLV09900 - Insured

This rule groups three child rules that perform validations on values for the insured party.

CLV09910 - Insured exists

If there is no insured party for the claim, this rule prevents the claim from going to ISO. There must be an insured party for the claim for ISO to be enabled.

CLV09920 - Name

This rule performs a series of tests on the name of the insured party. If the insured party is a person, the person must have a first and last name. If the insured party is a company, the company must have a name. If any of these tests fails, the rule prevents the claim from going to ISO. The appropriate name fields must be filled in for ISO to be enabled.

CLV09930 - Address

This rule performs a series of tests on the address of the insured party. If the address is missing the first line of the address or the city, the rule prevents the claim from going to ISO. Additionally, if the country is US, CA (Canada), or null (no country), the State field must be filled in. All these fields must be filled in for ISO to be enabled.

CLV091000 - Workers Comp

This rule filters for a workers' comp claim. Its child rules apply various tests for this type of claim.

CLV091010 - Claimant Exists

This rule checks to see if the claimant is missing. If there is no claimant, the rule prevents the claim from going to ISO. There must be a claimant for a workers' comp claim for ISO to be enabled.

CLV091020 - Name

If there is a claimant, the rule checks if the person has a first and last name. If either is missing, the rule prevents the claim from going to ISO. The claimant for a workers' comp claim must have a first and last name for ISO to be enabled.

CLV091030 - Address

If there is a claimant, this rule performs a series of tests on the address of the claimant. If the address is missing the first line of the address or the city, the rule prevents the claim from going to ISO. Additionally, if the country is US, CA (Canada), or null (no country), the State field must be filled in. All these fields must be filled in for ISO to be enabled.

CLV091030 - Injury Description

If the injury description is missing, the rule rejects the claim. A workers' comp claim must have an injury description for ISO to be enabled.

CLV091030 - Body Part

If either there are no body parts described or if the first body part's details are missing, the rule prevents the claim from going to ISO. A workers' comp claim must have at least one body part indicated for ISO to be enabled.

CLV091100 - Description

This rule evaluates if the claim is eligible to send search messages to the ISO system. If so and there is no content for the Description property, the rule prevents the claim from going to ISO. If it is to be searchable, a workers' comp claim must have a description for ISO to be enabled.

CLV10000 - Catastrophe

This rule determines if the claim is for a catastrophe. If so, the following child rules execute.

CLV10100 - Check Perils

This rule determines if the claim has a catastrophe peril. If it does, the claim's loss cause and loss type must match the peril's loss cause and loss type for the claim to be categorized as a catastrophe.

CLV10200 - Check Loss Date and Zone

This rule determines if:

- The claim's loss date is within the catastrophe's date range.
- The claim's location matches the zone type for which the catastrophe is valid.

Both conditions must be true for the claim to be categorized as a catastrophe.

CLV11000 - Subrogation Status And Liability

For a claim that is in subrogation, the total liability percentage for insured and adverse parties must equal 100%. This rule determines the actual percentage for this type of claim and rejects it if the percentage is not 100%.

CLV12000 - Injury diagnosis validity dates

This rule determines if the claim has an injury diagnosis. If so the child rules can run.

CLV120100 - WC

This rule determines if the start date and end date of each injury diagnosis for a workers' comp claim are within the valid dates for the injury code. If not, the claim is rejected.

CLV120200 - Default

This rule determines if the start date and end date of each injury diagnosis for a workers' comp claim are within the valid dates for the injury code. If not, the claim is rejected.

CLV13000 - Full Denial Reasons

This rule enforces a limit of five denial reasons for a workers' comp claim.

CLV14000 - Subrogation role changes

If this claim has a subrogation, the following child rules execute.

CLV14100 - Subrogation owner role removed

This rule prevents the user from directly removing the subrogation owner role from a user on the claim. Instead, the user must reassign the subrogation.

CLV14200 - Responsible party role removed

This rule prevents the user from directly removing the responsible party role from a claim contact. Instead, the user must remove the contact from the responsible parties list on the subrogation.

CLV15000 - Applicable Deductible CovTerm Changed

This rule checks if the claim loss cause has changed, which results in a change to the deductible coverage term. It prevents the claim from validating at load and save if a deductible has not been paid with the message:

The applicable deductible was updated as the loss cause has changed. You may need to inform insured about the changed deductible amount.

If a deductible has been paid, it prevents the claim from validating at new loss completion with the message:

Changing the loss cause may change the applicable deductible. Review the deductible amount on the exposure and any checks where deductible has been applied.

CLV16000 - ICDCode usage

This rule prevents the claim from validating at load and save if there is a discrepancy between ICD codes entered on medical treatments and codes on medical diagnoses, or, if a medical treatment code is entered but no medical diagnosis exists. It also prevents validation at load and save when adding ICD codes with different editions (such as ICD-9 and ICD-10) on the same claim.

CLV17000 - Medical Treatment Provider Validation

This rule prevents the claim from validating at load and save if the contact associated with a medical treatment on a workers' comp claim is not one of the following medical provider roles (in the ContactRole typelist):

- Doctor
- Primary Doctor
- Physical Therapist
- First Intake Doctor
- Occupational Therapist

Because the ClaimCenter user interface already prevents this from occurring, this rule is solely to prevent non-medical provider roles from being associated with medical treatments created using ClaimCenter Cloud API POST requests.

Contact Validation rules

Use the Contact Validation rules to ensure that contacts data stored is sufficient and valid for downstream processing. To access these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Validation > ContactValidationRules

This rule set contains the following rules. In the base configuration, Guidewire disables all the rules in this rule set.

Home phone country code must be valid

Guidewire disables this rule in the base configuration.

This rule checks to see if a home phone number is defined for the contact. If there is one, the rule verifies that the country code is valid. If it is not, the contact is rejected.

Work phone country code must be valid

Guidewire disables this rule in the base configuration.

This rule checks if a work phone number is defined for the contact. If there is one, the rule verifies that the country code is valid. If it is not, the contact is rejected.

Fax phone country code must be valid

Guidewire disables this rule in the base configuration.

This rule checks if a fax phone number is defined for the contact. If there is one, the rule verifies that the country code is valid. If it is not, the contact is rejected.

PreferredContactMethod cannot be null

Guidewire enables this rule in the base configuration.

This rule checks if a person contact has opted in to communications from the insurer, also known as a preferred digital contact method. If so, either the contact's mobile phone number or email address cannot be null. For example, if Ray Newton opts in to receive SMS (text) messages as his preferred contact method, his mobile number cannot be null.

Exposure Closed Validation rules

The Exposure Closed Validation rules execute whenever ClaimCenter closes an exposure. These rules execute:

- Before ClaimCenter commits the data to the database.
- Before ClaimCenter executes the Exposure Closed rule set.
- Before ClaimCenter executes the Exposure Preupdate rule set.
- Before ClaimCenter executes the Exposure Validation rule set.

You can use these rules to display validation errors or warnings if ClaimCenter must not close an exposure for some reason. To access these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Validation > ExposureClosedValidationRules

In the base configuration, this rule set contains the following rules.

ECV01000 - Open activities

If the exposure being closed has any open activities that are not permitted in closed claims, this rule prevents the exposure from being closed. Open activities not permitted in closed claims must be closed before closing the exposure.

ECV02000 - Injury-specific checks

This rule determines if the exposure being closed is for bodily injury. It is an example of a parent rule that could filter exposures for a set of child rules.

ECV03000 - Vehicle damage-specific checks

This rule determines if the exposure being closed is for vehicle damage. It is an example of a parent rule that could filter exposures for a set of child rules.

ECV04000 - Property-specific checks

This rule determines if the exposure being closed is for a property loss. It is an example of a parent rule that could filter exposures for a set of child rules.

ECV05000 - Content-specific checks

This rule determines if the exposure being closed is related to contents. It is an example of a parent rule that could filter exposures for a set of child rules.

ECV06000 - Loss of Use-specific checks

This rule determines if the exposure being closed is for loss of use. It is an example of a parent rule that could filter exposures for a set of child rules.

ECV07000 - Stop Closing Of Exposure With Unpaid Deductible

This rule determines if If this exposure's coverage has an unpaid deductible. If the exposure has at least one payment, it cannot be closed until the deductible is waived.

Exposure Reopened Validation rules

The Exposure Reopened Validation rules execute whenever a user reopens an exposure. The rules execute:

- Before ClaimCenter commits data to the database
- Before the Exposure Reopened rule set executes
- During the standard operation of the Preupdate and Validation rule sets.

Use these rules to raise validation errors or warnings if there is some reason to not reopen the exposure.

To access these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Validation > ExposureReopenedValidationRules

In the base configuration, this rule set contains the following rule.

ERV01000 - Claim closed

If the exposure being reopened is on a claim that is not open, this rule rejects the reopen action. An exposure cannot be reopened unless the claim is open.

Exposure Validation rules

Use the Exposure Validation rules to ensure that exposure data stored in ClaimCenter is sufficient and valid for downstream processing.

To access these rules, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Validation > ExposureValidationRules

In the base configuration, this rule set contains the following rules.

EXV01000 - Incident not null

This rule checks to see if an incident is not associated with the exposure and rejects the exposure if that is the case. An incident must be associated with an exposure for the exposure to be valid.

EXV02000 - Other coverage but no info

Guidewire disables this rule in the base configuration. The rule causes ClaimCenter to enforce that exposures fill in Other Coverage data during validation.

EXV03000 - ClaimContact Role Configuration

This rule determines if anything that is relevant to contact roles has changed for the exposure. If so, the rule checks for invalid role configurations for the exposure and reports each one it finds individually on the `Contacts` array.

EXV04000 - Coverage not null

Guidewire disables this rule in the base configuration. The restrict prevents the writing of checks for Workers' Comp exposures that have no appropriate coverage in the policy.

EXV05000 - ISO Validation

This rule tests if the current exposure has ISO enabled. If so, it passes the exposure to its child rules.

EXV05100 - Not draft

This rule checks the exposure to see if it is still in Draft state. If so, the rule prevents the exposure from going to ISO. The state must be at least ISO for ISO to be enabled.

EXV05200 - Primary Coverage

If there is no primary coverage for the exposure, this rule prevents the exposure from going to ISO. There must be exposure coverage for ISO to be enabled.

EXV05300 - Coverage Subtype

If there is no coverage subtype for the exposure, this rule prevents the exposure from going to ISO. There must be a coverage subtype for ISO to be enabled.

EXV05400 - Description

This rule determines first if the claim for the exposure has a loss type that is not workers' comp. If so, the rule then tests to see if there is a description of the incident. If there is no incident description, the rule prevents the exposure from going to ISO.

The rule tests for the following specific incident descriptions:

- Vehicle incident
- Fixed property incident
- Incident

EXV05500 - Non Workers Comp

This rule determines if the claim for the exposure has a loss type that is not workers' comp. If so, the following child rules execute.

EXV05510 - Claimant Exists

This rule tests to see if there is a claimant for the exposure. If there is no claimant, the rule prevents the exposure from going to ISO.

EXV05520 - Name

If there is a claimant, the rule checks if the person has a first and last name. If either is missing, the rule prevents the exposure from going to ISO. The claimant must have a first and last name for ISO to be enabled.

EXV05530 - Address

If there is a claimant, this rule performs a series of tests on the address of the claimant. If the address is missing the first line of the address or the city, the rule prevents the exposure from going to ISO. Additionally, if the country is US, CA (Canada), or null (no country), the State field must be filled in. All these fields must be filled in for ISO to be enabled.

EXV05600 - Bodily Injury Exposure

This rule determines if the exposure is of type bodily injury damage. If so, the following child rule executes.

EXV05610 - Injury

If there are no body parts described or if the first body part's details are missing, the rule prevents the exposure from going to ISO. A bodily injury exposure must have at least one body part indicated for ISO to be enabled.

EXV05700 - Vehicle Exposure

This rule determines if the exposure is of type vehicle damage. If so, the following child rules execute.

EXV05710 - Vehicle

This rule checks to see if any of the following vehicle incident fields are missing:

- `VehicleIncident.Vehicle`
- `VehicleIncident.Vehicle.Year`
- `VehicleIncident.Vehicle.Make`
- `VehicleIncident.Vehicle.Manufacturer`

If so, the rule prevents the exposure from going to ISO. A vehicle damage exposure must have at least the year and make specified for ISO to be enabled.

EXV05720 - Normal Vehicle

This rule tests the vehicle incident to see if the vehicle is *normal*: not an ATV, snowmobile, or boat. If so, the child rules execute.

EXV05721 - Vin Present

This rule tests the normal vehicle indicated for the incident to see if the vehicle identification number (VIN) exists. The loss party must not be a third party and the vehicle year must be after 1981 for this rule to apply. If there is no VIN number, the rule prevents the exposure from going to ISO.

EXV05722 - Vehicle Identification Number

This rule tests the normal vehicle indicated for the incident to verify that all the following conditions are true:

- The year is defined.
- The year is one that requires a VIN.
- The vehicle identification number is defined.
- The VIN has a valid format.

The rule uses constants defined in `gw.api.iso.ISOValidationConstants` to do these validations. If any of these tests fail, the rule prevents the exposure from going to ISO.

EXV05730 - Off-road Vehicle

This rule tests the vehicle incident to see if the vehicle is an ATV or snowmobile. If so, the child rules execute.

EXV05731 - Serial Number Present

This rule tests the off-road vehicle indicated for the incident to see if the vehicle serial number exists. If there is no serial number, the rule prevents the exposure from going to ISO.

EXV05732 - Off Road Type Present

This rule checks to see if the `OffRoadStyle` has been set for the vehicle associated with the incident. This value must be set for an ATV or snowmobile before the incident can go to ISO.

EXV05740 - Boat

This rule tests the vehicle incident to see if the vehicle is a boat. If so, the child rules execute.

EXV05741 - HIN Present

This rule tests the boat object indicated for the incident to verify that all the following conditions are true:

- The loss party is not a third party.
- The year is defined.
- The year is one that requires a hull identification number (HIN).
- The hull identification number is defined.

The rule uses constants defined in `gw.api.iso.ISOValidationConstants` to do these validations. If any of these tests fail, the rule prevents the exposure from going to ISO.

EXV05742 - HIN Format Correct

This rule tests the boat object indicated for the incident to verify that all the following conditions are true:

- The year is defined.
- The year is one that requires a hull identification number (HIN).
- The hull identification number is defined.
- The HIN has a valid format.

The rule uses constants defined in `gw.api.iso.ISOValidationConstants` to do these validations. If any of these tests fail, the rule prevents the exposure from going to ISO.

EXV05743 - Boat Type Present

This rule tests the vehicle incident to see if the type of boat is defined. If not, the rule prevents the exposure from going to ISO.

EXV05800 - LossOfUseDamage Exposure

This rule checks the exposure to see if the loss party or claimant is the same as the insured party. If not, the rule prevents the exposure from going to ISO.

EXV06000 - Settlement Date is Valid

This rule checks a workers' comp exposure to see if the settlement date is later than the current date—is in the future. If so, the rule prevents the exposure from going to ISO.

EXV07000 - Salvage

This rule checks the exposure to see if there was an amount recovered for salvage. If this amount is negative or zero, the rule rejects the exposure. A salvage recovery cannot be negative or zero, but it can be created as an ordinary payment instead.

EXV08000 - Incident changing

This rule prevents changing an exposure's incident if the exposure is related to one or more ClaimCenter services. This rule prevents the exposure from going to new loss completion.

EXV09000 - Deductibles

This rule ensures that deductible data validation enforcement is consistent whether an exposure's deductible is edited by a PATCH request submitted using ClaimCenter Cloud API or in the ClaimCenter user interface:

- If the overridden (modified) flag is set, or the deductible is waived, the edit reason cannot be null.
- Deductible amounts must be less than the coverage deductible amount on the policy.
- Deductible amount must be equal to its coverage's original amount when the deductible is not overridden (modified).
- The deductible cannot be waived if it is at least partially applied.

Note: For more information on deductibles, see the *Application Guide*.

Group Validation rules

ClaimCenter runs the Group Validation and Group Preupdate rules whenever it saves the Group entity. Any exceptions found during validation cause the bounding database transaction to roll back, thus effectively vetoing the update. In the base configuration, this rule set is empty.

Guidewire recommends that you use the LoadSave validation level for rules pertaining to validation for Users and Groups.

To access this rule set, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Validation > GroupValidationRules

Matter Closed Validation rules

The Matter Closed Validation rules execute whenever ClaimCenter closes a matter. These rules execute:

- Before ClaimCenter commits the data to the database.
- Before ClaimCenter executes the Matter Closed rule set.
- Before ClaimCenter executes the Matter Preupdate rule set.
- Before ClaimCenter executes the Matter Validation rule set.

Use these rules to raise validation errors or warnings if ClaimCenter must not close a matter for some reason.

To use this rule set, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Validation > MatterClosedValidationRules

In the base configuration, this rule set contains the following rule.

MCV01000 - Close Before Trial

This rule is provided as a test of validation warnings in Closed validation rules.

The rule determines if the trial date is in the future. If so, the matter cannot be closed because the trial has not yet occurred.

Matter Reopened Validation rules

The Matter Reopened Validation rules execute whenever a matter is reopened and before ClaimCenter commits the data to the database. The Matter Reopened Validation rules execute:

- Before ClaimCenter inserts a new or updated entity into the database.
- Before ClaimCenter executes the Matter Reopened rule set.
- Before ClaimCenter executes the Matter Preupdate rule set.
- Before ClaimCenter executes the Matter Validation rule set.

You can use these rules to raise validation errors or warnings if ClaimCenter must not reopen a matter for some reason. In the base configuration, this rule set is empty.

To access this rule set, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Validation > MatterReopenedValidationRules

Matter Validation rules

The rules in this rule set ensure that Matter data stored in ClaimCenter is sufficient and valid for downstream processing. ClaimCenter does not enforce validation levels on Matter objects, so the Matter Validation rule set always executes, regardless of the validation level.

To this rule set, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Validation > MatterValidationRules

In the base configuration, this rule set is empty.

Person Validation rules

The single rule in the Person rule set ensures that the phone number on the Person entity has valid countries defined for it. Guidewire disables this rule in the base configuration.

To this rule set, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Validation > PersonValidationRules

In the base configuration, this rule set contains the following rule.

Cell phone has valid country code

Guidewire disables this rule in the base configuration.

This rule checks to see if a cell phone number is defined for the contact. If there is one, the rule verifies that the country code is valid. If it is not, the contact is rejected.

Policy Validation rules

The rules in Policy Validation rule set ensure that policy data stored in ClaimCenter is sufficient and valid for downstream processing. ClaimCenter does not enforce validation levels on Policy objects, so the Policy Validation Rule Set always fires, regardless of the validation level.

To access this rule set, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Validation > PolicyValidationRules

In the base configuration, this rule set contains the following rules.

POV01000 - Effective and Expiration Dates

This rule checks to see if the effective date of the policy is later than the expiration date. If so, the rule prevents the policy from being saved. The policy effective date cannot be later than the policy expiration date.

POV02000 - Original Effective Date

This rule checks to see if an original effective date exists, which means that the policy effective date has changed. If so, the rule then checks to see if this original date is later than the policy effective date. If so, the rule prevents the policy from being saved. The original effective date cannot be later than the policy effective date.

POV03000 - Endorsement Dates

This rule checks each policy endorsement to see if the effective date of the endorsement is after its expiration date. If so, the rule prevents the policy from being saved. The effective date of an endorsement cannot be later than the expiration date of the endorsement.

Coinsurance and layering

These rules ensure data integrity as it relates to the integration of policy data defined in the policy coinsurance agreements that ClaimCenter receives and sends to the billing system (BillingCenter). These rules check if the claim's policy is valid and many of them produce a warning during FNOL (New Claim wizard) but an error upon check creation. By disabling these rules, data integrity cannot be guaranteed.

For more information on coinsurance in ClaimCenter, see the *ClaimCenter Application Guide*.

Several of these rules rely on a Gosu enhancement file `PolicyAgreementRulesEnhancement.gsx` where the following Boolean properties are defined:

- `SingleOurPartyAmongstParties`
- `SingleLeadRoleAmongstParties`
- `ConsistentSharePercentageForParties`
- `SharePercentageTotalNotAbove100ForParties`
- `ConsistentSharePercentageTotalBelow100ForParties`

The rules that use these properties give a warning at newloss system validation level so the user is made aware of the validation issues with the coinsurance data on the policy but the policy data can still be retrieved and saved to the database. However, an error occurs at payment system validation level to prevent payments until validation issues are addressed.

These properties are used in the following rules and are described below:

POV04000 - Coinsurance and Layering

This rule checks if the coinsurance feature flag is enabled and the claim's policy agreement is not null. If these conditions are true, the rule set proceeds to the child rule (POV04100). If these conditions are not true, the rule exits the rule set.

POV04100 - Coinsurance Agreements

This rule checks if the claim's policy is a coinsurance policy agreement. If true, the rule set proceeds to the next six child rules. If false, the rule set proceeds to the next rule (POV04200).

POV04110 - Single Our Party Amongst Parties

This rule checks if exactly only one insurer is identified as Our Party in the coinsurance policy agreement (as defined in the SingleOurPartyAmongstParties property). If not, the rule prevents the policy from being saved with an error at payment system validation level but gives a warning only at newloss system validation level.

POV04120 - Single Coinsurance Lead Amongst Parties

This rule checks if exactly only one insurer is identified as the Lead in the coinsurance policy agreement (as defined in the SingleLeadRoleAmongstParties property). If not, the rule prevents the policy from being saved with an error at payment system validation level but gives a warning only at newloss system validation level.

POV04130 - Each Party Share Percentage

This rule checks if each party's share on the agreement is greater than zero and less than or equal to 100% (as defined in the ConsistentSharePercentageForParties property). If not, the rule prevents the policy from being saved with an error at payment system validation level but gives a warning only at newloss system validation level.

POV04140 - Total Share Percentage Above 100%

This rule checks if the sum of all parties' shares on the agreement is greater than 100% (as defined in the SharePercentageTotalNotAbove100ForParties property). If it is, the rule prevents the policy from being saved with an error at payment system validation level but gives a warning only at newloss system validation level.

POV04150 - Total Share Percentage Below 100%

This rule checks if the sum of all parties' shares on the agreement is less than 100% (except if the coinsurance model is **Broker Collection**, **Own Claim Share** or **Our Role is Follow**) (as defined in the ConsistentSharePercentageTotalBelow100ForParties property which uses a typefilter in the CoinsuranceModel typelist). If it is, the rule prevents the policy from being saved with an error at payment system validation level but gives a warning only at newloss system validation level.

POV04160 - Policy Account Number is Required

This rule checks if the value of the policy's account number field (Policy.AccountNumber) is null. If it is, the rule prevents the policy from being saved with an error at payment system validation level but gives a warning only at newloss system validation level. The account number field is highlighted on the screen so the user can provide a value.

POV04200 - Non Coinsurance Agreements

This rule checks if the claim's policy is not a coinsurance policy agreement. If so, the rule's action is triggered. If not, the rule exits the rule set.

The rule action checks if the policy agreement's model is not null or the number of parties is greater than zero. If this is true, then, the policy agreement is not a coinsurance agreement, and therefore the coinsurance role, model, share, and parties cannot be defined. The rule prevents the policy from being saved with an error at payment system validation level but gives a warning only at newloss system validation level.

RIAgreement Validation rules

Use the rules in the RIAgreement Validation rule set to ensure that reinsurance agreement data is sufficient and valid.

To access this rule set, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Validation > RIAgreementValidationRules

In the base configuration, this rule set contains the following rules.

RIAV01000 - Required Fields

This rule serves as a parent to group the following child rules.

RIAV01005 - General

This rule determines if the reinsurance agreement is not a facultative proportional agreement. If not—for example, the agreement is a treaty that is non-proportional—the `TopOfLayer` field, represented in ClaimCenter as the coverage limit, must have a value. If this field is null, the agreement cannot be saved.

RIAV01010 - Non-Proportional

This rule first determines if the agreement is a non-proportional agreement. If so, there must be values for the attachment point and the ceded share. If either or both of these values are null, the agreement cannot be saved.

RIAV01020 - Proportional

This rule determines if the agreement is a proportional treaty. If so, its child rules execute.

RIAV01021 - Quota Share

This rule determines if the agreement is a quota share treaty. If so, the value of the ceded share must not be null. If it is, the agreement cannot be saved.

RIAV01022 - Surplus

This rule determines if the agreement is a surplus treaty. If so, the value of the attachment point must not be null. If it is, the agreement cannot be saved.

RIAV01030 - Agreement Group

This rule checks to see if the reinsurance group for the agreement is null, and, if so, if reinsurance codings are also not specified. If there are no reinsurance codings, the agreement is assumed to be new. A new reinsurance agreement must have a reinsurance group specified before the agreement can be saved.

RIAV02000 - Unique Agreement Name

This rule determines if the agreement is part of an agreement group. If so, the agreement must have a unique name in the group before it can be saved.

RIAV03000 - Proportional Agreement Percentage Total

This rule determines if the agreement is in an agreement group and if the agreement is proportional. If so, the total proportional share percentage of all the agreements in the group cannot be greater than 100%. If the total proportional share exceeds 100%, the agreement cannot be saved.

RITransactionSet Validation rules

You can use the RITransactionSet Validation rule set to ensure that reinsurance transaction data is sufficient and valid.

To access this rule set, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Validation > RITransactionSetValidationRules

In the base configuration, this rule set contains the following rules.

RITSV02000 - Ceded Reserves Are Positive

This rule retrieves all the agreements for the reinsurance ceded reserve transactions in the transaction set. It then determines for each agreement if either or both of the following conditions that will cause the rule to prevent the transaction from proceeding are true:

- Calculated (total non-adjusted) ceded reserves are less than zero.
- Total ceded reserves are less than zero.

RITSV03000 - Ceded Reserves Less Than Open Reserve

This rule retrieves all the agreements for the reinsurance ceded reserve transactions in the transaction set. It then determines for each agreement if either or both of the following conditions that will cause the rule to prevent the transaction from proceeding are true:

- There are calculated (total non-adjusted) ceded reserves (they are greater than zero) and they are greater than the open reserves.
- There are total ceded reserves (they are greater than zero) and they are greater than the open reserves.

RITSV04000 - RI Recoverables Are Positive

This rule retrieves all the agreements for the reinsurance recoverable transactions in the transaction set. It then determines for each agreement if either or both of the following conditions that will cause the rule to prevent the transaction from proceeding are true:

- Calculated (total non-adjusted) reinsurance recoverables are less than zero.
- Total reinsurance recoverables are less than zero.

RITSV05000 - RI Recoverables Less Than Payments

This rule retrieves all the agreements for the reinsurance recoverable transactions in the transaction set. It then determines for each agreement if either or both of the following conditions that will cause the rule to prevent the transaction from proceeding are true:

- There are calculated (total non-adjusted) reinsurance recoverables (they are greater than zero) and they are greater than the net amount paid.
- There are total reinsurance recoverables (they are greater than zero) and they are greater than the net amount paid.

ServiceRequest Validation rules

Use the rules in the Service Request rule set to ensure that service request data is valid for downstream processing. ClaimCenter does not enforce validation levels on ServiceRequest objects. Errors will cause validation to fail, regardless of the validation level.

To access this rule set, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Validation > ServiceRequestValidationRules

In the base configuration, this rule set contains the following rules.

SRVR01000 - Service Request History

This rule stops a service request and notifies the user if the data in the service request history is not consistent.

SRVR02000 - Service Request Invoice Currencies

This rule stops a service request and notifies the user if the currency of the service request invoice does not match the currency of the associated check.

In the base configuration, this currency compatibility check is also done for checks in the transaction validation rule set. The constraint is enforced on both sides, by service requests and checks, in case only one of the other validation rules is running during a commit.

TransactionSet Validation rules

ClaimCenter runs the Transaction Set Validation rules whenever ClaimCenter saves a TransactionSet object. ClaimCenter does not enforce validation levels on Transaction objects. Thus, the Transaction Set Validation rule set always fires, regardless of the validation level.

[See also](#)

- “Transaction Set validation rules” on page 201

To access this rule set, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Validation > TransactionSetValidationRules

In the base configuration, this rule set contains the following rules.

TXV01000 - Total payment not > Exposure limit

This rule either blocks completely or permits, with warnings, a payment that would cause one of the following conditions:

- Making a new payment and adding it to the payments already made would cause the Total Payments to exceed the Exposure limit.
- Making a new payment and adding it to the payments already made plus the remaining reserves would cause the Total Payments to exceed the exposure limit.

The action of the rule depends on the settings of its warning1 and warning2 variables.

TXV02000 - Check exp limit when increasing reserves

This rule executes whenever you attempt to create a new reserve. In the base configuration, the rule either blocks or permits, with warnings, a reserve increase that causes the total reserves to exceed the exposure limit. For example the sum of the combined current payments, the new reserve amount, and remaining reserve exceeds the exposure limit.

The action of the rule depends on the settings of its warning variable.

TXV03000 - Total Payments not > Incident limit

This rule executes whenever you attempt to create a new check. In the base configuration, the rule either blocks or permits with warnings a payment that causes either total payments or total payments and remaining reserves to exceed the incident limit.

The action of the rule depends on the settings of its warning1 and warning2 variables.

TXV04000 - Check incid limit when increasing reserves

This rule executes whenever you attempt to create a new reserve. In the base configuration, the rule either blocks or permits, with warnings, a reserve increase that causes the total reserves to exceed the incident limit. For example the sum of the combined current payments, the new reserve amount, and remaining reserve exceeds the incident limit.

The action of the rule depends on the settings of its warning variable.

TXV05000 - PIP

This rule executes whenever you attempt to save a check. In the base configuration, the rule determines whether adding this new payment would cause the total payments to exceed one of the following limits:

- PIPNonmedAgg – Non-medical aggregate limit
- PIPReplaceAgg – Lost wages and replacement services aggregate limit
- PIPPersonAgg – PIP per person aggregate limit
- PIPClaimAgg – PIP per claim aggregate limit

TXV06000 - Reserve threshold

This rule provides a hard-coded limit of 1,000,000 in claim currency for creating a single reserve.

TXV07000 - Default CheckSet Rules

This rule executes whenever you attempt to create a check. It requires that a payment that is to be sent to the payee and is not electronic have an address to which the check can be mailed.

TXV08000 - Check Aggregate Limits

This rule notifies the user if a transaction would result in exceeding an aggregate limit. In the base configuration, ClaimCenter displays a warning in these cases, but allows the transaction to go through.

See also “TXV08000 – Check aggregate limits” on page 206.

TXV10000 - Check

Guidewire disables this rule in the base configuration.

If enabled, this rule prevents the user from writing checks for workers' comp exposures that have no coverage or for which the exposure has not reached the payment validation level.

TXV11000 - Block Payments that Exceed Reserves

Guidewire disables this rule in the base configuration.

If enabled, this rule executes whenever you attempt to create a new check. If the check pays claim costs, the rule blocks the check if it would cause the claim amount to exceed the available reserves.

TXV12000 - Pending Reserves & Matching Pending Payments

This rule looks for new reserves on reserve lines that have pending payments. If the newly created reserves would make remaining reserves less than pending payments, the rule sends the user a warning message.

TXV13000 - Warning if Adverse Party pays non-Subro Recovery

This rule prevents a recovery paid by a responsible party from being created if the recovery transaction has not been assigned the subrogation category.

TXV14000 - Check Send Date Validation

Guidewire disables this rule in the base configuration.

If enabled, this rule executes whenever you attempt to create a check. It verifies that all checks in the check set are scheduled to be sent on business calendar days, and not on weekends or holidays.

TXV15000 - Financial Holds

This rule filters for a transaction set that has financial holds and either is a reserve set or is a check set that does not have deleted or canceled checks. If so, the following child rules execute for this transaction set.

TXV15100 - Coverage In Question

This rule determines if the reserve set or check set is for a claim that has coverage in question. If so, the rule prevents the transaction from going through and sends an appropriate message to the user.

TXV15200 - Incident Only

This rule determines if the reserve set or check set is for a claim that is incident only. If so, the rule prevents the transaction from going through and sends an appropriate message to the user.

TXV15300 - Unverified Policy

Guidewire disables this rule in the base configuration.

If enabled, the rule determines if the reserve set or check set is for a claim that does not have a verified policy. If so, the rule prevents the transaction from going through and sends an appropriate message to the user.

TXV16000 - New Vendor Management Rules

Guidewire disables this rule in the base configuration.

If enabled, the rule determines if the transaction is for a check set that has changed. If so, the child rule executes if enabled.

TXV16100 - Vendor payees should be linked and synched in CM

Guidewire disables this rule in the base configuration.

If enabled, the rule determines if the check is a new check, a check that has a change in payee, or a check that is being approved. If so and there is a vendor type payee on the check, the rule lets the check go through only if the vendor is linked and synchronized with ContactManager.

TXV17000 - Reserve Lines compatible with Service Requests

This rule validates all checks linked to a service request invoice. The rule determines if each check's set of reserve lines is compatible with at least one of the service request invoices.

TXV18000 - Check currencies compatible with Service Request Invoices

This rule validates the currency for a check that is associated with a service request invoice. It prevents the check from being saved if the check and invoice currencies do not match.

TXV24000 - Disallow When Policy Agreement is inconsistent

This rule is to prevent transactions on claims when the claim's policy agreement has not reached the ability to pay validation level.

User Validation rules

ClaimCenter runs the User Validation and User Preupdate rules on a save of the User entity. Any exceptions found during validation cause the bounding database transaction to roll back, thus effectively vetoing the update.

Guidewire recommends that you use the LoadSave validation level for rules pertaining to validation for users and groups. To access this rule set, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Validation > UserValidationRules

In the base configuration, this rule set is empty.

Workplan

A *workplan* adds initial activities to a Claim, Exposure or Matter object as a checklist of work that various people need to perform. Guidewire broadly divides these activities into the following categories:

- Activities that are performed a single time for the entire claim
These activities include contacting the insured, investigating the accident scene, and getting a police report. You can add these types of activities as part of the claim's workplan rules and associate them with the claim as a whole.
- Activities that are performed for each exposure or matter
For example, these activities can include contacting the individual claimants and getting a damage estimate for each vehicle for an auto claim. You can add these types of activities as part of the exposure or matter workplan rules and associate them with the exposure or matter.

To work with these rules, navigate in the Studio **Project** window to **configuration > config > Rule Sets > Workplan**.

Gosu methods to use with Workplan rules

The purpose of Workplan rule set is to add the most important or the most commonly overlooked activities to the workplan for the claim. These rules are available to the following business entities:

- Claim
- Exposure
- Matter

The following list describes the main methods to use in workplan rules.

createActivity

Adds an activity based on an activity pattern, but allows you to override some of the defaults.

createActivityFromPattern

Adds an activity based on an activity pattern, using all of the pattern's defaults.

hasDuplicateClaimNumbers

Returns **true** if there are other claims that are possible duplicates of the claim being checked.

The `Claim.DuplicateClaimNumbers` property contains a comma-delimited list of claim numbers that might be duplicates of the current claim, or `null` if no duplicates were found.

Claim Workplan rules

The Claim Workplan rules add initial activities to the claim as a checklist of work that various people need to perform on the claim.

To access this rule set, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Workplan > ClaimWorkplan

In the base configuration, this rule set contains the following rules.

CLW0100 - Not First and Final

This rule is the parent rule for the child rules that follow. It filters out claims that are first and final because that type of claim is not likely to need further work.

CLW01000 - Contact insured

This Gosu rule is disabled in the base configuration because it is handled in the Business Rules editor.

CLW02000 - Thirty day review

This Gosu rule is disabled in the base configuration because it is handled in the Business Rules editor.

CLW03000 - New users

This Gosu rule is disabled in the base configuration because it is handled in the Business Rules editor.

CLW03100 - Verify coverage

This Gosu rule is disabled in the base configuration because it is handled in the Business Rules editor.

CLW04000 - Auto claims

This Gosu rule is disabled in the base configuration because it is handled in the Business Rules editor.

CLW04100 - Scene inspection

This Gosu rule is disabled in the base configuration because it is handled in the Business Rules editor.

CLW04200 - New users

This Gosu rule is disabled in the base configuration because it is handled in the Business Rules editor.

CLW04210 - Police report

This Gosu rule is disabled in the base configuration because it is handled in the Business Rules editor.

CLW05000 - Property claims

This Gosu rule is disabled in the base configuration because it is handled in the Business Rules editor.

CLW05100 - Property inspection

This Gosu rule is disabled in the base configuration because it is handled in the Business Rules editor.

CLW05200 - New users

This Gosu rule is disabled in the base configuration because it is handled in the Business Rules editor.

CLW05210 - Police report

This Gosu rule is disabled in the base configuration because it is handled in the Business Rules editor.

CLW05300 - Verify coverage

This Gosu rule is disabled in the base configuration because it is handled in the Business Rules editor.

CLW06000 - Liability claims

This rule determines if the claim loss type is GL for its child rules. For example, in the base configuration, the line of business for this loss type can be Businessowners, General Liability, Other Liability, or Personal Umbrella.

CLW06100 - New users

This rule determines if the user experience level is low. If so, any child rules can run.

CLW07000 - Workers comp claims

This rule determines if the claim is a Workers' Compensation claim for its child rules.

CLW07100 - Initial claim acceptance

This rule creates an activity to determine compensability based on the claim loss date, the report date, and the Workers' Comp compensability parameters.

CLW07200 - Contact claimant

This rule creates an activity to contact the claimant for the Workers' Comp claim.

CLW07300 - New users

This Gosu rule is disabled in the base configuration because it is handled in the Business Rules editor.

CLW07310 - Get Employee Injury Notice

This Gosu rule is disabled in the base configuration because it is handled in the Business Rules editor.

CLW08000 - BizRules

This rule executes the business rules that are set to trigger on creation of a claim. The rule runs after the completion of the other Gosu Claim Workplan rules.

Exposure Workplan rules

The Exposure Workplan rules add initial activities to the exposure as a checklist of work that various people need to perform on the exposure.

IMPORTANT: If you are adding activities for each exposure, avoid adding redundant activities. For example, if the same person is a claimant on multiple exposures, do not add a separate activity to contact the claimant for each exposure.

To access this rule set, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Workplan > ExposureWorkplan

In the base configuration, this rule set contains the following rules.

EXW01000 - Contact claimant

This Gosu rule is disabled in the base configuration because it is handled in the Business Rules editor.

EXW02000 - Auto exposure

This Gosu rule is disabled in the base configuration because it is handled in the Business Rules editor.

EXW02100 - Vehicle inspection

This Gosu rule is disabled in the base configuration because it is handled in the Business Rules editor.

EXW03000 - Property exposure

This rule determines if the exposure type is property damage. If so, any child rules execute. There are no child rules in the base configuration.

EXW04000 - Injury exposure

This Gosu rule is disabled in the base configuration because it is handled in the Business Rules editor.

EXW04100 - Medical report

This Gosu rule is disabled in the base configuration because it is handled in the Business Rules editor.

EXW04200 - IME

This Gosu rule is disabled in the base configuration because it is handled in the Business Rules editor.

EXW05000 - WC Injury Exposure

This Gosu rule is disabled in the base configuration because it is handled in the Business Rules editor.

EXW05100 - Medical report

This Gosu rule is disabled in the base configuration because it is handled in the Business Rules editor.

EXW06000 - WC Lost Wages Exposure

This Gosu rule is disabled in the base configuration because it is handled in the Business Rules editor.

EXW06100 - Wage Statement

This Gosu rule is disabled in the base configuration because it is handled in the Business Rules editor.

EXW07000 - Homeowners Policy

This Gosu rule is disabled in the base configuration because it is handled in the Business Rules editor.

EXW07100 - Get list of damaged items

This Gosu rule is disabled in the base configuration because it is handled in the Business Rules editor.

EXW07200 - Contact insured about living expenses

This Gosu rule is disabled in the base configuration because it is handled in the Business Rules editor.

EXW07300 - Get property inspected

This Gosu rule is disabled in the base configuration because it is handled in the Business Rules editor.

EXW07400 - Get claimant medical reports

This Gosu rule is disabled in the base configuration because it is handled in the Business Rules editor.

EXW08000 - BizRules

This rule executes the business rules that are set to trigger on creation of an exposure. The rule runs after the completion of the other Gosu Exposure Workplan rules.

[See also](#)

- *Application Guide*

Matter Workplan rules

The Matter Workplan rules add initial activities to the matter as a checklist of work that various people need to perform on the matter. In the base configuration, there are no rules in this rule set.

To access this rule set, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Workplan > MatterWorkplan

Example Matter Workplan rule

In the following example Matter Workplan rule, the rule creates an activity requesting that the user handling the claim refer the matter to legal counsel for review.

```
CONDITION (matter : entity.Matter):  
return true  
  
ACTION (matter : entity.Matter, actions : gw.rules.Action):  
matter.Claim.createActivityFromPattern( null, ActivityPattern.finder  
    .getActivityPatternByCode( "review_legal" ) )
```


PolicyCenter rule set Categories

As part of the PolicyCenter base configuration, Guidewire provides sample rules in the following rule set categories.

Assignment

Determine the responsible party for an activity, for example.

Audit

Generate activities related to audits.

Event Message

Handle communication with integrated external applications.

Exception

Specify an action to take if an Activity or Job is overdue and enters the escalated state.

Renewal

Govern behavior at decision points in the Renewal job flow, for example, accepting or rejecting a renewal request.

Validation

Check for missing information or invalid data on non-policy-related (platform) objects.

Assignment

Guidewire refers to certain business entities as *assignable* entities. For assignable entities, it is possible:

- To determine the party responsible for that entity
- To assign that entity through the use of assignment methods

Guidewire divides the assignment rules into two categories:

- Global assignment rules
- Default assignment rules

In the base configuration, there is both a global and default assignment rule for each assignable entity.

[See also](#)

- For information on global and default assignment rules and assignment in general, see “Assignment in InsuranceSuite applications” on page 153.
- For a description of the available assignment methods, see “Assignment method reference” on page 165.

Audit

PolicyCenter supports the following types audits.

Final audit

Covers the entire policy term. A final audit begins on the policy effective date and ends on the policy expiration or cancellation date.

Premium reporting

A series of non-overlapping periodic audits that you schedule and bill within the coverage period.

Reporting Trend Analysis rule set

The Reporting Trend Analysis rule set checks to see if the reporting trend analysis ratio is within an acceptable range. In the default configuration, the acceptable range is from 90% to 110%. If the ratio is outside the acceptable range and the number of reporting days is greater than 60, the code creates a `RatioOutOfRange` activity and assigns it to an underwriter.

See also

- For information about reporting trend analysis, see the *Application Guide*.

Event Message rules

IMPORTANT: PolicyCenter runs the Event Message rules as part of the database bundle commit process. Only use these rules to create integration messages.

In the base configuration, there is a single rule set—Event Fired—in the Event Message rule category. The rules in this rule set:

- Perform event processing
- Generate messages about the occurrence of events

PolicyCenter calls the Event Fired rules if an entity involved in a bundle commit triggers an event of interest to a message destination. A message destination registers an event to indicate its interest in that event.

As part of the event processing:

- PolicyCenter runs the rules in the Event Fired rule set a single time for every event of interest to a message destination.
- PolicyCenter runs the Event Fired rule set a single time for each destination that is listening for that particular event. Thus, it is possible for PolicyCenter to run the Event Fired rules sets multiple times for each event, one time for each destination interested in that event.

Warning

Be extremely careful about modifying entity data in Event Fired rules and messaging plugin implementations. Use these rules to perform only the minimal data changes necessary for integration code. Entity changes in these code locations do not cause the application to run or re-run validation or preupdate rules. Therefore, do not change fields that might require those rules to re-run. Only change fields that are not modifiable from the user interface. For example, set custom data model extension flags only used by messaging code.

Guidewire does not support the following:

- Guidewire does not support (and, it is dangerous) to add or delete business entities from Event Fired rules or messaging plugins (even indirectly through other APIs).
- Guidewire does not support—even within the Event Message rule set—calling any message acknowledgment or skipping methods such as `reportAck`, `reportError`, or `skip`. Use those methods only within messaging plugins.

- Guidewire does not support creating messages outside of the Event Message rule set.

Messaging events

PolicyCenter automatically generates certain events for most top-level objects. Guidewire calls these events *standard* events. In general, this occurs for any addition, modification, or removal (or retirement) of a top-level entity. PolicyCenter automatically generates the following events on the Activity object:

- ActivityAdded
- ActivityChanged
- ActivityRemoved

It is also possible to create a custom event on an entity by using the `addEvent` method. This method takes a single parameter, `eventName`, which is a `String` value that sets the name of the event.

```
entity.addEvent(eventName)
```

IMPORTANT: Do not create custom code that generates update events for any entity type that can possibly be part of the archive bundle.

Message destinations and message events

You use the Studio **Messaging** editor to define message destinations and message events.

- A *message destination* is an external system to which it is possible to send messages.
- A *message event* is an abstract notification of a change in PolicyCenter that is of possible interest to an external system. For example, this can be adding, changing, or removing a Guidewire entity.

Using the Studio editor, it is possible to associate (register) one or more events with a particular message definition. For example, in the base configuration, PolicyCenter associates the following events with the `ContactMessageTransport` message destination (ID=67):

- ContactAdded
- ContactChanged
- ContactRemoved

If you modify, add, or remove a `Contact` object (among others), PolicyCenter generates the relevant event. Then, during a bundle commit of the `Contact` object, PolicyCenter notifies any Event Message rule interested in the event of the occurrence of the event. You can use this information to generate a message to send to an external system or to the system console for logging purposes, for example.

Message destinations and message plugins

Each message destination encapsulates all the necessary behavior for an external system, but uses three different plugin interfaces to implement the destination. Each plugin handles different parts of what a destination does. Thus:

- The message request plugin handles message pre-processing.
- The message transport plugin handles message transport.
- The message reply plugin handles message replies.

You register new messaging plugins in Studio first in the Plugins editor. After you create a new implementation, Studio prompts you for a plugin interface name, and, in some cases, a plugin name. Use that plugin name in the Messaging editor in Studio to register each destination.

Note: You must register your plugin in two different editors in Studio, first in the Plugin Registry, and then in the Messaging editor.

IMPORTANT: After the PolicyCenter application server starts, PolicyCenter initializes all message destinations. PolicyCenter saves a list of events for which each destination requested notifications. As this happens at system start-up, you must restart the PolicyCenter application if you change the list of events or destinations.

Generating messages

Use method `createMessage` to create the message text, which can be either a simple text message or a more involved constructed message. The following code is an example of a simple text message that uses the Gosu in-line dynamic template functionality to construct the message. Gosu in-line dynamic templates combine static text with values from variables or other calculations that Gosu evaluates at run time. For example, the following `createMessage` method call creates a message that lists the event name and the entity that triggered this rule set.

```
messageContext.createMessage("${messageContext.EventName} - ${messageContext.Root}")
```

The following is an example of a constructed message that provides more detailed information if an external submission batch completes successfully.

```
var batch = messageContext.Root as SubmissionGroup
var subs = batch.Submissions

var payload : String
payload = "External Submission Batch "
payload = payload + "(" + batch.Submissions.length + " Submissions):"

for (sub in subs) {
    payload = payload + " " + sub.JobNumber
}

messageContext.createMessage(payload)
```

Listening for object changes that trigger events

It is possible that you want to listen for a change in an object that does not automatically trigger an event if you update it. For example, suppose that you want to listen for a change to the `User` object (an update of the address, perhaps). However, in this case, the `User` object itself does not contain an address. Instead, PolicyCenter stores addresses as an array on `UserContact`, which is an extension of `Person`, which is a subtype of `Contact`, which points to `User`. Therefore, updating an address does not directly in itself touch the `User` object.

However, in an Event Message rule, you can listen for the `ContactChanged` event that PolicyCenter generates every time that the address changes. The following example illustrates this concept. (It prints out a message to the system console anytime that the address changes. In actual practice, you would use a different message destination, of course.)

```
USES
uses gw.api.util.ArrayUtil

CONDITION
//DestID 68 is the Console Message Logger
return messageContext.DestID == 68 and messageContext.EventName == "ContactChanged"

ACTION
var message = messageContext.createMessage( "Event: " + messageContext.EventName )
var contact = messageContext.Root as Contact
var fields = contact.PrimaryAddress.ChangedFields
print( ArrayUtil.toString( fields ) )
```

Example Event Fired rule

In the following example, the rule constructs a message containing the account number any time that the application adds an account.

```
CONDITION (messageContext : entity.MessageContext):
// Only fire if adding account
return messageContext.EventName == "AccountAdded"

ACTION (messageContext : entity.MessageContext, actions : gw.Rules.Action):
// Create handle to root object.
var account = messageContext.Root as Account
```

```
// Create the message
messageContext.createMessage("Account [" + account.AccountNumber + "] added")
```

Example asynchronous document storage failed Event Fired rule

Guidewire recommends that, in working with asynchronous document storage, that you create an Event Fired rule to handle the failure of the database to store a document properly. If a document fails to store properly, PolicyCenter creates a `FailedDocumentStore` event on `Document`.

To be useful, you need to add the `FailedDocumentStore` event to a messaging destination in `messaging-config.xml`. For example, you can add this event to message destination 324, which already tracks the `DocumentStore` event.

You then need to create an event rule to process the event as you see fit. You can, for example:

- Email a notification message to the creator of the `Document` entity.
- Create an activity on the primary or root object.
- Attempt to correct the document issues and create a new `DocumentStore` message.

The following example rule illustrates a possible Event Fired rule to handle the failure of the database to store a document.

```
CONDITION (messageContext : entity.MessageContext):
return messageContext.DestID == 324 and messageContext.EventName == "FailedDocumentStore"

ACTION (messageContext : entity.MessageContext, actions : gw.Rules.Action):
var document = messageContext.Root as Document
var fixedIssue = false
var sendEmail = false

if (fixedIssue) {
    messageContext.createMessage( "DocumentStore" )
}
else if (document.CreateUser.Contact.EmailAddress1 != null && sendEmail) {
    gw.api.email.EmailUtil.sendEmailWithBody(document,
    document.CreateUser.Contact.EmailAddress1,
    document.CreateUser.Contact.getDisplayName(),
    null, null, "Failed Document Store",
    //CREATE MESSAGE BODY HERE )
}
```

Exception

The Exception rule set category encompasses both exception rules and escalation rules.

- PolicyCenter runs *escalation* rules on entities that are active past a certain date.
- PolicyCenter runs *exception* rules on all instances of an entity with the intent of finding and handling instances with some unusual condition. (However, the query can prioritize some kinds of instances over others.) One common use of the exception rules is to find policies that look unusual for some reason and generate activities directing someone to deal with them.

Both rule types involve batch processes invoking rule sets on entities. You can configure whether and how often PolicyCenter runs these rule sets. See *InsuranceSuite Server Administration* for information on exception batch processes.

Activity Escalation rules

PolicyCenter associates two specific dates with an activity.

Due date The target date to complete this activity. If the activity is still open after this date, it becomes overdue.

Escalation date The date at which an open and overdue activity becomes escalated and needs urgent attention.

PolicyCenter runs scheduled process `activityesc` every 30 minutes (by default). This batch process runs against all open activities in PolicyCenter to find activities to escalate, using the following criteria:

- The activity has an escalation date that is non-null.
- The escalation date is in the past.
- PolicyCenter has not yet escalated the activity.

PolicyCenter marks each activity that meets the criteria as escalated. After the batch process runs, PolicyCenter runs the escalation rules for all the activities that have hit their escalation date.

This rule set is empty by default. However, you can use this rule set to specify what actions to take any time that the activity enters the escalated condition. For example, you can use this rule set to reassign escalated activities. In the following example, the rule re-assigns the activity to the group supervisor if the escalated activity priority is set to urgent.

```
CONDITION (activity : entity.Activity):
return activity.Priority == Priority.TC_URGENT

ACTION (activity : entity.Activity, actions : gw.Rules.Action):
activity.assign( activity.AssignedGroup, activity.AssignedGroup.Supervisor )
```

Group Exception rules

PolicyCenter runs the Group Exception rules on a scheduled basis to look for certain conditions on groups that might require further attention. You can use these Gosu rules to define the follow-up actions for each exception found. This rule set is empty in the base configuration. To work with this rule set, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Exception > GroupExceptionRules

In the base configuration, Guidewire schedules the Group Exception work queue to run the group exception rules on all groups in PolicyCenter every day at 4:00 a.m. However, Guidewire disables the Group Exception work queue by default. You must enable the work queue by setting the number of workers for this work queue to a number greater than zero in file `work-queue.xml`.

Policy Exception rules

In the base configuration, Guidewire disables the Policy Exception rules by disabling the batch processes that run these Policy Exception rules.

If enabled, the rules look for certain conditions on `PolicyPeriod` entities that possibly require further attention. You can then define follow-up actions for each exception found. First, PolicyCenter identifies `PolicyPeriod` entities that changed since the last inspection, or, that it has not inspected for a long time. PolicyCenter then runs these rules on each `PolicyPeriod` chosen.

This rule set is empty by default in the base configuration. Implementers are free to create rules in this rule set category as necessary. However, it is important to understand:

- PolicyCenter persists any change made to the `PolicyPeriod` or its related objects through this rule set to the database.
- PolicyCenter runs the rules in this rule set category—one at a time—over a potentially large number of `PolicyPeriod` entities. This can have an adverse impact on performance.

Validation and Policy Exception rules

The Policy Exception rules perform no implicit validation. Any validation that you want to perform you must execute explicitly by calling the appropriate validation method. Guidewire strongly recommends that you take care to validate only as needed, for performance reasons. For instance, if the only change to a `PolicyPeriod` entity is to raise an alert in the form of a Note, Activity, or email, do not validate.

Policy Exception rules and PolicyPeriod entities

PolicyCenter runs the Policy Exception rules over every PolicyPeriod entity in the database except for bound periods that are no longer the most recent model (`PolicyPeriod.MostRecentModel == false`). Thus, you can run exception rules on withdrawn, declined, and bound revisions.

Data model and the PolicyPeriod entity

To store the time at which it last ran the exception rules, PolicyCenter maintain a separate PolicyException entity with two fields of interest:

- A non-nullable foreign key to PolicyPeriod
- A datetime field named ExCheckTime indicating the time at which the exception rules last ran

A PolicyPeriod never has more than one associated PolicyException entity. If PolicyPeriod entity has no associated PolicyException entity, it means that the exception rules have not yet been run on that particular PolicyPeriod entity.

How PolicyCenter updates ExCheckTime

Any time a batch process runs exception rules on a given PolicyPeriod entity, the process does the following:

1. It runs the exception rules on the PolicyPeriod.
2. It finds the PolicyPeriod entity's associated PolicyException entity, or it creates a PolicyException entity if it does not find one.
3. It sets PolicyException.ExCheckTime to the current system clock time.
4. It saves (commits) the changes to the database.

How PolicyCenter schedules the Policy Exception rules

Three separate batch processes control the time at which PolicyCenter runs the exception rules.

OpenPolicyException	Runs on all PolicyPeriod entities that are in an <i>open</i> status—Draft, Quoted, and Binding, for example
---------------------	---

BoundPolicyException	Runs on all PolicyPeriod entities in a <i>bound</i> status—only those that are the latest in model time
----------------------	---

ClosedPolicyException	Runs on all PolicyPeriod entities in a <i>closed</i> status—Withdrawn, Declined, and Not Taken, for example
-----------------------	---

All three processes invoke the same rule set. Therefore, if you want the exception behavior to vary based on the PolicyPeriod status, you need to test that status within the rules.

In the base configuration, Guidewire disables these batch processes by default. To enable a batch process, remove the comments around its entry in `scheduler-config.xml`. If enabled, PolicyCenter runs the default entries at the following times:

- OpenPolicyException runs every night at 2 a.m.
- BoundPolicyException runs on the first Saturday of each month at 12:00 noon
- ClosedPolicyException runs on the first Sunday of each month at 12:00 noon

You can use configuration parameters in `config.xml` to limit how often PolicyCenter runs the exception rules against a single PolicyPeriod entity. Use the following parameters:

Batch process	Use parameter...	Default (days)
OpenPolicyException	OpenPolicyThresholdDays	1
BoundPolicyException	BoundPolicyThresholdDays	14

Batch process	Use parameter...	Default (days)
ClosedPolicyException	ClosedPolicyThresholdDays	14

These parameters limit how many days must pass before the exception rules run a second time on any given PolicyPeriod entity. For instance, suppose that a given PolicyPeriod has a withdrawn status and ClosedPolicyThresholdDays is set to 30. After PolicyCenter runs the exception rules on that PolicyPeriod, the ClosedPolicyException process does not run rules on that particular policy again until at least 30 days have passed. Thus, these configuration parameters are useful as throttles for performance.

The following table summarizes this information:

Process	Default Schedule	Criteria
OpenPolicyException	Every night at 2 a.m.	PolicyPeriod entities that are unlocked and that have a PolicyException whose ExCheckTime is earlier than OpenPolicyExceptionThresholdDays days ago.
BoundPolicyException	First Saturday noon	PolicyPeriod entities that are bound (and MostRecentModel == true) and have a PolicyException whose ExCheckTime is earlier than BoundPolicyExceptionThresholdDays days ago.
ClosedPolicyException	First Sunday noon	PolicyPeriod entities that are locked but not bound and have a PolicyException whose ExCheckTime is earlier than ClosedPolicyExceptionThresholdDays days ago.

Errors generated by the Policy Exception rules

Generally, if any of the exception rules throw an error, PolicyCenter logs the exception. PolicyCenter rolls back any changes made by the exception rules to that PolicyPeriod. (This is not true, however, if you made the changes in a separate bundle, which Guidewire *explicitly* does not recommend.) PolicyCenter handles each PolicyPeriod over which it runs the exception rules in a separate transaction. Therefore, if PolicyCenter encounters an error in PolicyPeriod B, it does not roll back changes to PolicyPeriod A.

A special loophole exists in the case of the ConcurrentDataChangeError error, which occurs any time that two separate transactions try to change the same data. This can happen quite easily in exception rules. For instance, imagine that someone is the process of editing a PolicyPeriod entity at the same time that the exception rules try to make changes to it. If there is a ConcurrentDataChangeError, the batch process still rolls back the changes, but remembers the ID of the PolicyPeriod. It then tries to run exception rules on it again at a later time. This is standard behavior across all exception rules.

Minimizing performance issues with the Policy Exception rules

It is extremely easy to configure the exception rules so as to create performance issues, because the rules run on all PolicyPeriod entities of a given type. To minimize performance issues, Guidewire recommends the following:

- Put as little expensive logic into the exception rules as possible. The suggested approach is to check for a condition (which is relatively cheap to inspect). Then, if the condition is true, perform a relatively low-cost action (such as creating an Activity).
- Only enable an exception process (OpenPolicyException, ClosedPolicyException, or BoundPolicyException) in scheduler-config.xml if you have Policy Exception rules for that particular policy exception type. For example, if you have bound policy exception rules, then only enable the BoundPolicyException process.
- Schedule the processes to run as rarely as possible, and use the highest values possible for a xxPolicyThresholdDays configuration parameter.

Premium Report Escalation rules

In the base configuration, PolicyCenter runs the Premium Report Exception rules at 4:30 a.m. every morning. This rule set contains one default rule in the base configuration. The rule generates an activity and attempts assign the activity to

the policy underwriter if the premium report is overdue. If the assignment attempt fails, the rule assigns the activity to the Default User.

User Exception rules

In the base configuration, UserException batch processing runs the User Exception rule sets on all users within PolicyCenter at 3:00 a.m. every day.

User Exception batch processing looks for certain conditions on users that possibly require further attention. You can then use these rules to define follow-up actions for each exception found. This rule set is empty by default in the base configuration.

First, PolicyCenter identifies users that have changed since the last inspection, or, users that it has not inspected for a certain period of time. PolicyCenter then runs User Exception rules on each chosen user.

Renewal

The Renewal rules implement business logic around policy renewal. A policy renewal takes place before the policy period ends, on an active policy. A policy renewal continues the insurance for another time period. It can include changes to the following:

- Coverage
- Form update
- Industry or class code
- Underwriting company

It is not possible to use policy renewal to reinstate a canceled policy.)

Renewal AutoUpdate

The Renewal AutoUpdate rule set triggers during creation of a Renewal job. You can use this rule set to perform any automatic changes to policy data that need to take place during a renewal.

In the following example, the rule updates building limits for all buildings on a BOP policy line renewal.

```
CONDITION (policyPeriod : entity.PolicyPeriod):
return policyPeriod.BOLineExists

ACTION (policyPeriod : entity.PolicyPeriod, actions : gw.Rules.Action):
for (bld in policyPeriod.BOLine.BOPLineLocations*.Buildings) {
    if (bld.BOPBuildingCov != null) {
        var increase = bld.BOPBuildingCov.BOPBldgAnnualIncreaseTerm.Value
        if (increase != 0) {
            bld.BOPBuildingCov.BOPBldgLimTerm.Value =
                bld.BOPBuildingCov.BOPBldgLimTerm.Value * (1 + (increase/100))
            if (bld.BOPPersonalPropCov != null) {
                bld.BOPPersonalPropCov.BOPBPPBldgLimTerm.Value =
                    bld.BOPPersonalPropCov.BOPBPPBldgLimTerm.Value * (1 + (increase/100))
            }
        }
    }
}
```

Validation

IMPORTANT: PolicyCenter does not permit you to modify objects during validation rule execution that require changes to the database. To allow this would make it impossible to ever completely validate data. (Data validation would be a ever-shifting target.)

PolicyCenter performs rule-based validation as it commits a data bundle containing a validatable entity to the database. Rules-based validation works with non-policy objects only. To validate policy-related objects, use class-based validation.

Many, if not most, of the validation rule sets are empty by default. Guidewire expects you to create your own rules to meet your business needs.

See also

- “PolicyCenter Rule-based validation” on page 215

Validation in PolicyCenter

Within PolicyCenter, use the validation rules to identify problems with data that the user must fix before PolicyCenter permits the user to submit the item for further processing. You can use validation rules to do the following:

- Ensure that the user enters data that makes sense
- Ensure that the user enters all necessary data
- Manage relationships between data fields

For example, one validation check verifies that a producer code (501-002554) has at least one role associated with it. If it does not, PolicyCenter displays a warning:

ProducerCode has no roles. A role is required to permit users to be assigned to '501-002554'.

Guidewire makes the following distinction between warnings and errors.

Warning Warnings are non-blocking, for information purposes only. The user can ignore (clear) a warning and continue.

Error Errors are blocking. The user cannot ignore or clear an error. The user must fix all errors before continuing.

Use the `reject` method—in one of its many forms—to prevent continued processing of the object, and to inform the user of the problem and how to correct it. The following table describes some of the more common forms of the `reject` method.

Method form	Description
<code>reject</code>	Indicates a problem and provides an error message, but does not point the user to a specific field.
<code>rejectField</code>	Indicates a problem with a particular field, provides an error message, and directs the user to the correct field to fix.
<code>rejectFieldWithFlowStep</code>	Similar to <code>rejectField</code> , but also provides a mechanism to filter out errors not associated with a particular Job wizard step (<code>flowStepId</code>). PolicyCenter uses this field only if validating against the default validation level. Otherwise, it ignores this field.

The method signature can take one of the following forms:

```
reject(errorLevel, strErrorReason, warningLevel, strWarningReason)
rejectField(strRelativeFieldPath, errorLevel, strErrorReason, warningLevel, strWarningReason )
rejectFieldWithFlowStep(strRelativeFieldPath, errorLevel, strErrorReason, warningLevel, strWarningReason,
    flowStepId)
```

The following table lists the `reject` method parameters and describes their use. Issues or problems that the rules identify are either errors (blocking) or warnings (non-blocking). You can indicate a failure as both an error and a warning simultaneously. However, if the failure is both an error and a warning, use different error and warning levels for the failure.

Method parameter	Description
<code>errorLevel</code>	Corresponding level effected by the validation error.
<code>flowStepId</code>	Job wizard step to which this error applies, if any. PolicyCenter uses this field only if validating against the default validation level. Otherwise, it ignores this field. If used, PolicyCenter filters out the validation error unless the user is on that particular wizard step.

Method parameter	Description
	- To specify multiple wizard steps, use a comma-separated list. - To indicate this step and all later steps, place a dash at the end of the <code>flowStepId</code>.
<code>strErrorReason</code>	Message to show to the user, indicating the reason for the error.
<code>strRelativeFieldPath</code>	Relative path from the root entity to the field that failed validation. Using a relative path (as opposed to a full path) sets a link to the problem field from within the message shown to the user. It also identifies the invalid field on-screen.
<code>strWarningReason</code>	Message to show within PolicyCenter to indicate the reason for the warning.
<code>warningLevel</code>	Corresponding level effected by the validation warning.

Validation Rules and Entity Types

If you define a validation rule, Guidewire recommends that you define it in a rule set declared for a specific entity type. For example, define it in a rule set that declares `Policy`, `User`, or one of the other business entities. Although it is possible to access other business entities within that rule (other than the declared type), *only* call the `reject` method on the declared entity in the rule set.

Studio does not detect violations of this guideline. Thus, during validation, it is possible that some traversal of the objects to be validated can reset that entity's previous result. If this traversal is before the rule engine executes the entity's own rules, PolicyCenter would lose this particular reject.

Constructing Error Text Strings

Guidewire recommends that you use the `DisplayKey` class to return the value to use for the `strErrorReason` and `strWarningReason`. For example, the following code returns the message, "User has duplicate roles".

```
USES:
uses gw.api.locale.DisplayKey

CONDITION (...):
...

ACTION (...):
DisplayKey.get("Java.Admin.User.DuplicateRoleError")
```

This also works with display keys that require a parameter or parameters. For example, `display_LanguageCode.properties` defines the following display key with placeholder `{0}`:

```
Java.UserDetail.Delete.IsSupervisorError = Cannot delete user because that user is the supervisor
of the following groups\: {0}
```

Code `DisplayKey.get("Java.UserDetail.Delete.IsSupervisorError", GroupName)` then returns the message (with the rule retrieving the value of *GroupName* already):

Cannot delete user because they are supervisor of the following groups: Western Region

PolicyCenter validatable entities

PolicyCenter defines the following non-policy objects as validatable entities that trigger rules-based validation:

- Account
- Activity
- Contact
- Group
- Organization

- ProducerCode
- Region
- User

PolicyCenter runs full validation (that transverses the validation graph) on the following entities, in the listed order, as they are modified:

- Account entities
- Group and User entities
- Activity entities

How an entity triggers Validation rules

For an entity to trigger the validation rules, it must implement the `Validatable` delegate. This is true for an entity in the base PolicyCenter configuration or for a custom entity that you create.

PolicyCenter executes rule-based validation automatically any time that there is a bundle commit (to the database) with a direct or indirect change to a validatable entity. For example, if a user saves a newly created or modified activity (one of the validatable entities), PolicyCenter runs the Activity validation rule set. If there are any errors, the commit fails and PolicyCenter displays error messages to the user.

PolicyCenter executes the validation rules on indirect changes based on the Boolean attribute `triggersValidation` on foreign keys and arrays defined in a validatable entity's metadata definition file. For example, in the base configuration, Guidewire defines the `Assignment` array off the `Account` entity as triggering validation. Any change to the `Assignment` array causes PolicyCenter to execute the Account validation rules during a bundle commit to the database.

Triggering Validation rules on custom entities

It is possible to create validation rules for custom entities, meaning those entities that you create yourself and that are not part of the base Guidewire PolicyCenter configuration.

For an entity to trigger a validation rule:

1. The entity must implement the `Validatable` delegate. This entity must be a root entity, not a subtype of an entity. In short, any extension entity that you create that you want to trigger a validation rule must implement the following code:

```
<implementsEntity name="Validatable"/>
```
2. A validation rule must exist in the *Validation* rule set category that conforms to the following naming convention:

```
<custom_entity_name>ValidationRules
```

To summarize, a validation rule must exist in a validation rule set that reflects the custom entity's name and ends with `ValidationRules`. In other words, you must create a rule set with the same name as the extension entity and add the word `ValidationRules` to it as a suffix. You then place your rules in this rule set.

For example, if you create an extension entity named `NewEntityExt`, then you need to create a rule set to hold your validation rules and name it:

```
NewEntityExtValidationRules
```

The validate method

Guidewire PolicyCenter provides a `validate` method that you can use to trigger validation on a number of important business entities. You can use this method any place that you can use Gosu, including user interface PCF pages, Gosu rules, and Gosu classes, for example:

```
entity.Account.validate()
```

The `validate` method returns a `ValidationResult` object. (PolicyCenter does not persist this object to the database.) You can use this object, for example, with the following base configuration Gosu function to display validation errors in the user interface:

```
gw.api.web.validation.ValidationUtil
    .showValidationErrorsOnCurrentTopLocation( ValidatableBean, ValidationResult, ValidationLevel )
```

Account Validation rule example

The Account rule set contains validation rules for Account objects (as expected). Accounts are a special type of validatable entities. PolicyCenter does not automatically validate Account objects as it validates other high-level entities. Unlike other objects, changes to policy data do not automatically trigger account validation on bundle commit in rules-based validation. PolicyCenter validates an Account entity only on bundle commits that actually affect the Account entity, such as in the **Account Setup** pages.

In the following example, the rule rejects an account if the account holder contact information does not contain a value for the work phone number.

```
CONDITION (account : entity.Account):
return account.AccountHolderContact.WorkPhone == null

ACTION (account : entity.Account, actions : gw.Rules.Action):
account.reject( TC_LOADSAVE, "You must provide a work phone number for the primary contact.", null, null)
```


Rule reports

The application provides the ability to generate the following types of useful reports.

Rule Repository report

This type of report lists and describes the existing Gosu rules. You generate this type of report by running the `gwb genRuleReport` build command from a command prompt.

Rule Execution report

This type of report provides information about the runtime performance of specific application code. You run this report from the Server Tools **Guidewire Profiler** screens.

Rule Chrono report

This type of report provides stack traces for rules triggered within the application. You run this report from the Server Tools **Guidewire Profiler** screens.

About the Rule Repository report

To facilitate working with the Gosu rules, the application provides a command prompt tool to generate a report describing all the existing rules. This tool generates the following:

- An XML file that contains the report information
- An XSLT file that provides a style sheet for the generated XML file

After you generate these files, it is possible to import the XML file into Microsoft Excel, for example. You can also provide a new style sheet to format the report to meet your business needs.

Generate a Rule Repository Report

Procedure

1. Open a command prompt and navigate to the application installation directory.
2. Execute the following command:

```
gwb genRuleReport
```

Results

This command generates the following files:

```
build/rules/RuleRepositoryReport.xml
```

```
build/rules/RuleRepositoryReport.xslt
```

About the Profiler Rule Execution report

The Guidewire Profiler provides information about the runtime performance of specific application code. It can also generate a report listing the rules that individual user actions trigger within the application. The Profiler is part of the Guidewire-provided Server Tools.

To view a Rule Execution report in Guidewire Profiler, you need to enable Web Profiling, then view the report in the **Profiler Analysis** screen.

To access the Server Tools, Guidewire Profiler, and the rule execution reports, you must have administrative privileges.

Generate a Profiler Rule Execution Report

Procedure

1. Log into the application using an administrative account.
2. Access the Server Tools and click **Guidewire Profiler** on the menu at the left-hand side of the screen.
3. On the Profiler **Configuration** page, click **Enable Web Profiling for this Session**.
This action enables profiling for the current session only.
4. Navigate back to the application screens.
5. Perform a task for which you want to view rule execution.
6. Upon completion of the task, return to Server Tools and reopen the application Profiler.
7. On the Profiler **Configuration** page, click **Profiler Analysis**.
This action opens the default **Stack Queries** analysis page.
8. Under **View Type**, select **Rule Execution**.

Understanding the Profiler Rule Execution Report

After you enable the Web profiler and perform activity within the application, the profiler **Profiler Analysis** screen displays zero, one, or multiple stack traces. The following list describes the meaning these stack traces.

Stack trace	Profiler displays...
None	A message stating that the Profiler did not find any stack traces. This happens if the actions in the application interface did not trigger any rules.
One	A single expanded stack trace. This stack trace lists, among other information, each rule set and rule that the application executed as a result of user actions within the application interface. The Profiler identifies each stack trace with the user action that created the stack trace. For example, if you create a new user within the application, you see <code>NewUser -> UserDetailsPage</code> for its stack name.
Multiple	A single expanded stack trace and multiple stack trace expansion buttons. There are multiple stack trace buttons if you perform multiple tasks in the interface. Click a stack trace button to access that particular stack trace and expand its details.

Each stack trace lists the following information about the profiled code:

- **Time**
- **Name**
- **Frame (ms)**
- **Elapsed (ms)**

- **Properties and Counters**

Within the stack trace, it is possible to:

- Expand a stack trace completely by clicking **Expand All**.
- Collapse a stack trace completely by clicking **Collapse All**.
- Partially expand or collapse a stack trace by clicking the + (plus) or – (minus) next a stack node.

The profiler lists the rule sets and rules that the application executed in the **Properties and Counters** column in the order in which they occurred.

View rule information in the Profiler Chrono report

About this task

It is possible to use the Guidewire Profiler **Chrono** report to view frames filtered out by the **Rule Execution** report.

Procedure

1. Log into the application using an administrative account.
2. Access the Server Tools and click **Guidewire Profiler** on the menu at the left-hand side of the screen.
3. Generate a rule execution report.
4. Make a note of the following information from the rule exception report:
 - Name of the session
 - Name of the stack
 - Time offset
5. Select **Guidewire Profiler > Profiler Analysis > Web**.
6. Under **Profiler Result**, select **Chrono** from the **View Type** drop-down list.
7. Select the desired session.
For example:
2016/05/05 09:23:25 web profiler session
8. Select the desired stack.
For example:
DesktopActivities > AssignActivity
9. As you select a stack, the **Profiler** page shows additional information about that stack at the bottom of the page.
10. Expand the **Time** node as needed to select the time offset that is of interest to you.

Assignment in InsuranceSuite applications

Note: In the following topics, the *application* refers to an InsuranceSuite application such as BillingCenter, ClaimCenter, PolicyCenter, or ContactManager when the information may apply to more than one InsuranceSuite application.

IMPORTANT: In the base configuration, the application provides assignment methods that take only a user and that assign the entity to the default group for that user. Guidewire deprecates these types of assignment methods. Do not use them. Guidewire recommends that you rework any existing assignment methods that only take a user into assignment methods that take both a user and a group.

At its core, the concept of assignment in the application is basically equivalent to ownership. The user to whom you assign an activity is the user who owns that activity, and who, therefore, has primary responsibility for it. For example, ownership of an Account, Claim, or Policy entity works in a similar fashion. Guidewire defines an entity that someone can own in this way as assignable.

Assignment in the application is usually made to a user and a group, as a pair (group, user). However, the assignment of a user is independent of the group. In other words, you can assign an entity to a user who is not a member of the specified group.

Assignment entity persistence

The application persists an assignment anytime that you persist the entity being assigned. Therefore, if you invoke the assignment methods on an entity from Gosu code, you must persist that entity. For example, you can persist an entity as part of a page commit in the application interface.

Primary and secondary assignment

The application distinguishes between two different types of assignment:

Primary assignment

Also known as user-based assignment, primary assignment assigns an entity to a single owner only. The entity must implement either the Assignable delegate or the BCAssignable (BillingCenter), CCAssignable (ClaimCenter), or PCAssignable (PolicyCenter) delegate, or both.

Secondary assignment

Also known as role-based assignment, secondary assignment assigns an object to a particular user role. Each role is held by a single user at a time, even though the user who holds that role can change over time. The entity must implement the `RoleAssignments` array.

Assignable entities in BillingCenter

In the BillingCenter base configuration, only the following assignable entities implement the `RoleAssignments` array:

- `DelProcessWorkflow`
- `Disbursement`
- `TroubleTicket`

All other assignable entities use primary assignment.

It is also possible for an assignable entity to be both primarily and secondarily assignable. For example, in the BillingCenter base configuration, the `TroubleTicket` entity implements the `Assignable` delegate, the `BCAssignable` delegate, and the `RoleAssignments` array.

Assignable entities in ClaimCenter

It is possible for an assignable entity to be both primarily and secondarily assignable. For example, in the ClaimCenter base configuration, the `Claim` entity implements the `Assignable` delegate, the `CCAssignable` delegate, and the `RoleAssignments` array.

Assignable entities in PolicyCenter

In the PolicyCenter base configuration, only the `Activity` and `UserRoleAssignment` entities implement primary assignment. All other assignable entities use secondary assignment.

Primary (user-based) assignment entities in ClaimCenter and PolicyCenter

The application uses *primary assignment* in the context of ownership. For example, only a single user (and group) can own an activity. Therefore, an `Activity` object is primarily assignable. Primary assignment takes place anytime that the application assigns an item to a single user.

To be available for user-based or primary assignment, an entity must implement one or both of the following delegates:

- `Assignable` delegate
- `CCAssignable` delegate (ClaimCenter)
- `PCAssignable` delegate (PolicyCenter)

In the base application configuration, the following objects implement the required delegates:

- `Activity`
- `Claim` (ClaimCenter)
- `Exposure` (ClaimCenter)
- `Matter` (ClaimCenter)
- `ServiceRequest` (ClaimCenter)
- `Subrogation` (ClaimCenter)
- `UserRoleAssignment`

`CCAssignable` (ClaimCenter) or `PCAssignable` (PolicyCenter) implements the `Assignable` delegate.

It is common for the application to implement the `Assignable` delegate in the main entity definition file and the `CCAssignable` (ClaimCenter) or `PCAssignable` (PolicyCenter) delegate in an entity extension file.

Secondary (role-based) assignment entities in ClaimCenter and PolicyCenter

The application uses *secondary assignment* in the context of user roles assigned to an entity that does not have a single owner. For example, an entity can have multiple roles associated with it as it moves through the application, with each role assigned to a specific person. As each of the roles can be held by only a single user, the application represents the relationship by an array of `UserRoleAssignment` entities. These `UserRoleAssignment` entities are primarily assignable and implement the `Assignable` delegate.

Thus, for an entity to be secondarily assignable, it must have an associated array of role assignments, the `RoleAssignments` array. This array contains the set of `UserRoleAssignment` objects.

In the application base configuration, the following objects contain a `RoleAssignments` array:

- Account (PolicyCenter)
- Claim (ClaimCenter)
- Exposure (ClaimCenter)
- Job (and all its subtypes) (PolicyCenter)
- Policy (PolicyCenter)

For example, the Claim entity's `RoleAssignments` array in ClaimCenter:

```
<array
  arrayentity="UserRoleAssignment"
  desc="The user role assignments for this claim."
  exportable="false"
  name="RoleAssignments"
  triggersValidation="true"/>
```

For example, the Job entity's `RoleAssignments` array in PolicyCenter:

```
<array
  arrayentity="JobUserRoleAssignment"
  cascadeDelete="true"
  desc="Role Assignments for this job."
  exportable="false"
  name="RoleAssignments"
  setterScriptability="all"/>
```

For example, an entity—such as Account—does not have a single owner. Instead, an account has a set of `User` objects associated with it, each of which has a particular `Role` with respect to the Account. As only a single `User` can hold each `Role`, PolicyCenter represents the relationships by a set of `UserRoleAssignment` entities.

To state this differently, it is possible to assign several different users to the same entity, but with different roles. It is also possible for the same user to have several roles on an entity. (Some common role types are Creator, Underwriter, and Producer.)

Determining the available roles on an Account, Job, or Policy object

PolicyCenter uses the following method to determine the available roles to display in the assignment drop-down as you edit an assignment on an account, job, or policy (the owner parameter):

```
gw.assignment.AssignmentUtil.filterAssignableRoles(owner, role)
```

The method filters the typekeys defined in the `UserRole` typelist by the specified role type (the `role` parameter). It returns `true` for those typekeys it determines are available in the assignment drop-down. For example, suppose that you start with the existing assignments on an account:

- Creator – Bruce Baker
- Producer – Bruce Baker
- Underwriter – Alice Applegate

If you want to add a new participant (assign a new role), then you need to do the following:

- Determine the set of roles in the `UserRole` typelist (Auditor, AuditExaminer, Creator, Producer, and so on).

- Pass each individual role type and the account (the owner in this case) to the `filterAssignableRoles` filter method.

As you call the method for the auditor role, the filter method determines that there is no currently existing user with that role on the account and returns `true`. PolicyCenter then displays the **Auditor** role in the assignment drop-down in the PolicyCenter interface. However, if you pass the producer role to the filter method, it determines that this role does currently exist on this account and returns `false`. Thus, PolicyCenter does not display the **Producer** role in the assignment drop-down in the PolicyCenter interface.

Permitting multiple users to share the same role on an entity

It is not possible—in the base configuration—for different users to share the same role on an entity. PolicyCenter enforces this behavior by setting a `UserRoleConstraint` category on each role in the `UserRole` typelist. For example, PolicyCenter associates the following `UserRoleConstraint` categories with the `AuditExaminer` role:

- `UserRoleConstraint.accountexclusive`
- `UserRoleConstraint.jobexclusive`
- `UserRoleConstraint.policyexclusive`

These constraints mandate that an account (or job or policy) can have at most one user assigned to the `AuditExaminer` role. Internally, the `filterAssignableRoles` method uses these constraints to enforce this behavior. Thus, to permit multiple users to share the same role on an entity, you need to remove that restriction (category) from that role type in the `UserRole` typelist. For example, to remove the restriction that an account can have only one associated user with the `AuditExaminer` role, simply remove that category (`UserRoleConstraint.accountexclusive`) from the `AuditExaminer` role.

To determine the categories set on a role, select a typecode (a role) from the `UserRole` typelist and view the **Categories** pane.

Secondary assignment and round-robin assignment

Secondary assignment uses the assignment owner to retrieve the round-robin state. What this means is that different secondary assignments on the same assignment owner can end up using the same round-robin state, and they can affect each other.

In general, if you use different search criteria for different secondary assignments, you do not encounter this problem as the search criteria is most likely different. However, if you want to want to make absolutely sure that different secondary assignments follow different round-robin states, then you need to extend the search criteria. In this case, add a flag column and set it to a different value for each different kind of secondary assignment.

Assignment within the Assignment rules

All assignment rules in the **Assignment** folder use the Assignment engine. In working within the context of the assignment rules, use the following assignment properties.

Primary assignment

`entity.CurrentAssignment`

This property returns the current assignment, regardless of whether you attempt to perform primary or secondary assignment. If you attempt to use it for secondary assignment, then the property returns the same object as the `CurrentRoleAssignment` property.

Note: `CurrentAssignment` is deprecated in ClaimCenter only. Do not use this property in ClaimCenter.

Secondary assignment

`entity.CurrentRoleAssignment`

This property returns `null` if you attempt to use it for primary assignment.

Secondary assignment and the Assignment rules in ClaimCenter

UserRoleAssignment entities share the rule set of their primary entities. Thus, the application assigns the UserRoleAssignment entities for an activity using the Activity Assignment rule sets, and so on. For example:

```
Activity.assignUserAndDefaultGroup( user )
```

It is important for rules to use this syntax, because it allows the application to use the rules (in this case) for both activities and activity-related UserRoleAssignment entities.

Secondary assignment and the Assignment rules in BillingCenter and PolicyCenter

UserRoleAssignment entities share the rule set of their primary entities. Thus, the application assigns the UserRoleAssignment entities for an activity using the Activity Assignment rule sets, and so on. It is for this reason that assignment statements in the rules always use the CurrentAssignment formulation, such as the following:

```
Activity.CurrentAssignment.assignUserAndDefaultGroup( user )
```

It is important for rules to use this syntax, because it allows the application to use the rules (in this case) for both activities and activity-related UserRoleAssignment entities.

Assignment outside the Assignment rules

It is possible to assign an entity outside of the assignment rules, which use the application Assignment engine. For example, in ClaimCenter, you can assign an entity in arbitrary Gosu code, outside of the assignment rules.

```
activity.assign(group, user)
```

In BillingCenter and PolicyCenter, if you assign an entity outside the Assignment engine, then you do not need to use `entity.currentAssignment` to retrieve the current assignment. Instead, you can use any of the methods that are available to entities that implement the Assignable delegate directly, for example:

```
activity.assign(group, user)
```

Global and default Assignment rules in BillingCenter and ClaimCenter

Guidewire provides a number of sample assignment rules in the base configuration. You can access these sample rules in Guidewire Studio, in the **Assignment** rule set category folder. All assignment that takes place in the context of the Assignment rules uses the application Assignment engine to execute the assignment.

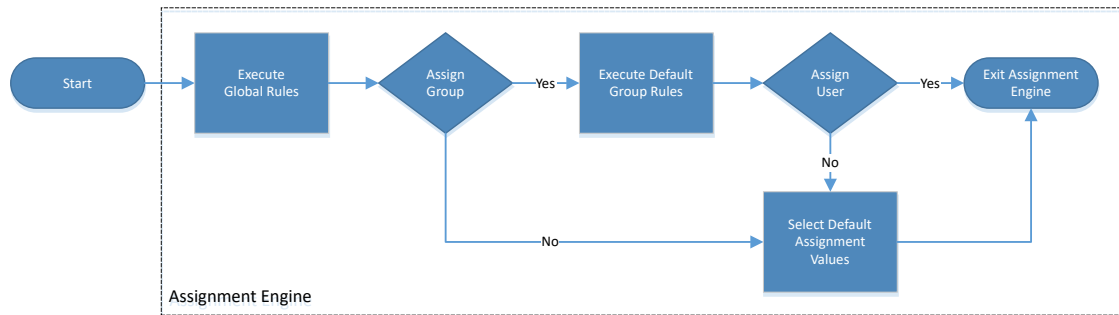
The application groups the Assignment rules into the following categories:

- Global assignment rules
- Default group assignment rules

In general, the application uses the global assignment rules to determine the group to which to assign the entity. It then runs the default group assignment rules to assign the entity to a user in the chosen group. The Assignment engine runs the assignment rules until it either completes the assignment or it runs out of rules to run.

Guidewire designs the assignment rules to trickle down through the group structure. For example, if a user assigns an activity using automated assignment, the Assignment engine runs a global activity assignment rule first. Typically, the global rule sets a group. Then, the Assignment engine calls the assignment rule set for that group. This rule set can either perform the final assignment or assign the activity to another group. If assigned to another group, the application calls the rule set for that group. This process continues until the Assignment engine completes the final assignment. (For simplicity, these examples use activity throughout, although the same logic applies to all assignable entities.)

The following graphic illustrates this process.



To support this process, the assignment engine contains the sense of *execution session* and *current group*. You cannot explicitly set these items.

- The *execution session* is one complete cycle through the assignment rules. In the Assignment engine flowchart, the execution session means all the processes that occur between the Start point and the Exit Assignment Engine termination point.
- The *current group* is the group to which the Assignment engine assigns the assignable entity after it executes an assignment rule set.

Unintended assignments

The Assignment engine sets the current group as it exits the rule set. To avoid unintended assignments, Guidewire recommends that you exit the rule set immediately after making a group assignment.

Global Assignment rules

The application triggers the Global assignment rules whenever you use auto-assignment to assign an entity. The main purpose of the Global assignment rules is to assign an entity (for example, an activity) to a specific group.

In the base configuration, BillingCenter provides a single default sample rule in each of the Global Activity and Trouble Ticket assignment rule sets. Each default rule attempts to assign the object to a group by round-robin assignment. BillingCenter triggers the global assignment rules whenever you use auto-assignment to assign an entity.

In the base configuration, ClaimCenter provides a number of sample rule sets in the Global assignment rules. Thus, there are is a rule set each to handle activities, claims, exposures, matters, service requests, and subrogation. The Assignment engine runs the rules in the Global rule set a single time, in a sequence set by the rule set hierarchy. If all of the initial Global rules fail to assign a group, the Assignment engine runs the Default rule set.

ClaimCenter runs rules in the Segmentation rule set category just prior to running assignment rules whenever you select automated assignment for a new claim or exposure. See “Segmentation” on page 100 for details.

While it is possible, Guidewire recommends that you do not make user assignments directly using the Global assignment rules. Instead, use these rules to trigger further group selection rules. For example, suppose that you make an assignment directly to a user who does not have the requisite job role, or requisite authority. In this case, there is no other rule that governs the situation, and the Assignment engine makes no assignment.

It is possible for you to implement your own assignment class and call it directly from within the application, if you choose. Guidewire does not restrict you to using the base configuration Assignment rules only.

Global assignment success or failure

After the Global assignment rules complete, the Assignment engine checks to see if the rules succeeded in assigning the entity to a group. The following list describes the possible assignment outcomes.

Rules assigned the entity to a group

The Assignment engine calls the Default Group assignment rule set for that object type and uses rules in that rule set to assign an owner for the object.

Rules did not assign the entity to a group

The Assignment engine assigns the object to the Default User and to the root group of the application group hierarchy. The Assignment engine then exits.

Default Group Assignment rules

The Assignment engine calls the Default Group assignment rules to assign an entity to a user, after the Global assignment rules assigned the entity to a group.

In the base configuration of BillingCenter, Guidewire provides a single sample rule in each of the Default Group Activity and Trouble Ticket rule sets. Each of these sample rules assigns the entity by round-robin assignment to a user in the specified group.

In the base configuration, ClaimCenter provides a number of sample rule sets in the Default Group assignment rules. Thus, there are a rule set each to handle activities, claims, exposures, matters, service requests, and subrogation.

Default group assignment success or failure

After the Default Group assignment rules complete, the Assignment engine checks to see if the rules succeeded in assigning the entity to a user. The following list describes the possible assignment outcomes.

Rules assigned the entity to a user

The Assignment engine exits as the assignment of the entity is complete.

BillingCenter: Rules did not assign the entity to a user

The Assignment engine assigns the entity to the supervisor of the current group.

ClaimCenter: Rules did not assign the entity to a user, but did assign the entity to a new group

The Assignment engine re-executes the Default Group rules. You can use the Default Group rules to reassign the entity to different groups at various levels in the ClaimCenter group hierarchy.

ClaimCenter: Rules did not assign the entity to either a user or a new group

The Assignment engine assigns the entity to the supervisor of the current group.

Assignment execution session in BillingCenter and ClaimCenter

Guidewire provides a `gw.api.assignment.AssignmentEngineUtil` class that provides several useful helper assignment methods. Do not use these helper methods as you create new assignment logic. Instead, use this class to provide a bridge between any existing deprecated assignment methods without a group parameter and the current assignment methods that do take a group parameter.

The following helper method, in particular is useful in that it returns the default group stored in the `ExecutionSession`:

```
getDefaultGroupIDFromExecutionSession
```

Formerly, a number of (now deprecated) assignment methods did not use the `GroupBase` parameter, and instead relied on the implicit Assignment engine state. As you can invoke these methods outside of the Assignment engine, you cannot rely on this implicit state during assignment. Therefore, Guidewire requires that all assignment methods take a `GroupBase` parameter. You can use the `getDefaultGroupIDFromExecutionSession` method to retrieve the current `GroupBase` value in `ExecutionSession`.

The returned `GroupBase` value from the `getDefaultGroupIDFromExecutionSession` method is the result of the last run of the rule set (global or default), *instead of the current value*. If a rule changes the assigned group but does not exit (by calling `actions.exit`), the assigned group is different from all the following rules in the current rule set.

If, instead, you want the currently assigned group on an object, an `Activity` object for example, then use the following:

ClaimCenter:
activity.AssignedGroup

BillingCenter:
activity.CurrentAssignment.AssignedGroup

The AssignedGroup (ClaimCenter) or CurrentAssignment.AssignedGroup (BillingCenter) method returns the currently assigned group on the current assignment. If a rule sets the AssignedGroup value and does not exit (does not call actions.exit), all the following rules use this value for the current group.

As the return value of getDefaultGroupIDFromExecutionSession can be null, you need to check for this condition.

Deprecated methods

Do not use the following deprecated methods on AssignmentEngineUtil:

- getCurrentGroupFromES
- getCurrentGroupIDFromES

Role assignment in PolicyCenter

In the default configuration of the PolicyCenter application, Guidewire assigns specific roles to specific entities. In actuality, there are many more role available that you can use for assignment. If you need—or want—more roles, you can create them. Role assignment is completely configurable.

Role assignment and Submission Jobs

Assigning submission jobs entails involves the following:

- Setting the producer for the submission job
- Setting the underwriter for the submission job

As PolicyCenter binds the submission, it passes the Underwriter, Producer, and Creator roles to the created Policy. (By default, PolicyCenter automatically assigns the roles of Creator and Requestor to the user that creates the initial policy submission.)

How PolicyCenter sets the producer during assignment

The process of assigning a producer to a submission job involves the following decision tree:

1. First, attempt to assign a user (with a user type of Producer) using the ProducerCode entered during job creation:
 - a. Check if the CurrentUser is a Producer for that ProducerCode. If so, preferentially choose that CurrentUser.
 - b. Otherwise, use round-robin on the Group to search for users with the required Producer user type.
2. If the assignment method cannot find a suitable user, try to set a Producer from the Account producer codes.
3. If the assignment method cannot find a valid producer, set the Producer to the unrestricted user (su) and log a warning that the method performed a default assignment only.

How PolicyCenter sets the underwriter during assignment

The process of assigning an underwriter to a submission job involves the following decision tree:

1. First, attempt to assign the PreferredUnderwriter from the ProducerCode as the default underwriter for the Submission job.
2. If the assignment method cannot find an underwriter, attempt to set an Underwriter from ProducerCode.Branch.
 - a. Check if the CurrentUser is an underwriter for that branch. If so, preferentially choose that CurrentUser.
 - b. Otherwise, use round-robin on the Group to find a user. If round-robin selects a user that does not have a user type of underwriter, look for any user in the group that is an underwriter. Guidewire strongly

recommends that you put only underwriters into the branch to take advantage of the load-balancing properties of round-robin assignment.

3. If the assignment method cannot find a valid user, set the Underwriter to the unrestricted user (su) and log a warning that the method performed a default assignment only.

Role assignment and non-Submission jobs

If the `ProducerCode` changes during any job, PolicyCenter updates the Policy with the new `ProducerCodeOfService`. For any of the following (new) jobs started from the policy, PolicyCenter adds the policy's current Underwriter and Producer role assignments to the job by default.

- Cancellation
- Policy Change
- Reinstatement
- Renewal
- Rewrite

Audit Job Assignment

An Audit job is a special case. Instead of assigning a producer or an underwriter, PolicyCenter assigns an auditor at the beginning of the job. PolicyCenter chooses this auditor from the default organization.

Account Job Assignment

Account assignment is relatively simple. PolicyCenter merely sets the Creator role to `CurrentUser` and sets nothing else.

Policy/Pre-renewal Direction

Guidewire exposes the assignment methods underlying the Pre-Renewal Direction process in the **Pre-Renewal Direction** pages. Use method `getSuggestedPreRenewalOwners` to populate the **Related To** drop-down list. Selecting a value assigns it to `PreRenewalOwner`. The end result of assigning the Pre-Renewal Direction is a role assignment to `PreRenewalOwner`. Thus, the Policy maintains the assignments of Underwriters and Producers *and* maintains the assignment of the Pre-Renewal Owner as well.

Pre-Renewal Owner is set using explicit assignment and property getter methods, which all exist in `gw.assignment.PolicyAssignmentEnhancement`:

- `PreRenewalOwner` property getter
- `PreRenewalOwner` property setter
- `getSuggestedPreRenewalOwners`

Activity Job Assignment

To create a new activity and assign it, use the `JobAssignmentEnhancement.createRoleActivity` method, which assigns the activity to the user with the given role on a job. This method has the following signature:

```
createRoleActivity(role, pattern, subject, description)
```

This creates an activity with the given activity pattern, then assigns a user with the given role to that activity.

Gosu support for assignment entities in PolicyCenter

To facilitate the process of assigning entities, Guidewire provides the following Gosu classes and enhancements in the `gw.assignment` package:

Class or enhancement	Description
AssignmentUtil	Contains shared code for checking roles, logging assignments, and setting the default user.
AuditAssignmentEnhancement	Assign an auditor to an Audit job.
JobAssignmentEnhancement	Contains shared job assignments such as underwriter and producer, pushes assignments from job to policy, and contains Activity assignment helpers.
PolicyAssignmentEnhancement	Contains Pre-Renewal Owner assignment and suggestion.
ProducerCodeAssignmentEnhancement	Retrieves the assignment group for a user within the ProducerCode.
UserAssignmentEnhancement	Retrieves the default assignment group from the user directly or for a given list of group types.

Assignment success or failure in BillingCenter and ClaimCenter

There are two general types of assignment methods:

- Assignment methods that execute in assignment rules in the context of the Assignment engine
- Assignment methods that execute in Gosu code outside of the Assignment engine

Guidewire recommends that you always determine if assignment actually succeeded:

- In general, if you call an assignment method directly and it was successful, then you need do nothing further. However, you need to take care if you call an assignment method that simply assigns the item to a group, such as one of the `assignGroup` methods. In this case, it is almost always necessary to call another assignment method to assign the item to an actual user.
- If you call an assignment method in an Assignment rule, you can exit the rule (not consider any further rules), or perform other actions before exiting. For information on the different ways to exit a rule, see “Exiting a Gosu rule” on page 22.

Assignment success or failure within the context of rules

Rule-based assignment methods that execute directly within an assignment rule return `true` if the method makes a valid assignment. If the assignment does not make an assignment, the method returns `false`. If the method returns `false`, it is the responsibility of the calling code to determine what to do. For example, the code can assign the item to the group supervisor or to invoke another assignment method.

Assignment success or failure outside the context of rules

IMPORTANT: Do not attempt to make direct assignments to user `defaultowner`.

It is possible to invoke the Assignment engine from outside of the assignment rules by calling the `autoAssign` method. Calling this method from outside the assignment rules invokes the Assignment engine automatically.

If you invoke assignment and it is not possible to assign the item to a user, the default assignment is to the group supervisor. In some cases, it is not possible to make an assignment to a group supervisor. For example, this can be if the assigned group has no supervisor. If it is not possible to assign the item to the group supervisor, then the application assigns the item to the application default owner (`defaultowner`).

The application provides user `defaultowner` as the assignee of last resort. This user’s first name is `Default` and the last name is `Owner`, with a user name of `defaultowner`. Guidewire recommends, as a business practice, that you have someone in the organization periodically search for outstanding work assigned to user `defaultowner`. If the search finds one of these assignments, the searcher must reassign these items to a proper owner. Guidewire also recommends

that the Rule Administrator investigate why the application did not assign an item of that type, so that you can correct any errors in the rules.

Logging assignment activity

Guidewire recommends that you log the action anytime that you use one of the assignment methods.

The following example in `BillingCenter`, called in an Assignment rule, shows how to log assignment activity.

```
uses gw.api.system.BCLoggerCategory
...

ACTION: (activity.Activity, actions : gw.rules.Action):

    var logger = BCLoggerCategory.ASSIGNMENT

    if ( activity.CurrentAssignment.assignUserByRoundRobin(false, activity.CurrentAssignment.AssignedGroup) ) {
        logger.info("Assigned to" + activity.AssignedUser.DisplayName)
        actions.exit()
    }
```

The following example in `ClaimCenter`, called in an Assignment rule, shows how to log assignment activity.

```
uses gw.api.system.CCLoggerCategory
...

ACTION: (activity.Activity, actions : gw.rules.Action):

    var logger = CCLoggerCategory.ASSIGNMENT

    if ( activity.assignUserByRoundRobin(false, activity.AssignedGroup) ) {
        logger.info("Assigned to" + activity.AssignedUser.DisplayName)
        actions.exit()
    }
```

Placing the exit action inside the `if` block forces the application to continue to the next assignment rule if the current assignment is not successful. A rule is not successful if it cannot choose a group or user to which to assign the item. It is important that you plan an effective exit from a rule. Otherwise, the application can make further unanticipated and unwanted assignments. If you perform assignment from an Assignment rule, you need only to use the `actions.exit` method to exit the rule.

Assignment success or failure in PolicyCenter

All assignment methods return a Boolean value that is `true` if `PolicyCenter` successfully makes an assignment, and `false` otherwise. It is your responsibility to determine what to do if `PolicyCenter` does not make an assignment. For example, you can assign the item to the group supervisor or invoke another assignment method.

IMPORTANT: Do not attempt to make direct assignments to defaultowner.

Assigning activities to roles

To assign activities to roles, `PolicyCenter` uses `AssignmentUtil.DefaultUser` as the default user to use if the assignment fails. You can modify `AssignmentUtil.DefaultUser` to change the default user throughout `PolicyCenter`. You can also modify individual PCF files to use a different default user than `AssignmentUtil.DefaultUser`.

Determining success or failure

Guidewire recommends that you always determine if the assignment actually succeeded. In general, if you call an assignment method directly and it is successful, then you need do nothing further. However, you need to take care if you call an assignment method that simply assigns the item to a group (one of the `assignGroup` methods, for example). In this case, it might be necessary to call another assignment method to assign the item to an actual user.

Logging assignment activity

Guidewire recommends also that you log the action anytime that you use one of the assignment methods. Class `gw.assignment.AssignmentUtil` provides several helper methods for logging assignment changes. These include the following:

- `assignAndLogUserRole`
- `logUserRoleAssignment`

To see examples of the use of these logging methods, see `JobAssignmentEnhancement` in the same package. For example, the following method uses `assignAndLogUserRole` to log information about the assignment process:

```
/**
 * Assign roles from the Policy to the Job
 */
function assignRolesFromPolicy() {
    var producer = this.Policy.getUserRoleAssignmentByRole( "producer" )
    if (producer != null) {
        AssignmentUtil.assignAndLogUserRole( this, producer.AssignedUser, producer.AssignedGroup, "producer",
            "Job.assignRolesFromPolicy()" )
    }

    var underwriter = this.Policy.getUserRoleAssignmentByRole( "underwriter" )
    if (underwriter != null) {
        AssignmentUtil.assignAndLogUserRole( this, underwriter.AssignedUser, underwriter.AssignedGroup,
            "underwriter", "Job.assignRolesFromPolicy()" )
    }
}
```

Assignment cascading in BillingCenter and ClaimCenter

For certain top-level assignable entities, the application needs to re-assign the related subobjects anytime that it assigns the top-level entity.

For example, in `BillingCenter`, if you reassign a trouble ticket, `BillingCenter` reassigns all open activities connected to that trouble ticket. This includes all activities currently assigned to the same group and user as the trouble ticket.

For example, in `ClaimCenter`, if you reassign a claim, `ClaimCenter` does the following:

- `ClaimCenter` reassigns all open activities connected to that claim, including all activities currently assigned to the same group and user as the claim. `ClaimCenter` assigns the activities connected to the claim only, and not the activities connected to any linked exposure.
- `ClaimCenter` reassigns all non-closed exposures for the claim currently assigned to the same group and user as the claim.

This cascading behavior is Guidewire application-specific. You cannot configure it:

- If you call the assignment methods through the Assignment engine, the Assignment engine cascades the assignments at the end of the process, as the engine exits with a completed assignment.
- If you call the assignment methods directly, then the assignment methods perform cascading of the assignment immediately as each method exits. For this reason, if you perform assignment outside of the Assignment rules, Guidewire strongly recommends that your Gosu code first determine the appropriate assignee (or set of assignees). It can then call a single assignment method on the assignee (or set of assignees). Do not rely on the recursive structure assumed by the assignment rules to perform assignment outside of the assignment rules.

Assignment events

Anytime that the assignment status changes on an assignment, the application creates an assignment event. The following events can trigger an assignment change event:

- `AssignmentAdded`

- AssignmentChanged
- AssignmentRemoved

The following list describes these events.

Old status	New status	Event	Code
Unassigned	Unassigned	None	None
Unassigned	Assigned	AssignmentAdded	Assignable.ASSIGNMENTADDED_EVENT
Assigned	Assigned	AssignmentChanged	Assignable.ASSIGNMENTCHANGED_EVENT
Assigned	Unassigned	AssignmentRemoved	Assignable.ASSIGNMENTREMOVED_EVENT

Note: A change can trigger the AssignmentChanged event for any number of reasons. For example, it can happen if a field such as the assigned user, the assigned group, or the date changes.

Assignment method reference

Note: For the latest information and description on assignment methods, consult the Gosudoc. You can also place the Studio cursor within a method signature and press **Ctrl+Q**.

Guidewire divides the assignment methods into the following general categories.

BillingCenter and ClaimCenter: Assignment by assignment engine

Assigns an object outside of the Assignment rules.

ClaimCenter: Claim-based assignment

Assigns an activity based on the claim owner.

ClaimCenter and PolicyCenter: Condition-based assignment

Determines a set of qualified assignees and then uses round-robin assignment to assign the object among the members of the set.

Dynamic assignment

Uses custom logic to perform automated assignment under more complex conditions.

BillingCenter and ClaimCenter: Group assignment

Assigns an object to a user group only.

Immediate assignment

Assigns an object directly to the specified user and group.

ClaimCenter: Manual assignment

Creates a new activity directing the specified user to manually perform (or review) the assignment.

ClaimCenter: Proximity-based assignment

Assigns an object based on its proximity to a specific geographical location.

ClaimCenter and PolicyCenter: Queue assignment

Assigns an activity to a queue for later assignment to an actual person.

Round-robin assignment

Rotates through a set of users, assigning work to each in sequence.

Assignment by assignment engine in BillingCenter and ClaimCenter

It is possible to invoke the Assignment engine, from outside the Assignment rules, to assign an object. To do so, use the `autoAssign` method. However, do not call this assignment method from within the Assignment rules themselves.

autoAssign

```
public boolean autoAssign()
```

This method invokes the rules-based Assignment engine to assign the entity that called it. Call this method outside of the Assignment rules if you want to invoke the Assignment engine to carry out assignment. For example, in ClaimCenter, you can call this method as part of entity creation to perform the initial assignment.

```
var actv = Claim.newActivity( null, null )
actv.autoAssign()
```

IMPORTANT: Do not call this assignment method in the context of the Assignment rules. The method invokes the Assignment engine. Calling it within the context of the Assignment rules can potentially create an infinite loop. If you attempt to do so, the application ignores the method call and outputs an error message.

Claim-based assignment in ClaimCenter

Guidewire provides several assignment methods that assign an activity based on the claim owner.

assignToClaimOwner

```
void assignToClaimOwner()
```

This method assigns the assignable entity to the user and group of the associated claim. *It is your responsibility to verify that the entity has a link to an assigned claim.* If the resulting assignment is invalid for any reason, the method throws `IllegalAssignmentException`. Currently, Guidewire only supports this method for use with an activity, exposure, or matter.

The following example assigns an exposure to the claim owner.

```
var exposure : Exposure
exposure.assignToClaimOwner()
```

Guidewire recommends, before attempting to assign the assignable, that you test if it is possible to assign the assignable using this method. Use the following property on the assignable to determine the owning claim for the assignable.

```
entity.OwningClaim
```

If the return value is `null`, then it is not possible to assign the assignable using the `assignToClaimOwner` method. Otherwise, it returns the owning claim object.

assignToClaimUserWithRole

```
boolean assignToClaimUserWithRole(userRole)
```

This method assigns the assignable entity based on the role associated with that entity. The method performs the following steps in searching for a user with a matching role:

1. If assigning an activity associated with an exposure, the method looks at roles associated with Exposures first.
2. It next attempts to match the supplied `userRole` with a Claim `UserRole`.
3. If not assigning an activity associated with an exposure, the method then searches for a match with a `UserRole` associated with an Exposure.

During the search:

- If the search finds a match at any step, it stops.
- If the match is unique, the method performs the assignment and returns `true`.
- If the match is not unique, the method returns `false`.
- If the search goes through all steps without finding a match, the method returns `false`.

In the base configuration, you can assign the entity to any one of the following roles:

- Attorney
- Doctor
- Independent Appraiser
- Nurse Case Manager
- Police
- Property Inspector
- Related User
- Repair Shop
- Reporter
- Salvage Owner
- SIU
- SIU Investigator
- Subrogation Owner
- Supervisor
- Vendor
- Vehicle Inspector

Note: Guidewire defines the roles that a user can have on an assignable object in typelist UserRole.

The following example assigns an exposure to a claim owner with a user role of `siuinvestigator`, a claim fraud specialist that investigates potential claim fraud.

```
var exposure : Exposure
exposure.assignToClaimUserWithRole("siuinvestigator")
```

Condition-based assignment in ClaimCenter and PolicyCenter

Condition-based assignment methods follow the same general pattern:

- They use a set of criteria to find a set of qualifying users, which can span multiple groups.
- They perform round-robin assignment among the resulting set of users.

It is important to note:

- The application ties the round-robin sequence to the set of criteria, not to the set of users. Thus, using the same set of restrictions to find a set of users re-uses the same round-robin sequence. However, two different sets of restrictions can result in *distinct* round-robin sequences, even if the set of resulting users is the same.
- The application does not use this kind of round-robin assignment with the load factors maintained in the application **Administration** screen. Those load factors are meaningful only within a single group, and condition-based assignment can span multiple groups.

assignByUserAttributes

```
boolean assignByUserAttributes(attributeBasedAssignmentCriteria, includeSubGroups,
currentGroup)
```

This method assigns an assignable item to the user who best matches the set of user attribute constraints defined in the `attributeBasedAssignmentCriteria` parameter.

If no user matches the criteria specified by the method, the method assigns the item to the item owner. For example, if the method cannot determine a user from the supplied criteria, it assigns an activity to the owner of the claim (in ClaimCenter) or policy (in PolicyCenter) associated with the activity.

The `AttributeBasedAssignmentCriteria` object contains the following fields:

Group	If set, restricts the search to the indicated group. This can be null.
AttributeCriteria	An array of <code>AttributeCriteriaElement</code> entities.

The `AttributeCriteriaElement` entities represent the conditions to be met. If more than one `AttributeCriteriaElement` entity is present, the method attempts to assign the assignable entity to those users who satisfy all of them. In other words, the method performs a Boolean AND operation on the restrictions.

The `AttributeCriteriaElement` entity has a number of fields, which are all optional. These fields can interact in very dependent ways, depending on the value of `UserField`.

Field	Description
UserField	The the AttributeCriteriaElement behaves differently depending on whether the UserField property contains an actual value: • If set, then UserField must be the name of a property on the User entity. The method imposes a search restriction using the Operator and Value fields to find users based on their value for this field. • If null, then the method imposes a search restriction based on attributes of the user. The exact restriction imposed can be more or less strict based on the other fields set:
AttributeField	If set, this is the name of a property on the Attribute entity. The method imposes a search restriction using the AttributeValue field to find users based on the user having the appropriate value for the named field for some attribute.
AttributeType	If set, then the method tightens the AttributeField-based restriction to Attributes only of the indicated type.
AttributeValue	If set, then the method restricts the search to users that have the specified AttributeValue only.
State	If set, then the method restricts the search to users that have an Attribute with the indicated value for State.
Value	If set, then the method restricts the search to users that have the specified Value for an Attribute that satisfies the other criteria.

You use the `currentGroup` and `includeSubGroups` parameters to further restrict the set of users under consideration to certain groups or subgroups. The `currentGroup` parameter can be null. If it is non-null, the assignment method uses the parameter for the following purposes:

1. The assignment method maintains separate round-robin states for the search criteria within each group. This is so that the application can use the method for group-specific assignment rotations.
2. If the method selects a user, it uses the supplied group to determine the best group for the assignment:
 - If the user is as a member of multiple subgroups under the supplied group, the method assigns the object to the closest group in the hierarchy to the supplied group.
 - If the user is a member of multiple groups at the same level of the group hierarchy, the method assigns the object to one of these groups randomly.
 - If the user is not a member of any subgroup under the supplied group, the method assigns the object to the supplied group.
 - If the supplied group value is null, the method assigns the object to the first group it finds of which the user is a member. The user must belong to at least one group for this assignment to succeed.

At times, it is important that you assign a particular claim (in ClaimCenter) or policy (in PolicyCenter) to a specific user, such as one who speaks French or one who has some sort of additional qualification. It is possible that these specially qualified users exist across an organization, rather than concentrated in a single group.

The following examples show searches in ClaimCenter and PolicyCenter for all of User entities who have an AttributeType of language and Attribute value of French.

ClaimCenter:

```
var attributeBasedAssignmentCriteria = new AttributeBasedAssignmentCriteria()
var frenchSpeaker= new AttributeCriteriaElement()
frenchSpeaker.AttributeType = UserAttributeType.TC_LANGUAGE
frenchSpeaker.AttributeField = "Name"
frenchSpeaker.AttributeValue = "French"
attributeBasedAssignmentCriteria.addToAttributeCriteria( frenchSpeaker )
activity.assignByUserAttributes(attributeBasedAssignmentCriteria , false, activity.AssignedGroup )
```

PolicyCenter:

```
var attributeBasedAssignmentCriteria = new AttributeBasedAssignmentCriteria()
var frenchSpeaker= new AttributeCriteriaElement()
```



```
frenchSpeaker.AttributeType = UserAttributeType.TC_LANGUAGE
frenchSpeaker.AttributeField = "Name"
frenchSpeaker.AttributeValue = "French"
attributeBasedAssignmentCriteria.addToAttributeCriteria( frenchSpeaker )
activity.CurrentAssignment.assignByUserAttributes(attributeBasedAssignmentCriteria , false,
activity.CurrentAssignment.AssignedGroup )
```

assignUserByLocation

```
boolean assignUserByLocation(address, includeSubGroups, currentGroup)
```

This method uses a location-based assigner to assign an assignable entity based on a given address. This is useful in the assignment of adjusters and accident appraisers, which is often done based on geographical territory ownership.

If no user matches the criteria specified by the method, the method assigns the item to the item owner. For example, if the method cannot determine a user from the supplied criteria, it assigns an activity to the owner of the claim (in ClaimCenter) or policy (in PolicyCenter) associated with the activity.

The method matches users first by zip, then by county, then by state. The first match wins. If one or more users match at a particular location level, then assignment performs round-robin assignment through that set, ignoring any matches at a lower level. For example, suppose the method finds no users that match by zip, but a few that match by county. In this case, the method performs round-robin assignment through the users that match by county and it ignores any others that match by state.

The assignUserByLocation method bases persistence in the round-robin assignment state on the specified location information. For this reason, it is preferable to use a partially completed location, such as one that includes only the zip code, rather than a specific house.

In ClaimCenter, the following example searches on the claimant's primary address location.

```
Claim.assignUserByLocation(Claim.Claimant.PrimaryAddress, true, Claim.AssignedGroup)
```

In PolicyCenter, the following example assigns a user based on the primary location of the account associated with the activity.

```
Activity.assignUserByLocation( Activity.Account.PrimaryLocation.Address, false,
Group( "default_data:1" /* Default Root Group */ ) )
```

assignUserByLocationAndAttributes

```
boolean assignUserByLocationAndAttributes(address, attributeBasedAssignmentCriteria,
includeSubGroups, currentGroup)
```

In the application, the assignUserByLocationAndAttribute method is a combination of the assignUserByLocation and assignByUserAttributes methods. You can use it apply both kinds of restrictions simultaneously. In a similar fashion to the assignUserByLocation method, you can use this method in situations in which the assignment needs to take a location into account. In ClaimCenter, this is the claim loss location. In PolicyCenter, this is the address of a policyholder. You can then impose additional restrictions, such as the ability to handle large dollar amounts or foreign languages, for example.

If no user matches the criteria specified by the method, the method assigns the item to the item owner. For example, if the method cannot determine a user from the supplied criteria, it assigns an activity to the owner of the claim (in ClaimCenter) or policy (in PolicyCenter) associated with the activity.

As with the assignByUserAttribute assignment method, if no user matches the criteria specified by the attributeBasedAssignmentCriteria, the method assigns the object to the owner of the object being assigned.

The following examples in ClaimCenter and PolicyCenter search for a French speaker that is closest to a given address.

ClaimCenter:

```
var attributeBasedAssignmentCriteria = new AttributeBasedAssignmentCriteria()
var frenchSpeaker = new AttributeCriteriaElement()
frenchSpeaker.AttributeType = UserAttributeType.TC_LANGUAGE
frenchSpeaker.AttributeValue = "french"
attributeBasedAssignmentCriteria.addToAttributeCriteria( frenchSpeaker )

activity.assignUserByLocationAndAttributes(claim.LossLocation,
```

```
attributeBasedAssignmentCriteria , false, activity.AssignedGroup)
```

PolicyCenter:

```
var attributeBasedAssignmentCriteria = new AttributeBasedAssignmentCriteria()
var frenchSpeaker = new AttributeCriteriaElement()
frenchSpeaker.AttributeType = UserAttributeType.TC_LANGUAGE
frenchSpeaker.AttributeValue = "french"
attributeBasedAssignmentCriteria.addToAttributeCriteria( frenchSpeaker )

activity.CurrentAssignment.assignUserByLocationAndAttributes(Activity.Account.PrimaryLocation.Address,
    attributeBasedAssignmentCriteria , false, activity.CurrentAssignment.AssignedGroup)
```

Dynamic assignment

Dynamic assignment provides a generic hook for you to implement your own assignment logic, which you can use to perform automated assignment under more complex conditions. For example, you can use dynamic assignment to implement your own version of load balancing assignment.

There are two dynamic methods available, one for users and the other for groups. Both the user- and group-assignment methods are exactly parallel, with the only difference being in the names of the various methods and interfaces.

```
public boolean assignGroupDynamically(dynamicGroupAssignmentStrategy)
public boolean assignUserDynamically(dynamicUserAssignmentStrategy)
```

These methods take a single argument. Make this argument a class that implements one of the following interfaces:

```
DynamicUserAssignmentStrategy
DynamicGroupAssignmentStrategy
```

Dynamic assignment flow

The `DynamicUserAssignmentStrategy` interface defines the following methods. (The Group version is equivalent.)

```
public Set getCandidateUsers(assignable, group, includeSubGroups)
public Set getLocksForAssignable(assignable, candidateUsers)
public GroupUser findUserToAssign(assignable, candidateGroups, locks)
boolean rollbackAssignment(assignable, assignedEntity)
Object getAssignmentToken(assignable)
```

The first three methods are the major methods on the interface. Your implementation of these interface methods must have the following assignment flow:

1. Call `DynamicUserAssignmentStrategy.getCandidateUsers`, which returns a set of assignable candidates.
2. Call `DynamicUserAssignmentStrategy.getLocksForAssignable`, passing in the set of candidates. It returns a set of entities. You must lock the rows in the database for these entities.
3. Open a new database transaction.
4. For each entity in the set of locks, lock that row in the transaction.
5. Call `DynamicUserAssignmentStrategy.findUserToAssign`, passing in the two sets generated in “step 1” and “step 2” previously. That method returns a `GroupUser` entity representing the user and group that you need to assign.
6. Commit the transaction, which results in the lock entities being updated and unlocked.

In ClaimCenter, dynamic assignment is not complete after these steps. Often, such as during FNOL intake or creating a new claim in a wizard, ClaimCenter performs assignment and updates workload information well before it saves the claim. If ClaimCenter cannot save the claim, the database still shows the increase in the user’s workload. The interface methods allow for the failure of the commit operation by adding one last final step.

Dynamic assignment is not complete after these steps. The interface methods allow for the failure of the commit operation by adding one last final step.

7. If the commit fails, roll back all changes made to the user information, if possible. If this is not possible, save the user name and reassign that user to the assignable item later, during a future save operation.

Dynamic assignment – required methods implementation

Any class that implements the `DynamicUserAssignmentStrategy` interface (or the Group version) must provide implementations of the following methods.

getCandidateUsers

Your implementation of the `getCandidateUsers` method must return the set of users to consider for assignment. (As elsewhere, the `Group` parameter establishes the root group to use to find the users under consideration. The `Boolean includeSubGroups` parameter indicates whether to include users belonging to descendant groups, or only those that are members of the parent group.)

getLocksForAssignable

The `getLocksForAssignable` method takes the set of users returned by `getCandidateUsers` and returns a set of entities that you must lock. By locked, Guidewire means that the current server node obtains the database rows corresponding to those entities, which must be persistent entities. Any other server nodes that needs to access these rows must wait until the assignment process finishes. Round-robin and dynamic assignment require this sort of locking to mandate that multiple nodes do not perform simultaneous assignments. This ensures that multiple nodes do not perform simultaneous assignments and assign multiple activities (for example) to the same person, instead of progressing through the set of candidates.

findUserToAssign

Your implementation of the `findUserToAssign` method must perform the actual assignment work, using the two sets of entities returned by the previous two methods. (That is, it takes a set of users and the set of entities for which you need to lock the database rows and performs that actual assignment.) This method must do the following:

- It makes any necessary state modifications (such as updating counters, and similar operations).
- It returns the `GroupUser` entity representing the selected User and Group.

Make any modifications to items such as load count, such as entities in the bundle of the assignable. This ensures that the application commits the modifications at the same time as it commits the assignment change.

rollbackAssignment

Guidewire provides the final two API methods to deal with situations in which, after the assignment flow, some problem in the bundle commit blocks the assignment. For example, this can happen if a validation rule caused a database rollback. However, at this point, the application has already updated the locked objects and committed the objects to the database (as in “step 6” in the assignment flow).

If the bundle commit does not succeed, the application calls the `rollbackAssignment` method automatically. Construct your implementation of this method to return `true` if it succeeds in rolling back the state numbers, and `false` otherwise.

In the event that the assignment does not get saved, you have the opportunity in your implementation of this method to re-adjust the load numbers.

getAssignmentToken

If the `rollbackAssignment` method returns `false`, then the application calls the `getAssignmentToken` method. Your implementation of this method must return some object that you can use to preserve the results of the assignment operation.

The basic idea is that in the event that it is not possible to commit an assignment, your logic does one of the following:

- The application rolls back any database changes that you made.
- The application preserves the assignment in the event that you invoke the assignment logic again.

Dynamic assignment – DynamicUserAssignmentStrategy implementation

As a very simple implementation of this API, Guidewire includes the following in the base configuration:

```
gw.api.LeastRecentlyModifiedAssignmentStrategy
```

The following code shows the implementation of the `LeastRecentlyModifiedAssignmentStrategy` class. This is a very simple application of the necessary concepts needed to create a working implementation. The class performs a very simple user selection, simply looking for the user that has gone the longest without modification.

Since the selection algorithm needs to inspect the user data to perform the assignment, the class returns the candidate users themselves as the set of entities to lock. This ensures that the assignment code can work without interference from other machines.

```
package gw.assignment.examples

uses gw.api.assignment.DynamicUserAssignmentStrategy
uses java.util.Set
uses java.util.HashSet

@Export
class LeastRecentlyModifiedAssignmentStrategy implements DynamicUserAssignmentStrategy {

    construct() { }

    override function getCandidateUsers(assignable:Assignable, group:Group, includeSubGroups:boolean) :
        Set {
        var users = (group.Users as Set<GroupUser>).map( \ groupUser -> groupUser.User )
        var result = new HashSet()
        result.addAll( users )
        return result
    }

    override function findUserToAssign(assignable:Assignable, candidates:Set, locks:Set) : GroupUser {
        var users = candidates as Set<User>
        var oldestModifiedUser = users.iterator().next()
        for (nextUser in users) {
            if (nextUser.UpdateTime < oldestModifiedUser.UpdateTime) {
                oldestModifiedUser = nextUser
            }
        }

        return oldestModifiedUser.GroupUsers[0]
    }

    override function getLocksForAssignable(assignable:Assignable, candidates:Set) : Set {
        return candidates
    }

    //Must return a unique token
    override function getAssignmentToken(assignable:Assignable) : Object {
        return "LeastRecentlyModifiedAssignmentStrategy_" + assignable
    }

    override function rollbackAssignment(assignable:Assignable, assignedEntity:Object) : boolean {
        return false
    }
}
```

Group assignment (within the Assignment rules) in BillingCenter and ClaimCenter

Methods that only assign a group are most useful if called in rules in the Global Assignment rule set, which are responsible for assigning a group only. In all other contexts, you need to assign both a user and a group. Therefore, outside the Group Assignment rules, it makes more sense to use one of the other types of assignment methods that perform both assignments.

Note: It is important to understand that the group methods do not assign the assignable item to a user. They simply pick a group to use during the rest of the assignment process. To complete the assignment, you must use one of the other assignment methods to assign the item to a user.

ClaimCenter: assignGroup

```
boolean assignGroup(GroupBase group)
```

Use this method to assign the indicated group. You use this method in situations in which you know the group to which you want to assign the object. This group assignment method ignores load factors.

The following example assigns an activity associated with a SIU (Special Investigation Unit) escalation to the “Western SIU” group.

```
var siuGroup = find (g in Group where g.Name == "Western SIU").AtMostOneRow
    if (activity.assignGroup(siuGroup)) {
        actions.exit()
    }
```

ClaimCenter: assignGroupByLocation

`boolean assignGroupByLocation(groupType, address, directChildrenOnly, group)`

This method uses a location-based assigner to assign an assignable entity based on a given address. This group assignment method ignores load factors. Use this assignment method to assign a group based on geographic closeness to a particular address. For example, use this method to assign the assignable entity to a group based in a processing office in a specific time zone.

The parameters for this assignment method have the following meaning:

<code>groupType</code>	• If the value of <code>groupType</code> is non-null, the method ignores (filters out) those groups that do not match the provided group type value. • If the value of <code>groupType</code> is null, the method ignores group type in the candidate pool of groups.
<code>address</code>	Using the supplied address, the method searches for the smallest zone that contains this address, which, in the United States, is the ZIP code (the postal code). If the method does not find a match, it expands the search to the next larger zone, which, again in the United States, is the county. If still not finding zone matches with the supplied address, the method expands the size of the zone again and searches for matches at the (U.S.) state level. The method continues this search zone expansion until it finds one or more zone matches with the supplied address. If the method finds multiple groups that match the criteria, it performs simple round-robin assignment among those groups. (Simple round-robin assignment does not use load factors.) The method bases this round-robin assignment on the zone type that the rule engine used to find the initial group match. For example, suppose that the method finds no groups that match by postal code, but does find a few groups that match by county. In that case, the assignment method performs round-robin assignment through the groups that match by county and ignores any groups that match by state.
<code>directChildrenOnly</code>	• If the value of <code>directChildrenOnly</code> is <code>true</code>, the method searches for groups directly underneath the provided group only. It does not search the children of those groups or any further descendants • If the value of <code>directChildrenOnly</code> is <code>false</code>, the method expands the candidate pool of groups to all descendants of the provided group.
<code>group</code>	• If the value of <code>group</code> is non-null, the method begins the search with the direct children of the specified group. ClaimCenter does not include the specified group itself in the search. • If the value of <code>group</code> is null, the method begins the search with the direct children of the root group. Depending on the size of the group hierarchy, this can potentially cause performance issues if <code>directChildrenOnly</code> is set to <code>false</code>

The `group`, `groupType`, and `directChildrenOnly` parameters work in conjunction with each other. For the sake of illustration, assume that `group` and `groupType` hold meaningful values and that `directChildrenOnly` is set to `true`.

1. The method looks at the children of the specified group and filters out any child groups that do not match the provided group type. If no child groups remain after filtering for group type, the method stops and does not assign the work item.
2. Of the remaining child groups (if any), the method looks for group locations that match the smallest zone of the provided address. Finding multiple groups that match at this zone level, the method uses simple round-robin to

assign the work item to a group. If no child group matches any zone of the provided address, at any level, the method stops and does not continue searching.

Now, suppose that `group` and `groupType` hold meaningful values, but `directChildrenOnly` is set to `false`.

1. In this case, the method includes in the candidate pool of groups all the descendant groups of the provided group. It removes from consideration any group that does not match the provided group type.
2. The method then uses the zone information of this set of groups to determine which of these groups match with the zone data of the provided address. Assuming multiple matches, the method uses simple round-robin assignment to assign the work item to a group.

The following code example uses the `getGroupTypeBasedOnClaimSegment` method to find the top two group choices for assigning this claim.

```
var result = libraries.Claimassignment.getGroupTypeBasedOnClaimSegment(claim.LossType, claim.Segment)
var primarygroupType = result[0]
var secondarygroupType = result[1]

if (claim.LossLocation != null) {
    if (claim.assignGroupByLocation(primarygroupType, claim.LossLocation, false,
        claim.AssignedGroup)) {
        actions.exit()
    }

    if (claim.assignGroupByLocation(secondarygroupType, claim.LossLocation, false,
        claim.AssignedGroup)) {
        actions.exit()
    }
}

if (claim.Insured.PrimaryAddress != null) {
    if (claim.assignGroupByLocation(primarygroupType, claim.Insured.PrimaryAddress,
        false, claim.AssignedGroup)) {
        actions.exit()
    }

    if (claim.assignGroupByLocation(secondarygroupType, claim.Insured.PrimaryAddress,
        false, claim.AssignedGroup)) {
        actions.exit()
    }
}
```

In this example, the assignment logic tries the following assignment options in the listed order until one succeeds:

- Attempt to assign the claim to a group of the primary group type that is near the claim's loss location.
- Attempt to assign the claim to a group of the secondary group type that is near the claim's loss location.
- Attempt to assign the claim to a group of the primary group type that is near the insured's primary address.
- Attempt to assign the claim to a group of the secondary group type that is near the insured's primary address.

ClaimCenter and BillingCenter: assignGroupByRoundRobin

```
boolean assignGroupByRoundRobin(groupType, includeSubGroups, group)
```

This assignment method assigns an item to a group (or one of its subgroups) using round-robin assignment. Use this method to distribute work (assignable items) among multiple groups. The `assignGroupByRoundRobin` method ignores any group for which the `Group.LoadFactor` value is zero (0).

The parameters for this assignment method have the following meaning:

<code>groupType</code>	• If the value of <code>groupType</code> is non-null, the method ignores (filters out) those groups that do not match the provided group type value. • If the value of <code>groupType</code> is null, the method ignores group type in the candidate pool of groups.
<code>includeSubGroups</code>	• If the value of <code>includeSubGroups</code> is true:

	◦ The method considers for assignment all descendants (subgroups) of the provided group. ◦ The method ignores the group load factor in making a round-robin assignment among the various groups. • If the value of <code>includeSubGroups</code> is <code>false</code>: ◦ The method considers for assignment the direct children of the provided group only. ◦ The method considers the load factor of each child group in making the round-robin assignment.
<code>group</code>	• If the value of <code>group</code> is non-null, the method begins the search with the direct children of the specified group. The application does not include the specified group itself in the search. • If the value of <code>group</code> is null, the method begins the search with the direct children of the root group. Depending on the size of the group hierarchy, this can potentially cause performance issues if <code>includeSubGroups</code> is set to <code>false</code>

In general, load factors have two purposes or uses:

- The first purpose is to provide a binary on/off switch. If the load factor for a user or group is set to zero (0), it disables assignment to that user or group. Conversely, if the load factor for a group or user is a non-zero value, it enables assignment to that group or user.
- The second purpose is to provide a means to compare the availability of groups or users for assignment. The use of load factors with round-robin assignment is only relevant if assigning within a single group. During round-robin assignment within a group, load factors determine how frequently a given group receives an assignment as compared to its sibling groups. For example, suppose that one group has a factor of 100 and its sibling group has a factor of 50. During round-robin assignment using load factors, the first group receives twice as many assignments as the second. The same logic holds true for round-robin assignment of users within a group as well. Again, load factors determine the how frequently a given user receives an assignment as compared with the other members of the group.

Whether the `assignGroupByRoundRobin` method uses the group load factor in making a round-robin assignment depends on the value of `includeSubGroups`.

- If `includeSubGroups` is `true`, the method ignores the load factor of each group in making the assignment.
- If `includeSubGroups` is `false`, the method considers the load factor of the group in making an assignment. Load factors are only relevant as a comparative within the group.

The `group`, `groupType`, and `includeSubGroups` parameters work in conjunction with each other. For the sake of illustration, assume that `group` and `groupType` hold meaningful values and that `includeSubGroups` is set to `false`.

1. The method looks at the direct children of the specified group and filters out any child groups that do not match the provided group type. If no child groups remain after filtering for group type, the method stops and does not assign the work item.
2. Of the remaining child groups (if any), the method uses round-robin assignment to assign the work item to a group. The method considers the load factor of each child group in making the round-robin assignment.

Now, suppose that `group` and `groupType` hold meaningful values, and `includeSubGroups` is set to `true`.

1. In this case, the method includes in the candidate pool of groups all the descendant groups of the provided group. It then filters out any groups from the candidate pool that do not match the provided group type.
2. The method uses round-robin assignment on the remaining groups to assign the work item to a group. The method does not consider the load factor of any group in making the round-robin assignment.

In `BillingCenter`, the following example assigns a troubleticket to one of the special handling groups within the currently assigned group.

```
TroubleTicket.assignGroupByRoundRobin(GroupType.TC_SPECIALHANDLING, true,
Activity.AssignedGroup)
```


In ClaimCenter, the following example assigns a complicated automobile claim to one of the child groups that handle complex claims within the currently assigned group.

```
claim.assignGroupByRoundRobin(GroupType.TC_AUTOCOMPLEX, false, claim.AssignedGroup)
```

Immediate assignment

Immediate assignment methods assign an object directly to the specified user or group.

BillingCenter and ClaimCenter: assign

```
boolean assign(group, user)
```

This method assigns the assignable entity to the specified group and user, the simplest of assignment operations. Use this method if you want a specific known user and group to own the entity in question.

In BillingCenter, in the following example, the assignment method assigns a trouble ticket to the owner of an already assigned trouble ticket.

```
var TroubleTickets= find (trbtkt in troubleTicket where trbtkt.TroubleTicketNumber contains "100")

for (trb in TroubleTickets) {
    if (trb != troubleTicket and trb.AssignedGroup == Group( "default_data:1" /* Default Root Group */ )
    and trb.AssignedUser != null) {
        if ( troubleTicket.CurrentAssignment.assign( trb.AssignedGroup, trb.AssignedUser ) ) {
            print( "##### This is the Default Group TroubleTicket Assignment rule "
+ gw.api.util.StringUtil.substring(actions.getRule().DisplayName,0, 8))
            print( "Assigned User is " + trb.AssignedUser)
            actions.exit()
        }
    }
}
```

In ClaimCenter, in the following example, the assignment method assigns another exposure to the owner of an already assigned exposure.

```
for (exp in Exposure.Claim.Exposures) {
    if (exp != Exposure and exp.AssignedGroup == Exposure.AssignedGroup
    and exp.AssignedUser != null) {
        if (Exposure.assign( exp.AssignedGroup, exp.AssignedUser ) ) {
            actions.exit()
        }
    }
}
```

assignUserAndDefaultGroup

```
boolean assignUserAndDefaultGroup(user)
```

This method assigns the assignable entity to the specified user, selecting a default group. The default group is generally the first group in the set of groups to which the user belongs. In general, use this method if a user only belongs to a single group, or if the assigned group really does not matter.

It is possible that the assigned group can affect visibility and permissions. Therefore, Guidewire recommends that use this method advisedly. For example, you might want to use this method only under the following circumstances:

- The users belong to only a single group.
- The assigned group has no security implications.

In ClaimCenter, the following example assigns an Activity to the current user and does not need to specify a group.

```
Activity.AssignUserAndDefaultGroup(User.util.CurrentUser)
```

In BillingCenter and PolicyCenter, the following example assigns an Activity to the current user and does not need to specify a group.

```
Activity.CurrentAssignment.AssignUserAndDefaultGroup(User.util.CurrentUser)
```

ClaimCenter: assignToIssueOwner

```
boolean assignToIssueOwner()
```


This method assigns the assignable entity to the “Issue Owner”. This concept only really applies to ClaimCenter, in which the issue owner is the owner of the associated Claim. Use this method to assign a subsidiary entity (for example, an Exposure or a Matter) to the owner of the primary entity, such as the Claim.

The following example assigns an exposure to the issue owner (the owner of the associated claim).

```
Exposure.assignToIssueOwner()
```

BillingCenter and ClaimCenter: assignToCreator

```
boolean assignToCreator(sourceEntity)
```

This method assigns the assignable entity to the user who created the supplied sourceEntity parameter.

In BillingCenter, the following example assigns an activity to the creator of a trouble ticket.

```
activity.CurrentAssignment.assignToCreator(troubleTicket)
```

In ClaimCenter, the following example assigns an exposure to creator of the claim associated with the exposure.

```
exposure.assignToCreator(exposure.Claim)
```

ClaimCenter: assignToPreviousOwner

```
boolean assignToPreviousOwner()
```

Assigns the entity to the previously assigned user. The method tracks the current group. If there is no current group defined, the method does nothing and logs a warning. Although not officially deprecated, Guidewire recommends that you not use this method.

Manual assignment in ClaimCenter

Manual assignment is slightly different from other assignment scenarios. Instead of assigning the assignable entity, manual assignment creates a new activity directing the specified user to manually perform (or review) the assignment. To be eligible to receive a manual assignment activity, a user must have the Review Assignment (actreviewassign) permission.

In the base configuration of ClaimCenter, the following roles all have the Review Assignment permission.

- Claim Supervisor
- New Loss Processing Supervisor
- Manager

IMPORTANT: Only use manual assignment methods, such as assignManually, in the Assignment rule set.

assignManually

```
boolean assignManually(responsibleUser)
```

This method designates a user to look at the assignable and manually assign it to someone. Use this method to flag the assignable for manual review and assignment. If you call this method, ClaimCenter does not actually assign the assignable entity *until* the designated user takes action to do so.

The following example assigns the claim to the group supervisor to assign manually.

```
Claim.assignManually(Claim.AssignedGroup.Supervisor)
```

confirmManually

```
boolean confirmManually(responsibleUser)
```

This method requests manual confirmation of the assignable entity's current assignment. It is roughly equivalent to assignManually, except that it assumes that a provisional assignment has already been made. ClaimCenter does not consider the assignment to be final until someone reviews and approves it.

Note: Items that you mark for manual confirmation appear in the **Pending Assignment** list of the supervisor of the assigned group.

The following example first assigns a claim provisionally to a user using round-robin assignment, then creates an activity for that user's supervisor to review the assignment and approve it.

```
Claim.assignUserByRoundRobin( false, Claim.AssignedGroup)
    Claim.confirmManually( Claim.AssignedGroup.Supervisor )
```

assignManuallybyRoundRobin

```
boolean assignManuallyByRoundRobin(group)
```

This method is similar to `assignManually`. However, this method shifts the responsibility for manual assignment to members of the specified group who have the `actreviewassign` permission. Use this method if the workload of manual assignment is too large for a single user. You would use this method to distribute the workload among members of a group that have the requisite permission.

For example, suppose that you have a team of supervisors responsible for assigning certain sensitive tasks to a set of Claims adjusters. You want manual assignment because you want the supervisors to select someone manually. But, you also want to rotate the responsibility for making assignment through the group of supervisors. This is different from the standard round-robin assignment, which rotates the assignment itself. This method rotates the designation of the person responsible for completing the manual assignment.

The following example moves the assignment responsibility among members of a particular group.

```
Claim.assignManuallybyRoundRobin(Claim.AssignedGroup)
```

confirmManuallyByRoundRobin

```
boolean confirmManuallyByRoundRobin(group)
```

This method is similar to `confirmManually`, except that it determines the responsibility for assignment confirmation through round-robin assignment among users in the specified group. Again, you would use a method of this type if the workload of assignment confirmation is too large for a single user.

The following example first assigns a claim to a user using round-robin assignment through members of a group. It then assigns responsibility for assignment confirmation to a member of that group.

```
Claim.assignUserByRoundRobin( false, Claim.AssignedGroup)
    Claim.confirmManuallybyRoundRobin( Claim.AssignedGroup)
```

Proximity-based assignment in ClaimCenter

Guidewire provides several proximity-based assignment methods. Each method provides a slightly different functionality. All proximity-based assignment methods are functions that the `CCAssignable` delegate interface defines.

IMPORTANT: To use the proximity assignment methods, you must integrate ClaimCenter with a geocoding server and geocode all users. For more information on proximity searches and geocoding, see the *Contact Management Guide*.

assignUserByLocationUsingProximitySearch

```
boolean assignUserByLocationUsingProximitySearch(address, includeSubGroups, currentGroup)
```

This method assigns the entity the user with the closest address to the specified address, as measured by a great-circle distance along the surface of the earth. To use this method, you must first geocode all user addresses in the database. You must also geocode the supplied address that serves as the center of the search, either in advance or at the time of the search. If the geocoding attempt fails for the supplied location, the method logs an error and returns `false`. (It also returns `false` if it unable to make an assignment.)

The `assignUserByLocationUsingProximitySearch` method performs the following search algorithm:

1. For each user in the specified group, the method computes the distance from each user location to the supplied location. (It uses the primary address of the Contact entity for each user.) The search begins with the users of the current group (currentGroup). It is an error if there is no current group.
2. If the includeSubGroups parameter is true, the method repeats this process with all of the descendant groups of the specified group.
3. The method then returns the user who is closest to the specified location. It does not perform round-robin assignment among the set of users.

This method takes the following parameters:

Parameter	Description
address	An address to use as the center of the search.
includeSubGroups	if true, then include users in any subgroups of the supplied group, and the users in the current group.
currentGroup	The group whose members to consider for assignment. This cannot be null.

The following example assigns a claim to someone that is closest (in distance) to the claimant's primary address.

```
Claim.assignUserByLocationUsingProximitySearch(Claim.claimant.PrimaryAddress, true,
Claim.AssignedGroup)
```

[assignUserByLocationUsingProximityAndAttributes](#)

```
boolean assignUserByLocationUsingProximityAndAttributes(address,
attributeBasedAssignmentCriteria, includeSubGroups, currentGroup)
```

This method is similar to the assignUserByLocationAndAttributes method, except that it also takes user attributes into account in the search algorithm.

The method uses a distance calculation and one or more attributes to select the closest user. It does not perform round-robin assignment among the set of users. You use this method, therefore, if you want to find the closest user with certain attributes.

This method takes the following parameters:

Parameter	Description
address	The location to use as the center of the search.
attributeBasedAssignmentCriteria	The user attributes to match against.
includeSubGroups	if true, then include users in any subgroups of the supplied group, and the users in the current group.
currentGroup	The group that includes the users to consider for the assignment

The following example searches for a French speaker that is closest to a given address. The example code first creates a new AttributeCriteriaElement object and assigns it a value of French. It then adds the new object to the attributeBasedAssignmentCriteria object and uses that object as part of the search criteria. The assignment method then tries to assign the activity to a French speaker that closest, in straight-line distance, to the primary address of the claimant on a claim.

```
var attributeBasedAssignmentCriteria = new AttributeBasedAssignmentCriteria()
var frenchSpeaker = new AttributeCriteriaElement()
frenchSpeaker.AttributeType = UserAttributeType.TC_LANGUAGE
frenchSpeaker.AttributeValue = "french"
attributeBasedAssignmentCriteria.addToAttributeCriteria( frenchSpeaker )
Activity.assignUserByLocationUsingProximityAndAttributes(Activity.Claimant.Person
.PrimaryAddress, attributeBasedAssignmentCriteria, false, Activity.AssignedGroup )
```

assignUserbyProximityWithSearchCriteria

```
public boolean assignUserByProximityWithSearchCriteria(usc, cap, includeSubGroups,
currentGroup)
```

This method assigns the entity to a user based on a *user search by proximity*. It uses the following search algorithm:

1. The method first geocodes the location that serves as the center of the search (if it is not already geocoded). You can then access this location through the following code:

```
usc.getContact().getProximitySearchCriteria()
```
2. The method compiles a list of users who satisfy the user search criteria. Use the specified *cap* to limit the number of users in this list. If you include proximity restrictions on the search, the method discards the users farthest from the search center.
3. The method uses the round-robin algorithm to pick one of the found users. This means that repeated, identical calls to this method rotate through the resulting set of users to find the user to return. The method bases the round-robin rotation on the exact *UserSearchCriteria* that you supply. Guidewire recommends that you use as general a location as possible (for example, a city, state or postal code, rather than a specific street address). This maximizes the benefit of the round-robin processing and reduces the load on the system.

This method takes the following parameters:

Parameter	Description
<i>usc</i>	An object of type <i>UserSearchCriteria</i> that sets the starting location for the proximity search. This cannot be null. The <i>UserSearchCriteria</i> object has a foreign key to a <i>ContactSearchCriteria</i> entity, which, in turn, has a foreign key to <i>Address</i> .
<i>cap</i>	Integer value that determines the number of users to return. • If greater than zero, it sets the maximum number of users to include in the round-robin assignment algorithm from the search results. • If zero or less, the method includes all users from the search results in the round-robin assignment algorithm. To always assign the entity to the closest user, set <i>cap</i> to 1.
<i>includeSubGroups</i>	If true, then include users in any subgroups of the supplied group, and the users in the current group. The method ignores this parameter if the <i>currentGroup</i> parameter is null.
<i>currentGroup</i>	The group whose members to consider for assignment. This can be null. If it is non-null, then the method uses it as part of the search.

IMPORTANT: This assignment method can be very slow compared to other assignment methods. Guidewire recommends that you consider its performance implications before using it.

To avoid errors in setting up the proximity search, Guidewire recommends that you use the following method:

```
gw.api.geocode.GeocodeScriptHelper.setupUserProximitySearch
```

This method sets up a number of specific fields on the search criteria. The *GeocodeScriptHelper* method returns a *UserSearchCriteria* object that other proximity assignment methods can use.

The *GeocodeScriptHelper* method has the following signature and parameters:

```
setupUserProximitySearch(searchCenter, isDistanceBased, number, unitOfDistance)
```

Parameter	Description
<i>searchCenter</i>	The location to use as the center of the search. This is an <i>Address</i> object.
<i>isDistanceBased</i>	• Distance-based searches – Set to true if you want to perform a distance-based search. This type of search finds users within n miles (kilometers) of the <i>searchCenter</i> location.

Parameter	Description
	• Ordinal-based searches – Set to false if you want to perform an ordinal-based search. This type of search finds the nearest n users to the searchCenter location (computed as the straight-line distance).
number	• Distance-based searches – The search radius in miles or kilometers for distance-based searches • Ordinal-based searches – The maximum number of results to return for an ordinal search.
unitOfDistance	• Distance-based searches – Indicates the unit (miles or kilometers) to use for distance-based searches. • Ordinal-based searches – Determines how ClaimCenter displays the results, ignored otherwise. This parameter takes its value from UnitOfDistance typelist. • UnitOfDistance.TC_KILOMETER • UnitOfDistance.TC_MILE

The following example sets up the proximity information to use in the search using the `Claim.LossLocation` as the supplied address, starting with a search radius of 10 miles. This is a distance-based search as the `isDistanceBased` parameter is set to true.

```
uses gw.api.geocode.GeocodeScriptHelper

// Set up a distance-based proximity search for claim assignment
if (claim.LossLocation != null) {
    var usc = GeocodeScriptHelper.setupUserProximitySearch(claim.LossLocation, true, 10,
        UnitOfDistance.TC_MILE)

    if (claim.assignUserByProximityWithSearchCriteria(usc, -1, true, claim.AssignedGroup)) {
        actions.exit() //Needed if using this method within the context of the assignment
    }
}
```

The following example sets up the proximity information to use in the search using a supplied address. This is a ordinal-based search as the `isDistanceBased` parameter is set to false. It returns the closest three users to the supplied address.

```
uses gw.api.geocode.GeocodeScriptHelper

// Set up a ordinal-based proximity search for claim assignment
var usc = GeocodeScriptHelper.setupUserProximitySearch(claim.LossLocation, false, 3, UnitOfDistance.TC_MILE)

//Set user city and postal code of search center location
usc.Contact.Address.City = claim.LossLocation.City
usc.Contact.Address.State = claim.LossLocation.State
usc.Contact.Address.PostalCode = claim.LossLocation.PostalCode

if (claim.assignUserByProximityWithSearchCriteria(usc, -1, true, claim.AssignedGroup)) {
    actions.exit() //Needed if using this method within the context of the assignment rules
}
```

assignUserByProximityWithAssignmentSearchCriteria

```
boolean assignUserByProximityWithAssignmentSearchCriteria(asc, cap, includeSubGroups,
    currentGroup)
```

This method is nearly identical to the `assignUserByProximityWithSearchCriteria` method, except that it uses an `AssignmentSearchCriteria` object rather than a `UserSearchCriteria` object. See that method for more information.

This method takes the following parameters:

Parameter	Description
asc	An object of type <code>gw.api.assignment.AssignmentSearchCriteria</code> that sets the starting location for the proximity search and the user attribute to match.
cap	Integer value that determines the number of users to return.

Parameter	Description
	• If greater than zero, it sets the maximum number of users to include in the round-robin assignment algorithm from the search results. • If 0 or less, the method includes all users from the search results in the round-robin assignment algorithm.
includeSubGroups	If true, then include users in any subgroups of the supplied group, and the users in the current group. The method ignores this parameter if the currentGroup parameter is null.
currentGroup	The group whose members to consider for assignment. This can be null. If it is non-null, then the method uses it as part of the search.

IMPORTANT: This assignment method can be very slow compared to other assignment methods. Guidewire recommends that you consider its performance implications before using it.

The following example attempts to assign a claim to a French speaker within 10 mile radius of the claim loss location. Notice that the code first uses the `CCGeocodeScriptHelper.setupAssignmentProximitySearch` method to set up the parameters for the proximity search. It then sets up the French-speaking attribute on the `CCUserSearchCriteria` object.

```
uses gw.api.geocode.CCGeocodeScriptHelper

// Set up an distance-based proximity search for claim assignment
if (claim.LossLocation != null) {

    var asc = CCGeocodeScriptHelper.setupAssignmentProximitySearch(Claim.LossLocation, true, 10,
        UnitOfDistance.TC_MILE )

    // Set the custom attribute match
    asc.CCUserSearchCriteria.AttributeName = "French"
    asc.CCUserSearchCriteria.AttributeValue = 1

    if (claim.assignUserByProximityWithAssignmentSearchCriteria(asc, -1, true,
        claim.AssignedGroup )) {
        actions.exit() //Needed if using this method within the context of the assignment rules
    } else {
        print ("#####: Assigning by proximity to Claimant Address = " + Claim.LossLocation + ": FAILED")
    }
} else { print ( "#####: Claim.LossLocation is NULL ..." ) }
```

Queue assignment in ClaimCenter and PolicyCenter

Each group in Guidewire ClaimCenter and PolicyCenter has a set of `AssignableQueue` entities associated with it. It is possible to modify the set of associated queues.

In ClaimCenter, in the base configuration, ClaimCenter assigns each group a queue named FNOL.

Guidewire supports assigning `Activity` entities only to queues. Guidewire deprecates the `assignToQueue` methods on `Assignable` in favor of the `assignActivityToQueue` method. This makes the restriction more clear.

If you assign an activity to a queue, you cannot simultaneously assign it to a user as well.

Within the application, an administrator can define and manage queues through the ClaimCenter or PolicyCenter **Administration** screen.

`assignActivityToQueue`

```
boolean assignActivityToQueue(queue, currentGroup)
```

Use this method to assign this activity to the specified queue. The activity entity then remains in the queue until someone or something reassigns it to a specific user. It is possible to assign an activity in the queue manually (by the group supervisor, for example) or for an individual to choose the activity from the queue.

To use this method, you need to define an `AssignableQueue` object.

You can use the name of the group to retrieve a queue attached to that group.

```
Activity.AssignedGroup.getQueue(queueName) //Returns an AssignableQueue object
```

```
Activity.AssignedGroup.AssignableQueues //Returns an array of AssignableQueue objects
```

In the first case, you need to know the name of the queue. You cannot do this directly, as there is no real unique identifier for a Queue outside of its group.

Note: In the ClaimCenter base configuration, each Group has an FNOL queue defined for it by default.

If you have multiple queues attached to a group, you can do something similar to the following to retrieve one of the queues. For example, use the first method if you do not know the name of the queue. Use the second method if you know the name of the queue.

```
var queue = Activity.AssignedGroup.AssignableQueues[0]
var queue = Activity.AssignedGroup.getQueue("QueueName")
```

You can then use the returned AssignableQueue object to assign an activity to that queue.

```
ClaimCenter: Activity.assignActivityToQueue(queue, group)
PolicyCenter: Activity.CurrentAssignment.assignActivityToQueue(queue, group)
```

Round-robin assignment

The round-robin algorithm rotates through a set of users, assigning work to each in sequence. It is important to understand that round-robin assignment is distinct from workload-based assignment. Round-robin assignment does not take the current number of entities assigned to any of the users into account.

You can use load factors (in some circumstances) to affect the frequency of assignment to one user or another. For example, suppose that person A has a load factor of 100 and person B has a load factor of 50. In this case, the round-robin algorithm selects person A for assignment twice as often as person B.

assignUserByRoundRobin

```
boolean assignUserByRoundRobin(includeSubGroups, currentGroup)
```

This method uses the round-robin user selection to choose the next user to receive the assignable from the current group or group tree. If the includeSubGroups parameter is true, the selector performs round-robin assignment not only through the direct children of the current group, but also through all of the parent group's subgroups.

For example, in ClaimCenter, suppose that you have a set of claims that you want to assign to a member of a particular group. If you want all of the users within that group and all of its descendent groups to share the workload, then you set the includeSubGroups parameter to true.

The following examples assign an activity to the next user in a set of users in a group.

- ClaimCenter:

```
activity.assignUserByRoundRobin( false, Activity.AssignedGroup )
```

- BillingCenter and PolicyCenter:

```
activity.CurrentAssignment.assignUserByRoundRobin( false, Activity.AssignedGroup )
```

Round-robin assignment of similar exposures in ClaimCenter

This assignment method is a type of round-robin assignment that takes into account similar exposures on a claim. This method is limited to assignment of exposures only, and is not applicable to any other entity.

assignUserByRoundRobinKeepSimilarTogether

```
boolean assignUserByRoundRobinKeepSimilarTogether(includeSubGroups, currentGroup)
```

When additional exposures are created on a claim, each additional exposure assigned using this method is evaluated if it is similar to an existing exposure. If it is, it is assigned to the same owner. If the exposure is not similar, the round robin sequence of assignment within the group takes place.

Note: Please be aware this method is distinct from the basic `assignUserByRoundRobin` method described in “Round-robin assignment” on page 183, which always assigns work in a sequence. Ensure you check which rules are called by this method versus the basic round robin method when assigning exposures.

In the base application, a similar exposure is an exposure against the same coverage subtype on the same claim. You can configure this logic to make your own determination of what counts as a similar exposure. See “Configuring the definition of a similar exposure” on page 184.

In the event a single user is assigned two or more exposures in this method, that user receives a *credit* for that similar assignment and is skipped the next time the round robin sequence attempts additional non-similar exposure assignment to that user.

Note: Credits and load factors are both taken into account when using this assignment method.

For example, on a personal auto claim, a claim is created that involved bodily injury during an auto accident. The first exposure created has a coverage subtype of **Liability - Bodily Injury** and is assigned to Andy Applegate using this round-robin assignment. A week later, the third-party driver involved in the accident complains of back pain and a second exposure is created on the claim, also with **Liability - Bodily Injury** coverage subtype. This assignment method assigns the second exposure not to the next user in the round-robin sequence but instead to Andy Applegate, because it is a similar exposure.

Andy is now credited with this second assignment. That way, if this round robin with similar exposures method cycles through the group and attempts to assign another non-similar exposure (such as **Collision**) to Andy on this claim, Andy is skipped because he's already been credited.

When using this method, credits are always incremented starting at the second exposure assigned. In this example, Andy's credit for this group and for this claim would be 1 because of the additional exposure assigned.

Note: If you want this assignment method to also apply to assignment within subgroups, set the `includeSubGroups` argument to `true`.

In the logs (at DEBUG level), when a credit occurs, the log shows a message such as the following:

```
Incremented SimilarAssignableCredits for user Andy Applegate to 1
```

In the logs (at DEBUG level), when this assignment method assigns a similar exposure, the log shows a message such as the following:

```
Assigned Similar Exposure to user: Andy Applegate
```

When users are skipped in the round robin cycle with this assignment because they already have already been credited with an exposure assignment, their credits are decremented. As always, any assignments received outside of the rule calling this method have no impact to this method's decisions.

To use this assignment method, configure Exposure Assignment rules. See “Global Exposure Assignment rules” on page 60.

Configuring the definition of a similar exposure

To configure the logic that determines what a similar exposure is, create a Gosu class that extends the `ExposureAssignSimilarMethodsImpl` Gosu plugin implementation. You can have multiple sets of criteria to determine a similar exposure in different situations such as different lines of business or using additional criteria such as cost type.

In the base plugin implementation, the determination of a similar exposure is made in the `isSimilar` function:

```
override function isSimilar(exposure : Exposure) : boolean {
    return exposure.CoverageSubType != null && Owner.CoverageSubType == exposure.CoverageSubType;
}
```


Configure similar exposures

About this task

To use your own definition of a similar exposure when using `assignUserByRoundRobinKeepSimilarTogether`, create a Gosu class that extends `ExposureAssignSimilarMethodsImpl` and includes the new definition of a similar exposure, extend the `Exposure` entity to implement this class, and modify the exposure assignment rules to call `exposure.assignUserByRoundRobinKeepSimilarTogether`.

In this example, the definition of a similar exposure is altered to be any exposure with the same claimant.

Procedure

1. In Guidewire Studio, create a new Gosu class in `gw.assignment` called `AssignExposureSameClaimantImpl`:

```
package gw.assignment
uses gw.api.assignment.ExposureAssignSimilarMethodsImpl
class AssignExposureSameClaimantImpl extends ExposureAssignSimilarMethodsImpl {
    construct(owner : Exposure) {
        super(owner)
    }

    override function isSimilar(exposure : Exposure) : boolean {
        return exposure.Claimant != null && Owner.Claimant == exposure.Claimant;
    }
}
```

2. Extend the `Exposure` entity by creating a new entity extension. In Guidewire Studio, navigate to **configuration > config > Extensions > Entity**.
3. Right-click on **Entity** and choose **New > Entity Extension**.
4. In the **Entity Extension** dialog window, scroll down in the list and choose **Exposure**. The `Exposure.etx` file is created and opens in a new tab in Guidewire Studio.
5. In your new `Exposure.etx` entity extension, implement the new Gosu class:

```
<?xml version="1.0"?>
<extension
  xmlns="http://guidewire.com/datamodel"
  entityName="Exposure">
  <implementsInterface
    iface="gw.api.assignment.AssignSimilarMethods"
    impl="gw.assignment.AssignExposureSameClaimantImpl"/>
</extension>
```

6. Modify or add `GlobalExposureAssignmentRules` to call `exposure.assignUserByRoundRobinKeepSimilarTogether`. To access Global Exposure Assignment Rules, navigate in Guidewire Studio to **configuration > config > Rule Sets > Assignment > GlobalExposureAssignmentRules**.

Note: This example uses a simplistic case where the default Gosu rule is altered to call the round-robin assignment of similar exposures method. It is recommended you check all other exposure assignment rules for where this method applies and change those as well.

7. In the `GEA03000` (Default) rule, comment out `exposure.assignToIssueOwner()` and add the following line below it:

```
//exposure.assignToIssueOwner()
exposure.assignUserByRoundRobinKeepSimilarTogether(false, exposure.getAssignedGroup())
```

If you want to include subgroups, add this line instead:

```
exposure.assignUserByRoundRobinKeepSimilarTogether(true, exposure.getAssignedGroup())
```

You must also alter `DGE02000` (Default - users in team) to use the `assignUserByRoundRobinKeepSimilarTogether()` method accordingly with subgroups.

Workload Count in ClaimCenter

In many cases, a ClaimCenter user can be a member of multiple work teams or groups. This means that you can assign a user to a claim review activity as a member of one group. You can then also assign that user a claim as a member of a different group. Therefore, in determining workload assignments, it is important to take the total (global) number of items assigned to an individual into account.

To this end, ClaimCenter provides a count of the total number of activities, claims, exposures, and matters assigned to an individual. You can access these total counts both through the ClaimCenter interface and through fields on the User object in Gosu.

Accessing workload count on the Team tab

The **Team** tab in ClaimCenter displays item counts for each user who is part of that team, organized into columns by each item type—claim, activity, and so on. However, these numbers are simply the item counts for that user in that group and do not include items assigned to that user as a member of another group. To identify the total number of items assigned to a user, ClaimCenter displays a global total for an item type in parentheses next to the subtotal number. You must be an administrator or a group supervisor to access the **Team** tab.

Accessing workload count in Gosu code

There are fields on the entities User and Group that determine the total number of items of a given type assigned to an individual or group. You can access these fields in Gosu code.

The fields are:

- `OpenActivityCount`
- `OpenClaimCount`
- `OpenExposureCount`
- `OpenMatterCount`

For example, the following Gosu code returns the total number of activities across all groups for the current owner of a claim:

```
var total = Claim.AssignedUser.OpenActivityCount
```

ClaimCenter updates the workload count numbers hourly while running statistics batch processing.

ClaimCenter rules-based validation

Note: Although the terms *object* and *entity* are not entirely identical, this topic uses these terms interchangeably.

ClaimCenter runs preupdate and validation rules every time that it commits data to the database. Guidewire calls a commit of data to the database a *database bundle commit*. The validation rules execute after ClaimCenter runs all preupdate callbacks and preupdate rules. ClaimCenter runs the validation rules as the last step before ClaimCenter actually writes data to the database.

Before applying rules during the bundle commit operation, ClaimCenter builds a validation object graph according to configuration settings. The graph contains possible targets for rule execution and both the preupdate and validation rules use the validation object graph.

Preupdate rules, unlike validation rules, can, and do, modify additional objects during execution. If a preupdate rule does modify additional objects, it is often necessary to construct these objects after the preupdate rules run and before the validation rules run.

If the preupdate rules:

- Modify an entity that is not in the commit bundle, ClaimCenter can possibly run additional validation rules if the entity being modified also triggers validation.
- Modify only entities that are already in the commit bundle, ClaimCenter runs the validation rules for the same set of entities on which the preupdate rules ran.
- Modify something that simply triggers validation for one of the top-level entities, there is no noticeable difference from those validation rules. ClaimCenter was already going to run these rules anyway.

The set of entities that the rules validate can be a superset of the entities for which preupdate rules are run. The preupdate rules are non-recursive—they do not run a second time on objects modified during execution of the preupdate rules. ClaimCenter adds any objects modified by the preupdate rules to the list of objects needing validation as described by the validation graph.

Overview of rules-based validation

For an entity to have preupdate or validation rules associated with it, the entity must be validatable. To be *validatable*, the entity must implement the `Validatable` delegate, defining an `implementsEntity` element with name set to `Validatable`.

In the base configuration, ClaimCenter provides a number of high-level entities that are validatable:

- `Claim`
- `RITransactionSet`
- `Group`

- | | | |
|---------------|---|----------|
| • Exposure | • ServiceRequest | • Region |
| • Matter | • TransactionSet (and all its subclasses) | • Person |
| • Policy | • Activity | • User |
| • RIAgreement | • Contact | |

Entity validation order

IMPORTANT: ClaimCenter does not use any predetermined order to validate entities of the same type, such as the exposures on a claim.

ClaimCenter evaluates entities in the order set in the `config.xml` configuration parameter `EntityValidationOrder`. Use this parameter to specify a comma-separated list of the validatable entities in the order in which you want ClaimCenter to perform validation. If you do not change this parameter, ClaimCenter performs entity validation by using the default order defined in the base configuration:

- Policy
- Claim
- Exposure
- Matter
- TransactionSet

ClaimCenter validates all other validatable entities not specified in the list after evaluating the listed entities in configuration parameter `EntityValidationOrder`, in no particular order.

About the validation graph

During database commit, ClaimCenter performs validation on the following:

- Any validatable entity that ClaimCenter updated or inserted into the validation graph.
- Any validatable entity that refers to an entity that ClaimCenter updated, inserted, or removed from the validation graph.

ClaimCenter constructs a virtual graph of the entities that reference a changed entity. This graph maps all the paths from each type of entity to the top-level validatable entities, such as `Claim` and `Exposure`. ClaimCenter queries these paths in the database or in memory to determine which validatable entities, if any, reference the entity that ClaimCenter inserted, updated, or removed from the validation graph.

ClaimCenter builds the validation graph by traversing the set of foreign keys and arrays that trigger validation. For example, if the data model marks the `Exposures` array on `Claim` as triggering validation, any change made to an exposure causes ClaimCenter to validate the claim as well.

ClaimCenter follows foreign keys and arrays that trigger validation through any links and arrays on the referenced entities down the object tree. For example, you might end up with a path like `Claim` → `ClaimContact` → `ContactAddress` → `Address`.

Note: To actually trigger validation, you must set each link in the chain—`Address`, `ContactAddress`, and `ClaimContact`—as triggering validation and set `Claim` as validatable.

ClaimCenter stores this path in reverse in the validation graph. Thus, if an address changes, ClaimCenter traverses `Address` → `ContactAddress` → `ClaimContact` → `Claim` to find any claims that reference a changed address.

How ClaimCenter traverses the validation graph

If an entity is validatable, ClaimCenter applies the preupdate and validation rules whenever you modify the entity contents directly. Suppose that you update an object linked to an object. For example, a foreign key links each of the following objects to the `User` object:

- Contact
- Credential
- UserSettings

If you update a user's credential, you might expect the preupdate and validation rules to execute before ClaimCenter commits the updates to the database. However, updating a user's credentials does not normally trigger rules for the container object, User in this case. The reason is that ClaimCenter implements a user's credentials as a foreign key from a User entity to a Credential entity.

To trigger preupdate and validation rules on linked objects, set the `triggersValidation` attribute of the foreign key to the object in the file defining the entity. For example, in `User.eti`, the following foreignkey elements have their `triggersValidation` attributes set to true:

```
Contact
Credential
```

If the container object implements `Validatable`, setting the `triggersValidation` attribute to true ensures that ClaimCenter runs the preupdate and validation rules on the linked object whenever you modify the object.

The `triggersValidation` attribute works from the direction of the element that the foreign key references. If `triggersValidation` is set to true on a foreign key or array element of entity Alpha that links to entity Beta, a change to Beta triggers validation in Alpha. For example, on the Policy object, `triggersValidation` is set to true on the array of ClaimContact objects. Therefore, a change to a ClaimContact object also causes ClaimCenter to validate the Policy object.

In the base configuration, ClaimCenter sets the validation triggers so that modifying a validatable entity in a bundle causes ClaimCenter to validate that entity and all its revisioning parents.

You can use the `triggersValidation` attribute on foreign keys and arrays that you add to custom subtypes, to custom entities, or to core entities by using extensions. The default value of this attribute is false if you do not set it specifically. If you want a new foreign key or array element of a validatable entity to trigger validation whenever its linked entities change, set `triggersValidation` to true.

How ClaimCenter works with owned arrays

In the ClaimCenter data model, an *owned array* is an array that is attached to a validatable entity and which has its `triggersValidation` attribute set to true.

ClaimCenter performs validation based on the validation graph. If an entity changes, ClaimCenter follows all the arrays and foreign keys that reference that entity and for which `triggersValidation` is true. It then uses the graph to find the validatable entities that ultimately reference the changed entity. In the case of owned arrays, ClaimCenter considers a change to the array as a change to the parent entity.

For example, the following data model illustrates this behavior:

- Entity Alpha has a foreign key to Beta
- Entity Beta has an owned array of Chiron
- Array Chiron has a foreign key to Delta

Both Alpha and Beta are validatable. Essentially, these relationships look like the following:

```
Alpha → Beta → Chiron[] → Delta
```

If you also mark both `Alpha → Beta` and `Chiron → Delta` as triggering validation, the following happens with the `Beta → Chiron` link:

- If the `Beta → Chiron` link has its `triggersValidation` element set to false, changes to Chiron cause ClaimCenter to validate Beta and Alpha. This evaluation occurs because ClaimCenter treats the change as a change to Beta directly. However, a change to Delta does not cause ClaimCenter to validate anything—the graph stops at Chiron. Because there is no change to Chiron, ClaimCenter does not consider Beta to have changed and performs no validation.

- If the Beta → Chiron link has its triggersValidation element set to true, changes to either Chiron or Delta cause ClaimCenter to validate both Beta and Alpha.

Top-level entities that trigger validation

To be validatable, an entity, subtype, delegate, or base object must implement the Validatable delegate by adding the following to the entity definition:

```
<implementsEntity name="Validatable"/>
```

The following table lists the entities that trigger validation in the base ClaimCenter configuration. All these entities implement the Validatable delegate. If a table cell is empty, there are no additional entities associated with that top-level entity that trigger validation in the base configuration.

Validatable entity	Foreign key entity	Array name	One-to-one entity
Activity			
Claim	Address	Activities	PropertyFireDamage
	ClaimWorkComp	ConcurrentEmpl	PropertyWaterDamage
	Policy	Contacts	SubrogationSummary
	PolicyLocation	ContribFactors	
		Documents	
		Evaluations	
		Exposures	
		Incidents	
		Matters	
		MetroReports	
		Negotiations	
		Notes	
		Officials	
		OtherBenefits	
		RIAgreementGroups	
		RoleAssignments	
		ServiceRequests	
		SIAnswerSet	
		Text	
Contact	Address	ContactAddresses	
		Reviews	
Exposure	Address	BenefitPeriods	
	Benefits	Documents	
	Coverage	IMEPerformed	
	EmploymentData	MedicalActions	
	Incident	Notes	
	RIAgreementGroup	OtherCoverageDet	
	StatCode	Roles	
		RoleAssignments	
		Settlements	
		Text	
Group			

Validatable entity	Foreign key entity	Array name	One-to-one entity
Matter		Roles StatusTypeLines	
Person	Address	ContactAddresses Reviews	
Policy		ClassCodes Contacts Coverages Endorsements PolicyLocations RiskUnits Roles StatCodes	
Region			
RIAgreement			
RITransactionSet		RITransactions	
ServiceRequest		History Invoices Messages Quotes	
TransactionSet			
User	Credential UserContact UserSettings	Attributes	

In many cases, an entity several levels down triggers validation on one of the top-level entities. For example, Claim preupdate and validation rules run after you update the address for a contact. The Contact entity definition has a `triggersValidation` foreign key attribute to the Address entity. The Claim entity also has a `triggersValidation` foreign key relationship with the Address entity. ClaimCenter follows the chain of triggering validations during a database commit.

Overriding validation triggers

In the base configuration, Guidewire sets validation to trigger on many top-level entities and on many of the arrays and foreign keys associated with them. You cannot modify the base metadata XML files in which these configurations are set. However, in some cases you can override the default validation behavior. For example:

- You want to trigger validation upon modification of an entity in the ClaimCenter base data model that does not currently trigger validation.
- You do not want to trigger validation on entities on which ClaimCenter performs validation in the base configuration.

You can override validation only on certain objects associated with a base configuration entity. In your own extension entities, you simply set `triggersValidation` to the appropriate value.

The following table lists the data objects for which you can override validation, and, the XML override element to use. See the *Data Model Configuration* documentation for information on working with these data elements and fields.

Data field	Override element
<array>	<array-override>
<foreignkey>	<foreignkey-override>
<onetoone>	<onetoone-override>

Validation performance issues

There are three ways in which validation can cause performance problems:

- The rules themselves have performance issues.
- There are too many objects being validated.
- The queries used to determine which objects to validate take too long.

The following list summarizes some of the common validation issues and how to mitigate each one.

Validating large amounts of administration objects

Guidewire recommends that you never mark foreign keys and arrays that point to administration objects, such as User, Group, and Catastrophe objects, as triggering validation. An administration object is any object that a large number of claims can reference. For example, suppose that you set the catastrophe field on Claim to trigger validation. Editing one catastrophe can then bring the entire application to a crawl as ClaimCenter validated hundreds or even thousands of claims. Setting this particular validation trigger would make editing catastrophe objects nearly impossible.

Length of query paths

In some cases, an entity can have a large number of foreign keys pointing at it. Triggering validation on the entity can cause performance problems, as ClaimCenter must follow each of those chains of relationships during validation. The longer the paths through the tables, the more expensive the queries that ClaimCenter executes whenever an object changes. Having a consistent direction for graph references helps to avoid performance problems.

Triggering validation on parent-pointing foreign keys on entities that have many possible owners, like ClaimContact, can result in much longer and unintended paths. For example, a number of entities point to ClaimContact, including Claim, Check, and MedicalTreatment. Each of these entities has links to other entities. Validating the entire web of relationships can have a serious negative performance impact.

In the default configuration, ClaimCenter minimizes this issue. However, if you modify the default configuration and add too many validation trigger overrides, you can introduce unintended performance issues.

Links between top-level objects

It is legal to have top-level entities trigger validation on each other. However, this use of validation triggers can unnecessarily increase the number of paths and the number of objects that ClaimCenter must validate on any particular commit.

Graph direction inconsistency

In general, Guidewire strongly recommends that you consistently order the validation graph to avoid problems:

- ClaimCenter considers arrays and foreign keys that represent some sort of containment as candidates for triggering validation. For instance, you can logically consider contacts and vehicles on a claim as part of the claim.
- ClaimCenter does not consider foreign keys that you reference as arrays or that are merely there to indicate association as candidates for triggering validation. For instance, an Incident on a ServiceRequest entity merely indicates association, rather than indicating that the incident is logically part of the service request.

Illegal links and arrays

Virtual foreign keys cannot trigger validation because ClaimCenter does not store them in the database. Attempting to use a virtual foreign key to trigger validation causes ClaimCenter to report an error at application startup.

Similarly, it is not possible to set any array or link property defined at the application level (such as the Driver link on Claim) to trigger validation.

ClaimCenter considers an array to be in the database if the foreign key that forms the array is in the database.

View the validation graph in the server log

About this task

It is possible to view a text version of the ClaimCenter validation graph for aid in debugging validation issues.

Procedure

1. In Guidewire Studio **Project** window, expand **configuration > config > logging**.
2. Open file log4j2-json.xml for editing.
3. Add the following entry to this file:

```
<!-- ClaimCenter Claim Validation Graph -->
<Logger name="Plugin.com.guidewire.pl.system.bundle.validation.ValidationTriggerGraphImpl" additivity="false"
level="debug">
<AppenderRef="ClaimValidationGraphLog"/>
</Logger>
```

Results

This procedure causes the validation graph to print during application deployment both as text in the system console and to log file /tmp/gwlogs/ClaimValidationGraphLog.log.

See also

- *InsuranceSuite Observability*

Detecting claim fraud in ClaimCenter

Guidewire provides a number of Gosu rules to assist in determining claim fraud, which you can use to trigger a special investigation of a suspicious claim. These rules identify certain characteristics of a claim that increase the suspicion of fraud and assign points to each of these characteristics. Depending on your business logic, you can:

- Assign different point values to these fraud characteristics based on their “severity”
- Set fraud triggers based on the presence of data, the absence of data, or the value of certain data

Using these rules, you can identify a point threshold after which the claim merits human review. A human reviewer (typically, the claim owner’s supervisor) can then determine whether to assign the claim to a Special Investigation Unit for further investigation.

Using Gosu rules to trigger claim fraud investigations provides a number of benefits, including:

- Providing a standardization of the process
- Enforcing business processes across the organization
- Assigning standardized weight to every characteristic evaluated
- Providing transparency of process
- Providing a consistent evaluation of all claims, rather than using intuition
- Keeping a record of the claim review, why the claim was chosen for review and the nature of the review

Many of these benefits can be important from a legal perspective.

The Special Investigation Details screen

In Guidewire ClaimCenter, you use the **Special Investigation Details** screen to view and track details of suspicious claims. This screen contains:

- A read-only list of conditions—Special Investigation (SI) triggers—that this claim violates
- An SI score field, which is a “counter” that tracks the SI points accumulated on this claim
- Fields to track escalation of the claim to the Special Investigation Unit (SIU), the fraud detection team

You control the user interface elements that a specific user can access and view through ClaimCenter permission settings.

Special Investigation trigger rules

Special Investigation (SI) trigger rules flag suspicious information or lack of information on claims. For example, the following occurrences all trigger SI rules:

- All the telephone fields for the contact on the exposure are null.
- The claim file does not record a police report or on-scene report.
- There was an unreasonable delay in reporting the loss, such as a delay of more than 30 days between the loss date and the claim report date.

During evaluation of the SI trigger rules, if a trigger rule evaluates to true and the rule has not yet been logged on the claim, then ClaimCenter:

- Adds the rule to the SI Triggers array
- Increments the SI score by the value specified in the rule actions

If the SI score reaches a defined threshold, ClaimCenter creates an activity for the claim handler's supervisor to review this particular claim. You can control this threshold value, which defaults to 5, by setting the value of the script parameter `SpecialInvestigation_CreateActivityForSupervisorThreshold`.

On receipt of the activity, a claim supervisor reviews the contents of the SI **Details** subtab and the details of the claim. The supervisor can choose to escalate the claim to the Special Investigation Unit. If the claim already has an associated investigator, ClaimCenter sends the activity to that individual.

SI rule evaluation sequence

The following sequence of events relating to special investigations occurs whenever you modify a claim or exposure.

1. ClaimCenter runs the Claim Preupdate or Exposure Preupdate rules.
2. ClaimCenter evaluates the SI-specific trigger rules to determine the following:
 - Are the conditions on the rule met?
 - Does the claim already contain a log of a particular trigger? If not already logged, ClaimCenter writes the trigger to the SI Triggers array and increments the SI score field on the claim in ClaimCenter.
3. ClaimCenter determines if the current SI score exceeds the threshold value set by script parameter `SpecialInvestigation_CreateActivityForSupervisorThreshold`.
 - If so, it creates a Special Investigations review activity for the supervisor of the claim owner.
 - If not, it starts the cycle again.
4. The claim supervisor reviews the claim and decides whether to escalate the claim to the Special Investigation Unit.
5. After ClaimCenter creates an SI escalation activity for the SI team, it assigns the activity to the SI investigator assigned to the claim if there is one. If a SI investigator is not currently assigned to this claim, then ClaimCenter does the following:
 - a. It calls the SI assignment rules to assign the SI escalation activity to the SI group.
 - b. It then uses round-robin assignment to assign the activity to a SI team member.
6. Finally, ClaimCenter adds the SI escalation activity owner to the claim (in the role of SIU Investigator).

Using the Special Investigation rules

It is necessary to evaluate the SI trigger rules at the correct stage of the claim's life cycle. For example, it is not possible to truly evaluate at the first report of the claim whether the claim is missing the police report or whether there are no witnesses. It is possible that this information is only available after a certain number of days have passed.

To handle these situations, the SI rules break the claim life cycle into stages. Different ClaimCenter rule sets handle different stages in the claim life cycle. The following list describes the order in which the rule sets run.

Rule set	Rules	More information
Claim Preupdate Rules	CPU03000 - Set SIU Life Cycle	"Set the initial life cycle stage" on page 197
Claim Exception Rules	CER01000 - Setting SIU Life Cycle State	"Advance the life cycle stage" on page 197
Claim Preupdate Rules	CPU04100 - SIU Life Cycle Stage 1	"Evaluating SI triggers" on page 198
Exposure Preupdate Rules	CPU04200 - SIU Life Cycle Stage 2 EPU03000 - SI Triggers	
Business Rules Editor	CPU5000 - Create Supervisor Review Activity	"Escalating a claim to a supervisor" on page 199
Default Group Activity Assignment Rules	DGA01000 - SI Assign claim review activity to supervisor	
Global Activity Assignment Rules	GAA01000 - SI - Assign Claim Review to Claim Owner's Group GAA02000 - SI - Assign SIU escalation activity to SIU group	"Assigning an activity to a SIU group" on page 200
Default Group Activity Assignment Rules	DGA02000 - SI Assign Escalation Activity to named SIU user DGA03000 - SI Default SI Escalation activity routing	"Assigning an activity to a SIU group" on page 200
Activity Postsetup Rules	APS01000 - SI - Add user in SIU Role to claim if necessary	"Adding an assigned user in a SIU role" on page 200

Set the initial life cycle stage

ClaimCenter uses claim property `SIULifeCycleState` to track a claim's stage.

Claim Preupdate rule CPU03000 - Set SIU Life Cycle sets the initial claim stage to 1 (step1) the first time that ClaimCenter runs the preupdate rules against a claim. To see this rule, navigate to the following location in Guidewire Studio **Project** window:

configuration > config > Rule Sets > Preupdate > ClaimPreupdate

ClaimCenter uses the value of a claim's state to determine which SIU (Special Investigation Unit) rules to run against the claim.

```
CONDITION (claim : entity.Claim):
return claim.SIULifeCycleState == null and Claim.State != TC_DRAFT

ACTION (claim : entity.Claim, actions : gw.rules.Action):
claim.SIULifeCycleState="step1"
```

Advance the life cycle stage

A rule in the Exception rule set category advances the claim stage based on the number of days that have passed since claim inception. ClaimCenter automatically runs the claim exception rules on a daily basis. In the base configuration, these rules run at 2:00 a.m. server time.

Claim exception rule CER01000 - Setting SIU Life Cycle State calls the ClaimCenter library method `SetSIULifeCycleState`, which actually sets the state. To see this rule, navigate to the following location in the Studio **Project** window:

configuration > config > Rule Sets > Exception > ClaimExceptionRules

The following table lists the default configuration for the various stages. You can customize this process by setting new values in the library method.

Stage	Number of days since creation
1	Less than 5
2	More than 5 and less than 20
3	More than 20

ClaimCenter evaluates claims not touched today on the standard rule schedule in the Exception rule set category.

Evaluating SI triggers

Each time that ClaimCenter runs the Claim Preupdate rules for a claim or the Exposure Preupdate rules for an exposure, ClaimCenter evaluates SI (Special Investigation) rules. For example, suppose that ClaimCenter pushes a claim to stage 2 during the evening's Exception process. Then, the next time that ClaimCenter evaluates the Claim Preupdate rules, it also evaluates the SI rules associated with stage 2.

Claim Preupdate SI rules for stage 1

Guidewire provides several sample stage 1 SI (Special Investigation) rules in the Claim Preupdate rule set. These rules add SIU life cycle points under various conditions. To see these rules, navigate to the following location in the Guidewire Studio **Project** window:

configuration > config > Rule Sets > Preupdate > ClaimPreupdate > CPU04000 - SI Triggers

The following list describes the base configuration SI rules.

CPU04100 - SIU Life Cycle Stage 1

Determines if the SIU Life Cycle stage is step1. If so, ClaimCenter evaluates the Stage 1 child rules that follow.

CPU04110 - SI - Unreasonable delay in reporting loss

If the claimant filed the claim more than 30 days after the reported loss date, this SI trigger rule increases the trigger value by a point value of 1. You can change the default number of days, the point value, the trigger description, and the additional information.

CPU04120 - SI - Claim within 15 days of policy inception

If the claimant filed the claim within 15 days of the policy issuance, this SI trigger rule increases the trigger value by a default value of 1. You can change the default number of days, the point value, the trigger description, and the additional information.

CPU04130 - SI - Claim within 30 days of policy inception

If the claimant filed the claim within 30 days of the policy issuance, this SI trigger rule increases the trigger value by a default value of 1. You can change the default number of days, the point value, the trigger description, and the additional information.

CPU04140 - SI - Claim loss is theft related

If the claim loss is theft-related, this SI trigger rule increases the trigger value by a default value of 1. You can change the point value, the trigger description, and the additional information.

Claim Preupdate SI rules for stage 2

Guidewire provides several sample stage 2 SI (Special Investigation) rules in the Claim Preupdate rule set. These rules add SIU life cycle points under various conditions. To see these rules, navigate to the following location in the Guidewire Studio **Project** window:

configuration > config > Rule Sets > Preupdate > ClaimPreupdate > CPU04000 - SI Triggers

The following list describes the base configuration SI rules.

CPU04200 - SIU Life Cycle Stage 2

This rule determines if the SIU Life Cycle stage is `step2`. If so, ClaimCenter evaluates the child Stage 2 rules that follow.

CPU04210 - SI - No on-scene reports of incident

If the police report is not available for this claim, this SI trigger rule increases the trigger value by a point value of 1. You can change the point value, the trigger description, and the additional information.

CPU04220 - SI - Minor not on policy

Guidewire disables this rule in the base configuration. If the driver was a minor and not listed on the policy, this SI trigger rule increases the trigger value by a default value of 1. You can change the point value, the trigger description, and the additional information.

Exposure Preupdate SIU rules

ClaimCenter provides sample Exposure Preupdate SI (Special Investigation) rules that you can enable to run every time there is an update or validation of an exposure. To see these rules, navigate to the following location in the Guidewire Studio **Project** window:

configuration > config > Rule Sets > Preupdate > ExposurePreupdate > EPU03000 - SI Triggers

The following list describes the base configuration SI rules.

EPU03000 - SI Triggers

A folder to group the Exposure Preupdate SIU rules.

EPU03100 - SI - Claimant has no phone number listed

Guidewire disables this rule in the base configuration. If the claimant has no phone number, this SI trigger rule increases the trigger value by a point value of 1. You can change the point value, the trigger description, and the additional information.

This rule is not active in the base configuration.

EPU03200 - SI - Claimant has POBOX address

Guidewire disables this rule in the base configuration. If the claimant has a primary address that is a post office box, this SI trigger rule increases the trigger value by a default value of 5. You can change the point value, the trigger description, and the additional information.

This rule is not active in the base configuration.

Escalating a claim to a supervisor

In the base configuration, ClaimCenter provides the following business rule to create a supervisor review activity based on the SIU score:

CPU05000 - SI - Create Supervisor Review Activity

You can find this business rule in the ClaimCenter **Administration** screens at the following location:

Business Settings > Business Rules > Activity Rules

This rule is available as both a Gosu rule in Studio and as a business rule within ClaimCenter. However, in the base configuration, Guidewire disables Gosu Claim Preupdate rule CPU05000 as the ClaimCenter business rule handles this functionality.

To view the disabled Gosu rule in Studio, navigate to the following location in the Guidewire Studio **Project** window:

configuration > config > Rule Sets > Preupdate > ClaimPreupdate > CPU05000- SI - Create Supervisor Review Activity

The business rule compares the SIU total score for the claim to the script parameter `SpecialInvestigation_CreateActivityForSupervisorThreshold`, which in the base configuration is set to a value of 5.

If the rule creates the activity for the supervisor, this action triggers the following SIU activity assignment rules.

1. A global activity assignment rule, GAA01000 - SI - Assign Claim Review to Claim Owner's Group, assigns the claim review activity to the claim owner's group.

2. A default group activity assignment rule, DGA01000 - SI - Assign Claim Review Activity to Supervisor, assigns the review activity to the supervisor of the group.
3. The group supervisor then reviews the claim and determines whether it requires further action.

Note: You can set the value for a script parameter on the **Administration** tab in Guidewire ClaimCenter.

Assigning an activity to a SIU group

After the escalation of a claim to the reviewing supervisor, the supervisor can review the claim further. The supervisor can also escalate the claim to a Special Investigation group to investigate the claim further.

ClaimCenter provides the following assignment rules to assist in this process.

GAA02000 - SI Assign SIU escalation activity to SIU group

If there is a Special Investigator associated with the claim, assign the SI escalation activity to that user's group. Otherwise, in the base configuration, the assignment goes to the Western SIU group.

If there is a Special Investigator associated with the claim, assign the SI escalation activity to that user's group. Otherwise, in the base configuration, the assignment goes to the Western SIU group.

DGA02000 - SI Assign Escalation Activity to named SIU user

If there is a Special Investigator associated with the claim, assign the SI escalation activity to the person with that role.

This rule is in the Default Group Activity Assignment rules.

DGA03000 - Default SI Escalation activity routing

If the previous rule does not result in an assignment, use round-robin assignment to assign the SI escalation activity to a member of the SI group.

This rule is in the Default Group Activity Assignment rules.

Adding an assigned user in a SIU role

As described in "Assigning an activity to a SIU group" on page 200, the ClaimCenter assignment rules assign an escalation activity to a member of the Special Investigation team. After these assignment rules run, ClaimCenter runs an additional Activity Postsetup rule. This rule associates the user with the suspect claim.

APS01000 - SI - Add user in SIU Role to claim if necessary

Determine if the assigned user has the SIU Investigator role. If not, add this user to the claim in that role.

Transaction Set validation in ClaimCenter

Insurance companies typically sell vehicle liability insurance as a package, with the insurer's liability limited to each person, each occurrence (incident), and total property damage. For example, a 100/300/50 package limits the maximum payout in an accident for which an insurer would be liable to the following:

- \$100,000 in Bodily Injury (BI) coverage for each person
- \$300,000 in Bodily Injury (BI) coverage for all persons
- \$50,000 for all Property Damage (PD)

By using the ClaimCenter transaction validation rules and library functions, you can track these limits and raise alerts if a transaction exceeds the exposure or incident limit. You can set these rules either to block the transaction from going through or to allow the transaction to continue, but with warnings.

Note: Guidewire provides the Transaction Set Validation rules as sample rules only. Customize these rules to meet your business logic as appropriate (for example, by modifying the reserve definition to meet your business needs).

Transaction Set validation rules

ClaimCenter provides sample transaction validation rules that block or warn the user from executing a financial transaction that exceeds the policy limits. There is another rule that is specific to PIP (Personal Injury Protection) that blocks the user from making payments that exceed the PIP aggregate limit ("TXV05000 – PIP" on page 204).

To access the Transaction Set Validation Rules, navigate to the following location in the Guidewire Studio **Project** window:

configuration > config > Rules Sets > Validation > TransactionSetValidationRules

The sample Transaction Set Validation rules execute correctly only if both of the following are true:

- The created exposure is tied to the coverage, which is visible in the **Exposure** screen.
- The corresponding coverages are defined at the vehicle level on the policy's **Vehicles** screen.

ClaimCenter runs the validation rules whenever it updates a ClaimCenter business entity (Claim, Exposure, or TransactionSet, for example) and commits data to the database.

These sample rules calculate payments values using *claim cost*, which includes only costs directly associated with the claim, and not ancillary costs such as expenses. The sample rules also calculate all values by using the financial calculations *RemainingReserves* value. The rule rules obtain this value by calling the *getRemainingReserves* method:

```
var RemainingReservesCache = gw.api.financials.FinancialsCalculationUtil.getRemainingReserves()
```

To work with a different type of reserve value, use a different getter method. For example, to access the *AvailableReserves* value, use the *getAvailableReserves* method:

```
var AvailableReservesCache = gw.api.financials.FinancialsCalculationUtil.getAvailableReserves()
```

TXV01000 – Total payments not greater than exposure limit

ClaimCenter executes the TXV01000 rule whenever you attempt to create a new check, for example:

```
transactionset.Subtype == "CheckSet" && transactionset.new && transactionset.ApprovalStatus == "unapproved"
```

In the base configuration, this rule has the following behavior.

Rule	Condition	Action
TXV01000	1. (New payment + Current Total Payments already made) > Exposure Limit	1. Error, block
	2. Above condition false <i>and</i> (New payment + Current Total Payments already made + Remaining Reserves) > Exposure Limit	2. Warn, but permit

Use rule TXV01000 to either block or permit with warnings a payment that would cause one of the following conditions:

- Making a new payment and adding it to the payments already made causes the total payments to exceed the Exposure limit.
- Making a new payment and adding it to the payments already made plus the remaining reserves causes the total payments to exceed the exposure limit. For this condition to be true, the previous condition must be false. Thus, this condition assumes that the remaining reserves is greater than the exposure limit.

The rule declares two variables, one for each listed condition, that determine the outcome of that condition.

Variable	Effect
warning1	true – warn, but permit; false – reject and block
warning2	true – warn, but permit; false – reject and block

Example 1: Payment Would Cause Total Payments to Exceed Exposure Limit

Suppose that you have created a claim with a Vehicle Damage exposure. The exposure limit is \$10,000 with a reserve of \$8,000. As you make payments (transactions 1 and 2), you deplete the reserve, but the total payout is still below both the exposure limit and the reserve limit. An attempted third payment now brings the payment total to \$10,900, which is over the exposure limit of \$10,000

Exposure Limit	Payment	Amount	Rule action
\$10,000	1	\$2,000	Permit the transaction as payment of \$2,000 < Exposure Limit of \$10,000
	2	\$5,500	Permit the transaction as \$2,000 + \$5,500 < \$10,000
	3	\$3,400	Block the transaction as \$2,000 + \$5,500 + \$3,400 > \$10,000
Total		\$10,900	

Depending on your business needs, you might want to raise an alert if a payment causes the payment total to exceed a reserve or exposure limit. In some cases, you might want to block the transaction altogether. The sample rules in the Transaction Set Validation rules handle either of these situations easily.

Example 2: Payment Plus Remaining Reserves Would Cause Total Payments to Exceed Exposure Limit

Exposure Limit	Remaining Reserves	Payment	Amount	Rule action
\$10,000	\$8,000	1	\$1,500	Permit the transaction as the payment of \$1,500 + \$8,000 (Remaining Reserves) < \$10,000 (Exposure limit)
	\$6,500			
		2	\$1,000	Permit the transaction as \$1,000 + \$1,500 + \$6,500 < \$10,000
	\$5,500			
		3	\$3,000	Warn the user as \$1,500 + \$1,000 + \$3,000 + \$5,500 > \$10,000, but permit the user to execute the transaction if desired

TXV02000 – Check exposure limit when increasing reserves

ClaimCenter executes rule TXV02000 whenever you attempt to create a new reserve or, more specifically, if you attempt to create a new TransactionSet of type ReserveSet, for example:

```
transactionset.Subtype == "ReserveSet" && transactionset.new
```

In this base configuration, rule TXV02000 has the following behavior.

Rule	Condition	Action
TXV02000	(New reserve amount + Current Total Payments + Remaining Reserves) > Exposure Limit	Warn, but permit

You can use rule TXV02000 to either block (reject completely) or permit (with warnings) a reserve increase. For example, you can reject a reserve increase that would cause the sum of the combined current payments, the new reserve amount, and remaining reserve to exceed the exposure limit. To illustrate, suppose that the exposure limit is \$10,000 and the initial reserve limit is \$8,500, and you are attempting to make a payment of \$1,000. The remaining reserve after this transaction equals \$7,500. Increasing the reserve by \$2,000 would create a combined total of \$10,500 (\$7,500 + \$2,000 + \$1,000), which exceeds the exposure limit.

Exposure Limit	Remaining Reserves	Payments	Rule Action
\$10,000	\$8,500	\$1,000	Permit the transaction as \$1,000 + \$8,500 < \$10,000, leaving a Remaining Reserves of \$7,500
	\$7,500		
	\$7,500 + \$2,000 = \$9,500		Block (or permit with warnings) as attempting to increase the reserves by \$2,000 to \$9,500 fails (\$9,500 + \$1,000 > \$10,000)

You set the outcome of rule TXV02000, to block or warn, by setting the variable warning in the rule.

Variable Value

warning true – warn, but permit; false – reject and block

TXV03000 – Total payments not greater than incident limit

ClaimCenter executes rule TVX03000 if you attempt to create a new check.

```
transactionset.Subtype == TC_CHECKSET && transactionset.new && transactionset.ApprovalStatus == TC_UNAPPROVED
```

In the base configuration, rule TVX03000 has the following behaviour.

Rule	Condition	Action
TXV03000	1. (New payment + Current Total Payments already made for all exposures tied to the same coverage) > Incident Limit	1. Error, block
	2. Above condition false <i>and</i> (New payment + Current Total Payments for all exposures tied to the same coverage + Remaining Reserves) > Incident Limit	2. Warn, but permit

This sample rule works in a similar manner to rule TXV01000, except that it applies to incident limits, not exposure limits.

Use rule TXV03000 to either block or permit, with warnings, a payment that would cause one of the listed conditions.

The rule declares two variables, one for each listed condition, that determine the outcome of that condition.

Variable Value

warning1 true – warn, but permit; false – reject and block

warning2 true – warn, but permit; false – reject and block

Note that rule TXV03000 handles one vehicle per incident only.

TXV04000 – Check incident limit when increasing reserves

ClaimCenter executes rule TXV04000 whenever you attempt to create a new reserve.

```
transactionset.Subtype == TC_RESERVESET && transactionset.new
```

In the base configuration, rule TXV04000 has the following behavior.

Rule	Condition	Action
TXV04000	(New reserve amount + Current Total Payments for all exposures tied to the same coverage + Remaining Reserves) > Incident Limit	Warn, but permit

This sample rule works in a similar manner to TXV02000, except that it applies to incident limits, not exposure limits.

You set the outcome of rule TXV04000, either block or warn, by setting the variable warning in the rule.

Variable Value

warning true – warn, but permit; false – reject and block

Note that rule TXV04000 handles one vehicle per incident only.

TXV05000 – PIP

ClaimCenter executes rule TVX05000 whenever it validates a check.

```
TransactionSet.Subtype == TC_CHECKSET
```

In the base configuration, rule TVX05000 has the following behavior.

Rule	Condition	Action
TXV05000	Payments on PIP coverages exceed one of the four defined PIP Aggregate Limits	Warn, but permit

This sample rule determines whether adding this new amount to the payments already made for this exposure would cause the total payments to exceed one of the following limits.

Limit	Description
PIPNonmedAgg	Non-medical aggregate limit
PIPReplaceAgg	Lost wages and replacement services aggregate limit
PIPPersonAgg	PIP per person aggregate limit
PIPClaimAgg	PIP per claim aggregate limit

Note: ClaimCenter calculates these aggregate limits in Claim enhancement method `sumForPIPAgg`.

Gosu code within rule TVX05000 looks at each limit and determines whether the new payment total exceeds a specific aggregate limit. If so, the rule permits the transaction, but raises a warning within ClaimCenter. The user must then explicitly cancel or continue the transaction. For example, if the new payment causes the payment total to exceed the `PIPPersonAgg` limit, it raises the following warning

```
TransactionSet.reject( null, null, "external", "The PIP Per Person Aggregate Limit ($"
+ (each.Exposure.Coverage as VehicleCoverage).PersonAggLimit
+ ") has been exceeded for this coverage, " + each.Exposure.Coverage.Type
+ ". Do you still want to continue? )
```

At the same time, the rule sets a Boolean flag to indicate that the sum of all payments, including this new one, now exceeds one of the aggregate limits. There are four of these flags, corresponding to the four aggregate limits:

- `PIPNonmedAggLimitReached`
- `PIPReplaceAggLimitReached`
- `PIPPersonAggLimitReached`
- `PIPClaimAggLimitReached`

Other Gosu rules can access these values as properties on the Exposure entity. For example, a rule in the Exposure Preupdate rule set could use the following code to determine if the aggregate limit for each person has been reached for this exposure:

```
var PersonAggLimitReached = exposure.PIPPersonAggLimitReached
```

TXV06000 – Reserve threshold

In the base configuration, the TXV06000 rule imposes currency-specific hardcoded thresholds for reserve creation. Thresholds are only available for the currencies provided in the base application. The purpose of these thresholds is to limit the amount of a single reserve (`ReserveSet`) when creating reserves in ClaimCenter. If the threshold is exceeded, an error appears and the transaction is not committed.

Threshold amounts in the base configuration are as follows:

Supported Currency	Threshold
USD	1,000,000
EUR	1,000,000
GBP	1,000,000

Supported Currency	Threshold
CAD	1,000,000
AUD	1,000,000
JPY	100,000,000
RUB	100,000,000

The threshold values for the supported currencies can be configured in the `ReserveSetMaxValidator.gs` Gosu class. For unsupported currencies not in the previous list, the default threshold value is 1,000,000. Unsupported currencies can be added in the Gosu class.

Note: The `ReserveSetMaxValidator.gs` file is located in `gsrsrc/gw/financials/ReserveSetMaxValidator.gs`.

TXV08000 – Check aggregate limits

ClaimCenter executes the TXV08000 rule to notify users whenever it determines that an amount exceeds an aggregate limit, or it deems the value invalid. In the base configuration, ClaimCenter displays a warning in these cases, but allows the transaction to go through.

Rule	Condition	Action
TXV08000	Transaction results in an aggregate limit being exceeded or marked invalid	Warn, but permit

It is possible to mark aggregate limits as invalid in the following ways:

- Whenever a staging table load brings in new transactions that impact an existing or newly loaded aggregate limit. In this case, ClaimCenter marks the aggregate limits on the corresponding policy period as invalid.
- Whenever a system administrator requests a recalculation of all aggregate limit values and the `AggLimitCalc` batch process, which runs in the background, is not complete. In this scenario, ClaimCenter initially marks all aggregate limits as invalid. As the rebuild process progresses through the database, ClaimCenter marks each aggregate limit as valid, one by one.

TXV13000 – Warning if adverse party pays non-subrogation recovery

The TXV13000 rule validates transactions. It prevents ClaimCenter from saving a new transaction if both of the following are true:

- The transaction involves subrogation.
- Any of its adverse party contacts has made a recovery payment that is not for subrogation.

Rule	Condition	Action
TXV13000	Checks whether the following conditions are all true:	Error, block
	• Is the transaction new? • Is the transaction a recovery? • Does the transaction involve subrogation.	

If all the listed conditions are true, ClaimCenter collects all the adverse party contacts and determines for each of these contacts if it has the subrogation recovery category. If any of these contacts does not have this category, ClaimCenter blocks the transaction and sends a message to the user.

TXV17000 – Reserve lines compatible with service requests

ClaimCenter executes the TXV17000 rule whenever it validates a check. It validates all checks linked to a service request invoice to determine if the check's set of reserve lines is compatible with at least one of the service request invoices.

It is compatible if one of the following conditions is met:

- One of the associated service requests is claim-level.
- Each exposure-level payment shares an incident with one of the associated service requests.

Rule	Condition	Action
TXV17000	Check's reserve line is compatible with one of the service request's invoices.	Error, block

TXV18000 – Currencies compatible with service requests

The TXV18000 rule validates currencies associated with a service request invoice.

Rule	Condition	Action
TXV18000	Check and service request invoice currencies match.	Error, block

TXV24000 - Disallow When Policy Agreement is Inconsistent

ClaimCenter executes the TXV24000 rule to prevent transactions on claims when the claim's policy agreement has not reached the ability to pay validation level. Claims with a policy agreement are used exclusively by coinsurance claims.

In these cases, the following error is shown:

The policy has only reached the '{0}' validation level, which is insufficient. Please validate the policy against '{1}' to find issues preventing it from reaching that level.

Rule	Condition	Action
TXV24000	Checks whether the following conditions are all true: • Does the claim's policy have a coinsurance agreement? • Is the policy validation level priority greater than the Ability to pay validation level priority (100)? Note: The validation level priority value is a numeric value where 100 is the highest level (Ability to pay) and 400 is the lowest level (Load and save).	Error, block

See also

- “About ClaimCenter validation levels” on page 108

sumForPIPAgg enhancement methods

ClaimCenter Gosu rules use the sumForPIPAgg enhancement methods on the Claim entity in working with PIP-related transactions. To access the sumForPIPAgg enhancement class, navigate to the following location in the Guidewire Studio **Project** window:

configuration > gsrc > libraries

Several of the sumForPIPAgg enhancement methods, in turn, contain method calls to other enhancement methods. In the base configuration, these enhancement methods calculate the aggregates sums of the following coverage types per incident:

Aggregate method	Sums these coverage types
sumForPIPClaimAgg	• PIP Medical • PIP Rehabilitation

Aggregate method	Sums these coverage types
	• PIP Lost Wages • PIP Funeral • PIP Death • PIP Extraordinary Medical
sumForPIPNonmedAgg	• PIP Rehabilitation • PIP Lost Wages • PIP Funeral • PIP Death
sumForPIPPersonAgg	• PIP Medical • PIP Rehabilitation • PIP Lost Wages • PIP Funeral • PIP Death • PIP Extraordinary Medical
sumForPIPReplaceAgg	PIP Lost Wages

The TXV05000 sample rule uses these summing methods as methods on the Exposure entity, for example:

```
each.Exposure.Claim.sumForPIPNonmedAgg(Claimant) > (each.Exposure.Coverage as
VehicleCoverage).NonmedAggLimit)
```


Working with queues in ClaimCenter

An *activity queue* is a list of activities that are ready to be chosen and performed by users of ClaimCenter. An activity must have an assigned owner before it is possible to complete or skip the activity.

Assignment review activities

ClaimCenter does not route activities of type *assignment review* to a queue, even if you explicitly call for this behavior. Therefore, these types of actives do not show in the **Queue** tab in ClaimCenter. Instead, ClaimCenter displays these activities in the **Pending Assignments** screen.

Example: Using a queue to handle FNOL claims

The following example shows how to use a queue to handle FNOL claims. If a claim enters ClaimCenter as an FNOL (First Notification of Loss), it triggers the ClaimCenter Global Assignment rules. However, in this example, the assignment rules do not assign the claim. Instead, these rules generate review activities, which ClaimCenter then places in a queue for response by a team of people who manually assign incoming claims.

ClaimCenter associates every queue with a single group, or potentially with a group and its subgroups. ClaimCenter automatically creates a default FNOL queue for each group in the hierarchy. This queue is not visible to the subgroups, by default.

This example performs the following sequence of actions, using a specific Gosu rule at each step to perform the necessary functionality.

Action	More information
Assign the incoming FNOL claim to an intake group to make an assignment decision on that claim.	“Assign a claim to an intake group” on page 210
Assign the claim to the group’s supervisor. This “parks” the claim until the final assignment determination.	“Assign a claim to a group supervisor” on page 210
Create the FNOL review activity.	“Create a FNOL review activity” on page 211
Assign the FNOL review activity to the intake group designated in the first assignment rule.	“Assign a FNOL review activity to an intake group” on page 211
Place the review activity on the FNOL queue of the designated intake group, from which members of the group then pull a review activity	“Assign a FNOL review activity to a queue” on page 212

Assign a claim to an intake group

Before you begin

Before proceeding, review “Example: Using a queue to handle FNOL claims” on page 209.

About this task

Global Claim Assignment rule ASGCDefault - Assign to Claim Intake Units assigns an incoming claim to a specific group or groups. This group consists of users who make the assignment decisions. This example uses two groups:

- FNOL Review Group - WC
- FNOL Review Group - GL

There are no conditions for this rule, so the rule fires for every claim that comes into ClaimCenter.

Procedure

1. In the ClaimCenter **Project** window, expand the following navigation path:
configuration > config > Rule Sets > Assignment > GlobalClaimAssingmentRules
2. Create the following new rule:
ASGCDefault - Assign to Claim Intake Units
3. Ensure that this rule is the first rule in the rule hierarchy.
4. Populate the rule with the following code:

```
CONDITION (claim : entity.Claim) :
    return true

ACTION (claim : entity.Claim, actions : gw.rules.Action) :
    var FNOLreview : Group //Some FNOL review group

    if (claim.LossType == TC_WC ) {
        if ( claim.assignGroup( FNOLreview ) ) { //Claim assigned to FNOL Review Group - WC
            actions.exit() }
        } else {
            if (claim.assignGroup( FNOLreview ) ) { //Claim assigned to FNOL Review Group - GL
                actions.exit()
            }
        }
    }
```

What to do next

After completing this procedure, proceed to “Assign a claim to a group supervisor” on page 210.

Assign a claim to a group supervisor

Before you begin

Before proceeding, complete “Assign a claim to an intake group” on page 210.

About this task

Default Group Claim Assignment rule ASDGCDefault - Assign to Claim Group Supervisor assigns the claim to the supervisor of the group. There are no conditions for this rule, so the rule fires for every claim that comes into ClaimCenter.

Procedure

1. In the ClaimCenter **Project** window, expand the following navigation path:
configuration > config > Rule Sets > Assignment > DefaultGroupClaimAssignmentRules
2. Create the following new rule:
ASDGCDefault - Assign to Claim Group Supervisor

3. Ensure that this rule is the first rule in the rule hierarchy.
4. Populate the rule with the following code:

```
CONDITION (claim : entity.Claim)
    return true

ACTION (claim : entity.Claim, actions : gw.rules.Action)
    claim.assign(claim.AssignedGroup, claim.AssignedGroup.Supervisor)
```

What to do next

After completing this procedure, proceed to “Create a FNOL review activity” on page 211.

Create a FNOL review activity

Before you begin

Before proceeding, complete “Assign a claim to a group supervisor” on page 210.

About this task

Claim Workplan rule WPWC03 - FNOL Review creates the review activity, which ClaimCenter then places in the FNOL queue. This activity tells the user to review and assign the claim. There are no conditions for this rule, so the rule fires for every claim that comes into ClaimCenter.

Procedure

1. Create an activity pattern with the code `fnol_review`. In the ClaimCenter **Administration** tab, navigate to the **Business Settings > Activity Patterns** screen.
2. In the ClaimCenter **Project** window, expand the following navigation path:
configuration > config > Rule Sets > Workplan > ClaimWorkplan
3. Create the following new rule:
WPWC03 - FNOL Review
4. Ensure that this rule is the last rule in the rule hierarchy.
5. Populate the rule with the following code:

```
CONDITION (claim : entity.Claim):
    return true

ACTION (claim : entity.Claim, actions : gw.rules.Action):
    claim.createActivityFromPattern(null, ActivityPattern.finder.getActivityPatternByCode("fnol_review"))
```

What to do next

After completing this procedure, proceed to “Assign a FNOL review activity to an intake group” on page 211.

Assign a FNOL review activity to an intake group

Before you begin

Before proceeding, complete “Create a FNOL review activity” on page 211.

About this task

Global Activity Assignment rule ASGA30 - FNOL Review Assignments assigns the activity to the group that has already been assigned to the claim. The condition on the rule ensures that the rule assigns only this specific type of activity. Otherwise, ClaimCenter would assign all activities that trigger this rule to the queue. Notice that the activity assignment happens as part of the `if` condition, which returns `true` if successful.

Procedure

1. In the ClaimCenter **Project** window, expand the following navigation path:
configuration > config > Rule Sets > Assignment > GlobalActivityAssignmentRules
2. Create the following new rule:
ASGA30 - FNOL Review Assignments
3. Ensure that this rule is the first rule in the rule hierarchy.
4. Populate the rule with the following code:

```
CONDITION (activity : entity.Activity, actions : gw.rules.Action) :
    return activity.ActivityPattern.Code == "fnol_review"

ACTION (activity : entity.Activity, actions : gw.rules.Action) :
    if ( activity.assignGroup( activity.claim.AssignedGroup ) ) {
        actions.exit()
    }
```

What to do next

After completing this procedure, proceed to “Assign a FNOL review activity to a queue” on page 212.

Assign a FNOL review activity to a queue

Before you begin

Before proceeding, complete “Assign a FNOL review activity to an intake group” on page 211.

About this task

Default Group Activity Assignment rule ASDGA01 - Assign New Claim Activity to Queue adds the review activity to the queue. This rule condition also specifies the correct activity type.

Note: ClaimCenter defines the default FNOL queue as `AssignableQueues[0]`.

Procedure

1. In the ClaimCenter **Project** window, expand the following navigation path:
configuration > config > Rule Sets > Assignment > DefaultGroupActivityAssignmentRules
2. Create the following new rule:
ASDGA01 - Assign New Claim Activity to Queue
3. Ensure that this rule is the first rule in the rule hierarchy.
4. Populate the rule with the following code:

```
CONDITION (activity : entity.Activity) :
    return activity.ActivityPattern.Code == "fnol_review"

ACTION (activity : entity.Activity, actions : gw.rules.Actions) :
    activity.assignActivityToQueue( activity.AssignedGroup.AssignableQueues[0],
        activity.AssignedGroup)
    actions.exit()
```

Example: Using a queue to void/stop checks

You can configure ClaimCenter to use a queue to handle Void/Stop Payment activity requests. In this scenario, ClaimCenter does not allow random adjusters to void or stop checks. Instead, individual adjusters create an activity requesting a void/stop of a check. The activity enters a queue from which a member of the group of accountants takes the activity and completes it.

This example uses two rules that assign an activity that an end-user created manually. Therefore, there is no need for claim assignment or workplan rules, as there is in “Example: Using a queue to handle FNOL claims” on page 209. All

that is necessary are the two activity assignment rules. However, you must first create a Stop/Void queue if one does not exist.

This example performs the following sequence of actions.

Step action	More information
Create the void/stop queue.	"Create a stop/void queue" on page 213
The first rule assigns an activity of type <code>void_or_stop_check</code> to a particular group, the Accountants group.	"Assign an activity to the appropriate group" on page 213
The second rule adds the activity to the void/stop queue.	"Add an activity to a queue" on page 214

Create a stop/void queue

Before you begin

Before proceeding, review "Example: Using a queue to void/stop checks" on page 212.

About this task

ClaimCenter associates each queue with a single group, or potentially with a group and its subgroups. Therefore, the first step in this example is to associate the group that is responsible for voiding/stopping payments with the void/stop queue. If a void/stop queue does not exist, you must create one. If the group responsible for voiding/stopping payments does not exist, you must define it first as well.

If the appropriate queue does not exist, use the ClaimCenter **Administration** tab to create one.

Procedure

1. Log into Guidewire ClaimCenter using an administrative account.
2. Select the **Administration** tab.
3. In the left-hand navigation pane, select a specific group in the group hierarchy tree.
This example uses the Accountants group to handle the voiding and stopping of checks. The members of this group are the only ones assigned the proper role to void/stop payments.
4. Then, select the **Queues** tab for that group.
5. Click **Edit** and add the queue.

What to do next

After completing this procedure, proceed to "Assign an activity to the appropriate group" on page 213.

Assign an activity to the appropriate group

Before you begin

Before proceeding, complete "Create a stop/void queue" on page 213.

About this task

In this second step of the example of using a queue to void or stop checks, you create a rule that assigns an activity if appropriate. Global Activity Assignment rule ASGA40 - Void/Stop Payments checks to see if the activity is of type `void_or_stop_check`. If it is, the rule assigns the activity to the Accounts group.

Procedure

1. In the ClaimCenter **Project** window, expand the following navigation path:
configuration > config > Rule Sets > Assignment > GlobalActivityAssignmentRules

2. Create the following new rule:
ASDGA01 - Assign New Claim Activity to Queue
3. Ensure that this rule is the first rule in the rule hierarchy.
4. Populate the rule with the following code:

```
CONDITION (activity : entity.Activity) :
    return activity.ActivityPattern.Code == "void_or_stop_check"

ACTION (activity : entity.Activity, actions : gw.rules.Actions) :
    if (activity.assignGroup( Group( "cc:96" /* Accountants */ ) ) ) {
        actions.exit()
    }
```

This rule example uses a place holder group for demonstration purposes.

What to do next

After completing this procedure, proceed to “Add an activity to a queue” on page 214.

Add an activity to a queue

Before you begin

Before proceeding, complete “Assign an activity to the appropriate group” on page 213.

About this task

In this third step of the example of using a queue to void or stop checks, you create a rule that adds an activity to a queue. Default Group Activity Assignment rule ASDGA40 - Assign Void/Stop Activity to Queue adds the activity to the queue. The rule condition requires that the activity be of the correct activity type.

Note: The rule refers to `AssignableQueues[1]` because this queue is presumably the first new queue added to this group. If you have multiple queues on a group, you need to choose the correct one. `AssignableQueues[0]` refers to the default FNOL queue that ClaimCenter creates whenever you create a new group.

Procedure

1. In the ClaimCenter **Project** window, expand the following navigation path:
configuration > config > Rule Sets > Assignment > DefaultGroupActivityAssignmentRules
2. Create the following new rule:
ASDGA40 - Assign Void/Stop Activity to Queue
3. Ensure that this rule is the first rule in the rule hierarchy.
4. Populate the rule with the following code:

```
CONDITION (activity : entity.Activity) :
    return activity.ActivityPattern.Code == "void_or_stop_check"

ACTION (activity : entity.Activity, actions : gw.rules.Action) :
    activity.assignActivityToQueue( activity.AssignedGroup.AssignableQueues[1],
        activity.AssignedGroup)
    actions.exit()
```

PolicyCenter Rule-based validation

Rules-based validation uses Gosu rules that you define through the Studio Rules editor. Rules-based validation works on non-policy objects only, objects that are not part of the policy graph.

For an entity to have validation rules associated with it, the entity must be validatable. The entity must implement the `Validatable` delegate using `<implementsEntity name="Validatable"/>` in the entity definition. In the base configuration, Guidewire PolicyCenter comes preconfigured with a number of high-level entities that trigger validation (meaning that they are validatable). These entities are:

- Account
- Activity
- Contact
- Group
- Organization
- ProducerCode
- Region
- User

PolicyCenter validates these entities in no particular order.

Commit-cycle validation

Rule-based validation is also known as *commit-cycle* validation because it occurs only during a data bundle commit.

More information

See the following topics:

- The discussion of PolicyCenter validation in the *PolicyCenter Configuration* documentation.
- The discussion of the “Validation” on page 143 rule set.

About the validation graph

During database commit, PolicyCenter performs validation on the following items:

- Any validatable entity that PolicyCenter updated or inserted into the validation graph.
- Any validatable entity that refers to an entity that PolicyCenter updated, inserted, or removed from the validation graph.

PolicyCenter gathers the entities that reference a changed entity into a virtual graph. This graph maps all the paths from each type of entity to the top-level validatable entities like Account and ProducerCode. PolicyCenter queries these paths in the database or in memory to determine which validatable entities, if any, reference the entity that PolicyCenter inserted, updated, or removed from the validation graph.

PolicyCenter determines the validation graph by traversing the set of foreign keys and arrays that trigger validation. For example, the data model marks the ProducerCodeRoles array on ProducerCode as triggering validation. Therefore, any changes made to a producer code role causes PolicyCenter to validate the producer code as well.

PolicyCenter follows foreign keys and arrays that triggers validation through any links and arrays on the referenced entities down the object tree. For example, you might end up with a path like Policy → Contact → ContactAddress → Address.

Note: To actually trigger validation, you must set each link in the chain—Address, ContactAddress, and Contact—as triggering validation and set Policy as validatable.

PolicyCenter stores this path in reverse in the validation graph. Thus, if an address changes, PolicyCenter traverses the tree in reverse order from address to contact address. It then moves to the policy contact, and finally to policy (Address → ContactAddress → PolicyContact → Policy).

How PolicyCenter traverses the validation graph

If a entity is validatable, PolicyCenter applies the pre-update and validation rules any time that you modify the entity contents directly. Suppose that you update an object linked to another object. For example, a foreign key links each of the following objects to the User object:

- Contact
- Credential
- UserSettings

If you update a user's credential, you might reasonably expect the pre-update and validation rules to execute before PolicyCenter commits the updates to the database. However, updating a user's credentials does not normally trigger rules to the container object (User, in this case). The reason, implementation (at the metadata level) of a user's credential is a pointer from a User entity to a Credential entity.

The standard way to trigger the pre-update and validation rules on linked objects is by setting the triggersValidation pointer (foreign key) to the object in the metadata XML file. For example, in User.eti, you see the following:

```
<entity xmlns="http://guidewire.com/datamodel" ... entity="User">
  ...
  <implementsEntity name="Validatable"/>
  ...
  <foreignkey columnName="ContactID"
    desc="Contact entry related to the user."
    fkentity="UserContact"
    name="Contact"
    ...
    triggersValidation="true"/>
  <foreignkey columnName="CredentialID"
    desc="Security credential for the user."
    fkentity="Credential"
    name="Credential"
    ...
    triggersValidation="true"/>
  ...
</entity>
```

Setting the triggersValidation attribute to true ensures that PolicyCenter runs the pre-update and validation rules on the linked object any time that you modify it. (This is only true if the container object implements the Validatable delegate.)

In the base configuration, Guidewire sets the validation triggers so that modifying a validatable entity in a bundle causes PolicyCenter to validate that entity and all its parents.

You can use the `triggersValidation` attribute on foreign keys and arrays that you add to custom subtypes, to custom entities, or to core entities by using extensions. The default value of this attribute is `false` if you do not set it specifically. If you want a new foreign key or array element of a validatable entity to trigger validation whenever its linked entities change, set `triggersValidation` to `true`.

How PolicyCenter works with owned arrays

In the PolicyCenter data model, an *owned array* is an array that is attached to a validatable entity and which has its `triggersValidation` attribute set to `true`. PolicyCenter treats owned arrays in a special fashion.

PolicyCenter performs validation based on the validation graph. If an entity changes, PolicyCenter follows all the arrays and foreign keys that reference that entity and for which `triggersValidation="true"`. It then walks up the graph to find the validatable entities that ultimately references the changed entity. In the case of owned arrays, PolicyCenter considers any change to the array as a change to the parent entity.

Consider the following data model:

- Entity A has a foreign key to B
- Entity B has an owned array of C
- Array C has a foreign key to D

Suppose that both A and B are validatable. These relationships look like:

```
A → B → C[] → D
```

Suppose that you also mark both $A \rightarrow B$ and $C \rightarrow D$ as triggering validation. What happens with the $B \rightarrow C$ link?

- If the $B \rightarrow C$ array has its `triggersValidation` element set to `false`, changes to C cause PolicyCenter to validate B and A. This is because PolicyCenter treats the change as a change to B directly. A change to D does not cause PolicyCenter to validate anything. The graph stops at C. Because C is not actually changed, PolicyCenter does not consider B to have changed, and performs no validation.
- If the $B \rightarrow C$ link has its `triggersValidation` element set to `true`, changes to either C or D cause PolicyCenter to validate both B and A.

Top-level entities that trigger full validation

WARNING: Guidewire places the base entity definition files in the PolicyCenter installation modules/`configuration/config/metadata/entity` directory. Do not modify these files in any way. Any attempt to modify files in this directory can cause damage to the PolicyCenter application severe enough to prevent the application from starting thereafter.

To be validatable, an entity (or subtype, or delegate, or base object) must implement the `Validatable` delegate by adding the following to the entity definition:

```
<implementsEntity name="Validatable"/>DynamicGroupAssignmentStrategy
```

The following table lists those entities that trigger validation in the base PolicyCenter configuration. If a table cell is empty, then there are no additional entities associated with that top-level entity that trigger validation (`triggersValidation="true"`) in the base configuration.

Validatable entity	Foreign key entity	Array name
Account		RoleAssignments
Activity		
Contact	Address	ContactAddresses
Group		

Validatable entity	Foreign key entity	Array name
Organization		
ProducerCode		ProducerCodeRoles
Region		
User	Credential UserContact UserSettings	Attributes

Overriding validation triggers

In the base configuration, Guidewire sets validation to trigger on many top-level entities and on many of the arrays and foreign keys associated with them. You cannot modify the base metadata XML files in which these configurations are set. However, there can be occasions in which you want to override the default validation behavior. For example:

- You want to trigger validation upon modification of an entity in the PolicyCenter base data model that does not currently trigger validation.
- You do not want to trigger validation on entities on which PolicyCenter performs validation in the base configuration.

You can only override validation on certain objects associated with a base configuration entity. (It is not necessary to override your own extension entities as you simply set `triggersValidation` to the desired value as you define the entity.)

The following tables lists the data objects for which you can override validation, and, the XML override element to use. See the *Data Model Configuration* for information on these data objects.

Data field	Override element
<array>	<array-override>
<foreignkey>	<foreignkey-override>
<onetoone>	<onetoone-override>

Validation trigger example

In the base configuration, PolicyCenter runs the validation rules on the Account object during database commit. Guidewire defines the Account entity in `Account.eti`. The PolicyCenter base configuration first defines the Account entity as *validatable*, then defines the RoleAssignments array as triggering validation.

```
<entity entity="Account" table="account" type="retireable" lockable="true">
  ...
  <implementsEntity name="Validatable"/>
  ...
  <array arrayentity="UserRoleAssignment"
    cascadeDelete="true"
    desc="Role Assignments for this account."
    exportable="false"
    name="RoleAssignments"
    triggersValidation="true"/>
  ...
</entity>
```

Validation performance issues

There are three ways in which validation can cause performance problems:

- The rules themselves are too performance intensive.

- There are too many objects being validated.
- The queries used to determine which objects to validate take too long.

The following list summarizes some of the common validation issues and how to mitigate each one.

Validating large amounts of administration objects

Guidewire recommends that you never mark foreign keys and arrays that point to administration objects—such as User or Group objects—as triggering validation. An administration object is any object that a large number of other policies can reference. For example, suppose that you set the Group field on Policy to trigger validation. Editing one policy can then bring the entire application to a crawl as PolicyCenter validated hundreds or even thousands of policies. Setting this particular validation trigger would make editing policy objects nearly impossible.

Length of query paths

In some cases, an entity can have a large number of foreign keys pointing at it. Triggering validation on the entity can cause performance problems, as PolicyCenter must follow each of those chains of relationships during validation. The longer the paths through the tables, the more expensive the queries that PolicyCenter executes any time that an object changes. Having a consistent direction for graph references helps to avoid this.

Triggering validation on parent-pointing foreign keys on entities that have many possible owners (like account contact) can result in much longer and unintended paths. For example, a number of entities point to AccountContact. Each of these entities then contains links to other entities. Validating the entire web of relationships can have a serious negative performance impact.

Guidewire designs the PolicyCenter default configuration to minimize this issue. However, heavy modification of the default configuration using validation trigger overrides can introduce unintended performance issues.

Links between top-level objects

It is legal to have top-level entities trigger validation on each other. However, this use of validation triggers can unnecessarily increase the number of paths and the number of objects that PolicyCenter must validate on any particular commit.

Graph direction inconsistency

In general, Guidewire strongly recommends that you consistently order the validation graph to avoid problems:

- PolicyCenter consider arrays and foreign keys that represent some sort of containment as candidates for triggering validation.
- PolicyCenter does not consider foreign keys that you reference as arrays or that are merely there to indicate association as candidates for triggering validation.

Illegal links and arrays

Virtual foreign keys cannot trigger validation because PolicyCenter does not store them in the database. Attempting to use a virtual foreign key to trigger validation causes PolicyCenter to report an error at application startup. Similarly, it is not possible to set any array or link property defined at the application level (such as the Driver link on Policy) to trigger validation.

PolicyCenter considers an array to be in the database if the foreign key that forms the array is in the database.

View a text version of the validation graph

About this task

It is possible to view a text version of the PolicyCenter validation graph for aid in debugging validation issues.

Procedure

1. In PolicyCenter Studio **Project** window, expand **configuration > config > logging**.
2. Open file `log4j2-json.xml` for editing.
3. Add the following entry to this file:

```
<!-- PolicyCenter Claim Validation Graph -->
<Logger name="Plugin.com.guidewire.pl.system.bundle.validation.ValidationTriggerGraphImpl" additivity="false"
```

```
level="debug">  
<AppenderRef="PolicyValidationGraphLog"/>  
</Logger>
```

Results

This action causes the validation graph to print during application deployment both as text in the system console and to the following log file:

```
/tmp/gwlogs/PolicyValidationGraphLog.log
```

See also

- *InsuranceSuite Observability*